

Dat Mon - Food Delivery Android

1.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	11
2.1 Namespace List	11
3 Hierarchical Index	13
3.1 Class Hierarchy	13
4 Class Index	17
4.1 Class List	17
5 File Index	21
5.1 File List	21
6 Directory Documentation	23
6.1 app/app/src/main/java/com/example/myapp/adapters Directory Reference	23
6.2 app/app/src/main/java/com/example/myapp/api Directory Reference	24
6.3 app Directory Reference	24
6.4 app/app Directory Reference	24
6.5 backend Directory Reference	25
6.6 app/app/src/main/java/com Directory Reference	25
6.7 app/app/src/main/java/com/example Directory Reference	26
6.8 app/app/src/main/java Directory Reference	26
6.9 app/app/src/main Directory Reference	27
6.10 app/app/src/main/java/com/example/myapp Directory Reference	27
6.11 app/app/src/main/java/com/example/myapp/screens Directory Reference	28
6.12 app/app/src Directory Reference	29
7 Namespace Documentation	31
7.1 Package AddressAdapter	31
7.2 Package AddressAdapter.AddressViewHolder	31
7.3 Package ApiService	31
7.4 Package AppCompatActivity	31
7.4.1 Function Documentation	32
7.4.1.1 bindOrder()	32
7.4.1.2 bindOrderCard()	33
7.4.1.3 bindTopTwoOrders()	33
7.4.1.4 displayMenuItems()	34
7.4.1.5 displayRestaurants()	34
7.4.1.6 if() [1/2]	34
7.4.1.7 if() [2/2]	35
7.4.1.8 loadOrders()	35
7.4.1.9 onCreate()	35
7.4.1.10 openOrderDetail()	37

7.4.1.11 reorderCurrentOrder()	37
7.4.1.12 resolveUserIdForPoints()	37
7.4.1.13 searchByCategory()	38
7.4.1.14 searchMenuItems()	39
7.4.1.15 searchRestaurants()	39
7.4.1.16 show()	40
7.4.1.17 startActivity()	40
7.4.1.18 updateAddress()	41
7.4.1.19 updateOrderStatusToCODConfirmed()	42
7.4.2 Variable Documentation	42
7.4.2.1 currentAddress	42
7.4.2.2 edtPhone	42
7.4.2.3 firstOrderId	42
7.4.2.4 foodId	43
7.4.2.5 icCart	43
7.4.2.6 icHeart	43
7.4.2.7 icHome	43
7.4.2.8 icProfile	43
7.4.2.9 isRequestingVnPay	43
7.4.2.10 món	43
7.4.2.11 menuItems	44
7.4.2.12 nay	44
7.4.2.13 orderId	44
7.4.2.14 pointsCard2	44
7.4.2.15 pointsCard3	44
7.4.2.16 pointsCard4	44
7.4.2.17 recyclerView	44
7.4.2.18 restaurant	45
7.4.2.19 restaurantId	45
7.4.2.20 restaurants	45
7.4.2.21 rvAddresses	45
7.4.2.22 rvCart	45
7.4.2.23 rvNotifications	45
7.4.2.24 rvPreOrderCart	45
7.4.2.25 selectedPromotion	46
7.5 auth Namespace Reference	46
7.5.1 Detailed Description	46
7.5.2 Function Documentation	46
7.5.2.1 create_access_token()	46
7.5.2.2 get_current_user()	47
7.5.2.3 get_password_hash()	47
7.5.2.4 verify_password()	48
7.5.3 Variable Documentation	48

7.5.3.1 ACCESS_TOKEN_EXPIRE_MINUTES	48
7.5.3.2 ALGORITHM	49
7.5.3.3 oauth2_scheme	49
7.5.3.4 pwd_context	49
7.5.3.5 SECRET_KEY	49
7.6 Package CartItemAdapter	49
7.7 Package CartItemAdapter.CartItemViewHolder	49
7.7.1 Function Documentation	50
7.7.1.1 CartItemViewHolder()	50
7.8 chatbot_service Namespace Reference	50
7.8.1 Detailed Description	51
7.8.2 Variable Documentation	51
7.8.2.1 GEMINI_API_KEY	51
7.9 Package ChatMessageAdapter	51
7.10 Package ChatMessageAdapter.MessageViewHolder	51
7.11 check_db Namespace Reference	51
7.11.1 Variable Documentation	51
7.11.1.1 conn	51
7.11.1.2 cursor	51
7.11.1.3 fcm	52
7.11.1.4 notifs	52
7.11.1.5 tables	52
7.12 database Namespace Reference	52
7.12.1 Detailed Description	52
7.12.2 Function Documentation	52
7.12.2.1 get_db()	52
7.12.3 Variable Documentation	53
7.12.3.1 Base	53
7.12.3.2 engine	53
7.12.3.3 SessionLocal	53
7.12.3.4 SQLALCHEMY_DATABASE_URL	53
7.13 Package FAQAdapter	53
7.14 Package FAQAdapter.FAQViewHolder	53
7.15 Package FcmTokenRegistrar	53
7.16 firebase_utils Namespace Reference	53
7.16.1 Detailed Description	54
7.16.2 Function Documentation	54
7.16.2.1 __upload_image_sync()	54
7.16.2.2 init_firebase()	55
7.16.2.3 upload_image()	55
7.16.3 Variable Documentation	55
7.16.3.1 FIREBASE_CREDENTIALS_PATH	55
7.16.3.2 FIREBASE_STORAGE_BUCKET	56

7.17 Package FirebaseMessagingService	56
7.17.1 Function Documentation	56
7.17.1.1 onMessageReceived()	56
7.17.1.2 showNotification()	56
7.18 main Namespace Reference	57
7.18.1 Detailed Description	58
7.18.2 Function Documentation	58
7.18.2.1 build_menuitem_with_reviews()	58
7.18.2.2 chat_with_bot()	59
7.18.2.3 create_address()	60
7.18.2.4 create_menu_item()	61
7.18.2.5 create_notification()	61
7.18.2.6 create_order()	62
7.18.2.7 create_payment()	63
7.18.2.8 create_restaurant()	63
7.18.2.9 create_review()	64
7.18.2.10 create_user()	64
7.18.2.11 login_for_access_token()	65
7.18.2.12 mark_notification_as_read()	66
7.18.2.13 read_all_menu_items()	66
7.18.2.14 read_categories()	67
7.18.2.15 read_faqs()	67
7.18.2.16 read_menu_item_detail()	68
7.18.2.17 read_order_detail()	68
7.18.2.18 read_promotions()	69
7.18.2.19 read_restaurant()	69
7.18.2.20 read_restaurant_menu()	70
7.18.2.21 read_restaurants()	70
7.18.2.22 read_root()	70
7.18.2.23 read_user_addresses()	71
7.18.2.24 read_user_notifications()	71
7.18.2.25 read_user_orders()	71
7.18.2.26 read_user_profile_summary()	72
7.18.2.27 read_users()	72
7.18.2.28 read_users_me()	72
7.18.2.29 search_menu_items_by_category_name()	73
7.18.2.30 search_menu_items_by_name()	73
7.18.2.31 search_restaurants_by_name()	74
7.18.2.32 update_address()	74
7.18.2.33 update_device_token()	75
7.18.2.34 update_order_status()	75
7.18.2.35 vnpay_return()	76
7.18.3 Variable Documentation	77

7.18.3.1 allow_credentials	77
7.18.3.2 allow_headers	77
7.18.3.3 allow_methods	77
7.18.3.4 allow_origins	78
7.18.3.5 app	78
7.18.3.6 bind	78
7.18.3.7 host	78
7.18.3.8 port	78
7.19 Package MenuItemAdapter	78
7.20 Package MenuItemAdapter.MenuItemViewHolder	78
7.21 Package MenuItemAdapterSmall	78
7.22 Package MenuItemAdapterSmall.MenuItemViewHolder	79
7.23 models Namespace Reference	79
7.23.1 Detailed Description	79
7.24 Package NetworkConfig	79
7.25 notification_service Namespace Reference	79
7.25.1 Detailed Description	79
7.25.2 Function Documentation	80
7.25.2.1 notify_user()	80
7.25.2.2 send_fcm_notification()	80
7.26 Package NotificationAdapter	81
7.27 Package NotificationAdapter.NotificationViewHolder	81
7.28 payment_service Namespace Reference	81
7.28.1 Detailed Description	82
7.28.2 Function Documentation	82
7.28.2.1 generate_vnpay_url()	82
7.28.3 Variable Documentation	82
7.28.3.1 VNP_HASH_SECRET	82
7.28.3.2 VNP_RETURN_URL	83
7.28.3.3 VNP_TMNCODE	83
7.28.3.4 VNP_URL	83
7.29 Package RestaurantAdapter	83
7.30 Package RestaurantAdapter.RestaurantViewHolder	83
7.31 Package RetrofitClient	83
7.32 schemas Namespace Reference	83
7.32.1 Detailed Description	85
7.33 view_data Namespace Reference	85
7.33.1 Detailed Description	85
7.33.2 Function Documentation	85
7.33.2.1 get_all_tables()	85
7.33.2.2 view_all_data()	86
7.33.2.3 view_specific_table()	86
7.33.2.4 view_table_data()	87

7.33.3 Variable Documentation	88
7.33.3.1 limit	88
7.33.3.2 table_name	88
8 Class Documentation	89
8.1 models.Address Class Reference	89
8.1.1 Detailed Description	90
8.1.2 Member Data Documentation	90
8.1.2.1 __tablename__	90
8.1.2.2 detail	90
8.1.2.3 id	90
8.1.2.4 orders	90
8.1.2.5 phone	91
8.1.2.6 user	91
8.1.2.7 userid	91
8.2 schemas.Address Class Reference	91
8.2.1 Detailed Description	92
8.2.2 Member Data Documentation	92
8.2.2.1 model_config	92
8.3 schemas.AddressBase Class Reference	93
8.3.1 Detailed Description	93
8.3.2 Member Data Documentation	94
8.3.2.1 phone	94
8.4 schemas.AddressCreate Class Reference	94
8.4.1 Detailed Description	95
8.5 schemas.AddressUpdate Class Reference	95
8.5.1 Detailed Description	96
8.5.2 Member Data Documentation	96
8.5.2.1 detail	96
8.5.2.2 phone	96
8.6 models.Category Class Reference	96
8.6.1 Detailed Description	97
8.6.2 Member Data Documentation	97
8.6.2.1 __tablename__	97
8.6.2.2 description	97
8.6.2.3 id	98
8.6.2.4 menu_items	98
8.6.2.5 name	98
8.6.2.6 type	98
8.7 schemas.Category Class Reference	98
8.7.1 Detailed Description	99
8.7.2 Member Data Documentation	99
8.7.2.1 model_config	99

8.8 schemas.CategoryBase Class Reference	100
8.8.1 Detailed Description	100
8.8.2 Member Data Documentation	101
8.8.2.1 description	101
8.8.2.2 type	101
8.9 schemas.CategoryCreate Class Reference	101
8.9.1 Detailed Description	102
8.10 chatbot_service.ChatBotService Class Reference	102
8.10.1 Detailed Description	103
8.10.2 Constructor & Destructor Documentation	103
8.10.2.1 __init__()	103
8.10.3 Member Function Documentation	103
8.10.3.1 _call_gemini_sdk()	103
8.10.3.2 get_app_context()	104
8.10.3.3 get_response()	104
8.10.4 Member Data Documentation	105
8.10.4.1 _call_gemini_sdk	105
8.10.4.2 client	105
8.10.4.3 db	105
8.10.4.4 user_id	106
8.11 models.ChatMessage Class Reference	106
8.11.1 Detailed Description	107
8.11.2 Member Data Documentation	107
8.11.2.1 __tablename__	107
8.11.2.2 id	107
8.11.2.3 message	107
8.11.2.4 senderrole	107
8.11.2.5 sentat	107
8.11.2.6 session	107
8.11.2.7 sessionid	108
8.12 schemas.ChatMessage Class Reference	108
8.12.1 Detailed Description	109
8.12.2 Member Data Documentation	109
8.12.2.1 model_config	109
8.13 schemas.ChatMessageBase Class Reference	109
8.13.1 Detailed Description	110
8.14 schemas.ChatMessageCreate Class Reference	110
8.14.1 Detailed Description	111
8.15 models.ChatSession Class Reference	111
8.15.1 Detailed Description	112
8.15.2 Member Data Documentation	112
8.15.2.1 __tablename__	112
8.15.2.2 createdat	112

8.15.2.3 id	113
8.15.2.4 messages	113
8.15.2.5 notifications	113
8.15.2.6 status	113
8.15.2.7 user	113
8.15.2.8 userid	113
8.16 schemas.ChatSession Class Reference	114
8.16.1 Detailed Description	114
8.16.2 Member Data Documentation	114
8.16.2.1 messages	114
8.16.2.2 model_config	115
8.17 schemas.ChatSessionCreate Class Reference	115
8.17.1 Detailed Description	115
8.18 models.Delivery Class Reference	116
8.18.1 Detailed Description	116
8.18.2 Member Data Documentation	117
8.18.2.1 __tablename__	117
8.18.2.2 deliverytime	117
8.18.2.3 id	117
8.18.2.4 order	117
8.18.2.5 orderid	117
8.18.2.6 shipper	117
8.18.2.7 shipperid	117
8.18.2.8 status	118
8.19 schemas.Delivery Class Reference	118
8.19.1 Detailed Description	119
8.19.2 Member Data Documentation	119
8.19.2.1 model_config	119
8.20 schemas.DeliveryBase Class Reference	119
8.20.1 Detailed Description	120
8.20.2 Member Data Documentation	120
8.20.2.1 deliverytime	120
8.21 schemas.DeliveryCreate Class Reference	121
8.21.1 Detailed Description	122
8.22 models.FAQ Class Reference	122
8.22.1 Detailed Description	123
8.22.2 Member Data Documentation	123
8.22.2.1 __tablename__	123
8.22.2.2 answer	123
8.22.2.3 id	123
8.22.2.4 isactive	123
8.22.2.5 question	123
8.22.2.6 user_faqs	124

8.23 schemas.FAQ Class Reference	124
8.23.1 Detailed Description	125
8.23.2 Member Data Documentation	125
8.23.2.1 model_config	125
8.24 schemas.FAQBase Class Reference	125
8.24.1 Detailed Description	126
8.24.2 Member Data Documentation	126
8.24.2.1 isactive	126
8.25 schemas.FAQCreate Class Reference	127
8.25.1 Detailed Description	128
8.26 models.LoyaltyPoint Class Reference	128
8.26.1 Detailed Description	129
8.26.2 Member Data Documentation	129
8.26.2.1 __tablename__	129
8.26.2.2 id	129
8.26.2.3 menu_item	129
8.26.2.4 menuitemid	129
8.26.2.5 points	129
8.26.2.6 updatedat	129
8.26.2.7 user	130
8.26.2.8 userid	130
8.27 schemas.LoyaltyPoint Class Reference	130
8.27.1 Detailed Description	131
8.27.2 Member Data Documentation	131
8.27.2.1 model_config	131
8.28 schemas.LoyaltyPointBase Class Reference	132
8.28.1 Detailed Description	132
8.28.2 Member Data Documentation	133
8.28.2.1 points	133
8.29 schemas.LoyaltyPointCreate Class Reference	133
8.29.1 Detailed Description	134
8.30 models.MenuItem Class Reference	134
8.30.1 Detailed Description	135
8.30.2 Member Data Documentation	135
8.30.2.1 __tablename__	135
8.30.2.2 category	135
8.30.2.3 categoryid	135
8.30.2.4 description	136
8.30.2.5 id	136
8.30.2.6 image_url	136
8.30.2.7 is_available	136
8.30.2.8 name	136
8.30.2.9 order_items	136

8.30.2.10 price	136
8.30.2.11 restaurant	136
8.30.2.12 restaurantid	137
8.30.2.13 reviews	137
8.30.2.14 social_shares	137
8.31 schemas.MenuItem Class Reference	137
8.31.1 Detailed Description	138
8.31.2 Member Data Documentation	138
8.31.2.1 model_config	138
8.32 schemas.MenuItemBase Class Reference	139
8.32.1 Detailed Description	139
8.32.2 Member Data Documentation	139
8.32.2.1 description	139
8.32.2.2 image_url	140
8.32.2.3 is_available	140
8.33 schemas.MenuItemCreate Class Reference	140
8.33.1 Detailed Description	141
8.34 schemas.MenuItemWithRestaurant Class Reference	142
8.34.1 Detailed Description	143
8.34.2 Member Data Documentation	143
8.34.2.1 model_config	143
8.34.2.2 restaurant_name	143
8.35 schemas.MenuItemWithReviews Class Reference	143
8.35.1 Detailed Description	144
8.35.2 Member Data Documentation	144
8.35.2.1 avg_rating	144
8.35.2.2 model_config	145
8.35.2.3 restaurant_name	145
8.35.2.4 reviews	145
8.36 models.Notification Class Reference	145
8.36.1 Detailed Description	146
8.36.2 Member Data Documentation	146
8.36.2.1 __tablename__	146
8.36.2.2 content	146
8.36.2.3 createdat	146
8.36.2.4 id	147
8.36.2.5 isread	147
8.36.2.6 order	147
8.36.2.7 orderid	147
8.36.2.8 session	147
8.36.2.9 sessionid	147
8.36.2.10 title	147
8.36.2.11 type	147

8.36.2.12 user	148
8.36.2.13 userid	148
8.37 schemas.Notification Class Reference	148
8.37.1 Detailed Description	149
8.37.2 Member Data Documentation	149
8.37.2.1 model_config	149
8.38 schemas.NotificationBase Class Reference	150
8.38.1 Detailed Description	150
8.38.2 Member Data Documentation	151
8.38.2.1 isread	151
8.38.2.2 orderid	151
8.38.2.3 sessionid	151
8.39 schemas.NotificationCreate Class Reference	151
8.39.1 Detailed Description	152
8.40 schemas.Order Class Reference	153
8.40.1 Detailed Description	154
8.40.2 Member Data Documentation	154
8.40.2.1 model_config	154
8.40.2.2 order_items	154
8.41 schemas.OrderBase Class Reference	154
8.41.1 Detailed Description	155
8.41.2 Member Data Documentation	155
8.41.2.1 preorderdate	155
8.41.2.2 preordertime	155
8.42 schemas.OrderCreate Class Reference	155
8.42.1 Detailed Description	156
8.42.2 Member Data Documentation	156
8.42.2.1 order_items	156
8.43 schemas.OrderDetail Class Reference	157
8.43.1 Detailed Description	158
8.43.2 Member Data Documentation	158
8.43.2.1 address_detail	158
8.43.2.2 order_items	158
8.43.2.3 restaurant_name	158
8.44 models.OrderItem Class Reference	158
8.44.1 Detailed Description	159
8.44.2 Member Data Documentation	159
8.44.2.1 __tablename__	159
8.44.2.2 id	159
8.44.2.3 menu_item	160
8.44.2.4 menuitemid	160
8.44.2.5 order	160
8.44.2.6 orderid	160

8.44.2.7 price	160
8.44.2.8 quantity	160
8.45 schemas.OrderItem Class Reference	161
8.45.1 Detailed Description	161
8.45.2 Member Data Documentation	162
8.45.2.1 model_config	162
8.46 schemas.OrderItemBase Class Reference	162
8.46.1 Detailed Description	163
8.47 schemas.OrderItemCreate Class Reference	163
8.47.1 Detailed Description	164
8.48 schemas.OrderItemDetail Class Reference	164
8.48.1 Detailed Description	164
8.48.2 Member Data Documentation	165
8.48.2.1 image_url	165
8.48.2.2 menuitem_name	165
8.49 models.Orders Class Reference	165
8.49.1 Detailed Description	166
8.49.2 Member Data Documentation	166
8.49.2.1 __tablename__	166
8.49.2.2 address	166
8.49.2.3 addressid	166
8.49.2.4 createdat	167
8.49.2.5 delivery	167
8.49.2.6 id	167
8.49.2.7 notifications	167
8.49.2.8 order_items	167
8.49.2.9 payments	167
8.49.2.10 preorderdate	167
8.49.2.11 preordertime	167
8.49.2.12 restaurant	168
8.49.2.13 restaurantid	168
8.49.2.14 status	168
8.49.2.15 totalprice	168
8.49.2.16 used_promotions	168
8.49.2.17 user	168
8.49.2.18 userid	168
8.50 schemas.OrderStatusUpdate Class Reference	169
8.50.1 Detailed Description	169
8.51 models.Payment Class Reference	170
8.51.1 Detailed Description	170
8.51.2 Member Data Documentation	171
8.51.2.1 __tablename__	171
8.51.2.2 id	171

8.51.2.3 method	171
8.51.2.4 order	171
8.51.2.5 orderid	171
8.51.2.6 status	171
8.52 schemas.Payment Class Reference	172
8.52.1 Detailed Description	172
8.52.2 Member Data Documentation	173
8.52.2.1 model_config	173
8.53 schemas.PaymentBase Class Reference	173
8.53.1 Detailed Description	174
8.54 schemas.PaymentCreate Class Reference	174
8.54.1 Detailed Description	175
8.55 models.Promotion Class Reference	175
8.55.1 Detailed Description	176
8.55.2 Member Data Documentation	176
8.55.2.1 __tablename__	176
8.55.2.2 code	176
8.55.2.3 discounttype	176
8.55.2.4 discountvalue	176
8.55.2.5 expiredate	176
8.55.2.6 id	176
8.55.2.7 minordervalue	177
8.55.2.8 status	177
8.55.2.9 used_promotions	177
8.56 schemas.Promotion Class Reference	177
8.56.1 Detailed Description	178
8.56.2 Member Data Documentation	178
8.56.2.1 model_config	178
8.57 schemas.PromotionBase Class Reference	179
8.57.1 Detailed Description	179
8.58 schemas.PromotionCreate Class Reference	180
8.58.1 Detailed Description	180
8.59 models.Restaurant Class Reference	181
8.59.1 Detailed Description	182
8.59.2 Member Data Documentation	182
8.59.2.1 __tablename__	182
8.59.2.2 address	182
8.59.2.3 close_time	182
8.59.2.4 description	182
8.59.2.5 id	182
8.59.2.6 image_url	182
8.59.2.7 menu_items	183
8.59.2.8 name	183

8.59.2.9 open_time	183
8.59.2.10 orders	183
8.59.2.11 phone_number	183
8.59.2.12 rating	183
8.59.2.13 reviews	183
8.59.2.14 social_shares	183
8.59.2.15 status	184
8.60 schemas.Restaurant Class Reference	184
8.60.1 Detailed Description	185
8.60.2 Member Data Documentation	185
8.60.2.1 menu_items	185
8.60.2.2 model_config	185
8.61 schemas.RestaurantBase Class Reference	186
8.61.1 Detailed Description	186
8.61.2 Member Data Documentation	187
8.61.2.1 address	187
8.61.2.2 close_time	187
8.61.2.3 description	187
8.61.2.4 image_url	187
8.61.2.5 open_time	187
8.61.2.6 rating	187
8.62 schemas.RestaurantCreate Class Reference	188
8.62.1 Detailed Description	189
8.63 models.Review Class Reference	189
8.63.1 Detailed Description	190
8.63.2 Member Data Documentation	190
8.63.2.1 __tablename__	190
8.63.2.2 comment	190
8.63.2.3 id	190
8.63.2.4 menu_item	190
8.63.2.5 menuitemid	190
8.63.2.6 order	190
8.63.2.7 orderid	191
8.63.2.8 rating	191
8.63.2.9 restaurant	191
8.63.2.10 restaurantid	191
8.63.2.11 user	191
8.63.2.12 userid	191
8.64 schemas.Review Class Reference	192
8.64.1 Detailed Description	193
8.64.2 Member Data Documentation	193
8.64.2.1 model_config	193
8.65 schemas.ReviewBase Class Reference	193

8.65.1 Detailed Description	194
8.65.2 Member Data Documentation	194
8.65.2.1 comment	194
8.65.2.2 menuitemid	194
8.65.2.3 restaurantid	194
8.66 schemas.ReviewCreate Class Reference	195
8.66.1 Detailed Description	196
8.67 schemas.ReviewResponse Class Reference	196
8.67.1 Detailed Description	197
8.67.2 Member Data Documentation	197
8.67.2.1 comment	197
8.67.2.2 model_config	197
8.67.2.3 user_name	197
8.68 models.Shipper Class Reference	197
8.68.1 Detailed Description	198
8.68.2 Member Data Documentation	198
8.68.2.1 __tablename__	198
8.68.2.2 deliveries	198
8.68.2.3 description	199
8.68.2.4 id	199
8.68.2.5 name	199
8.68.2.6 phone	199
8.68.2.7 rating	199
8.68.2.8 status	199
8.69 schemas.Shipper Class Reference	200
8.69.1 Detailed Description	201
8.69.2 Member Data Documentation	201
8.69.2.1 model_config	201
8.70 schemas.ShipperBase Class Reference	201
8.70.1 Detailed Description	202
8.70.2 Member Data Documentation	202
8.70.2.1 description	202
8.70.2.2 rating	202
8.71 schemas.ShipperCreate Class Reference	202
8.71.1 Detailed Description	203
8.72 models.SocialShare Class Reference	203
8.72.1 Detailed Description	204
8.72.2 Member Data Documentation	204
8.72.2.1 __tablename__	204
8.72.2.2 context	205
8.72.2.3 createdat	205
8.72.2.4 id	205
8.72.2.5 menu_item	205

8.72.2.6 menuitemid	205
8.72.2.7 platform	205
8.72.2.8 restaurant	205
8.72.2.9 restaurantid	205
8.72.2.10 sharetype	206
8.72.2.11 user	206
8.72.2.12 userid	206
8.73 schemas.SocialShare Class Reference	206
8.73.1 Detailed Description	207
8.73.2 Member Data Documentation	207
8.73.2.1 model_config	207
8.74 schemas.SocialShareBase Class Reference	208
8.74.1 Detailed Description	208
8.74.2 Member Data Documentation	209
8.74.2.1 context	209
8.75 schemas.SocialShareCreate Class Reference	209
8.75.1 Detailed Description	210
8.76 schemas.Token Class Reference	210
8.76.1 Detailed Description	211
8.77 schemas.TokenData Class Reference	211
8.77.1 Detailed Description	212
8.77.2 Member Data Documentation	212
8.77.2.1 email	212
8.78 models.UsedPromotion Class Reference	212
8.78.1 Detailed Description	213
8.78.2 Member Data Documentation	213
8.78.2.1 __tablename__	213
8.78.2.2 id	213
8.78.2.3 order	213
8.78.2.4 orderid	213
8.78.2.5 promotion	214
8.78.2.6 promotionid	214
8.78.2.7 usedat	214
8.79 schemas.UsedPromotion Class Reference	214
8.79.1 Detailed Description	215
8.79.2 Member Data Documentation	215
8.79.2.1 model_config	215
8.80 schemas.UsedPromotionBase Class Reference	216
8.80.1 Detailed Description	216
8.81 schemas.UsedPromotionCreate Class Reference	217
8.81.1 Detailed Description	217
8.82 models.User Class Reference	218
8.82.1 Detailed Description	219

8.82.2 Member Data Documentation	219
8.82.2.1 __tablename__	219
8.82.2.2 addresses	219
8.82.2.3 chat_sessions	219
8.82.2.4 email	219
8.82.2.5 id	219
8.82.2.6 loyalty_points	219
8.82.2.7 name	220
8.82.2.8 notifications	220
8.82.2.9 orders	220
8.82.2.10 password	220
8.82.2.11 phone	220
8.82.2.12 reviews	220
8.82.2.13 role	220
8.82.2.14 social_shares	220
8.82.2.15 user_devices	221
8.82.2.16 user_faqs	221
8.82.2.17 username	221
8.83 schemas.User Class Reference	221
8.83.1 Detailed Description	222
8.83.2 Member Data Documentation	222
8.83.2.1 model_config	222
8.84 schemas.UserBase Class Reference	223
8.84.1 Detailed Description	223
8.85 schemas.UserCreate Class Reference	224
8.85.1 Detailed Description	224
8.86 models.UserDevice Class Reference	225
8.86.1 Detailed Description	225
8.86.2 Member Data Documentation	226
8.86.2.1 __tablename__	226
8.86.2.2 device_token	226
8.86.2.3 device_type	226
8.86.2.4 id	226
8.86.2.5 last_active	226
8.86.2.6 user	226
8.86.2.7 userid	226
8.87 schemas.UserDevice Class Reference	227
8.87.1 Detailed Description	228
8.87.2 Member Data Documentation	228
8.87.2.1 model_config	228
8.88 schemas.UserDeviceBase Class Reference	228
8.88.1 Detailed Description	229
8.88.2 Member Data Documentation	229

8.88.2.1 device_type	229
8.89 schemas.UserDeviceCreate Class Reference	229
8.89.1 Detailed Description	230
8.90 models.UserFAQ Class Reference	230
8.90.1 Detailed Description	231
8.90.2 Member Data Documentation	231
8.90.2.1 ____tablename____	231
8.90.2.2 faq	231
8.90.2.3 faqid	232
8.90.2.4 id	232
8.90.2.5 user	232
8.90.2.6 userid	232
8.90.2.7 viewedat	232
8.91 schemas.UserFAQ Class Reference	233
8.91.1 Detailed Description	233
8.91.2 Member Data Documentation	234
8.91.2.1 model_config	234
8.92 schemas.UserFAQBase Class Reference	234
8.92.1 Detailed Description	235
8.93 schemas.UserFAQCreate Class Reference	235
8.93.1 Detailed Description	236
8.94 schemas.UserProfileSummary Class Reference	236
8.94.1 Detailed Description	236
9 File Documentation	237
9.1 app/app/src/main/java/com/example/myapp/adapters/AddressAdapter.kt File Reference	237
9.2 AddressAdapter.kt	237
9.3 app/app/src/main/java/com/example/myapp/adapters/ChatMessageAdapter.kt File Reference	238
9.4 ChatMessageAdapter.kt	238
9.5 app/app/src/main/java/com/example/myapp/adapters/FAQAdapter.kt File Reference	238
9.6 FAQAdapter.kt	239
9.7 app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapter.kt File Reference	239
9.8 MenuItemAdapter.kt	239
9.9 app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapterSmall.kt File Reference	241
9.10 MenuItemAdapterSmall.kt	241
9.11 app/app/src/main/java/com/example/myapp/adapters/MyFirebaseMessagingService.kt File Reference	242
9.12 MyFirebaseMessagingService.kt	242
9.13 app/app/src/main/java/com/example/myapp/adapters/NotificationAdapter.kt File Reference	243
9.14 NotificationAdapter.kt	243
9.15 app/app/src/main/java/com/example/myapp/adapters/RestaurantAdapter.kt File Reference	244

9.16 RestaurantAdapter.kt	244
9.17 app/app/src/main/java/com/example/myapp/api/ApiService.kt File Reference	245
9.18 ApiService.kt	245
9.19 app/app/src/main/java/com/example/myapp/api/FcmTokenRegistrar.kt File Reference	249
9.20 FcmTokenRegistrar.kt	249
9.21 app/app/src/main/java/com/example/myapp/api/NetworkConfig.kt File Reference	251
9.22 NetworkConfig.kt	251
9.23 app/app/src/main/java/com/example/myapp/api/RetrofitClient.kt File Reference	251
9.24 RetrofitClient.kt	252
9.25 app/app/src/main/java/com/example/myapp/screens/activity_cart.kt File Reference	252
9.26 activity_cart.kt	253
9.27 app/app/src/main/java/com/example/myapp/screens/add_shipping_address.kt File Reference	254
9.28 add_shipping_address.kt	254
9.29 app/app/src/main/java/com/example/myapp/screens/cart.kt File Reference	255
9.30 cart.kt	256
9.31 app/app/src/main/java/com/example/myapp/screens/CartItemAdapter.kt File Reference	257
9.32 CartItemAdapter.kt	258
9.33 app/app/src/main/java/com/example/myapp/screens/chatbot.kt File Reference	258
9.34 chatbot.kt	259
9.35 app/app/src/main/java/com/example/myapp/screens/customer_support.kt File Reference	260
9.36 customer_support.kt	260
9.37 app/app/src/main/java/com/example/myapp/screens/delivered_order_details.kt File Reference	261
9.38 delivered_order_details.kt	261
9.39 app/app/src/main/java/com/example/myapp/screens/detail_support.kt File Reference	263
9.40 detail_support.kt	263
9.41 app/app/src/main/java/com/example/myapp/screens/discouts.kt File Reference	264
9.42 discouts.kt	264
9.43 app/app/src/main/java/com/example/myapp/screens/edit_shipping_address.kt File Reference	265
9.44 edit_shipping_address.kt	265
9.45 app/app/src/main/java/com/example/myapp/screens/food_detail.kt File Reference	267
9.46 food_detail.kt	268
9.47 app/app/src/main/java/com/example/myapp/screens/home.kt File Reference	271
9.48 home.kt	271
9.49 app/app/src/main/java/com/example/myapp/screens/list_restaurant.kt File Reference	273
9.50 list_restaurant.kt	274
9.51 app/app/src/main/java/com/example/myapp/screens/notification.kt File Reference	276
9.52 notification.kt	276
9.53 app/app/src/main/java/com/example/myapp/screens/order.kt File Reference	277
9.54 order.kt	278
9.55 app/app/src/main/java/com/example/myapp/screens/order_history.kt File Reference	282
9.56 order_history.kt	283

9.57	app/app/src/main/java/com/example/myapp/screens/OrderTracking.kt File Reference . .	284
9.58	OrderTracking.kt	284
9.59	app/app/src/main/java/com/example/myapp/screens/OrderTrackingDetail.kt File Reference	289
9.60	OrderTrackingDetail.kt	290
9.61	app/app/src/main/java/com/example/myapp/screens/payment_methods.kt File Reference	294
9.62	payment_methods.kt	294
9.63	app/app/src/main/java/com/example/myapp/screens/payment_successful.kt File Reference	296
9.64	payment_successful.kt	296
9.65	app/app/src/main/java/com/example/myapp/screens/PointsDetail.kt File Reference . . .	297
9.66	PointsDetail.kt	298
9.67	app/app/src/main/java/com/example/myapp/screens/pre_order.kt File Reference	299
9.68	pre_order.kt	300
9.69	app/app/src/main/java/com/example/myapp/screens/pre_order_cart.kt File Reference .	302
9.70	pre_order_cart.kt	302
9.71	app/app/src/main/java/com/example/myapp/screens/profile.kt File Reference	304
9.72	profile.kt	305
9.73	app/app/src/main/java/com/example/myapp/screens/restaurant_profile.kt File Reference	306
9.74	restaurant_profile.kt	306
9.75	app/app/src/main/java/com/example/myapp/screens/savedeliveryaddress.kt File Reference	308
9.76	savedeliveryaddress.kt	308
9.77	app/app/src/main/java/com/example/myapp/screens/share.kt File Reference	310
9.78	share.kt	310
9.79	app/app/src/main/java/com/example/myapp/screens/share_restaurant.kt File Reference	311
9.80	share_restaurant.kt	311
9.81	app/app/src/main/java/com/example/myapp/screens/shipping_address_added_successfully.kt File Reference	312
9.82	shipping_address_added_successfully.kt	312
9.83	app/app/src/main/java/com/example/myapp/screens/shipping_address_fixed_successfully.kt File Reference	313
9.84	shipping_address_fixed_successfully.kt	313
9.85	app/app/src/main/java/com/example/myapp/screens/signin.kt File Reference	314
9.86	signin.kt	314
9.87	app/app/src/main/java/com/example/myapp/screens/signup.kt File Reference	316
9.88	signup.kt	316
9.89	app/app/src/main/java/com/example/myapp/screens/start.kt File Reference	318
9.90	start.kt	318
9.91	app/app/src/main/java/com/example/myapp/screens/VNPayActivity.kt File Reference .	319
9.92	VNPayActivity.kt	319
9.93	backend/auth.py File Reference	320
9.94	auth.py	320
9.95	backend/chatbot_service.py File Reference	321
9.96	chatbot_service.py	322
9.97	backend/check_db.py File Reference	323

9.98	check_db.py	323
9.99	backend/database.py File Reference	324
9.100	database.py	324
9.101	backend/firebase_utils.py File Reference	324
9.102	firebase_utils.py	325
9.103	backend/main.py File Reference	326
9.104	main.py	327
9.105	backend/models.py File Reference	336
9.106	models.py	337
9.107	backend/notification_service.py File Reference	340
9.108	notification_service.py	340
9.109	backend/payment_service.py File Reference	341
9.110	payment_service.py	342
9.111	backend/schemas.py File Reference	342
9.112	schemas.py	344
9.113	backend/view_data.py File Reference	349
9.114	view_data.py	349
	Index	351

Chapter 1

Directory Hierarchy

1.1 Directories

adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
app	24
app	24
src	29
main	27
java	26
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28

activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
app	24
src	29
main	27
java	26
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254

cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
backend	25
auth.py	320
chatbot_service.py	321
check_db.py	323
database.py	324
firebase_utils.py	324
main.py	326
models.py	336
notification_service.py	340
payment_service.py	341
schemas.py	342
view_data.py	349
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245

FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discouts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255

CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discouts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
java	26
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263

discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
main	27
java	26
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271

list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238
FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296

PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discouts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319
src	29
main	27
java	26
com	25
example	26
myapp	27
adapters	23
AddressAdapter.kt	237
ChatMessageAdapter.kt	238

FAQAdapter.kt	238
MenuItemAdapter.kt	239
MenuItemAdapterSmall.kt	241
MyFirebaseMessagingService.kt	242
NotificationAdapter.kt	243
RestaurantAdapter.kt	244
api	24
ApiService.kt	245
FcmTokenRegistrar.kt	249
NetworkConfig.kt	251
RetrofitClient.kt	251
screens	28
activity_cart.kt	252
add_shipping_address.kt	254
cart.kt	255
CartItemAdapter.kt	257
chatbot.kt	258
customer_support.kt	260
delivered_order_details.kt	261
detail_support.kt	263
discounts.kt	264
edit_shipping_address.kt	265
food_detail.kt	267
home.kt	271
list_restaurant.kt	273
notification.kt	276
order.kt	277
order_history.kt	282
OrderTracking.kt	284
OrderTrackingDetail.kt	289
payment_methods.kt	294
payment_successful.kt	296
PointsDetail.kt	297
pre_order.kt	299
pre_order_cart.kt	302
profile.kt	304
restaurant_profile.kt	306
savedeliveryaddress.kt	308
share.kt	310
share_restaurant.kt	311
shipping_address_added_successfully.kt	312
shipping_address_fixed_successfully.kt	313
signin.kt	314
signup.kt	316
start.kt	318
VNPayActivity.kt	319

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

AddressAdapter	31
AddressAdapter.AddressViewHolder	31
ApiService	31
AppCompatActivity	31
auth	46
CartItemAdapter	49
CartItemAdapter.CartItemViewHolder	49
chatbot_service	50
ChatMessageAdapter	51
ChatMessageAdapter.MessageViewHolder	51
check_db	51
database	52
FAQAdapter	53
FAQAdapter.FAQViewHolder	53
FcmTokenRegistrar	53
firebase_utils	53
FirebaseMessagingService	56
main	57
MenuItemAdapter	78
MenuItemAdapter.MenuItemViewHolder	78
MenuItemAdapterSmall	78
MenuItemAdapterSmall.MenuItemViewHolder	79
models	79
NetworkConfig	79
notification_service	79
NotificationAdapter	81
NotificationAdapter.NotificationViewHolder	81
payment_service	81
RestaurantAdapter	83
RestaurantAdapter.RestaurantViewHolder	83
RetrofitClient	83
schemas	83
view_data	85

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Base	
models.Address	89
models.Category	96
models.ChatMessage	106
models.ChatSession	111
models.Delivery	116
models.FAQ	122
models.LoyaltyPoint	128
models.MenuItem	134
models.Notification	145
models.OrderItem	158
models.Orders	165
models.Payment	170
models.Promotion	175
models.Restaurant	181
models.Review	189
models.Shipper	197
models.SocialShare	203
models.UsedPromotion	212
models.User	218
models.UserDevice	225
models.UserFAQ	230
BaseModel	
schemas.AddressBase	93
schemas.Address	91
schemas.AddressCreate	94
schemas.AddressUpdate	95
schemas.CategoryBase	100
schemas.Category	98
schemas.CategoryCreate	101
schemas.ChatMessageBase	109
schemas.ChatMessage	108
schemas.ChatMessageCreate	110
schemas.ChatSession	114
schemas.ChatSessionCreate	115

schemas.DeliveryBase	119
schemas.Delivery	118
schemas.DeliveryCreate	121
schemas.FAQBase	125
schemas.FAQ	124
schemas.FAQCreate	127
schemas.LoyaltyPointBase	132
schemas.LoyaltyPoint	130
schemas.LoyaltyPointCreate	133
schemas.MenuItemBase	139
schemas.MenuItem	137
schemas.MenuItemCreate	140
schemas.MenuItemWithRestaurant	142
schemas.MenuItemWithReviews	143
schemas.NotificationBase	150
schemas.Notification	148
schemas.NotificationCreate	151
schemas.OrderBase	154
schemas.Order	153
schemas.OrderCreate	155
schemas.OrderDetail	157
schemas.OrderItemBase	162
schemas.OrderItem	161
schemas.OrderItemCreate	163
schemas.OrderItemDetail	164
schemas.OrderStatusUpdate	169
schemas.PaymentBase	173
schemas.Payment	172
schemas.PaymentCreate	174
schemas.PromotionBase	179
schemas.Promotion	177
schemas.PromotionCreate	180
schemas.RestaurantBase	186
schemas.Restaurant	184
schemas.RestaurantCreate	188
schemas.ReviewBase	193
schemas.Review	192
schemas.ReviewCreate	195
schemas.ReviewResponse	196
schemas.ShipperBase	201
schemas.Shipper	200
schemas.ShipperCreate	202
schemas.SocialShareBase	208
schemas.SocialShare	206
schemas.SocialShareCreate	209
schemas.Token	210
schemas.TokenData	211
schemas.UsedPromotionBase	216
schemas.UsedPromotion	214
schemas.UsedPromotionCreate	217
schemas.UserBase	223
schemas.User	221
schemas.UserCreate	224
schemas.UserDeviceBase	228
schemas.UserDevice	227
schemas.UserDeviceCreate	229

schemas.UserFAQBase	234
schemas.UserFAQ	233
schemas.UserFAQCreate	235
schemas.UserProfileSummary	236
chatbot_service.ChatBotService	102

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

models.Address	89
schemas.Address	91
schemas.AddressBase	93
schemas.AddressCreate	94
schemas.AddressUpdate	95
models.Category	96
schemas.Category	98
schemas.CategoryBase	100
schemas.CategoryCreate	101
chatbot_service.ChatBotService	102
models.ChatMessage	106
schemas.ChatMessage	108
schemas.ChatMessageBase	109
schemas.ChatMessageCreate	110
models.ChatSession	111
schemas.ChatSession	114
schemas.ChatSessionCreate	115
models.Delivery	116
schemas.Delivery	118
schemas.DeliveryBase	119
schemas.DeliveryCreate	121
models.FAQ	122
schemas.FAQ	124
schemas.FAQBase	125
schemas.FAQCreate	127
models.LoyaltyPoint	128
schemas.LoyaltyPoint	130
schemas.LoyaltyPointBase	132
schemas.LoyaltyPointCreate	133
models.MenuItem	134
schemas.MenuItem	137
schemas.MenuItemBase	139
schemas.MenuItemCreate	140
schemas.MenuItemWithRestaurant	142
schemas.MenuItemWithReviews	143

models.Notification	145
schemas.Notification	148
schemas.NotificationBase	150
schemas.NotificationCreate	151
schemas.Order	153
schemas.OrderBase	154
schemas.OrderCreate	155
schemas.OrderDetail	157
models.OrderItem	158
schemas.OrderItem	161
schemas.OrderItemBase	162
schemas.OrderItemCreate	163
schemas.OrderItemDetail	164
models.Orders	165
schemas.OrderStatusUpdate	169
models.Payment	170
schemas.Payment	172
schemas.PaymentBase	173
schemas.PaymentCreate	174
models.Promotion	175
schemas.Promotion	177
schemas.PromotionBase	179
schemas.PromotionCreate	180
models.Restaurant	181
schemas.Restaurant	184
schemas.RestaurantBase	186
schemas.RestaurantCreate	188
models.Review	189
schemas.Review	192
schemas.ReviewBase	193
schemas.ReviewCreate	195
schemas.ReviewResponse	196
models.Shipper	197
schemas.Shipper	200
schemas.ShipperBase	201
schemas.ShipperCreate	202
models.SocialShare	203
schemas.SocialShare	206
schemas.SocialShareBase	208
schemas.SocialShareCreate	209
schemas.Token	210
schemas.TokenData	211
models.UsedPromotion	212
schemas.UsedPromotion	214
schemas.UsedPromotionBase	216
schemas.UsedPromotionCreate	217
models.User	218
schemas.User	221
schemas.UserBase	223
schemas.UserCreate	224
models.UserDevice	225
schemas.UserDevice	227
schemas.UserDeviceBase	228
schemas.UserDeviceCreate	229
models.UserFAQ	230
schemas.UserFAQ	233
schemas.UserFAQBase	234
schemas.UserFAQCreate	235

schemas.UserProfileSummary	236
--	---------------------

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

app/app/src/main/java/com/example/myapp/adapters/AddressAdapter.kt	237
app/app/src/main/java/com/example/myapp/adapters/ChatMessageAdapter.kt	238
app/app/src/main/java/com/example/myapp/adapters/FAQAdapter.kt	238
app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapter.kt	239
app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapterSmall.kt	241
app/app/src/main/java/com/example/myapp/adapters/MyFirebaseMessagingService.kt	242
app/app/src/main/java/com/example/myapp/adapters/NotificationAdapter.kt	243
app/app/src/main/java/com/example/myapp/adapters/RestaurantAdapter.kt	244
app/app/src/main/java/com/example/myapp/api/ApiService.kt	245
app/app/src/main/java/com/example/myapp/api/FcmTokenRegistrar.kt	249
app/app/src/main/java/com/example/myapp/api/NetworkConfig.kt	251
app/app/src/main/java/com/example/myapp/api/RetrofitClient.kt	251
app/app/src/main/java/com/example/myapp/screens/activity_cart.kt	252
app/app/src/main/java/com/example/myapp/screens/add_shipping_address.kt	254
app/app/src/main/java/com/example/myapp/screens/cart.kt	255
app/app/src/main/java/com/example/myapp/screens/CartItemAdapter.kt	257
app/app/src/main/java/com/example/myapp/screens/chatbot.kt	258
app/app/src/main/java/com/example/myapp/screens/customer_support.kt	260
app/app/src/main/java/com/example/myapp/screens/delivered_order_details.kt	261
app/app/src/main/java/com/example/myapp/screens/detail_support.kt	263
app/app/src/main/java/com/example/myapp/screens/discouts.kt	264
app/app/src/main/java/com/example/myapp/screens/edit_shipping_address.kt	265
app/app/src/main/java/com/example/myapp/screens/food_detail.kt	267
app/app/src/main/java/com/example/myapp/screens/home.kt	271
app/app/src/main/java/com/example/myapp/screens/list_restaurant.kt	273
app/app/src/main/java/com/example/myapp/screens/notification.kt	276
app/app/src/main/java/com/example/myapp/screens/order.kt	277
app/app/src/main/java/com/example/myapp/screens/order_history.kt	282
app/app/src/main/java/com/example/myapp/screens/OrderTracking.kt	284
app/app/src/main/java/com/example/myapp/screens/OrderTrackingDetail.kt	289
app/app/src/main/java/com/example/myapp/screens/payment_methods.kt	294
app/app/src/main/java/com/example/myapp/screens/payment_successful.kt	296
app/app/src/main/java/com/example/myapp/screens/PointsDetail.kt	297
app/app/src/main/java/com/example/myapp/screens/pre_order.kt	299
app/app/src/main/java/com/example/myapp/screens/pre_order_cart.kt	302

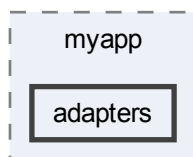
app/app/src/main/java/com/example/myapp/screens/ profile.kt	304
app/app/src/main/java/com/example/myapp/screens/ restaurant_profile.kt	306
app/app/src/main/java/com/example/myapp/screens/ savedeliveryaddress.kt	308
app/app/src/main/java/com/example/myapp/screens/ share.kt	310
app/app/src/main/java/com/example/myapp/screens/ share_restaurant.kt	311
app/app/src/main/java/com/example/myapp/screens/ shipping_address_added_successfully.kt	312
app/app/src/main/java/com/example/myapp/screens/ shipping_address_fixed_successfully.kt	313
app/app/src/main/java/com/example/myapp/screens/ signin.kt	314
app/app/src/main/java/com/example/myapp/screens/ signup.kt	316
app/app/src/main/java/com/example/myapp/screens/ start.kt	318
app/app/src/main/java/com/example/myapp/screens/ VNPayActivity.kt	319
backend/ auth.py	320
backend/ chatbot_service.py	321
backend/ check_db.py	323
backend/ database.py	324
backend/ firebase_utils.py	324
backend/ main.py	326
backend/ models.py	336
backend/ notification_service.py	340
backend/ payment_service.py	341
backend/ schemas.py	342
backend/ view_data.py	349

Chapter 6

Directory Documentation

6.1 app/app/src/main/java/com/example/myapp/adapters Directory Reference

Directory dependency graph for adapters:



Files

- file [AddressAdapter.kt](#)
- file [ChatMessageAdapter.kt](#)
- file [FAQAdapter.kt](#)
- file [MenuItemAdapter.kt](#)
- file [MenuItemAdapterSmall.kt](#)
- file [MyFirebaseMessagingService.kt](#)
- file [NotificationAdapter.kt](#)
- file [RestaurantAdapter.kt](#)

6.2 app/app/src/main/java/com/example/myapp/api Directory Reference

Directory dependency graph for api:



Files

- file [ApiService.kt](#)
- file [FcmTokenRegistrar.kt](#)
- file [NetworkConfig.kt](#)
- file [RetrofitClient.kt](#)

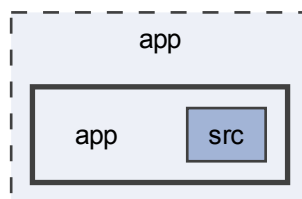
6.3 app Directory Reference

Directories

- directory [app](#)

6.4 app/app Directory Reference

Directory dependency graph for app:



Directories

- directory [src](#)

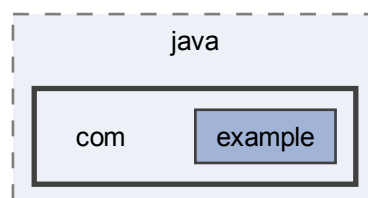
6.5 backend Directory Reference

Files

- file [auth.py](#)
- file [chatbot_service.py](#)
- file [check_db.py](#)
- file [database.py](#)
- file [firebase_utils.py](#)
- file [main.py](#)
- file [models.py](#)
- file [notification_service.py](#)
- file [payment_service.py](#)
- file [schemas.py](#)
- file [view_data.py](#)

6.6 app/app/src/main/java/com Directory Reference

Directory dependency graph for com:

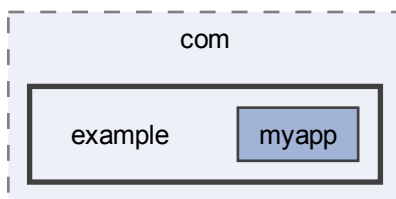


Directories

- directory [example](#)

6.7 app/app/src/main/java/com/example Directory Reference

Directory dependency graph for example:

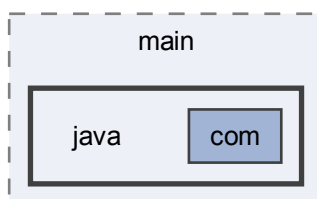


Directories

- directory [myapp](#)

6.8 app/app/src/main/java Directory Reference

Directory dependency graph for java:

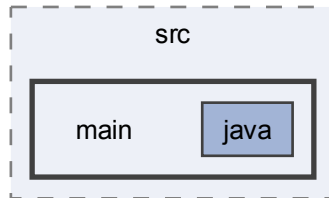


Directories

- directory [com](#)

6.9 app/app/src/main Directory Reference

Directory dependency graph for main:

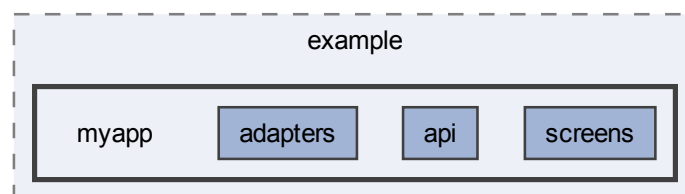


Directories

- directory [java](#)

6.10 app/app/src/main/java/com/example/myapp Directory Reference

Directory dependency graph for myapp:

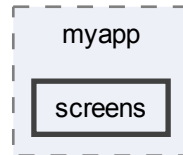


Directories

- directory [adapters](#)
- directory [api](#)
- directory [screens](#)

6.11 app/app/src/main/java/com/example/myapp/screens Directory Reference

Directory dependency graph for screens:

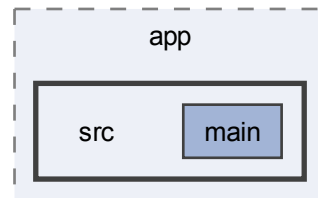


Files

- file [activity_cart.kt](#)
- file [add_shipping_address.kt](#)
- file [cart.kt](#)
- file [CartItemAdapter.kt](#)
- file [chatbot.kt](#)
- file [customer_support.kt](#)
- file [delivered_order_details.kt](#)
- file [detail_support.kt](#)
- file [discounts.kt](#)
- file [edit_shipping_address.kt](#)
- file [food_detail.kt](#)
- file [home.kt](#)
- file [list_restaurant.kt](#)
- file [notification.kt](#)
- file [order.kt](#)
- file [order_history.kt](#)
- file [OrderTracking.kt](#)
- file [OrderTrackingDetail.kt](#)
- file [payment_methods.kt](#)
- file [payment_successful.kt](#)
- file [PointsDetail.kt](#)
- file [pre_order.kt](#)
- file [pre_order_cart.kt](#)
- file [profile.kt](#)
- file [restaurant_profile.kt](#)
- file [savedeliveryaddress.kt](#)
- file [share.kt](#)
- file [share_restaurant.kt](#)
- file [shipping_address_added_successfully.kt](#)
- file [shipping_address_fixed_successfully.kt](#)
- file [signin.kt](#)
- file [signup.kt](#)
- file [start.kt](#)
- file [VNPayActivity.kt](#)

6.12 app/app/src Directory Reference

Directory dependency graph for src:



Directories

- directory [main](#)

Chapter 7

Namespace Documentation

7.1 Package AddressAdapter

Packages

- package [AddressViewHolder](#)

7.2 Package AddressAdapter.AddressViewHolder

7.3 Package ApiService

7.4 Package AppCompatActivity

Functions

- lateinit var Vui lòng chọn ít nhất Toast.LENGTH_SHORT. [show](#) () return @setOnClickListener }
- lateinit var chatbot::class.java [startActivity](#) (intent) } } private fun loadFAQs()
- var [if](#) ([orderId](#) > 0)
- fun [bindOrder](#) (order:OrderDetailResponse)
- fun [reorderCurrentOrder](#) ()
- override fun [onCreate](#) (savedInstanceState:Bundle?)
- lateinit var [if](#) ([addressId](#)==1)
- fun [updateAddress](#) (phone:String, address:String)
- fun [searchMenuItems](#) (query:String)
- fun [searchByCategory](#) (query:String)
- fun [displayMenuItems](#) ()
- fun [searchRestaurants](#) (query:String)
- fun [displayRestaurants](#) ()
- fun [bindTopTwoOrders](#) (orders:List< OrderResponse >)
- fun [openOrderDetail](#) (orderId:Int?)
- fun [updateOrderStatusToCODConfirmed](#) (orderId:Int)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val activity_ cart::class.java val profile::class.java fun [loadOrders](#) ()
- fun [bindOrderCard](#) (titleId:Int, pointsId:Int, order:OrderResponse?, fallbackTitlePrefix:String)
- fun [resolveUserIdForPoints](#) ()

Variables

- lateinit var [rvCart](#)
- lateinit var [edtPhone](#)
- lateinit var Vui lòng chọn ít nhất [món](#)
- lateinit var [recyclerView](#)
- lateinit var mình là trợ lý ảo của ứng dụng đặt món. Mình có thể giúp gì cho đơn hàng của bạn hôm nay
- var [orderId](#)
- var [selectedPromotion](#)
- var [menuItems](#)
- var [restaurants](#)
- lateinit var [rvNotifications](#)
- var [currentAddress](#)
- var [firstOrderId](#)
- var [isRequestingVnPay](#)
- var [firstOrderId](#) val [pointsCard2](#)
- var [firstOrderId](#) val secondOrderId val [pointsCard3](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val [pointsCard4](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val [icHome](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val [icHeart](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val [icCart](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val activity_cart::class.java val [icProfile](#)
- lateinit var [rvPreOrderCart](#)
- var [restaurant](#)
- lateinit var [rvAddresses](#)
- var [foodId](#)
- var [restaurantId](#)

7.4.1 Function Documentation

7.4.1.1 bindOrder()

```
fun AppCompatActivity.bindOrder (
    order: OrderDetailResponse ) [private]
```

Liên kết dữ liệu đơn hàng lên giao diện.

Hiển thị tiêu đề với mã đơn hàng và ngày tạo, danh sách món ăn, tổng số lượng, địa chỉ giao hàng và thành tiền.

Parameters

order	Chi tiết đơn hàng từ API
-------	--------------------------

Definition at line 128 of file [delivered_order_details.kt](#).

```
00128         : OrderDetailResponse) {
00129     loadedOrderDetail = order
00130     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00131     val title = findViewById<TextView>(R.id.tvTitle)
00132     val content = findViewById<TextView>(R.id.tvOrderContent)
```



```

00133
00134     val totalQuantity = order.order_items.sumOf { it.quantity }
00135     val itemSummary = if (order.order_items.isEmpty()) {
00136         "(Không có chi tiết món)"
00137     } else {
00138         order.order_items.joinToString("\n") {
00139             val itemName = it.menuitem_name ?: "Món #${it.menuitemid}"
00140             if (it.image_url.isNullOrEmpty()) {
00141                 "- $itemName x${it.quantity}"
00142             } else {
00143                 "- $itemName x${it.quantity}"
00144             }
00145         }
00146     }
00147
00148     title.text = "Chi tiết đơn hàng #${order.id} ngày ${formatDate(order.createdat)}"
00149     content.text = "${itemSummary}\nTổng số món: $totalQuantity\nĐịa chỉ: ${order.address_detail ?
    "N/A"}\nPhương thức thanh toán: Thanh toán online/cash\nVoucher: Không có\nThành tiền:
    ${fmt.format(order.totalprice)}đ"
00150 }

```

7.4.1.2 bindOrderCard()

```

fun AppCompatActivity.bindOrderCard (
    titleId:Int ,
    pointsId:Int ,
    order:OrderResponse? ,
    fallbackTitlePrefix:String ) [private]

```

Gán dữ liệu đơn hàng vào thẻ giao diện.

Hiển thị tên đơn hàng và điểm tích lũy (tổng giá / 1000). Nếu đơn hàng là null, hiển thị giá trị mặc định.

Parameters

titleId	ID resource của TextView hiển thị tên đơn hàng.
pointsId	ID resource của TextView hiển thị điểm tích lũy.
order	Đối tượng đơn hàng cần hiển thị, có thể null.
fallbackTitlePrefix	Tiền tố tiêu đề khi đơn hàng hợp lệ.

Definition at line 163 of file [PointsDetail.kt](#).

```

00163         : Int, pointsId: Int, order: OrderResponse?, fallbackTitlePrefix: String) {
00164     val titleView: TextView = findViewById(titleId)
00165     val pointsView: TextView = findViewById(pointsId)
00166
00167     if (order == null) {
00168         titleView.text = "Chưa có đơn hàng"
00169         pointsView.text = "0"
00170         return
00171     }
00172
00173     titleView.text = "$fallbackTitlePrefix${order.id}"
00174     pointsView.text = (order.totalprice / 1000).toString()
00175 }

```

7.4.1.3 bindTopTwoOrders()

```

fun AppCompatActivity.bindTopTwoOrders (
    orders:List< OrderResponse > ) [private]

```

Gán dữ liệu 2 đơn hàng gần nhất lên giao diện.

Hiển thị ngày, mã đơn, trạng thái cho mỗi đơn. Lưu firstOrderId và secondOrderId để mở chi tiết. Làm mờ nút "Xem chi tiết" nếu không có đơn hàng.

Parameters

orders	Danh sách đơn hàng đã sắp xếp
--------	-------------------------------

Definition at line 128 of file [order_history.kt](#).

```

00128             : List<OrderResponse>) {
00129     val tvDate1: TextView = findViewById(R.id.tvDate1)
00130     val tvDate2: TextView = findViewById(R.id.tvDate2)
00131     val tvViewDetails1: TextView = findViewById(R.id.tvViewDetails1)
00132     val tvViewDetails2: TextView = findViewById(R.id.tvViewDetails2)
00133
00134     val first = orders.getOrNull(0)
00135     val second = orders.getOrNull(1)
00136
00137     firstOrderId = first?.id
00138     secondOrderId = second?.id
00139
00140     tvDate1.text = first?.let { "${formatDate(it.createdat)} - #${it.id} - ${it.status}" } ?: "Chưa có đơn hàng"
00141     tvDate2.text = second?.let { "${formatDate(it.createdat)} - #${it.id} - ${it.status}" } ?: "Chưa có đơn hàng"
00142
00143     tvViewDetails1.alpha = if (first != null) 1f else 0.4f
00144     tvViewDetails2.alpha = if (second != null) 1f else 0.4f
00145 }
```

7.4.1.4 displayMenuItems()

```
fun AppCompatActivity.displayMenuItems () [private]
```

Hiển thị danh sách món ăn lên RecyclerView.

Tạo [MenuItemAdapter](#) mới với dữ liệu menuItems hiện tại và gán cho RecyclerView.

Definition at line 241 of file [home.kt](#).

```

00241     {
00242     menuItemAdapter = MenuItemAdapter(this, menuItems)
00243     rvMenuItems.adapter = menuItemAdapter
00244 }
```

7.4.1.5 displayRestaurants()

```
fun AppCompatActivity.displayRestaurants () [private]
```

Hiển thị danh sách nhà hàng lên RecyclerView.

Tạo mới [RestaurantAdapter](#) với danh sách hiện tại và gán cho RecyclerView.

Definition at line 206 of file [list_restaurant.kt](#).

```

00206     {
00207     restaurantAdapter = RestaurantAdapter(this, restaurants)
00208     rvRestaurants.adapter = restaurantAdapter
00209 }
```

7.4.1.6 if() [1/2]

```
lateinit var AppCompatActivity.if (
    addressId == -1) [private]
```

Definition at line 64 of file [edit_shipping_address.kt](#).

```

00064     {
00065     Toast.makeText(this, "Không tìm thấy địa chỉ", Toast.LENGTH_SHORT).show()
00066     finish()
00067     return
00068 }
```

7.4.1.7 if() [2/2]

```
var AppCompatActivity.if (
    orderId ,
    0 ) [private]
```

Definition at line 59 of file [delivered_order_details.kt](#).

```
00059      {
00060      loadOrderDetail(orderId)
00061      }
```

7.4.1.8 loadOrders()

```
var firstOrderId val secondOrderId val thirdOrderId val fourthOrderId val home.class.java val Wishlist clicked val activity←
_cart.class.java val profile.class.java fun AppCompatActivity.loadOrders () [private]
```

Tải danh sách đơn hàng từ API.

Gọi API lấy đơn hàng theo userId, lọc các đơn đã hoàn thành, sắp xếp theo thời gian tạo giảm dần, và hiển thị lên 4 thẻ.

Definition at line 122 of file [PointsDetail.kt](#).

```
00122      {
00123      val userId = resolveUserIdForPoints()
00124      RetrofitClient.apiService.getUserOrders(userId).enqueue(object : Callback<List<OrderResponse>» {
00125      override fun onResponse(call: Call<List<OrderResponse>», response: Response<List<OrderResponse>») {
00126      if (!response.isSuccessful || response.body().isEmpty()) {
00127      Toast.makeText(this@PointsDetail, "Không có đơn hàng để đánh giá", Toast.LENGTH_SHORT).show()
00128      return
00129      }
00130
00131      val completedOrders = response.body()!!
00132      .filter { it.status == "completed" }
00133      .sortedByDescending { it.createdAt }
00134
00135      firstOrderId = completedOrders.getOrNull(0)?.id
00136      secondOrderId = completedOrders.getOrNull(1)?.id
00137      thirdOrderId = completedOrders.getOrNull(2)?.id
00138      fourthOrderId = completedOrders.getOrNull(3)?.id
00139
00140      bindOrderCard(R.id.tvDishName1, R.id.tvPoints1, completedOrders.getOrNull(0), "Đơn hàng #")
00141      bindOrderCard(R.id.tvDishName2, R.id.tvPoints2, completedOrders.getOrNull(1), "Đơn hàng #")
00142      bindOrderCard(R.id.tvDishName3, R.id.tvPoints3, completedOrders.getOrNull(2), "Đơn hàng #")
00143      bindOrderCard(R.id.tvDishName4, R.id.tvPoints4, completedOrders.getOrNull(3), "Đơn hàng #")
00144      }
00145
00146      override fun onFailure(call: Call<List<OrderResponse>», t: Throwable) {
00147      Toast.makeText(this@PointsDetail, "Không tải được đơn hàng", Toast.LENGTH_SHORT).show()
00148      }
00149      })
00150      }
```

7.4.1.9 onCreate()

```
override fun AppCompatActivity.onCreate (
    savedInstanceState: Bundle? )
```

Khởi tạo Activity, thiết lập giao diện và các sự kiện click.

Thiết lập nút quay lại (đóng Activity) và nút chuyển đến chatbot.

Khởi tạo Activity, cập nhật trạng thái đơn COD và thiết lập giao diện.

Nếu order_id hợp lệ, gọi API cập nhật trạng thái thành "confirmed". Thiết lập nút trở về trang chủ với FLAG_ACTIVITY_CLEAR_TASK, các nút điều hướng (profile, home, giỏ hàng).

Khởi tạo màn hình hồ sơ.

Đọc thông tin người dùng từ SharedPreferences, gọi API lấy điểm tích lũy, gán sự kiện click cho tất cả các nút điều hướng và chức năng.

Parameters

savedInstanceState	Trạng thái đã lưu của Activity (nếu có)
--------------------	---

Khởi tạo màn hình thông báo thêm địa chỉ thành công.

Thiết lập nút "Trở về trang chủ" với FLAG_ACTIVITY_CLEAR_TASK để xóa ngăn xếp Activity, cùng thanh điều hướng dưới cùng.

Parameters

savedInstanceState	Trạng thái đã lưu trước đó (nếu có).
--------------------	--------------------------------------

Khởi tạo màn hình thông báo sửa địa chỉ thành công.

Thiết lập nút "Trở về trang chủ" với FLAG_ACTIVITY_CLEAR_TASK để xóa ngăn xếp Activity, cùng thanh điều hướng dưới cùng.

Parameters

savedInstanceState	Trạng thái đã lưu trước đó (nếu có).
--------------------	--------------------------------------

Khởi tạo màn hình đăng nhập.

Thiết lập layout, gán sự kiện click cho nút quay lại và nút đăng nhập. Xác thực đầu vào, gọi API đăng nhập, lưu token và chuyển màn hình.

Parameters

savedInstanceState	Trạng thái đã lưu của Activity (nếu có)
--------------------	---

Khởi tạo màn hình đăng ký.

Thiết lập layout, gán sự kiện click cho nút quay lại, nút chuyển sang đăng nhập và nút đăng ký. Xác thực dữ liệu đầu vào trước khi gọi API đăng ký.

Parameters

savedInstanceState	Trạng thái đã lưu của Activity (nếu có)
--------------------	---

Definition at line 31 of file [detail_support.kt](#).

```

00031         : Bundle?) {
00032     super.onCreate(savedInstanceState)
00033     setContentView(R.layout.detailsupport)
00034
00035     // click btnBack
00036     val btnBack: ImageView = findViewById(R.id.btn_back)
00037     btnBack.setOnClickListener {
00038         finish() // quay lại màn hình trước (start)
00039     }
00040
00041     // click btn_chatbot
00042     val btnchatbot: View = findViewById(R.id.btnChatbot)
00043     btnchatbot.setOnClickListener {
00044         val intent = Intent(this, chatbot::class.java)
00045         startActivity(intent)
00046     }
00047 }
```

7.4.1.10 openOrderDetail()

```
fun AppCompatActivity.openOrderDetail (
    orderId: Int? ) [private]
```

Mở màn hình chi tiết đơn hàng.

Parameters

<code>orderId</code>	ID đơn hàng cần xem chi tiết. Nếu null, hiển thị Toast lỗi.
----------------------	---

Definition at line 152 of file `order_history.kt`.

```
00152         : Int?) {
00153     if (orderId == null) {
00154         Toast.makeText(this, "Không có đơn để xem", Toast.LENGTH_SHORT).show()
00155         return
00156     }
00157     val intent = Intent(this, delivered_order_details::class.java)
00158     intent.putExtra("order_id", orderId)
00159     startActivity(intent)
00160 }
```

7.4.1.11 reorderCurrentOrder()

```
fun AppCompatActivity.reorderCurrentOrder () [private]
```

Xử lý chức năng đặt lại đơn hàng.

Chuyển sang màn hình đặt hàng (order) với `reorder_order_id` để tự động tải lại các món ăn từ đơn hàng cũ. Hiển thị Toast nếu đơn hàng chưa được tải xong.

Definition at line 159 of file `delivered_order_details.kt`.

```
00159         {
00160     val sourceOrder = loadedOrderDetail
00161     if (sourceOrder == null) {
00162         Toast.makeText(this, "Đơn hàng chưa tải xong", Toast.LENGTH_SHORT).show()
00163         return
00164     }
00165
00166     // Thay vì gọi API tạo đơn ngay, ta chuyển sang màn hình Thanh toán (Order)
00167     // Màn hình Order đã có sẵn logic nhận "reorder_order_id" để tự tải lại các món cũ
00168     val intent = Intent(this, order::class.java)
00169     intent.putExtra("reorder_order_id", sourceOrder.id)
00170     startActivity(intent)
00171 }
```

7.4.1.12 resolveUserIdForPoints()

```
fun AppCompatActivity.resolveUserIdForPoints () [private]
```

Xác định ID người dùng cho màn hình điểm tích lũy.

Đọc `user_id` từ `SharedPreferences`. Nếu không hợp lệ, trả về 1 làm mặc định.

Returns

ID của người dùng hiện tại. Chuyển đến màn hình chi tiết đơn hàng.

Truyền tên đơn hàng và mã đơn hàng sang màn hình OrderTrackingDetail. Hiển thị Toast nếu thiếu mã đơn hàng.

Parameters

dishName	Tên món ăn hoặc đơn hàng.
orderId	ID đơn hàng, có thể null.

Definition at line 184 of file [PointsDetail.kt](#).

```

00184         : Int {
00185     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00186     val userId = sharedPref.getInt("user_id", -1)
00187     if (userId > 0) return userId
00188
00189     val userName = sharedPref.getString("user_name", "") ?: ""
00190     if (userName.contains("Trung", ignoreCase = true) || userName.isBlank()) {
00191         return 1
00192     }
00193     return 1
00194 }
00195
00205 private fun navigateToOrderDetail(dishName: String, orderId: Int?) {
00206     if (orderId == null) {
00207         Toast.makeText(this, "Thiếu mã đơn hàng để đánh giá", Toast.LENGTH_SHORT).show()
00208         return
00209     }
00210     val intent = Intent(this, OrderTrackingDetail::class.java)
00211     intent.putExtra("order_name", dishName)
00212     intent.putExtra("order_id", orderId)
00213     startActivity(intent)
00214 }

```

7.4.1.13 searchByCategory()

```

fun AppCompatActivity.searchByCategory (
    query:String ) [private]

```

Tìm kiếm món ăn theo danh mục.

Được gọi khi tìm kiếm theo tên không có kết quả. Hiển thị Toast nếu không tìm thấy kết quả nào.

Parameters

query	Từ khóa tìm kiếm danh mục
-------	---------------------------

Definition at line 211 of file [home.kt](#).

```

00211         : String) {
00212     searchJob = RetrofitClient.apiService.searchMenuItemsByCategory(query)
00213     searchJob?.enqueue(object : Callback<List<MenuItem>» {
00214         override fun onResponse(call: Call<List<MenuItem>», response: Response<List<MenuItem>») {
00215             if (response.isSuccessful) {
00216                 val results = response.body() ?: emptyList()
00217                 menuItems = results
00218                 displayMenuItems()
00219                 if (results.isEmpty()) {
00220                     Toast.makeText(this@home, "Không tìm thấy món ăn", Toast.LENGTH_SHORT).show()
00221                 }
00222             } else {
00223                 Toast.makeText(this@home, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00224             }
00225         }
00226     })
00227     override fun onFailure(call: Call<List<MenuItem>», t: Throwable) {
00228         if (!call.isCanceled) {
00229             Toast.makeText(this@home, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00230         }
00231     }
00232 })
00233 }

```

7.4.1.14 searchMenuItems()

```
fun AppCompatActivity.searchMenuItems (
    query:String ) [private]
```

Tìm kiếm món ăn theo tên.

Hủy yêu cầu tìm kiếm cũ, gửi yêu cầu mới với từ khóa. Nếu không tìm thấy theo tên, tự động chuyển sang tìm theo danh mục.

Parameters

query	Từ khóa tìm kiếm
-------	------------------

Definition at line 173 of file [home.kt](#).

```
00173         : String) {
00174     // Cancel any ongoing search
00175     searchJob?.cancel()
00176
00177     // First search by name
00178     searchJob = RetrofitClient.apiService.searchMenuItemsByName(query)
00179     searchJob?.enqueue(object : Callback<List<MenuItem>» {
00180         override fun onResponse(call: Call<List<MenuItem>», response: Response<List<MenuItem>») {
00181             if (response.isSuccessful) {
00182                 val results = response.body() ?: emptyList()
00183                 if (results.isNotEmpty()) {
00184                     menuItems = results
00185                     displayMenuItems()
00186                 } else {
00187                     // If no results by name, search by category
00188                     searchByCategory(query)
00189                 }
00190             } else {
00191                 Toast.makeText(this@home, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00192             }
00193         }
00194     })
00195     override fun onFailure(call: Call<List<MenuItem>», t: Throwable) {
00196         if (!call.isCanceled) {
00197             Toast.makeText(this@home, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00198         }
00199     }
00200 }
00201 }
```

7.4.1.15 searchRestaurants()

```
fun AppCompatActivity.searchRestaurants (
    query:String ) [private]
```

Tìm kiếm nhà hàng theo tên từ API.

Hủy tìm kiếm trước đó, gọi API searchRestaurantsByName với query. Hiển thị Toast nếu không tìm thấy kết quả.

Parameters

query	Từ khóa tìm kiếm tên nhà hàng
-------	-------------------------------

Definition at line 174 of file [list_restaurant.kt](#).

```
00174         : String) {
00175     // Cancel any ongoing search
00176     searchJob?.cancel()
00177 }
```

```

00178     searchJob = RetrofitClient.apiService.searchRestaurantsByName(query)
00179     searchJob?.enqueue(object : Callback<List<Restaurant>» {
00180         override fun onResponse(call: Call<List<Restaurant>», response: Response<List<Restaurant>») {
00181             if (response.isSuccessful) {
00182                 val results = response.body() ?: emptyList()
00183                 restaurants = results
00184                 displayRestaurants()
00185                 if (results.isEmpty()) {
00186                     Toast.makeText(this@list_restaurant, "Không tìm thấy nhà hàng", Toast.LENGTH_SHORT).show()
00187                 }
00188             } else {
00189                 Toast.makeText(this@list_restaurant, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00190             }
00191         }
00192     })
00193     override fun onFailure(call: Call<List<Restaurant>», t: Throwable) {
00194         if (!call.isCanceled) {
00195             Toast.makeText(this@list_restaurant, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00196         }
00197     }
00198 })
00199 }

```

7.4.1.16 show()

lateinit var Vui lòng chọn ít nhất Toast.LENGTH_SHORT. AppCompatActivity.show () [private]

7.4.1.17 startActivity()

lateinit var class home.java AppCompatActivity.startActivity (
 intent) [private]

Tải danh sách câu hỏi thường gặp từ API.

Gọi API getFAQs và tạo [FAQAdapter](#) với dữ liệu nhận được. Hiển thị Toast nếu tải thất bại.

Tải chi tiết đơn hàng từ API theo ID.

Gọi API getOrderDetail và hiển thị dữ liệu lên giao diện. Hiển thị Toast lỗi nếu tải thất bại.

Parameters

orderId	ID của đơn hàng cần tải
-------------------------	-------------------------

Gọi API lấy tất cả món ăn từ server.

Hủy yêu cầu tìm kiếm đang chạy trước khi gửi yêu cầu mới. Cập nhật danh sách menuItems và hiển thị lên RecyclerView. Hiển thị Toast lỗi nếu kết nối thất bại.

Tải toàn bộ danh sách nhà hàng từ API.

Hủy bất kỳ tìm kiếm đang chạy trước khi gọi API getRestaurants(). Cập nhật RecyclerView khi nhận dữ liệu thành công.

Tải danh sách đơn hàng từ API.

Lấy userId, gọi API getUserOrders, lọc các đơn có trạng thái "completed", sắp xếp theo ngày giảm dần, và hiển thị 2 đơn gần nhất.

Definition at line 68 of file [customer_support.kt](#).

```
00078     {
```



```

00079         RetrofitClient.apiService.getFAQs().enqueue(object : Callback<List<FAQ> {
00080             override fun onResponse(call: Call<List<FAQ>, response: Response<List<FAQ>) {
00081                 if (response.isSuccessful) {
00082                     val faqs = response.body() ?: emptyList()
00083                     adapter = FAQAdapter(faqs)
00084                     recyclerView.adapter = adapter
00085                 } else {
00086                     Toast.makeText(this@customer_support,
00087                         "Lỗi tải FAQ: ${response.code()}", Toast.LENGTH_SHORT).show()
00088                 }
00089             }
00090         })
00091         override fun onFailure(call: Call<List<FAQ>, t: Throwable) {
00092             Toast.makeText(this@customer_support,
00093                 "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00094         }
00095     })
00096 }

```

7.4.1.18 updateAddress()

```

fun AppCompatActivity.updateAddress (
    phone:String ,
    address:String ) [private]

```

Gọi API cập nhật địa chỉ giao hàng.

Tạo Retrofit client, xây dựng request với số điện thoại và địa chỉ mới, gọi API updateAddress. Chuyển sang màn hình thành công nếu cập nhật thành công.

Parameters

phone	Số điện thoại liên hệ mới.
address	Địa chỉ giao hàng mới.

Definition at line 169 of file [edit_shipping_address.kt](#).

```

00169         : String, address: String) {
00170         val retrofit = Retrofit.Builder()
00171             .baseUrl("http://10.0.2.2:8000/") // Android emulator localhost
00172             .addConverterFactory(GsonConverterFactory.create())
00173             .build()
00174
00175         val apiService = retrofit.create(ApiService::class.java)
00176
00177         val addressUpdateRequest = AddressUpdateRequest(
00178             detail = address,
00179             phone = phone
00180         )
00181
00182         apiService.updateAddress(addressId, addressUpdateRequest).enqueue(object : Callback<Address> {
00183             override fun onResponse(call: Call<Address>, response: Response<Address>) {
00184                 if (response.isSuccessful) {
00185                     // Thành công - chuyển sang màn hình thông báo
00186                     Toast.makeText(this@edit_shipping_address, "Cập nhật địa chỉ thành công!",
00187                         Toast.LENGTH_SHORT).show()
00188                     val intent = Intent(this@edit_shipping_address, shipping_address_fixed_successfully::class.java)
00189                     startActivity(intent)
00190                     finish()
00191                 } else {
00192                     Toast.makeText(this@edit_shipping_address, "Không thể cập nhật địa chỉ",
00193                         Toast.LENGTH_SHORT).show()
00194                 }
00195             }
00196         })
00197         override fun onFailure(call: Call<Address>, t: Throwable) {
00198             Toast.makeText(this@edit_shipping_address, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00199         }
00200     })
00201 }

```

7.4.1.19 updateOrderStatusToCODConfirmed()

```
fun AppCompatActivity.updateOrderStatusToCODConfirmed (
    orderId: Int ) [private]
```

Cập nhật trạng thái đơn hàng COD thành "confirmed".

Gọi API updateOrderStatus để xác nhận đơn hàng COD, từ đó backend sẽ gửi thông báo đến khách hàng. Hiển thị Toast nếu cập nhật thất bại.

Parameters

<code>orderId</code>	ID đơn hàng cần cập nhật trạng thái
----------------------	-------------------------------------

Definition at line 94 of file [payment_successful.kt](#).

```
00094         : Int) {
00095     RetrofitClient.apiService.updateOrderStatus(orderId, OrderStatusUpdateRequest("confirmed"))
00096     .enqueue(object : Callback<OrderResponse> {
00097         override fun onResponse(call: Call<OrderResponse>, response: Response<OrderResponse>) {
00098             if (response.isSuccessful) {
00099                 // Thông báo sẽ được gửi từ backend
00100             } else {
00101                 Toast.makeText(this@payment_successful, "Cập nhật đơn hàng thất bại",
00102                     Toast.LENGTH_SHORT).show()
00103             }
00104         }
00105         override fun onFailure(call: Call<OrderResponse>, t: Throwable) {
00106             Toast.makeText(this@payment_successful, "Lỗi mạng", Toast.LENGTH_SHORT).show()
00107         }
00108     })
00109 }
```

7.4.2 Variable Documentation

7.4.2.1 currentAddress

```
var AppCompatActivity.currentAddress [private]
```

Địa chỉ giao hàng hiện tại của người dùng

Definition at line 49 of file [order.kt](#).

7.4.2.2 edtPhone

```
lateinit var AppCompatActivity.edtPhone [private]
```

EditText nhập số điện thoại liên hệ

Definition at line 37 of file [add_shipping_address.kt](#).

7.4.2.3 firstOrderId

```
var AppCompatActivity.firstOrderId [private]
```

ID của đơn hàng đầu tiên (gần nhất)

ID của đơn hàng thứ nhất

Definition at line 40 of file [order_history.kt](#).

7.4.2.4 foodId

```
var AppCompatActivity.foodId [private]
```

ID của món ăn cần chia sẻ

Definition at line 32 of file [share.kt](#).

7.4.2.5 icCart

```
var firstOrderId val secondOrderId val thirdOrderId val fourthOrderId val home.class.java val Wishlist clicked val AppCompatActivity.icCart [private]
```

Definition at line 104 of file [PointsDetail.kt](#).

7.4.2.6 icHeart

```
var firstOrderId val secondOrderId val thirdOrderId val fourthOrderId val home.class.java val AppCompatActivity.icHeart [private]
```

Definition at line 98 of file [PointsDetail.kt](#).

7.4.2.7 icHome

```
var firstOrderId val secondOrderId val thirdOrderId val fourthOrderId val AppCompatActivity.icHome [private]
```

Definition at line 92 of file [PointsDetail.kt](#).

7.4.2.8 icProfile

```
var firstOrderId val secondOrderId val thirdOrderId val fourthOrderId val home.class.java val Wishlist clicked val activity<->_cart.class.java val AppCompatActivity.icProfile [private]
```

Definition at line 110 of file [PointsDetail.kt](#).

7.4.2.9 isRequestingVnPay

```
var AppCompatActivity.isRequestingVnPay [private]
```

Definition at line 46 of file [payment_methods.kt](#).

7.4.2.10 món

```
lateinit var Vui lòng chọn ít nhất AppCompatActivity.món [private]
```

Definition at line 103 of file [cart.kt](#).

7.4.2.11 menuItems

```
var AppCompatActivity.menuItems [private]
```

Danh sách tất cả món ăn lấy từ API

Definition at line 46 of file [home.kt](#).

7.4.2.12 nay

lateinit var mình là trợ lý ảo của ứng dụng đặt món. Mình có thể giúp gì cho đơn hàng của bạn hôm AppCompatActivity.nay [private]

Definition at line 75 of file [chatbot.kt](#).

7.4.2.13 orderId

```
var AppCompatActivity.orderId [private]
```

Definition at line 45 of file [delivered_order_details.kt](#).

7.4.2.14 pointsCard2

```
var firstOrderId val AppCompatActivity.pointsCard2 [private]
```

Definition at line 71 of file [PointsDetail.kt](#).

7.4.2.15 pointsCard3

```
var firstOrderId val secondOrderId val AppCompatActivity.pointsCard3 [private]
```

Definition at line 78 of file [PointsDetail.kt](#).

7.4.2.16 pointsCard4

```
var firstOrderId val secondOrderId val thirdOrderId val AppCompatActivity.pointsCard4 [private]
```

Definition at line 85 of file [PointsDetail.kt](#).

7.4.2.17 recyclerView

```
lateinit var AppCompatActivity.recyclerView [private]
```

Definition at line 44 of file [chatbot.kt](#).

7.4.2.18 restaurant

```
var AppCompatActivity.restaurant [private]
```

Definition at line 43 of file [restaurant_profile.kt](#).

7.4.2.19 restaurantId

```
var AppCompatActivity.restaurantId [private]
```

ID của nhà hàng cần chia sẻ

Definition at line 29 of file [share_restaurant.kt](#).

7.4.2.20 restaurants

```
var AppCompatActivity.restaurants [private]
```

Definition at line 52 of file [list_restaurant.kt](#).

7.4.2.21 rvAddresses

```
lateinit var AppCompatActivity.rvAddresses [private]
```

RecyclerView hiển thị danh sách địa chỉ

Definition at line 38 of file [savedeliveryaddress.kt](#).

7.4.2.22 rvCart

```
lateinit var AppCompatActivity.rvCart [private]
```

RecyclerView hiển thị danh sách giỏ hàng

Definition at line 36 of file [activity_cart.kt](#).

7.4.2.23 rvNotifications

```
lateinit var AppCompatActivity.rvNotifications [private]
```

RecyclerView hiển thị danh sách thông báo

Definition at line 40 of file [notification.kt](#).

7.4.2.24 rvPreOrderCart

```
lateinit var AppCompatActivity.rvPreOrderCart [private]
```

RecyclerView hiển thị danh sách món ăn đặt trước

Definition at line 58 of file [pre_order_cart.kt](#).

7.4.2.25 selectedPromotion

var AppCompatActivity.selectedPromotion [private]

Definition at line 39 of file [discouts.kt](#).

7.5 auth Namespace Reference

Functions

- [verify_password](#) (plain_password, hashed_password)
- [get_password_hash](#) (password)
- [create_access_token](#) (dict data, Optional[timedelta] expires_delta=None)
- [get_current_user](#) (str token=Depends([oauth2_scheme](#)), Session db=Depends([database.get_db](#)))

Variables

- [SECRET_KEY](#) = os.getenv("SECRET_KEY")
- [ALGORITHM](#) = os.getenv("ALGORITHM", "HS256")
- [ACCESS_TOKEN_EXPIRE_MINUTES](#) = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))
- [pwd_context](#) = CryptContext(schemes=["bcrypt"], deprecated="auto")
- [oauth2_scheme](#) = OAuth2PasswordBearer(tokenUrl="token")

7.5.1 Detailed Description

Module: auth.py

Xác thực và phân quyền người dùng.

Cung cấp JWT token, mã hóa mật khẩu bcrypt, và dependency lấy user hiện tại.

7.5.2 Function Documentation

7.5.2.1 create_access_token()

```
auth.create_access_token (
    dict data,
    Optional[timedelta] expires_delta = None)
```

Tạo JWT access token với payload và thời gian hết hạn.

Definition at line 42 of file `auth.py`.

```

00042 def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
00043     """Tạo JWT access token với payload và thời gian hết hạn."""
00044     to_encode = data.copy()
00045     if expires_delta:
00046         expire = datetime.utcnow() + expires_delta
00047     else:
00048         expire = datetime.utcnow() + timedelta(minutes=15)
00049     to_encode.update({"exp": expire})
00050     encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
00051     return encoded_jwt
00052

```

Here is the caller graph for this function:



7.5.2.2 `get_current_user()`

```

auth.get_current_user (
    str token = Depends(oauth2_scheme),
    Session db = Depends(database.get_db))

```

Dependency: giải mã JWT token và trả về user hiện tại. Raise 401 nếu token không hợp lệ.

Definition at line 53 of file `auth.py`.

```

00053 async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(database.get_db)):
00054     """Dependency: giải mã JWT token và trả về user hiện tại. Raise 401 nếu token không hợp lệ."""
00055     credentials_exception = HTTPException(
00056         status_code=status.HTTP_401_UNAUTHORIZED,
00057         detail="Could not validate credentials",
00058         headers={"WWW-Authenticate": "Bearer"},
00059     )
00060     try:
00061         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
00062         email: str = payload.get("sub")
00063         if email is None:
00064             raise credentials_exception
00065     except JWTError:
00066         raise credentials_exception
00067     user = db.query(models.User).filter(models.User.email == email).first()
00068     if user is None:
00069         raise credentials_exception
00070     return user

```

7.5.2.3 `get_password_hash()`

```

auth.get_password_hash (
    password)

```

Hash mật khẩu plaintext bằng bcrypt.

Definition at line 38 of file [auth.py](#).

```
00038 def get_password_hash(password):
00039     """Hash mật khẩu plaintext bằng bcrypt."""
00040     return pwd_context.hash(password)
00041
```

Here is the caller graph for this function:



7.5.2.4 verify_password()

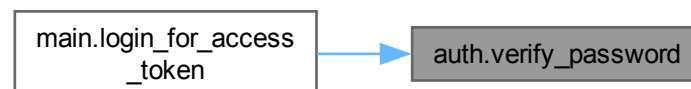
```
auth.verify_password (
    plain_password,
    hashed_password)
```

Xác thực mật khẩu plaintext với mật khẩu đã hash bằng bcrypt.

Definition at line 34 of file [auth.py](#).

```
00034 def verify_password(plain_password, hashed_password):
00035     """Xác thực mật khẩu plaintext với mật khẩu đã hash bằng bcrypt."""
00036     return pwd_context.verify(plain_password, hashed_password)
00037
```

Here is the caller graph for this function:



7.5.3 Variable Documentation

7.5.3.1 ACCESS_TOKEN_EXPIRE_MINUTES

```
auth.ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))
```

Definition at line 27 of file [auth.py](#).

7.5.3.2 ALGORITHM

```
auth.ALGORITHM = os.getenv("ALGORITHM", "HS256")
```

Definition at line 25 of file [auth.py](#).

7.5.3.3 oauth2_scheme

```
auth.oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
```

Definition at line 32 of file [auth.py](#).

7.5.3.4 pwd_context

```
auth.pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

Definition at line 30 of file [auth.py](#).

7.5.3.5 SECRET_KEY

```
auth.SECRET_KEY = os.getenv("SECRET_KEY")
```

Definition at line 23 of file [auth.py](#).

7.6 Package CartItemAdapter

Packages

- package [CartItemViewHolder](#)

7.7 Package CartItemAdapter.CartItemViewHolder

Functions

- class [CartItemViewHolder](#) (itemView:android.view.View)

7.7.1 Function Documentation

7.7.1.1 CartItemViewHolder()

class CartItemAdapter.CartItemViewHolder.CartItemViewHolder (
 itemView:android.view. View)

ViewHolder chứa các view con cho mỗiCartItem.

Bao gồm hình ảnh, tên món, số lượng và giá tiền. ImageView hiển thị hình ảnh món ăn

TextView hiển thị tên món ăn

TextView hiển thị số lượng

TextView hiển thị tổng giá tiền

Gán dữ liệuCartItem vào các view con.

Hiển thị tên, số lượng, tổng giá tiền và hình ảnh (qua Picasso).

Parameters

cartItem	Đối tượngCartItem chứa dữ liệu cần hiển thị.
----------	--

Definition at line 1 of file [CartItemAdapter.kt](#).

```

00035         : android.view.View) : RecyclerView.ViewHolder(itemView) {
00037     private val imgCartItem: ImageView = itemView.findViewById(R.id.imgCartItem)
00039     private val tvCartItemName: TextView = itemView.findViewById(R.id.tvCartItemName)
00041     private val tvCartItemQty: TextView = itemView.findViewById(R.id.tvCartItemQty)
00043     private val tvCartItemPrice: TextView = itemView.findViewById(R.id.tvCartItemPrice)
00044
00052     fun bind(cartItem: CartItem) {
00053         tvCartItemName.text = cartItem.name
00054         tvCartItemQty.text = "SL: ${cartItem.qty}"
00055
00056         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00057         val totalPrice = cartItem.qty * cartItem.price
00058         tvCartItemPrice.text = "${fmt.format(totalPrice)}đ"
00059
00060         if (!cartItem.imageUrl.isNullOrEmpty()) {
00061             Picasso.get()
00062                 .load(cartItem.imageUrl)
00063                 .placeholder(R.drawable.placeholder_loading)
00064                 .error(R.drawable.pngwing)
00065                 .into(imgCartItem)
00066         } else {
00067             imgCartItem.setImageResource(R.drawable.pngwing)
00068         }
00069     }
00070 }
```

7.8 chatbot_service Namespace Reference

Classes

- class [ChatBotService](#)

Variables

- [GEMINI_API_KEY](#) = os.getenv("GEMINI_API_KEY")

7.8.1 Detailed Description

Module: `chatbot_service.py`

Service chatbot sử dụng Google Gemini AI để trả lời câu hỏi của người dùng.
Tự động lấy dữ liệu nhà hàng, món ăn và đơn hàng từ DB làm ngữ cảnh cho AI.

7.8.2 Variable Documentation

7.8.2.1 GEMINI_API_KEY

```
chatbot_service.GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

Definition at line 18 of file `chatbot_service.py`.

7.9 Package ChatMessageAdapter

Packages

- package `MessageViewHolder`

7.10 Package ChatMessageAdapter.MessageViewHolder

7.11 check_db Namespace Reference

Variables

- `conn` = `sqlite3.connect('test.db')`
- `cursor` = `conn.cursor()`
- `tables` = `cursor.fetchall()`
- `fcm` = `cursor.fetchall()`
- `notifs` = `cursor.fetchall()`

7.11.1 Variable Documentation

7.11.1.1 conn

```
check_db.conn = sqlite3.connect('test.db')
```

Definition at line 2 of file `check_db.py`.

7.11.1.2 cursor

```
check_db.cursor = conn.cursor()
```

Definition at line 3 of file `check_db.py`.

7.11.1.3 fcm

```
check_db.fcm = cursor.fetchall()
```

Definition at line 9 of file [check_db.py](#).

7.11.1.4 notifs

```
check_db.notifs = cursor.fetchall()
```

Definition at line 12 of file [check_db.py](#).

7.11.1.5 tables

```
check_db.tables = cursor.fetchall()
```

Definition at line 6 of file [check_db.py](#).

7.12 database Namespace Reference

Functions

- [get_db\(\)](#)

Variables

- [SQLALCHEMY_DATABASE_URL](#) = os.getenv("DATABASE_URL", "postgresql://postgres:password@localhost/food_delivery")
- [engine](#) = create_engine([SQLALCHEMY_DATABASE_URL](#))
- [SessionLocal](#) = sessionmaker(autocommit=False, autoflush=False, bind=[engine](#))
- [Base](#) = declarative_base()

7.12.1 Detailed Description

Module: database.py

Cấu hình kết nối SQLAlchemy và session cho cơ sở dữ liệu PostgreSQL.
Cung cấp engine, SessionLocal và hàm get_db() dependency cho FastAPI.

7.12.2 Function Documentation

7.12.2.1 get_db()

```
database.get_db()
```

FastAPI dependency: tạo session database cho mỗi request, tự động đóng sau khi hoàn tất.

Definition at line 27 of file [database.py](#).

```
00027 def get_db():
00028     """FastAPI dependency: tạo session database cho mỗi request, tự động đóng sau khi hoàn tất."""
00029     db = SessionLocal()
00030     try:
00031         yield db
00032     finally:
00033         db.close()
```

7.12.3 Variable Documentation

7.12.3.1 Base

```
database.Base = declarative_base()
```

Definition at line 25 of file [database.py](#).

7.12.3.2 engine

```
database.engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

Definition at line 19 of file [database.py](#).

7.12.3.3 SessionLocal

```
database.SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Definition at line 22 of file [database.py](#).

7.12.3.4 SQLALCHEMY_DATABASE_URL

```
database.SQLALCHEMY_DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:password@localhost/↔  
food_delivery")
```

Definition at line 16 of file [database.py](#).

7.13 Package FAQAdapter

Packages

- package [FAQViewHolder](#)

7.14 Package FAQAdapter.FAQViewHolder

7.15 Package FcmTokenRegistrar

7.16 firebase_utils Namespace Reference

Functions

- [init_firebase](#) ()
- [_upload_image_sync](#) (file_content, filename)
- [upload_image](#) (file_content, filename)

Variables

- `FIREBASE_CREDENTIALS_PATH` = `os.getenv("FIREBASE_CREDENTIALS_PATH")`
- `FIREBASE_STORAGE_BUCKET` = `os.getenv("FIREBASE_STORAGE_BUCKET")`

7.16.1 Detailed Description

Module: `firebase_utils.py`

Khởi tạo Firebase Admin SDK và upload ảnh lên Firebase Storage.
Hỗ trợ upload bất đồng bộ qua `anyio` để không chặn event loop.

7.16.2 Function Documentation

7.16.2.1 `_upload_image_sync()`

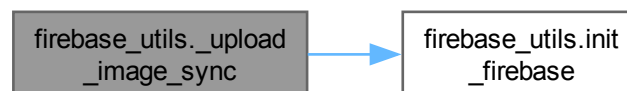
```
firebase_utils._upload_image_sync (
    file_content,
    filename) [protected]
```

Upload ảnh đồng bộ lên Firebase Storage, trả về URL công khai.

Definition at line 38 of file `firebase_utils.py`.

```
00038 def _upload_image_sync(file_content, filename):
00039     """Upload ảnh đồng bộ lên Firebase Storage, trả về URL công khai."""
00040     if not firebase_admin._apps:
00041         if not init_firebase():
00042             return None
00043
00044     try:
00045         bucket = storage.bucket()
00046         ext = filename.split('.')[-1]
00047         unique_filename = f"images/{uuid.uuid4()}.{ext}"
00048         blob = bucket.blob(unique_filename)
00049         blob.upload_from_string(file_content, content_type=f"image/{ext}")
00050         blob.make_public()
00051         return blob.public_url
00052     except Exception as e:
00053         print(f"Error uploading image: {e}")
00054         return None
00055
```

Here is the call graph for this function:



7.16.2.2 init_firebase()

firebase_utils.init_firebase ()

Khởi tạo Firebase Admin SDK với credentials từ file service account.

Definition at line 22 of file [firebase_utils.py](#).

```

00022 def init_firebase():
00023     """Khởi tạo Firebase Admin SDK với credentials từ file service account."""
00024     if not FIREBASE_CREDENTIALS_PATH or not os.path.exists(FIREBASE_CREDENTIALS_PATH):
00025         print(f"Warning: Firebase credentials not found at {FIREBASE_CREDENTIALS_PATH}. Image upload will fail.")
00026         return False
00027
00028     try:
00029         cred = credentials.Certificate(FIREBASE_CREDENTIALS_PATH)
00030         firebase_admin.initialize_app(cred, {
00031             'storageBucket': FIREBASE_STORAGE_BUCKET
00032         })
00033         return True
00034     except Exception as e:
00035         print(f"Error initializing Firebase: {e}")
00036         return False
00037

```

Here is the caller graph for this function:



7.16.2.3 upload_image()

firebase_utils.upload_image (

file_content,

filename)

Bản async của hàm upload ảnh, không gây nghẽn event loop.

Definition at line 56 of file [firebase_utils.py](#).

```

00056 async def upload_image(file_content, filename):
00057     """Bản async của hàm upload ảnh, không gây nghẽn event loop."""
00058     return await anyio.to_thread.run_sync(_upload_image_sync, file_content, filename)

```

7.16.3 Variable Documentation

7.16.3.1 FIREBASE_CREDENTIALS_PATH

firebase_utils.FIREBASE_CREDENTIALS_PATH = os.getenv("FIREBASE_CREDENTIALS_PATH")

Definition at line 18 of file [firebase_utils.py](#).

7.16.3.2 FIREBASE_STORAGE_BUCKET

```
firebase_utils.FIREBASE_STORAGE_BUCKET = os.getenv("FIREBASE_STORAGE_BUCKET")
```

Definition at line 20 of file [firebase_utils.py](#).

7.17 Package FirebaseMessagingService

Functions

- override fun [onMessageReceived](#) (remoteMessage:RemoteMessage)
- fun [showNotification](#) (title:String, body:String)

7.17.1 Function Documentation

7.17.1.1 onMessageReceived()

```
override fun FirebaseMessagingService.onMessageReceived (
    remoteMessage:RemoteMessage )
```

Definition at line 26 of file [MyFirebaseMessagingService.kt](#).

```
00026             : RemoteMessage) {
00027     super.onMessageReceived(remoteMessage)
00028
00029     // 1. Lấy dữ liệu tiêu đề và nội dung
00030     val title = remoteMessage.notification?.title ?: "Thông báo từ Trung"
00031     val body = remoteMessage.notification?.body ?: "Nội dung trống"
00032
00033     showNotification(title, body)
00034 }
```

7.17.1.2 showNotification()

```
fun FirebaseMessagingService.showNotification (
    title:String ,
    body:String ) [private]
```

Hiển thị thông báo trên thiết bị. Tạo NotificationChannel cho Android 8.0+ và xây dựng notification với tiêu đề, nội dung và mức ưu tiên cao.

Parameters

title	Tiêu đề của thông báo.
body	Nội dung của thông báo.

Definition at line 44 of file [MyFirebaseMessagingService.kt](#).

```
00044             : String, body: String) {
00045     val channelId = "food_delivery_channel"
00046     val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
00047
00048     // 2. Tạo Kênh (Channel) cho Android đời mới
00049     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
00050         val channel = NotificationChannel(
00051             channelId,
```



```

00052         "Thông báo đơn hàng",
00053         NotificationManager.IMPORTANCE_HIGH
00054     ).apply {
00055         description = "Kênh nhận thông báo đặt đồ ăn"
00056     }
00057     notificationManager.createNotificationChannel(channel)
00058 }
00059
00060 // 3. Xây dựng giao diện thông báo
00061 val notification = NotificationCompat.Builder(this, channelId)
00062     .setSmallIcon(R.drawable.pngwing) // Dùng icon pngwing bạn đã có
00063     .setContentTitle(title)
00064     .setContentText(body)
00065     .setPriority(NotificationCompat.PRIORITY_HIGH)
00066     .setAutoCancel(true)
00067     .build()
00068
00069 // 4. Hiển thị lên màn hình
00070 notificationManager.notify(System.currentTimeMillis().toInt(), notification)
00071 }

```

7.18 main Namespace Reference

Functions

- [read_root](#) ()
- [login_for_access_token](#) (OAuth2PasswordRequestForm form_data=Depends(), Session db=Depends(get_db))
- [create_user](#) (schemas.UserCreate user, Session db=Depends(get_db))
- [read_users_me](#) (models.User current_user=Depends(auth.get_current_user))
- [update_device_token](#) (str token=Form(...), str device_type=Form("android"), models.User current_user=Depends(auth.get_current_user), Session db=Depends(get_db))
- [read_users](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [create_address](#) (schemas.AddressCreate address, Session db=Depends(get_db))
- [read_user_addresses](#) (int user_id, Session db=Depends(get_db))
- [update_address](#) (int address_id, schemas.AddressUpdate address, Session db=Depends(get_db))
- [read_categories](#) (Session db=Depends(get_db))
- [read_promotions](#) (bool active_only=True, Session db=Depends(get_db))
- [chat_with_bot](#) (Gửi, tin, nhắn, đến, chatbot, AI, và, nhận, phản, hồi, schemas.ChatMessageCreate chat_input, Optional[int] session_id=None, models.User current_user=Depends(auth.get_current_user), Session db=Depends(get_db))
- [create_restaurant](#) (Tạo, nhà, hàng, mới, với, ảnh, upload, lên, Firebase, Storage, str name=Form(...), Optional[str] address=Form(None), str phone_number=Form(...), str status=Form(...), Optional[str] description=Form(None), Optional[UploadFile] image=File(None), Session db=Depends(get_db))
- [read_restaurants](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [search_restaurants_by_name](#) (str name, int skip=0, int limit=100, Session db=Depends(get_db))
- [read_restaurant](#) (int restaurant_id, Session db=Depends(get_db))
- [create_menu_item](#) (Tạo, món, ăn, mới, với, ảnh, upload, lên, Firebase, Storage, str name=Form(...), Decimal price=Form(...), Optional[str] description=Form(None), int restaurantid=Form(...), int categoryid=Form(...), Optional[UploadFile] image=File(None), Session db=Depends(get_db))
- [build_menuitem_with_reviews](#) (models.MenuItem item, Session db)
- [read_all_menu_items](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [read_restaurant_menu](#) (int restaurant_id, Session db=Depends(get_db))
- [search_menu_items_by_name](#) (str name, int skip=0, int limit=100, Session db=Depends(get_db))
- [search_menu_items_by_category_name](#) (str category_name, int skip=0, int limit=100, Session db=Depends(get_db))
- [read_menu_item_detail](#) (int menu_item_id, Session db=Depends(get_db))
- [vnipay_return](#) (Request request, Session db=Depends(get_db))

- [create_payment](#) (Tạo, URL, thanh, toán, VNPay, cho, đơn, hàng, str order_id, int amount, Request request)
- [create_order](#) ([schemas.OrderCreate](#) order, Session db=Depends(get_db))
- [read_user_orders](#) (int user_id, Session db=Depends(get_db))
- [read_order_detail](#) (int order_id, Session db=Depends(get_db))
- [update_order_status](#) (int order_id, [schemas.OrderStatusUpdate](#) payload, Session db=Depends(get_db))
- [create_review](#) ([schemas.ReviewCreate](#) review, Session db=Depends(get_db))
- [read_user_profile_summary](#) (int user_id, Session db=Depends(get_db))
- [read_faqs](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [read_user_notifications](#) (int user_id, Session db=Depends(get_db))
- [create_notification](#) ([schemas.NotificationCreate](#) notification, Session db=Depends(get_db))
- [mark_notification_as_read](#) (int notification_id, Session db=Depends(get_db))

Variables

- [bind](#)
- [app](#) = FastAPI(title="Đặt Món Food Delivery API")
- [allow_origins](#)
- [allow_credentials](#)
- [allow_methods](#)
- [allow_headers](#)
- [host](#)
- [port](#)

7.18.1 Detailed Description

Module: `main.py`

FastAPI application chính của hệ thống Đặt Món.

Định nghĩa tất cả REST API endpoints: auth, user, restaurant, menu, order, payment, chatbot, notification.

7.18.2 Function Documentation

7.18.2.1 `build_menuitem_with_reviews()`

```
main.build_menuitem_with_reviews (
    models.MenuItem item,
    Session db)
```

Helper function to build MenuItem with reviews and avg_rating.

Definition at line 319 of file [main.py](#).

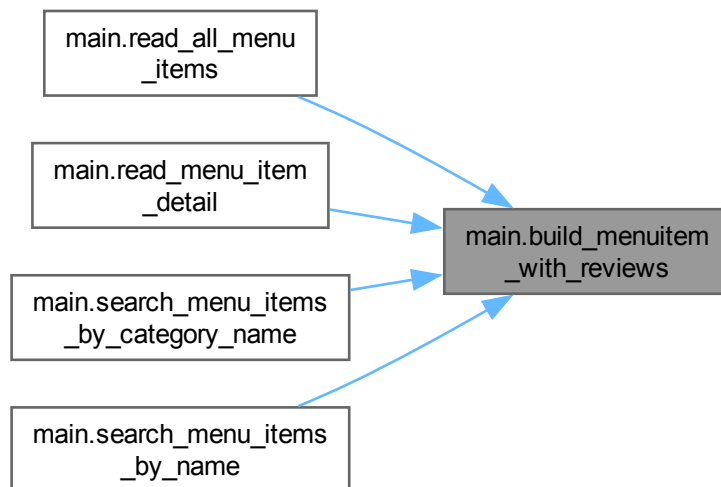
```
00319 def build_menuitem_with_reviews(item: models.MenuItem, db: Session):
00320     """Helper function to build MenuItem with reviews and avg_rating."""
00321     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == item.restaurantid).first()
00322     restaurant_name = restaurant.name if restaurant else ""
00323
00324     # Fetch reviews for this menu item
00325     reviews = db.query(models.Review).filter(models.Review.menuitemid == item.id).all()
00326     reviews_data = []
00327     total_rating = 0
00328
00329     for review in reviews:
```

```

00330     user = db.query(models.User).filter(models.User.id == review.userid).first()
00331     user_name = user.name if user else "Unknown"
00332     reviews_data.append({
00333         "id": review.id,
00334         "rating": review.rating,
00335         "comment": review.comment,
00336         "userid": review.userid,
00337         "user_name": user_name
00338     })
00339     total_rating += review.rating
00340
00341     avg_rating = total_rating / len(reviews) if reviews else 0.0
00342
00343     return {
00344         "id": item.id,
00345         "name": item.name,
00346         "image_url": item.image_url,
00347         "price": item.price,
00348         "is_available": item.is_available,
00349         "description": item.description,
00350         "restaurantid": item.restaurantid,
00351         "categoryid": item.categoryid,
00352         "restaurant_name": restaurant_name,
00353         "reviews": reviews_data,
00354         "avg_rating": avg_rating
00355     }
00356
00357
00358 @app.get("/menu-items/", response_model=List[schemas.MenuItemWithReviews])

```

Here is the caller graph for this function:



7.18.2.2 chat_with_bot()

```

main.chat_with_bot (
    Gủi,
    tin,
    nhắn,
    đén,
    chatbot,
    AI,

```

```

    và,
    nhận,
    phản,
    hồi,
    schemas.ChatMessageCreate chat_input,
    Optional[int] session_id = None,
    models.User current_user = Depends(auth.get_current_user),
    Session db = Depends(get_db))

```

Definition at line 190 of file [main.py](#).

```

00196 ):
00197     # 1. Tìm hoặc tạo ChatSession
00198     if session_id:
00199         session = db.query(models.ChatSession).filter(models.ChatSession.id == session_id).first()
00200         if not session:
00201             raise HTTPException(status_code=404, detail="Chat Session not found")
00202     else:
00203         session = models.ChatSession(status="active", userid=current_user.id)
00204         db.add(session)
00205         db.commit()
00206         db.refresh(session)
00207
00208     # 2. Lưu tin nhắn người dùng
00209     user_msg = models.ChatMessage(senderrole="user", message=chat_input.message, sessionid=session.id)
00210     db.add(user_msg)
00211     db.commit() # Lưu trước để AI có thể đọc lịch sử nếu cần (trong tương lai)
00212
00213     # 3. Gọi AI xử lý
00214     bot_service = ChatBotService(db, user_id=current_user.id)
00215     bot_response_text = await bot_service.get_response(chat_input.message)
00216
00217     # 4. Lưu phản hồi của AI
00218     bot_msg = models.ChatMessage(senderrole="bot", message=bot_response_text, sessionid=session.id)
00219     db.add(bot_msg)
00220
00221     db.commit()
00222     db.refresh(bot_msg)
00223
00224     return bot_msg
00225
00226
00227 # --- RESTAURANT ENDPOINTS ---
00228
00229
00230
00231 @app.post("/restaurants/", response_model=schemas.Restaurant)

```

7.18.2.3 create_address()

```

main.create_address (
    schemas.AddressCreate address,
    Session db = Depends(get_db))

```

Tạo địa chỉ giao hàng mới cho người dùng.

Definition at line 135 of file [main.py](#).

```

00135 def create_address(address: schemas.AddressCreate, db: Session = Depends(get_db)):
00136     """Tạo địa chỉ giao hàng mới cho người dùng."""
00137     db_address = models.Address(detail=address.detail, phone=address.phone, userid=address.userid)
00138     db.add(db_address)
00139     db.commit()
00140     db.refresh(db_address)
00141     return db_address
00142
00143
00144 @app.get("/users/{user_id}/addresses/", response_model=List[schemas.Address])

```

7.18.2.4 create_menu_item()

```

main.create_menu_item (
    Tạo,
    món,
    ăn,
    mới,
    với,
    ảnh,
    upload,
    lên,
    Firebase,
    Storage,
    str name = Form(...),
    Decimal price = Form(...),
    Optional[str] description = Form(None),
    int restaurantid = Form(...),
    int categoryid = Form(...),
    Optional[UploadFile] image = File(None),
    Session db = Depends(get_db))

```

Definition at line 290 of file [main.py](#).

```

00299 ):
00300     image_url = None
00301     if image:
00302         content = await image.read()
00303         image_url = await upload_image(content, image.filename)
00304
00305     db_item = models.MenuItem(
00306         name=name,
00307         price=price,
00308         description=description,
00309         restaurantid=restaurantid,
00310         categoryid=categoryid,
00311         image_url=image_url
00312     )
00313     db.add(db_item)
00314     db.commit()
00315     db.refresh(db_item)
00316     return db_item
00317
00318

```

7.18.2.5 create_notification()

```

main.create_notification (
    schemas.NotificationCreate notification,
    Session db = Depends(get_db))

```

Tạo thông báo mới và gửi push notification.

Definition at line 743 of file [main.py](#).

```

00743 def create_notification(notification: schemas.NotificationCreate, db: Session = Depends(get_db)):
00744     """Tạo thông báo mới và gửi push notification."""
00745     # Sử dụng notify_user để tạo thông báo - nó sẽ vừa lưu vào DB vừa gửi push
00746     notify_user(
00747         db=db,
00748         user_id=notification.userid,
00749         title=notification.title,
00750         body=notification.content,
00751         order_id=notification.orderid
00752     )
00753
00754     # Lấy thông báo vừa tạo để trả về
00755     db_notification = db.query(models.Notification).filter(
00756         models.Notification.userid == notification.userid,

```

```

00757     models.Notification.title == notification.title
00758 ).order_by(models.Notification.id.desc()).first()
00759
00760     return db_notification if db_notification else {
00761         "id": 0,
00762         "title": notification.title,
00763         "type": notification.type,
00764         "content": notification.content,
00765         "isread": notification.isread,
00766         "createdat": datetime.now(),
00767         "userid": notification.userid,
00768         "orderid": notification.orderid,
00769         "sessionid": notification.sessionid
00770     }
00771
00772
00773 @app.put("/notifications/{notification_id}/read")

```

7.18.2.6 create_order()

```

main.create_order (
    schemas.OrderCreate order,
    Session db = Depends(get_db))

```

Tạo đơn hàng mới kèm danh sách món ăn và gửi thông báo.

Definition at line 521 of file [main.py](#).

```

00521 def create_order(order: schemas.OrderCreate, db: Session = Depends(get_db)):
00522     """Tạo đơn hàng mới kèm danh sách món ăn và gửi thông báo."""
00523     db_order = models.Orders(
00524         status=order.status,
00525         preorderdate=order.preorderdate,
00526         preordertime=order.preordertime,
00527         totalprice=order.totalprice,
00528         restaurantid=order.restaurantid,
00529         addressid=order.addressid,
00530         userid=order.userid
00531     )
00532     db.add(db_order)
00533     db.commit()
00534     db.refresh(db_order)
00535
00536     for item in order.order_items:
00537         db_order_item = models.OrderItem(
00538             quantity=item.quantity,
00539             price=item.price,
00540             menuitemid=item.menuitemid,
00541             orderid=db_order.id
00542         )
00543         db.add(db_order_item)
00544
00545     db.commit()
00546     db.refresh(db_order)
00547
00548     # Nếu là COD (status="confirmed"), gửi thông báo ngay lúc tạo đơn hàng
00549     if db_order.status == "confirmed":
00550         try:
00551             total_formatted = f"{db_order.totalprice:.0f}".replace(".", ",")
00552             notify_user(
00553                 db=db,
00554                 user_id=db_order.userid,
00555                 title="Đơn hàng được đặt thành công! ",
00556                 body=f"Đơn hàng #{db_order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ quán xác
nhận.",
                 order_id=db_order.id
00557             )
00558         except Exception as e:
00559             print(f"Error sending notification: {e}")
00560
00561     return db_order
00562
00563
00564
00565 @app.get("/users/{user_id}/orders/", response_model=List[schemas.Order])

```

7.18.2.7 create_payment()

```
main.create_payment (
    Tạo,
    URL,
    thanh,
    toán,
    VNPay,
    cho,
    đơn,
    hàng,
    str order_id,
    int amount,
    Request request)
```

Definition at line 496 of file [main.py](#).

```
00501 ):
00502     ip_address = request.client.host
00503
00504
00505     payment_url = generate_vnpay_url(
00506         order_id=str(order_id),
00507         amount=amount,
00508         ip_address=ip_address
00509     )
00510
00511
00512     return {
00513         "payment_url": payment_url
00514     }
00515
00516
00517 # --- ORDER ENDPOINTS ---
00518
00519
00520 @app.post("/orders/", response_model=schemas.Order)
```

7.18.2.8 create_restaurant()

```
main.create_restaurant (
    Tạo,
    nhà,
    hàng,
    mới,
    với,
    ảnh,
    upload,
    lên,
    Firebase,
    Storage,
    str name = Form(...),
    Optional[str] address = Form(None),
    str phone_number = Form(...),
    str status = Form(...),
    Optional[str] description = Form(None),
    Optional[UploadFile] image = File(None),
    Session db = Depends(get_db))
```

Definition at line 232 of file [main.py](#).

```
00241 ):
00242     image_url = None
00243     if image:
00244         content = await image.read()
00245         image_url = await upload_image(content, image.filename)
```

```

00246
00247 db_restaurant = models.Restaurant(
00248     name=name,
00249     address=address,
00250     phone_number=phone_number,
00251     status=status,
00252     description=description,
00253     image_url=image_url
00254 )
00255 db.add(db_restaurant)
00256 db.commit()
00257 db.refresh(db_restaurant)
00258 return db_restaurant
00259
00260
00261 @app.get("/restaurants/", response_model=List[schemas.Restaurant])

```

7.18.2.9 create_review()

```

main.create_review (
    schemas.ReviewCreate review,
    Session db = Depends(get_db))

```

Tạo đánh giá cho đơn hàng và cập nhật điểm trung bình nhà hàng.

Definition at line 660 of file [main.py](#).

```

00660 def create_review(review: schemas.ReviewCreate, db: Session = Depends(get_db)):
00661     """Tạo đánh giá cho đơn hàng và cập nhật điểm trung bình nhà hàng."""
00662     order = db.query(models.Orders).filter(models.Orders.id == review.orderid).first()
00663     if not order:
00664         raise HTTPException(status_code=404, detail="Order not found")
00665
00666     if order.userid != review.userid:
00667         raise HTTPException(status_code=403, detail="Review does not belong to this user")
00668
00669     order_item = db.query(models.OrderItem).filter(models.OrderItem.orderid == order.id).first()
00670     if not order_item:
00671         raise HTTPException(status_code=400, detail="Order has no items to review")
00672
00673     menuitemid = review.menuitemid or order_item.menuitemid
00674     restaurantid = review.restaurantid or order.restaurantid
00675
00676     db_review = models.Review(
00677         rating=review.rating,
00678         comment=review.comment,
00679         orderid=review.orderid,
00680         menuitemid=menuitemid,
00681         restaurantid=restaurantid,
00682         userid=review.userid,
00683     )
00684     db.add(db_review)
00685     db.commit()
00686
00687     # Tính lại điểm đánh giá trung bình của nhà hàng
00688     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == restaurantid).first()
00689     if restaurant:
00690         all_reviews = db.query(models.Review).filter(models.Review.restaurantid == restaurantid).all()
00691         if all_reviews:
00692             avg_rating = sum(r.rating for r in all_reviews) / len(all_reviews)
00693             restaurant.rating = int(round(avg_rating))
00694             db.add(restaurant)
00695             db.commit()
00696
00697     db.refresh(db_review)
00698     return db_review
00699
00700
00701 @app.get("/users/{user_id}/profile-summary", response_model=schemas.UserProfileSummary)

```

7.18.2.10 create_user()

```

main.create_user (
    schemas.UserCreate user,
    Session db = Depends(get_db))

```


Đăng ký tài khoản người dùng mới.

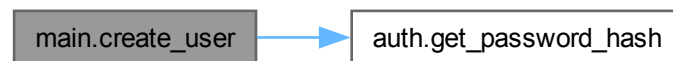
Definition at line 75 of file [main.py](#).

```

00075 def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
00076     """Đăng ký tài khoản người dùng mới."""
00077     # Check if user exists
00078     db_user = db.query(models.User).filter(models.User.email == user.email).first()
00079     if db_user:
00080         raise HTTPException(status_code=400, detail="Email already registered")
00081
00082     hashed_password = auth.get_password_hash(user.password)
00083     db_user = models.User(
00084         name=user.name,
00085         username=user.username,
00086         email=user.email,
00087         password=hashed_password,
00088         phone=user.phone,
00089         role=user.role
00090     )
00091     db.add(db_user)
00092     db.commit()
00093     db.refresh(db_user)
00094     return db_user
00095
00096
00097 @app.get("/users/me/", response_model=schemas.User)

```

Here is the call graph for this function:



7.18.2.11 login_for_access_token()

```

main.login_for_access_token (
    OAuth2PasswordRequestForm form_data = Depends(),
    Session db = Depends(get_db))

```

Đăng nhập bằng username/password, trả về JWT access token.

Definition at line 55 of file [main.py](#).

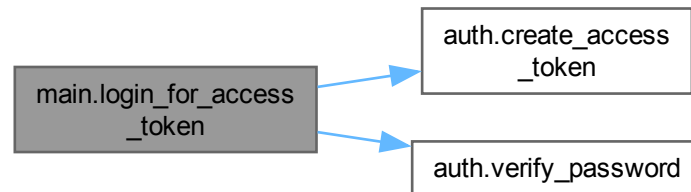
```

00055 async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends(), db: Session =
    Depends(get_db)):
00056     """Đăng nhập bằng username/password, trả về JWT access token."""
00057     user = db.query(models.User).filter(models.User.username == form_data.username).first()
00058     if not user or not auth.verify_password(form_data.password, user.password):
00059         raise HTTPException(
00060             status_code=status.HTTP_401_UNAUTHORIZED,
00061             detail="Incorrect username or password",
00062             headers={"WWW-Authenticate": "Bearer"},
00063         )
00064     access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
00065     access_token = auth.create_access_token(
00066         data={"sub": user.email}, expires_delta=access_token_expires
00067     )
00068     return {"access_token": access_token, "token_type": "bearer"}
00069
00070
00071 # --- USER ENDPOINTS ---
00072

```

```
00073
00074 @app.post("/users/", response_model=schemas.User)
```

Here is the call graph for this function:



7.18.2.12 mark_notification_as_read()

```
main.mark_notification_as_read (
    int notification_id,
    Session db = Depends(get_db))
```

Đánh dấu thông báo đã đọc.

Definition at line 774 of file [main.py](#).

```
00774 def mark_notification_as_read(notification_id: int, db: Session = Depends(get_db)):
00775     """Đánh dấu thông báo đã đọc."""
00776     db_notification = db.query(models.Notification).filter(models.Notification.id == notification_id).first()
00777     if not db_notification:
00778         raise HTTPException(status_code=404, detail="Notification not found")
00779
00780     db_notification.isread = True
00781     db.commit()
00782     return {"message": "Notification marked as read"}
00783
00784
```

7.18.2.13 read_all_menu_items()

```
main.read_all_menu_items (
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))
```

Lấy danh sách tất cả món ăn từ tất cả nhà hàng, bao gồm reviews.

Definition at line 359 of file [main.py](#).

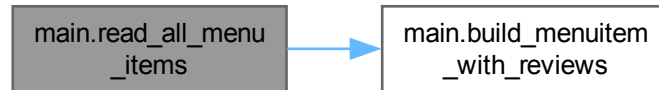
```
00359 def read_all_menu_items(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00360     """Lấy danh sách tất cả món ăn từ tất cả nhà hàng, bao gồm reviews."""
00361     menu_items = db.query(models.MenuItem).offset(skip).limit(limit).all()
00362
00363     result = []
00364     for item in menu_items:
00365         item_dict = build_menuitem_with_reviews(item, db)
```

```

00366         result.append(item_dict)
00367     return result
00368
00369
00370 @app.get("/restaurants/{restaurant_id}/menu/", response_model=List[schemas.MenuItem])

```

Here is the call graph for this function:



7.18.2.14 read_categories()

```

main.read_categories (
    Session db = Depends(get_db))

```

Lấy danh sách tất cả danh mục món ăn.

Definition at line 172 of file [main.py](#).

```

00172 def read_categories(db: Session = Depends(get_db)):
00173     """Lấy danh sách tất cả danh mục món ăn."""
00174     return db.query(models.Category).all()
00175
00176
00177 @app.get("/promotions/", response_model=List[schemas.Promotion])

```

7.18.2.15 read_faqs()

```

main.read_faqs (
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))

```

Lấy danh sách các câu hỏi thường gặp (FAQ).

Definition at line 728 of file [main.py](#).

```

00728 def read_faqs(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00729     """Lấy danh sách các câu hỏi thường gặp (FAQ)."""
00730     return db.query(models.FAQ).filter(models.FAQ.isactive == True).offset(skip).limit(limit).all()
00731
00732
00733 # --- NOTIFICATION ENDPOINTS ---
00734
00735
00736 @app.get("/users/{user_id}/notifications/", response_model=List[schemas.Notification])

```

7.18.2.16 read_menu_item_detail()

```
main.read_menu_item_detail (
    int menu_item_id,
    Session db = Depends(get_db))
```

Lấy chi tiết món ăn bao gồm reviews.

Definition at line 414 of file [main.py](#).

```
00414 def read_menu_item_detail(menu_item_id: int, db: Session = Depends(get_db)):
00415     """Lấy chi tiết món ăn bao gồm reviews."""
00416     item = db.query(models.MenuItem).filter(models.MenuItem.id == menu_item_id).first()
00417     if not item:
00418         raise HTTPException(status_code=404, detail="Menu item not found")
00419
00420     return build_menuitem_with_reviews(item, db)
00421 # --- VNPAY RETURN ---
00422 @app.get("/vnpay_return")
```

Here is the call graph for this function:



7.18.2.17 read_order_detail()

```
main.read_order_detail (
    int order_id,
    Session db = Depends(get_db))
```

Lấy chi tiết đơn hàng kèm tên nhà hàng, địa chỉ và danh sách món.

Definition at line 572 of file [main.py](#).

```
00572 def read_order_detail(order_id: int, db: Session = Depends(get_db)):
00573     """Lấy chi tiết đơn hàng kèm tên nhà hàng, địa chỉ và danh sách món."""
00574     order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00575     if not order:
00576         raise HTTPException(status_code=404, detail="Order not found")
00577
00578     restaurant_name = order.restaurant.name if order.restaurant else None
00579     address_detail = order.address.detail if order.address else None
00580
00581     order_items = []
00582     for item in order.order_items:
00583         order_items.append({
00584             "id": item.id,
00585             "quantity": item.quantity,
00586             "price": item.price,
00587             "menuitemid": item.menuitemid,
00588             "menuitem_name": item.menu_item.name if item.menu_item else None,
00589             "image_url": item.menu_item.image_url if item.menu_item else None
00590         })
00591
00592     return {
```

```

00593         "id": order.id,
00594         "status": order.status,
00595         "preorderdate": order.preorderdate,
00596         "preordertime": order.preordertime,
00597         "totalprice": order.totalprice,
00598         "restaurantid": order.restaurantid,
00599         "addressid": order.addressid,
00600         "userid": order.userid,
00601         "createdat": order.createdat,
00602         "restaurant_name": restaurant_name,
00603         "address_detail": address_detail,
00604         "order_items": order_items
00605     }
00606
00607
00608 @app.put("/orders/{order_id}/status", response_model=schemas.Order)

```

7.18.2.18 read_promotions()

```

main.read_promotions (
    bool active_only = True,
    Session db = Depends(get_db))

```

Lấy danh sách mã khuyến mãi, mặc định chỉ lấy mã đang active.

Definition at line 178 of file [main.py](#).

```

00178 def read_promotions(active_only: bool = True, db: Session = Depends(get_db)):
00179     """Lấy danh sách mã khuyến mãi, mặc định chỉ lấy mã đang active."""
00180     query = db.query(models.Promotion)
00181     if active_only:
00182         query = query.filter(models.Promotion.status.ilike("active"))
00183     return query.all()
00184
00185
00186 # --- CHAT ENDPOINTS ---
00187
00188
00189 @app.post("/chat/", response_model=schemas.ChatMessage)

```

7.18.2.19 read_restaurant()

```

main.read_restaurant (
    int restaurant_id,
    Session db = Depends(get_db))

```

Lấy chi tiết nhà hàng theo ID.

Definition at line 278 of file [main.py](#).

```

00278 def read_restaurant(restaurant_id: int, db: Session = Depends(get_db)):
00279     """Lấy chi tiết nhà hàng theo ID."""
00280     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == restaurant_id).first()
00281     if not restaurant:
00282         raise HTTPException(status_code=404, detail="Restaurant not found")
00283     return restaurant
00284
00285
00286 # --- MENU ITEM ENDPOINTS ---
00287
00288
00289 @app.post("/menu-items/", response_model=schemas.MenuItem)

```

7.18.2.20 read_restaurant_menu()

```
main.read_restaurant_menu (  
    int restaurant_id,  
    Session db = Depends(get_db))
```

Lấy danh sách món ăn của một nhà hàng.

Definition at line 371 of file [main.py](#).

```
00371 def read_restaurant_menu(restaurant_id: int, db: Session = Depends(get_db)):  
00372     """Lấy danh sách món ăn của một nhà hàng."""  
00373     return db.query(models.MenuItem).filter(models.MenuItem.restaurantid == restaurant_id).all()  
00374  
00375  
00376 @app.get("/menu-items/search/", response_model=List[schemas.MenuItemWithReviews])
```

7.18.2.21 read_restaurants()

```
main.read_restaurants (  
    int skip = 0,  
    int limit = 100,  
    Session db = Depends(get_db))
```

Lấy danh sách tất cả nhà hàng (phân trang).

Definition at line 262 of file [main.py](#).

```
00262 def read_restaurants(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):  
00263     """Lấy danh sách tất cả nhà hàng (phân trang)."""  
00264     restaurants = db.query(models.Restaurant).offset(skip).limit(limit).all()  
00265     return restaurants  
00266  
00267  
00268 @app.get("/restaurants/search/", response_model=List[schemas.Restaurant])
```

7.18.2.22 read_root()

```
main.read_root ()
```

Endpoint gốc, trả lời chào mừng API.

Definition at line 46 of file [main.py](#).

```
00046 def read_root():  
00047     """Endpoint gốc, trả lời chào mừng API."""  
00048     return {"message": "Welcome to Đặt Món Food Delivery API"}  
00049  
00050  
00051 # --- AUTH ENDPOINTS ---  
00052  
00053  
00054 @app.post("/token", response_model=schemas.Token)
```

7.18.2.23 read_user_addresses()

```
main.read_user_addresses (
    int user_id,
    Session db = Depends(get_db))
```

Lấy danh sách địa chỉ giao hàng của người dùng.

Definition at line 145 of file [main.py](#).

```
00145 def read_user_addresses(user_id: int, db: Session = Depends(get_db)):
00146     """Lấy danh sách địa chỉ giao hàng của người dùng."""
00147     return db.query(models.Address).filter(models.Address.userid == user_id).all()
00148
00149
00150 @app.put("/addresses/{address_id}/", response_model=schemas.Address)
```

7.18.2.24 read_user_notifications()

```
main.read_user_notifications (
    int user_id,
    Session db = Depends(get_db))
```

Lấy danh sách thông báo của người dùng.

Definition at line 737 of file [main.py](#).

```
00737 def read_user_notifications(user_id: int, db: Session = Depends(get_db)):
00738     """Lấy danh sách thông báo của người dùng."""
00739     return db.query(models.Notification).filter(models.Notification.userid ==
    user_id).order_by(models.Notification.createdat.desc()).all()
00740
00741
00742 @app.post("/notifications/", response_model=schemas.Notification)
```

7.18.2.25 read_user_orders()

```
main.read_user_orders (
    int user_id,
    Session db = Depends(get_db))
```

Lấy danh sách đơn hàng của người dùng.

Definition at line 566 of file [main.py](#).

```
00566 def read_user_orders(user_id: int, db: Session = Depends(get_db)):
00567     """Lấy danh sách đơn hàng của người dùng."""
00568     return db.query(models.Orders).filter(models.Orders.userid == user_id).all()
00569
00570
00571 @app.get("/orders/{order_id}/detail", response_model=schemas.OrderDetail)
```

7.18.2.26 read_user_profile_summary()

```
main.read_user_profile_summary (
    int user_id,
    Session db = Depends(get_db))
```

Lấy tóm tắt hồ sơ người dùng: điểm, đơn hàng đã giao, tổng chi tiêu.

Definition at line 702 of file [main.py](#).

```
00702 def read_user_profile_summary(user_id: int, db: Session = Depends(get_db)):
00703     """Lấy tóm tắt hồ sơ người dùng: điểm, đơn hàng đã giao, tổng chi tiêu."""
00704     user = db.query(models.User).filter(models.User.id == user_id).first()
00705     if not user:
00706         raise HTTPException(status_code=404, detail="User not found")
00707
00708     orders = db.query(models.Orders).filter(models.Orders.userid == user_id).all()
00709     delivered_orders = [o for o in orders if (o.status or "").lower() in {"delivered", "completed", "done"}]
00710     total_spent = sum(o.totalprice or 0 for o in orders)
00711
00712     # Quy ước điểm đơn giản để Android hiển thị động thay cho số cứng.
00713     points = total_spent // 10000
00714
00715     return {
00716         "user_id": user.id,
00717         "user_name": user.name,
00718         "points": points,
00719         "delivered_orders": len(delivered_orders),
00720         "total_spent": total_spent
00721     }
00722
00723
00724 # --- FAQ ENDPOINTS ---
00725
00726
00727 @app.get("/faqs/", response_model=List[schemas.FAQ])
```

7.18.2.27 read_users()

```
main.read_users (
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))
```

Lấy danh sách tất cả người dùng (phân trang).

Definition at line 125 of file [main.py](#).

```
00125 def read_users(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00126     """Lấy danh sách tất cả người dùng (phân trang)."""
00127     users = db.query(models.User).offset(skip).limit(limit).all()
00128     return users
00129
00130
00131 # --- ADDRESS ENDPOINTS ---
00132
00133
00134 @app.post("/addresses/", response_model=schemas.Address)
```

7.18.2.28 read_users_me()

```
main.read_users_me (
    models.User current_user = Depends(auth.get_current_user))
```

Lấy thông tin người dùng hiện tại từ JWT token.

Definition at line 98 of file [main.py](#).

```
00098 async def read_users_me(current_user: models.User = Depends(auth.get_current_user)):
00099     """Lấy thông tin người dùng hiện tại từ JWT token."""
00100     return current_user
00101
00102
00103 @app.post("/devices/token")
```


7.18.2.29 search_menu_items_by_category_name()

```
main.search_menu_items_by_category_name (
    str category_name,
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))
```

Tìm kiếm món ăn theo tên danh mục (không phân biệt hoa thường).

Definition at line 391 of file [main.py](#).

```
00391 def search_menu_items_by_category_name(category_name: str, skip: int = 0, limit: int = 100, db: Session =
    Depends(get_db)):
00392     """Tìm kiếm món ăn theo tên danh mục (không phân biệt hoa thường)."""
00393     # Tìm category theo tên
00394     categories = db.query(models.Category).filter(
00395         models.Category.name.ilike(f"%{category_name}%")
00396     ).all()
00397     category_ids = [cat.id for cat in categories]
00398
00399     if not category_ids:
00400         return []
00401
00402     menu_items = db.query(models.MenuItem).filter(
00403         models.MenuItem.categoryid.in_(category_ids)
00404     ).offset(skip).limit(limit).all()
00405
00406     result = []
00407     for item in menu_items:
00408         item_dict = build_menuitem_with_reviews(item, db)
00409         result.append(item_dict)
00410     return result
00411
00412
00413 @app.get("/menu-items/{menu_item_id}/", response_model=schemas.MenuItemWithReviews)
```

Here is the call graph for this function:



7.18.2.30 search_menu_items_by_name()

```
main.search_menu_items_by_name (
    str name,
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))
```

Tìm kiếm món ăn theo tên (không phân biệt hoa thường).

Definition at line 377 of file [main.py](#).

```

00377 def search_menu_items_by_name(name: str, skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00378     """Tìm kiếm món ăn theo tên (không phân biệt hoa thường)."""
00379     menu_items = db.query(models.MenuItem).filter(
00380         models.MenuItem.name.ilike(f"%{name}%")
00381     ).offset(skip).limit(limit).all()
00382
00383     result = []
00384     for item in menu_items:
00385         item_dict = build_menuitem_with_reviews(item, db)
00386         result.append(item_dict)
00387     return result
00388
00389
00390 @app.get("/menu-items/category/search/", response_model=List[schemas.MenuItemWithReviews])

```

Here is the call graph for this function:



7.18.2.31 search_restaurants_by_name()

```

main.search_restaurants_by_name (
    str name,
    int skip = 0,
    int limit = 100,
    Session db = Depends(get_db))

```

Tìm kiếm nhà hàng theo tên (không phân biệt hoa thường).

Definition at line 269 of file [main.py](#).

```

00269 def search_restaurants_by_name(name: str, skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00270     """Tìm kiếm nhà hàng theo tên (không phân biệt hoa thường)."""
00271     restaurants = db.query(models.Restaurant).filter(
00272         models.Restaurant.name.ilike(f"%{name}%")
00273     ).offset(skip).limit(limit).all()
00274     return restaurants
00275
00276
00277 @app.get("/restaurants/{restaurant_id}/", response_model=schemas.Restaurant)

```

7.18.2.32 update_address()

```

main.update_address (
    int address_id,
    schemas.AddressUpdate address,
    Session db = Depends(get_db))

```

Cập nhật địa chỉ giao hàng.

Definition at line 151 of file [main.py](#).

```

00151 def update_address(address_id: int, address: schemas.AddressUpdate, db: Session = Depends(get_db)):
00152     """Cập nhật địa chỉ giao hàng."""
00153     db_address = db.query(models.Address).filter(models.Address.id == address_id).first()
00154     if not db_address:
00155         raise HTTPException(status_code=404, detail="Address not found")
00156
00157     # Cập nhật các trường
00158     if address.detail is not None:
00159         db_address.detail = address.detail
00160     if address.phone is not None:
00161         db_address.phone = address.phone
00162
00163     db.commit()
00164     db.refresh(db_address)
00165     return db_address
00166
00167
00168 # --- CATEGORY ENDPOINTS ---
00169
00170
00171 @app.get("/categories/", response_model=List[schemas.Category])

```

7.18.2.33 update_device_token()

```

main.update_device_token (
    str token = Form(...),
    str device_type = Form("android"),
    models.User current_user = Depends(auth.get_current_user),
    Session db = Depends(get_db))

```

Android App gọi API này để gửi FCM Token lên Server.

Definition at line 104 of file [main.py](#).

```

00109 ):
00110     """Android App gọi API này để gửi FCM Token lên Server."""
00111     # Kiểm tra xem token này đã có chưa
00112     db_device = db.query(models.UserDevice).filter(models.UserDevice.device_token == token).first()
00113     if not db_device:
00114         db_device = models.UserDevice(device_token=token, device_type=device_type, userid=current_user.id)
00115         db.add(db_device)
00116     else:
00117         db_device.userid = current_user.id
00118         # Cập nhật thời gian hoạt động
00119
00120     db.commit()
00121     return {"message": "Device token updated successfully"}
00122
00123
00124 @app.get("/users/", response_model=List[schemas.User])

```

7.18.2.34 update_order_status()

```

main.update_order_status (
    int order_id,
    schemas.OrderStatusUpdate payload,
    Session db = Depends(get_db))

```

Cập nhật trạng thái đơn hàng và gửi thông báo tương ứng.

Definition at line 609 of file [main.py](#).

```

00609 def update_order_status(order_id: int, payload: schemas.OrderStatusUpdate, db: Session = Depends(get_db)):
00610     """Cập nhật trạng thái đơn hàng và gửi thông báo tương ứng."""
00611     order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00612     if not order:
00613         raise HTTPException(status_code=404, detail="Order not found")

```

```

00614
00615 old_status = order.status
00616 order.status = payload.status
00617 db.commit()
00618 db.refresh(order)
00619
00620 print(f"DEBUG: Update order {order_id}: {old_status} -> {payload.status}")
00621
00622 # Tạo thông báo khi trạng thái đơn hàng thay đổi
00623 try:
00624     notification_title = ""
00625     notification_body = ""
00626
00627     if payload.status in ["paid", "confirmed"] and old_status not in ["paid", "confirmed"]:
00628         notification_title = "Đơn hàng được đặt thành công! "
00629         total_formatted = f"{order.totalprice:,.0f}".replace(".", ",")
00630         notification_body = f"Đơn hàng #{order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ
quản xác nhận."
00631         print(f"DEBUG: Sending notification for order {order_id}")
00632     elif payload.status == "delivering":
00633         notification_title = "Đơn hàng đang giao "
00634         notification_body = f"Đơn hàng #{order.id} đang trên đường giao đến bạn"
00635     elif payload.status == "completed":
00636         notification_title = "Đơn hàng đã giao "
00637         notification_body = f"Đơn hàng #{order.id} đã giao thành công. Cảm ơn đã đặt hàng!"
00638     elif payload.status == "cancelled":
00639         notification_title = "Đơn hàng bị hủy "
00640         notification_body = f"Đơn hàng #{order.id} đã bị hủy"
00641
00642     # Nếu có thay đổi trạng thái, tạo thông báo
00643     if notification_title:
00644         print(f"DEBUG: Sending '{notification_title}' to user {order.userid}")
00645         notify_user(
00646             db=db,
00647             user_id=order.userid,
00648             title=notification_title,
00649             body=notification_body,
00650             order_id=order.id
00651         )
00652         print(f"DEBUG: Notification sent successfully")
00653     except Exception as e:
00654         print(f"Lỗi khi tạo thông báo trạng thái: {str(e)}")
00655
00656     return order
00657
00658
00659 @app.post("/reviews/", response_model=schemas.Review)

```

7.18.2.35 vnpay_return()

```

main.vnpay_return (
    Request request,
    Session db = Depends(get_db))

```

Callback từ VNPay sau khi thanh toán, cập nhật trạng thái đơn hàng.

Definition at line 423 of file [main.py](#).

```

00423 async def vnpay_return(request: Request, db: Session = Depends(get_db)):
00424     """Callback từ VNPay sau khi thanh toán, cập nhật trạng thái đơn hàng."""
00425
00426
00427     params = dict(request.query_params)
00428
00429
00430     response_code = params.get("vnp_ResponseCode")
00431     order_ref = params.get("vnp_TxnRef") # Format: DH{order_id}
00432
00433     # Trích xuất order_id từ reference (VD: "DH123" -> 123)
00434     try:
00435         # Nếu là định dạng "DH{id}"
00436         if order_ref and order_ref.startswith("DH"):
00437             order_id = int(order_ref[2:])
00438         else:
00439             order_id = int(order_ref) if order_ref else 0
00440     except (ValueError, AttributeError):
00441         order_id = 0
00442

```

```

00443
00444     if response_code == "00":
00445         # Thanh toán thành công - cập nhật đơn hàng trong DB
00446         if order_id > 0:
00447             order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00448             if order:
00449                 order.status = "paid"
00450
00451                 # Tạo Payment record
00452                 payment_record = models.Payment(
00453                     status="success",
00454                     method="vnpay",
00455                     orderid=order.id
00456                 )
00457                 db.add(payment_record)
00458                 db.commit()
00459                 db.refresh(order)
00460
00461                 # Gửi notification
00462                 total_formatted = f"{order.totalprice:,.0f}".replace(".", ", ")
00463                 notify_user(
00464                     db=db,
00465                     user_id=order.userid,
00466                     title="Đơn hàng được đặt thành công! ",
00467                     body=f"Đơn hàng #{order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ quán xác
nhận.",
00468                     order_id=order.id
00469                 )
00470
00471             return {
00472                 "status": "success",
00473                 "message": f"Thanh toán thành công cho đơn {order_ref}"
00474             }
00475         else:
00476             # Thanh toán thất bại - cập nhật Payment record
00477             if order_id > 0:
00478                 payment_record = models.Payment(
00479                     status="failed",
00480                     method="vnpay",
00481                     orderid=order_id
00482                 )
00483                 db.add(payment_record)
00484                 db.commit()
00485
00486             return {
00487                 "status": "failed",
00488                 "message": "Thanh toán thất bại"
00489             }
00490
00491     # --- PAYMENT ENDPOINTS ---
00492
00493
00494
00495 @app.get("/create-payment")

```

7.18.3 Variable Documentation

7.18.3.1 allow_credentials

main.allow_credentials

Definition at line 39 of file [main.py](#).

7.18.3.2 allow_headers

main.allow_headers

Definition at line 41 of file [main.py](#).

7.18.3.3 allow_methods

main.allow_methods

Definition at line 40 of file [main.py](#).

7.18.3.4 `allow__origins`

`main.allow__origins`

Definition at line [38](#) of file [main.py](#).

7.18.3.5 `app`

`main.app = FastAPI(title="Đặt Món Food Delivery API")`

Definition at line [32](#) of file [main.py](#).

7.18.3.6 `bind`

`main.bind`

Definition at line [29](#) of file [main.py](#).

7.18.3.7 `host`

`main.host`

Definition at line [787](#) of file [main.py](#).

7.18.3.8 `port`

`main.port`

Definition at line [787](#) of file [main.py](#).

7.19 Package MenuItemAdapter

Packages

- package [MenuItemViewHolder](#)

7.20 Package MenuItemAdapter.MenuItemViewHolder

7.21 Package MenuItemAdapterSmall

Packages

- package [MenuItemViewHolder](#)

7.22 Package MenuItemAdapterSmall.MenuItemViewHolder

7.23 models Namespace Reference

Classes

- class [Category](#)
- class [FAQ](#)
- class [User](#)
- class [Shipper](#)
- class [Promotion](#)
- class [Restaurant](#)
- class [MenuItem](#)
- class [Address](#)
- class [Orders](#)
- class [OrderItem](#)
- class [Payment](#)
- class [Delivery](#)
- class [Review](#)
- class [Notification](#)
- class [ChatSession](#)
- class [ChatMessage](#)
- class [UserDevice](#)
- class [UserFAQ](#)
- class [UsedPromotion](#)
- class [LoyaltyPoint](#)
- class [SocialShare](#)

7.23.1 Detailed Description

Module: models.py

Định nghĩa tất cả SQLAlchemy ORM models cho cơ sở dữ liệu ứng dụng đặt món.
Bao gồm 19 model: User, Restaurant, MenuItem, Orders, Payment, Delivery, Review, v.v.

7.24 Package NetworkConfig

7.25 notification_service Namespace Reference

Functions

- [send_fcm_notification](#) (str token, str title, str body, dict data=None)
- [notify_user](#) (Session db, int user_id, str title, str body, int order_id=None)

7.25.1 Detailed Description

Module: notification_service.py

Service gửi thông báo đẩy qua Firebase Cloud Messaging (FCM).
Lưu thông báo vào database và gửi push notification đến tất cả thiết bị của người dùng.

7.25.2 Function Documentation

7.25.2.1 notify_user()

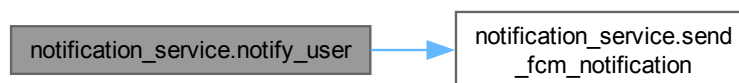
```
notification_service.notify_user (
    Session db,
    int user_id,
    str title,
    str body,
    int order_id = None)
```

Gửi thông báo cho tất cả thiết bị của một người dùng.

Definition at line 36 of file [notification_service.py](#).

```
00036 def notify_user(db: Session, user_id: int, title: str, body: str, order_id: int = None):
00037     """Gửi thông báo cho tất cả thiết bị của một người dùng."""
00038     # 1. Lưu thông báo vào bảng Notification trong DB
00039     db_notification = models.Notification(
00040         title=title,
00041         content=body,
00042         type="order_update",
00043         userid=user_id,
00044         orderid=order_id,
00045         isread=False
00046     )
00047     db.add(db_notification)
00048     db.commit()
00049
00050     # 2. Lấy danh sách FCM Token của người dùng đó
00051     devices = db.query(models.UserDevice).filter(models.UserDevice.userid == user_id).all()
00052
00053     # 3. Gửi thông báo thực tới điện thoại
00054     for device in devices:
00055         send_fcm_notification(
00056             token=device.device_token,
00057             title=title,
00058             body=body,
00059             data={"order_id": str(order_id)} if order_id else None
00060         )
```

Here is the call graph for this function:



7.25.2.2 send_fcm_notification()

```
notification_service.send_fcm_notification (
    str token,
    str title,
    str body,
    dict data = None)
```

Gửi thông báo tới một thiết bị Android cụ thể qua FCM.

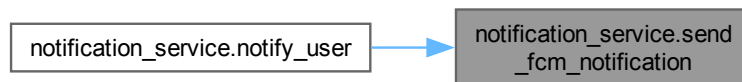
Definition at line 14 of file `notification_service.py`.

```

00014 def send_fcm_notification(token: str, title: str, body: str, data: dict = None):
00015     """Gửi thông báo tới một thiết bị Android cụ thể qua FCM."""
00016     if not firebase_admin._apps:
00017         init_firebase()
00018
00019     message = messaging.Message(
00020         notification=messaging.Notification(
00021             title=title,
00022             body=body,
00023         ),
00024         data=data,
00025         token=token,
00026     )
00027
00028     try:
00029         response = messaging.send(message)
00030         print(f"Successfully sent message: {response}")
00031         return True
00032     except Exception as e:
00033         print(f"Error sending FCM message: {e}")
00034         return False
00035

```

Here is the caller graph for this function:



7.26 Package NotificationAdapter

Packages

- package `NotificationViewHolder`

7.27 Package NotificationAdapter.NotificationViewHolder

7.28 payment_service Namespace Reference

Functions

- `generate_vnpay_url` (str order_id, int amount, str ip_address)

Variables

- str `VNP_URL` = "https://sandbox.vnpayment.vn/paymentv2/vpcpay.html"
- str `VNP_TMNCODE` = "2HON5OA2"
- str `VNP_HASH_SECRET` = "UGDPB7W11M55W6UTQ0N69X36I2VIE0UL"
- str `VNP_RETURN_URL` = "http://localhost:8000/vnpay_return"

7.28.1 Detailed Description

Module: `payment_service.py`

Service tạo URL thanh toán VNPay (sandbox).

Xây dựng payment URL với chữ ký HMAC-SHA512 để xác thực giao dịch.

7.28.2 Function Documentation

7.28.2.1 `generate_vnpay_url()`

```
payment_service.generate_vnpay_url (
    str order_id,
    int amount,
    str ip_address)
```

Tạo URL thanh toán VNPay với chữ ký HMAC-SHA512 cho đơn hàng.

Definition at line 23 of file [payment_service.py](#).

```
00023 def generate_vnpay_url(order_id: str, amount: int, ip_address: str):
00024     """Tạo URL thanh toán VNPay với chữ ký HMAC-SHA512 cho đơn hàng."""
00025
00026     params = {
00027         "vnp_Version": "2.1.0",
00028         "vnp_Command": "pay",
00029         "vnp_TmnCode": VNP_TMNCODE,
00030         "vnp_Amount": int(amount) * 100,
00031         "vnp_CurrCode": "VND",
00032         "vnp_TxnRef": order_id,
00033         "vnp_OrderInfo": f"Thanh toan don hang {order_id}",
00034         "vnp_OrderType": "other",
00035         "vnp_Locale": "vn",
00036         "vnp_ReturnUrl": VNP_RETURN_URL,
00037         "vnp_IpAddr": ip_address,
00038         "vnp_CreateDate": datetime.now().strftime("%Y%m%d%H%M%S"),
00039     }
00040
00041
00042     # sort params
00043     sorted_params = sorted(params.items())
00044
00045
00046     query_string = urllib.parse.urlencode(sorted_params)
00047
00048
00049     # tạo hash
00050     hash_value = hmac.new(
00051         VNP_HASH_SECRET.encode(),
00052         query_string.encode(),
00053         hashlib.sha512
00054     ).hexdigest()
00055
00056
00057     payment_url = f"{VNP_URL}?{query_string}&vnp_SecureHash={hash_value}"
00058
00059
00060     return payment_url
```

7.28.3 Variable Documentation

7.28.3.1 `VNP_HASH_SECRET`

```
str payment_service.VNP_HASH_SECRET = "UGDPB7W11M55W6UTQ0N69X36I2VIE0UL"
```

Definition at line 17 of file [payment_service.py](#).

7.28.3.2 VNP_RETURN_URL

```
str payment_service.VNP_RETURN_URL = "http://localhost:8000/vnpay_return"
```

Definition at line 18 of file [payment_service.py](#).

7.28.3.3 VNP_TMNCODE

```
str payment_service.VNP_TMNCODE = "2HON5OA2"
```

Definition at line 16 of file [payment_service.py](#).

7.28.3.4 VNP_URL

```
str payment_service.VNP_URL = "https://sandbox.vnpayment.vn/paymentv2/vpcpay.html"
```

Definition at line 15 of file [payment_service.py](#).

7.29 Package RestaurantAdapter

Packages

- package [RestaurantViewHolder](#)

7.30 Package RestaurantAdapter.RestaurantViewHolder

7.31 Package RetrofitClient

7.32 schemas Namespace Reference

Classes

- class [CategoryBase](#)
- class [CategoryCreate](#)
- class [Category](#)
- class [MenuItemBase](#)
- class [MenuItemCreate](#)
- class [MenuItem](#)
- class [MenuItemWithRestaurant](#)
- class [RestaurantBase](#)
- class [RestaurantCreate](#)
- class [Restaurant](#)
- class [UserBase](#)
- class [UserCreate](#)
- class [User](#)
- class [Token](#)

- class [TokenData](#)
- class [AddressBase](#)
- class [AddressCreate](#)
- class [AddressUpdate](#)
- class [Address](#)
- class [OrderItemBase](#)
- class [OrderItemCreate](#)
- class [OrderItem](#)
- class [OrderBase](#)
- class [OrderCreate](#)
- class [Order](#)
- class [OrderItemDetail](#)
- class [OrderDetail](#)
- class [OrderStatusUpdate](#)
- class [UserProfileSummary](#)
- class [ChatMessageBase](#)
- class [ChatMessageCreate](#)
- class [ChatMessage](#)
- class [ChatSessionCreate](#)
- class [ChatSession](#)
- class [UserDeviceBase](#)
- class [UserDeviceCreate](#)
- class [UserDevice](#)
- class [UserFAQBase](#)
- class [UserFAQCreate](#)
- class [UserFAQ](#)
- class [FAQBase](#)
- class [FAQCreate](#)
- class [FAQ](#)
- class [ShipperBase](#)
- class [ShipperCreate](#)
- class [Shipper](#)
- class [PromotionBase](#)
- class [PromotionCreate](#)
- class [Promotion](#)
- class [PaymentBase](#)
- class [PaymentCreate](#)
- class [Payment](#)
- class [DeliveryBase](#)
- class [DeliveryCreate](#)
- class [Delivery](#)
- class [ReviewBase](#)
- class [ReviewCreate](#)
- class [Review](#)
- class [ReviewResponse](#)
- class [MenuItemWithReviews](#)
- class [NotificationBase](#)
- class [NotificationCreate](#)
- class [Notification](#)
- class [UsedPromotionBase](#)
- class [UsedPromotionCreate](#)
- class [UsedPromotion](#)
- class [LoyaltyPointBase](#)
- class [LoyaltyPointCreate](#)
- class [LoyaltyPoint](#)
- class [SocialShareBase](#)
- class [SocialShareCreate](#)
- class [SocialShare](#)

7.32.1 Detailed Description

Module: schemas.py

Định nghĩa tất cả Pydantic v2 schemas cho request/response validation.

Bao gồm schema cho Category, MenuItem, Restaurant, User, Order, Payment, Review, v.v.

7.33 view_data Namespace Reference

Functions

- [get_all_tables](#) ()
- [view_table_data](#) (Session db, model_class, str [table_name](#), int [limit](#)=10)
- [view_all_data](#) ()
- [view_specific_table](#) (str [table_name](#), int [limit](#)=20)

Variables

- [table_name](#) = sys.argv[1]
- int [limit](#) = int(sys.argv[2]) if len(sys.argv) > 2 else 20

7.33.1 Detailed Description

Script đã sửa để hiển thị chi tiết thuộc tính của từng bản ghi

7.33.2 Function Documentation

7.33.2.1 get_all_tables()

view_data.get_all_tables ()

Lấy danh sách tất cả các bảng

Definition at line 15 of file [view_data.py](#).

```
00015 def get_all_tables():
00016     """Lấy danh sách tất cả các bảng"""
00017     inspector = inspect(engine)
00018     return inspector.get_table_names()
00019
```

7.33.2.2 view_all_data()

view_data.view_all_data ()

Xem dữ liệu tất cả các bảng

Definition at line 55 of file [view_data.py](#).

```
00055 def view_all_data():
00056     """Xem dữ liệu tất cả các bảng"""
00057     db = SessionLocal()
00058     try:
00059         print("\n" + "="*60)
00060         print("DỮ LIỆU HIỆN TẠI TRONG DATABASE")
00061         print("="*60)
00062         tables = [
00063             (Category, "category"), (FAQ, "faq"), (User, "User"),
00064             (Shipper, "shipper"), (Promotion, "promotion"), (Restaurant, "restaurant"),
00065             (MenuItem, "menuitem"), (Address, "address"), (Orders, "orders"),
00066             (OrderItem, "orderitem"), (Payment, "payment"), (Delivery, "delivery"),
00067             (Review, "review"), (Notification, "notification"), (ChatSession, "chatsession"),
00068             (ChatMessage, "chatmessage"), (UserDevice, "userdevice"), (UserFAQ, "userfaq"),
00069             (UsedPromotion, "usedpromotion"), (LoyaltyPoint, "loyaltypoint"), (SocialShare, "socialshare"),
00070         ]
00071         for model_class, table_name in tables:
00072             view_table_data(db, model_class, table_name, limit=20)
00073     finally:
00074         db.close()
00075
```

Here is the call graph for this function:



7.33.2.3 view_specific_table()

view_data.view_specific_table (
 str table_name,
 int limit = 20)

Xem dữ liệu của một bảng cụ thể theo tên.

Definition at line 76 of file [view_data.py](#).

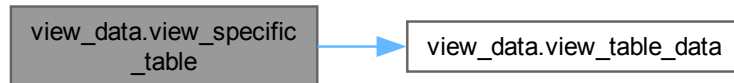
```
00076 def view_specific_table(table_name: str, limit: int = 20):
00077     """Xem dữ liệu của một bảng cụ thể theo tên."""
00078     db = SessionLocal()
00079     try:
00080         table_map = {
00081             "category": Category, "faq": FAQ, "user": User, "shipper": Shipper,
00082             "promotion": Promotion, "restaurant": Restaurant, "menuitem": MenuItem,
00083             "address": Address, "orders": Orders, "orderitem": OrderItem,
00084             "payment": Payment, "delivery": Delivery, "review": Review,
00085             "notification": Notification, "chatsession": ChatSession, "chatmessage": ChatMessage,
00086             "userdevice": UserDevice, "userfaq": UserFAQ, "usedpromotion": UsedPromotion,
00087             "loyaltypoint": LoyaltyPoint, "socialshare": SocialShare,
00088         }
00089         model_class = table_map.get(table_name.lower())
```

```

00090         if not model_class:
00091             print(f"Không tìm thấy bảng '{table_name}'")
00092         return
00093     view_table_data(db, model_class, table_name, limit)
00094 finally:
00095     db.close()
00096

```

Here is the call graph for this function:



7.33.2.4 view_table_data()

```

view_data.view_table_data (
    Session db,
    model_class,
    str table_name,
    int limit = 10)

```

Xem dữ liệu của một bảng với đầy đủ thuộc tính

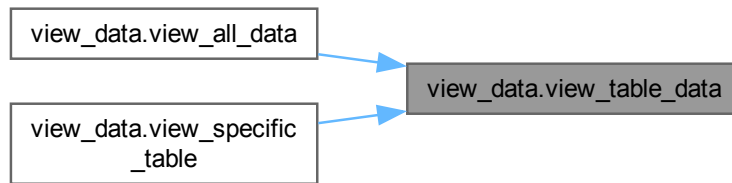
Definition at line 20 of file [view_data.py](#).

```

00020 def view_table_data(db: Session, model_class, table_name: str, limit: int = 10):
00021     """Xem dữ liệu của một bảng với đầy đủ thuộc tính"""
00022     print(f"\n{'='*60}")
00023     print(f"BẢNG: {table_name}")
00024     print(f"{'='*60}")
00025
00026     try:
00027         total = db.query(model_class).count()
00028         print(f"Tổng số bản ghi: {total}")
00029
00030         records = db.query(model_class).limit(limit).all()
00031
00032         if not records:
00033             print("Không có dữ liệu trong bảng này.")
00034             return
00035
00036         print(f"\nHiển thị {len(records)} bản ghi đầu tiên:")
00037         print("-" * 60)
00038
00039         # SỬA ĐOẠN NÀY ĐỂ HIỂN THỊ THUỘC TÍNH
00040         for i, record in enumerate(records, 1):
00041             # Lấy danh sách các cột thực tế từ Model
00042             columns = [column.key for column in inspect(record).mapper.column_attrs]
00043
00044             # Tạo dictionary chứa dữ liệu của các cột đó
00045             data = {col: getattr(record, col) for col in columns}
00046
00047             # In ra dưới dạng đẹp hơn
00048             print(f"[{i}] {data}")
00049
00050     except Exception as e:
00051         print(f"Lỗi khi truy vấn bảng {table_name}: {e}")
00052
00053 # ... (Các hàm view_all_data và view_specific_table giữ nguyên như cũ) ...
00054

```

Here is the caller graph for this function:



7.33.3 Variable Documentation

7.33.3.1 limit

```
int view_data.limit = int(sys.argv[2]) if len(sys.argv) > 2 else 20
```

Definition at line [101](#) of file [view_data.py](#).

7.33.3.2 table_name

```
view_data.table_name = sys.argv[1]
```

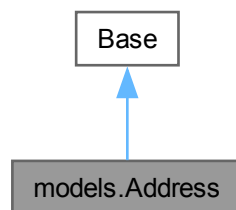
Definition at line [100](#) of file [view_data.py](#).

Chapter 8

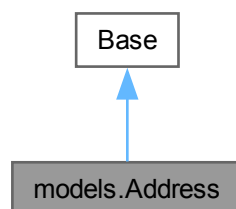
Class Documentation

8.1 models.Address Class Reference

Inheritance diagram for models.Address:



Collaboration diagram for models.Address:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `detail` = `Column(Text)`
- `phone` = `Column(String(255))`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `user` = `relationship("User", back_populates="addresses")`
- `orders` = `relationship("Orders", back_populates="address")`

Static Private Attributes

- `str __tablename__ = "address"`

8.1.1 Detailed Description

ORM model cho bảng address, địa chỉ giao hàng của người dùng.

Definition at line 111 of file [models.py](#).

8.1.2 Member Data Documentation

8.1.2.1 `__tablename__`

`str models.Address.__tablename__ = "address"` [static], [private]

Definition at line 113 of file [models.py](#).

8.1.2.2 `detail`

`models.Address.detail = Column(Text)` [static]

Definition at line 115 of file [models.py](#).

8.1.2.3 `id`

`models.Address.id = Column(Integer, primary_key=True, index=True)` [static]

Definition at line 114 of file [models.py](#).

8.1.2.4 `orders`

`models.Address.orders = relationship("Orders", back_populates="address")` [static]

Definition at line 120 of file [models.py](#).

8.1.2.5 phone

```
models.Address.phone = Column(String(255)) [static]
```

Definition at line 116 of file [models.py](#).

8.1.2.6 user

```
models.Address.user = relationship("User", back_populates="addresses") [static]
```

Definition at line 119 of file [models.py](#).

8.1.2.7 userid

```
models.Address.userid = Column(Integer, ForeignKey("User.id")) [static]
```

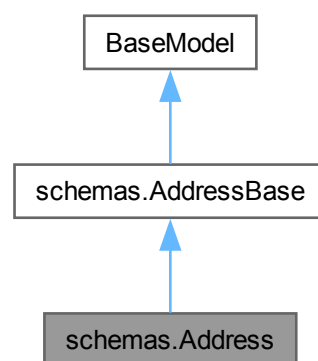
Definition at line 117 of file [models.py](#).

The documentation for this class was generated from the following file:

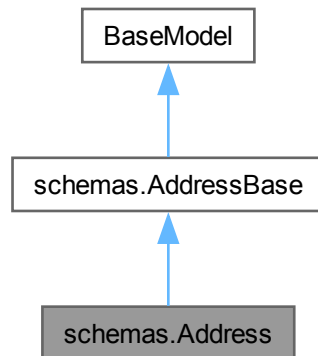
- [backend/models.py](#)

8.2 schemas.Address Class Reference

Inheritance diagram for schemas.Address:



Collaboration diagram for `schemas.Address`:



Static Public Attributes

- int `model_config` = `ConfigDict(from_attributes=True)`

Static Public Attributes inherited from `schemas.AddressBase`

- Optional `phone` = `None`

8.2.1 Detailed Description

Definition at line 107 of file `schemas.py`.

8.2.2 Member Data Documentation

8.2.2.1 `model_config`

```
int schemas.Address.model_config = ConfigDict(from_attributes=True) [static]
```

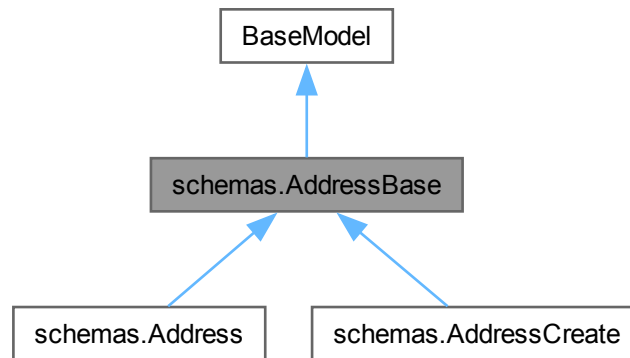
Definition at line 109 of file `schemas.py`.

The documentation for this class was generated from the following file:

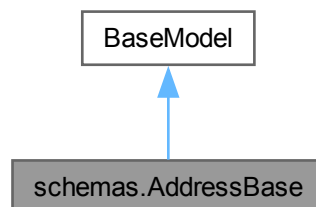
- `backend/schemas.py`

8.3 schemas.AddressBase Class Reference

Inheritance diagram for schemas.AddressBase:



Collaboration diagram for schemas.AddressBase:



Static Public Attributes

- Optional `phone` = None

8.3.1 Detailed Description

Schema cơ sở cho địa chỉ giao hàng.

Definition at line 94 of file `schemas.py`.

8.3.2 Member Data Documentation

8.3.2.1 phone

Optional `schemas.AddressBase.phone = None` [static]

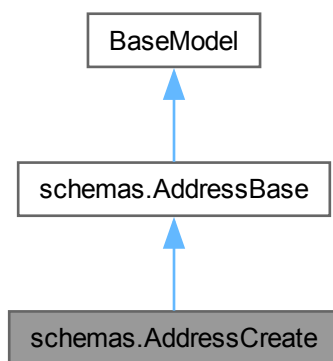
Definition at line 97 of file [schemas.py](#).

The documentation for this class was generated from the following file:

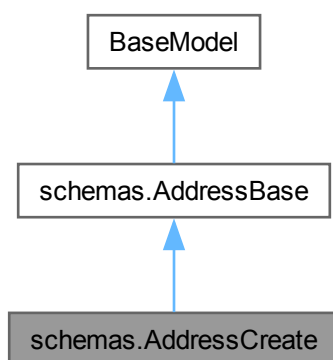
- [backend/schemas.py](#)

8.4 schemas.AddressCreate Class Reference

Inheritance diagram for `schemas.AddressCreate`:



Collaboration diagram for `schemas.AddressCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.AddressBase](#)

- Optional [phone](#) = None

8.4.1 Detailed Description

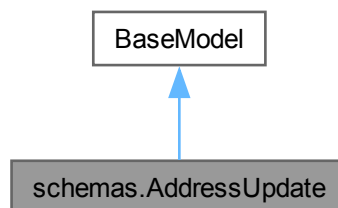
Definition at line 100 of file [schemas.py](#).

The documentation for this class was generated from the following file:

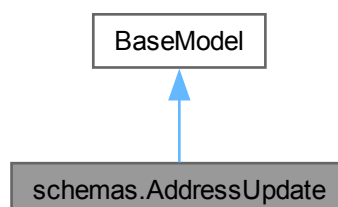
- backend/[schemas.py](#)

8.5 schemas.AddressUpdate Class Reference

Inheritance diagram for schemas.AddressUpdate:



Collaboration diagram for schemas.AddressUpdate:



Static Public Attributes

- Optional [detail](#) = None
- Optional [phone](#) = None

8.5.1 Detailed Description

Definition at line [103](#) of file [schemas.py](#).

8.5.2 Member Data Documentation

8.5.2.1 detail

Optional `schemas.AddressUpdate.detail = None` [static]

Definition at line [104](#) of file [schemas.py](#).

8.5.2.2 phone

Optional `schemas.AddressUpdate.phone = None` [static]

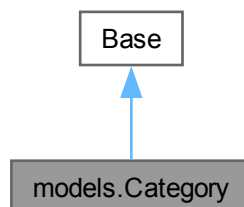
Definition at line [105](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

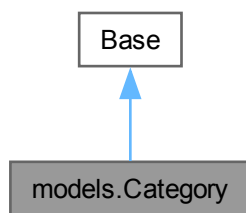
- [backend/schemas.py](#)

8.6 models.Category Class Reference

Inheritance diagram for `models.Category`:



Collaboration diagram for models.Category:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `name` = `Column(String(255), nullable=False)`
- `type` = `Column(String(255))`
- `description` = `Column(Text)`
- `menu_items` = `relationship("MenuItem", back_populates="category")`

Static Private Attributes

- `str __tablename__ = "category"`

8.6.1 Detailed Description

ORM model cho bảng category, phân loại món ăn.

Definition at line 12 of file `models.py`.

8.6.2 Member Data Documentation

8.6.2.1 `__tablename__`

`str models.Category.__tablename__ = "category"` [static], [private]

Definition at line 14 of file `models.py`.

8.6.2.2 `description`

`models.Category.description = Column(Text)` [static]

Definition at line 18 of file `models.py`.

8.6.2.3 id

```
models.Category.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 15 of file [models.py](#).

8.6.2.4 menu_items

```
models.Category.menu_items = relationship("MenuItem", back_populates="category") [static]
```

Definition at line 19 of file [models.py](#).

8.6.2.5 name

```
models.Category.name = Column(String(255), nullable=False) [static]
```

Definition at line 16 of file [models.py](#).

8.6.2.6 type

```
models.Category.type = Column(String(255)) [static]
```

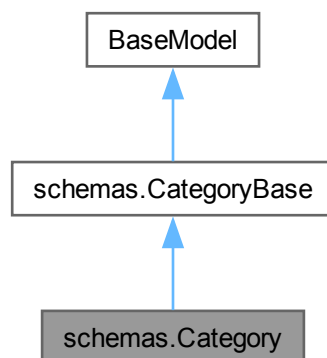
Definition at line 17 of file [models.py](#).

The documentation for this class was generated from the following file:

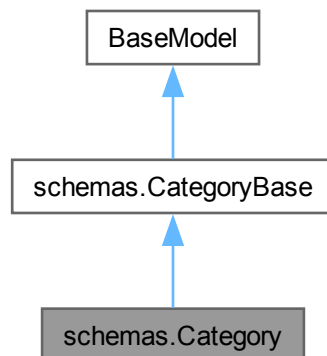
- [backend/models.py](#)

8.7 schemas.Category Class Reference

Inheritance diagram for schemas.Category:



Collaboration diagram for schemas.Category:



Static Public Attributes

- int `model_config` = `ConfigDict(from__attributes=True)`

Static Public Attributes inherited from [schemas.CategoryBase](#)

- Optional `type` = `None`
- Optional `description` = `None`

8.7.1 Detailed Description

Definition at line 24 of file [schemas.py](#).

8.7.2 Member Data Documentation

8.7.2.1 model_config

```
int schemas.Category.model_config = ConfigDict(from__attributes=True) [static]
```

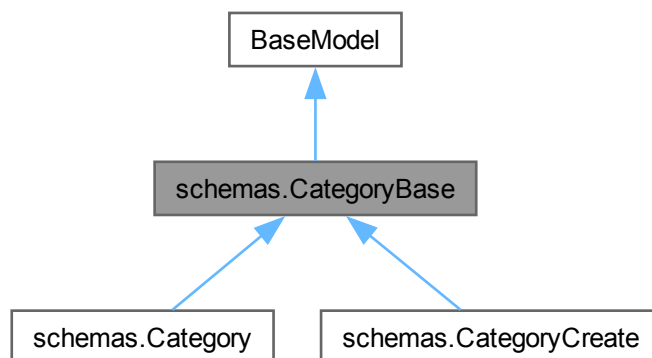
Definition at line 26 of file [schemas.py](#).

The documentation for this class was generated from the following file:

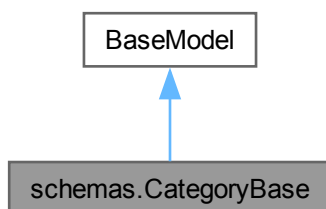
- [backend/schemas.py](#)

8.8 schemas.CategoryBase Class Reference

Inheritance diagram for schemas.CategoryBase:



Collaboration diagram for schemas.CategoryBase:



Static Public Attributes

- Optional `type` = None
- Optional `description` = None

8.8.1 Detailed Description

Schema cơ sở cho danh mục món ăn.

Definition at line 15 of file `schemas.py`.

8.8.2 Member Data Documentation

8.8.2.1 description

Optional schemas.CategoryBase.description = None [static]

Definition at line 19 of file [schemas.py](#).

8.8.2.2 type

Optional schemas.CategoryBase.type = None [static]

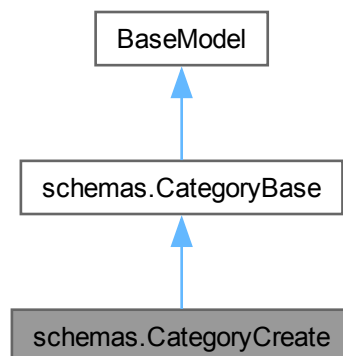
Definition at line 18 of file [schemas.py](#).

The documentation for this class was generated from the following file:

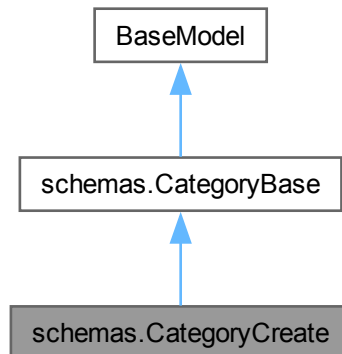
- [backend/schemas.py](#)

8.9 schemas.CategoryCreate Class Reference

Inheritance diagram for schemas.CategoryCreate:



Collaboration diagram for `schemas.CategoryCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.CategoryBase](#)

- Optional `type` = None
- Optional `description` = None

8.9.1 Detailed Description

Definition at line 21 of file [schemas.py](#).

The documentation for this class was generated from the following file:

- [backend/schemas.py](#)

8.10 chatbot__service.ChatBotService Class Reference

Public Member Functions

- `__init__` (self, Session `db`, int `user_id`=None)
- `get_app_context` (self)
- `get_response` (self, str `user_message`)

Public Attributes

- `db` = db
- `user_id` = user_id
- `client` = `genai.Client(api_key=GEMINI_API_KEY)`

Protected Member Functions

- [_call_gemini_sdk](#) (self, str prompt)

Protected Attributes

- [_call_gemini_sdk](#)

8.10.1 Detailed Description

Service xử lý chatbot AI, kết nối Google Gemini và truy vấn database.

Definition at line 20 of file [chatbot_service.py](#).

8.10.2 Constructor & Destructor Documentation

8.10.2.1 `__init__()`

```
chatbot_service.ChatBotService.__init__(
    self,
    Session db,
    int user_id = None)
```

Definition at line 23 of file [chatbot_service.py](#).

```
00023 def __init__(self, db: Session, user_id: int = None):
00024     self.db = db
00025     self.user_id = user_id
00026     # Khởi tạo Client bằng SDK chính thức
00027     if GEMINI_API_KEY:
00028         self.client = genai.Client(api_key=GEMINI_API_KEY)
00029     else:
00030         self.client = None
00031
```

8.10.3 Member Function Documentation

8.10.3.1 `_call_gemini_sdk()`

```
chatbot_service.ChatBotService._call_gemini_sdk (
    self,
    str prompt) [protected]
```

Hàm đồng bộ gọi AI thông qua SDK.

Definition at line 70 of file [chatbot_service.py](#).

```
00070 def _call_gemini_sdk(self, prompt: str):
00071     """Hàm đồng bộ gọi AI thông qua SDK."""
00072     try:
00073         # Nâng cấp lên model thế hệ 3 mới nhất
00074         response = self.client.models.generate_content(
00075             model='gemini-3-flash-preview',
00076             contents=prompt
00077         )
00078         return response.text
00079     except genai_errors.ClientError as e:
00080         error_code = getattr(e, 'code', None)
00081         error_message = str(e)
00082
00083         # Xử lý lỗi 429 - Quota exceeded
00084         if error_code == 429 or 'RESOURCE_EXHAUSTED' in error_message:
00085             print(f"Lỗi 429 - Quota exceeded")
00086             raise Exception("QUOTA_EXCEEDED")
00087         else:
00088             print(f"Lỗi SDK Gemini: {e}")
00089             raise e
00090     except Exception as e:
00091         print(f"Lỗi SDK Gemini: {e}")
00092         raise e
00093
```

8.10.3.2 get_app_context()

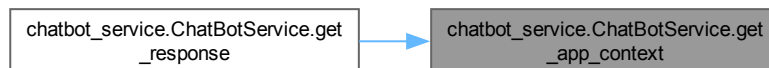
```
chatbot_service.ChatBotService.get_app_context (
    self)
```

Lấy dữ liệu từ DB để làm ngữ cảnh cho AI.

Definition at line 32 of file [chatbot_service.py](#).

```
00032 def get_app_context(self):
00033     """Lấy dữ liệu từ DB để làm ngữ cảnh cho AI."""
00034     try:
00035         restaurants = self.db.query(models.Restaurant).all()
00036         menu_items = self.db.query(models.MenuItem).all()
00037         # Lấy đơn hàng theo user_id nếu có, ngược lại lấy tất cả
00038         if self.user_id:
00039             orders = self.db.query(models.Orders).filter(models.Orders.userid == self.user_id).all()
00040         else:
00041             orders = self.db.query(models.Orders).all()
00042
00043         context = "Dưới đây là thông tin về ứng dụng Đặt Món:\n\n"
00044
00045         context += "NHÀ HÀNG:\n"
00046         for r in restaurants:
00047             context += f"- {r.name}: {r.description} (Địa chỉ: {r.address}, Trạng thái: {r.status})\n"
00048
00049         context += "\nMÓN ĂN:\n"
00050         for m in menu_items:
00051             res_name = next((r.name for r in restaurants if r.id == m.restaurantid), "Không rõ")
00052             context += f"- {m.name}: Giá {m.price}đ, Mô tả: {m.description} (Tại nhà hàng: {res_name})\n"
00053
00054         context += "\nĐƠN HÀNG:\n"
00055         for o in orders:
00056             user_name = "Không rõ"
00057             try:
00058                 user = self.db.query(models.User).filter(models.User.id == o.userid).first()
00059                 if user:
00060                     user_name = user.name
00061             except:
00062                 pass
00063             context += f"- Đơn hàng #{o.id}: Trạng thái {o.status}, Tổng tiền {o.totalprice}đ, Khách hàng: {user_name}\n"
00064
00065         return context
00066     except Exception as e:
00067         print(f"Lỗi lấy dữ liệu DB: {e}")
00068     return "Hiện chưa có thông tin cụ thể."
00069
```

Here is the caller graph for this function:



8.10.3.3 get_response()

```
chatbot_service.ChatBotService.get_response (
    self,
    str user_message)
```

Nhận tin nhắn người dùng, xây dựng prompt và gọi Gemini AI trả lời.

Definition at line 94 of file [chatbot_service.py](#).

```

00094     async def get_response(self, user_message: str):
00095         """Nhận tin nhắn người dùng, xây dựng prompt và gọi Gemini AI trả lời."""
00096         if not self.client:
00097             return "Xin lỗi, ChatBot chưa được cấu hình API Key trong file .env."
00098
00099         app_context = self.get_app_context()
00100
00101         prompt = f"""
00102         Bạn là trợ lý ảo của ứng dụng 'Đặt Món' - Food Delivery Việt Nam.
00103
00104         {app_context}
00105
00106         HƯỚNG DẪN:
00107         1. Trả lời dựa trên dữ liệu nhà hàng, món ăn và đơn hàng được cung cấp ở trên.
00108         2. Thân thiện, ngắn gọn, trả lời bằng tiếng Việt.
00109         3. Nếu khách hàng hỏi về đơn hàng, hãy kiểm tra thông tin đơn hàng trong dữ liệu.
00110
00111         Câu hỏi khách hàng: {user_message}
00112         """
00113
00114         try:
00115             # Chạy trong thread pool để đảm bảo non-blocking cho FastAPI
00116             return await anyio.to_thread.run_sync(self._call_gemini_sdk, prompt)
00117         except Exception as e:
00118             error_message = str(e)
00119             if "QUOTA_EXCEEDED" in error_message:
00120                 return "Xin lỗi, hệ thống AI tạm thời quá tải do đã đạt giới hạn sử dụng miễn phí. Vui lòng thử lại sau 1-2
phút hoặc liên hệ admin để nâng cấp API."
00121             return "Rất tiếc, AI đang gặp một chút trục trặc. Bạn thử lại sau nhé!"

```

Here is the call graph for this function:



8.10.4 Member Data Documentation

8.10.4.1 _call_gemini_sdk

chatbot_service.ChatBotService._call_gemini_sdk [protected]

Definition at line 116 of file [chatbot_service.py](#).

8.10.4.2 client

chatbot_service.ChatBotService.client = genai.Client(api_key=[GEMINI_API_KEY](#))

Definition at line 28 of file [chatbot_service.py](#).

8.10.4.3 db

chatbot_service.ChatBotService.db = db

Definition at line 24 of file [chatbot_service.py](#).

8.10.4.4 user_id

```
chatbot_service.ChatBotService.user_id = user_id
```

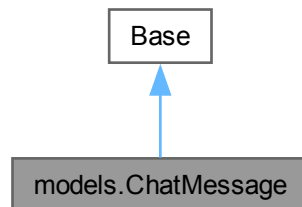
Definition at line 25 of file [chatbot_service.py](#).

The documentation for this class was generated from the following file:

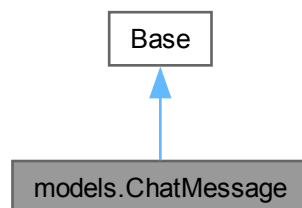
- [backend/chatbot_service.py](#)

8.11 models.ChatMessage Class Reference

Inheritance diagram for models.ChatMessage:



Collaboration diagram for models.ChatMessage:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `senderrole` = `Column(String(255))`
- `message` = `Column(Text)`
- `sentat` = `Column(DateTime, server_default=func.now())`
- `sessionid` = `Column(Integer, ForeignKey("chatsession.id"))`
- `session` = `relationship("ChatSession", back_populates="messages")`

Static Private Attributes

- str `__tablename__` = "chatmessage"

8.11.1 Detailed Description

ORM model cho bảng chatmessage, tin nhắn trong phiên trò chuyện.

Definition at line 222 of file [models.py](#).

8.11.2 Member Data Documentation

8.11.2.1 `__tablename__`

str models.ChatMessage.__tablename__ = "chatmessage" [static], [private]

Definition at line 224 of file [models.py](#).

8.11.2.2 `id`

models.ChatMessage.id = Column(Integer, primary_key=True, index=True) [static]

Definition at line 225 of file [models.py](#).

8.11.2.3 `message`

models.ChatMessage.message = Column(Text) [static]

Definition at line 227 of file [models.py](#).

8.11.2.4 `senderrole`

models.ChatMessage.senderrole = Column(String(255)) [static]

Definition at line 226 of file [models.py](#).

8.11.2.5 `sentat`

models.ChatMessage.sentat = Column(DateTime, server_default=func.now()) [static]

Definition at line 228 of file [models.py](#).

8.11.2.6 `session`

models.ChatMessage.session = relationship("ChatSession", back_populates="messages") [static]

Definition at line 231 of file [models.py](#).

8.11.2.7 sessionid

```
models.ChatMessage.sessionid = Column(Integer, ForeignKey("chatsession.id")) [static]
```

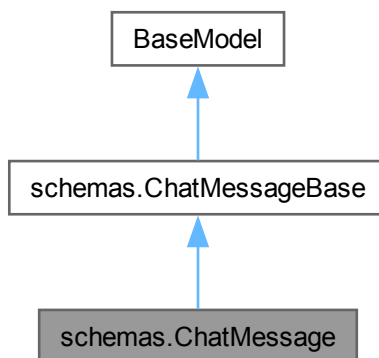
Definition at line 229 of file [models.py](#).

The documentation for this class was generated from the following file:

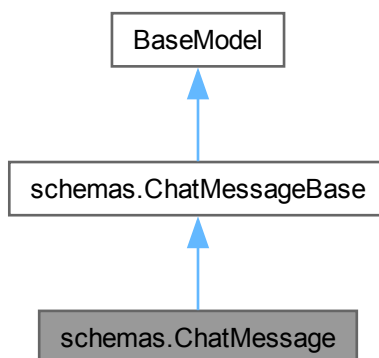
- backend/[models.py](#)

8.12 schemas.ChatMessage Class Reference

Inheritance diagram for schemas.ChatMessage:



Collaboration diagram for schemas.ChatMessage:



Static Public Attributes

- `int model_config = ConfigDict(from_attributes=True)`

8.12.1 Detailed Description

Definition at line 181 of file [schemas.py](#).

8.12.2 Member Data Documentation

8.12.2.1 model_config

```
int schemas.ChatMessage.model_config = ConfigDict(from_attributes=True) [static]
```

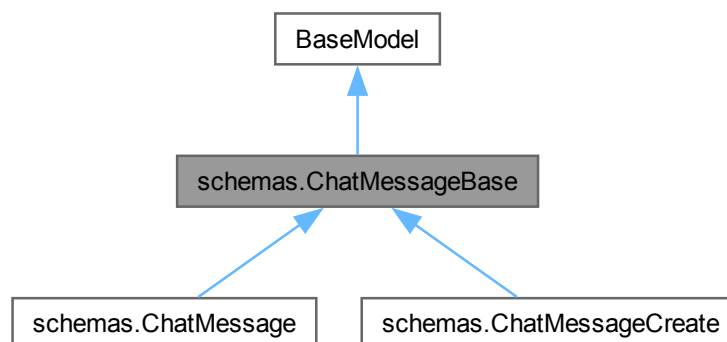
Definition at line 186 of file [schemas.py](#).

The documentation for this class was generated from the following file:

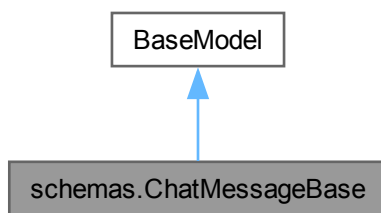
- [backend/schemas.py](#)

8.13 schemas.ChatMessageBase Class Reference

Inheritance diagram for `schemas.ChatMessageBase`:



Collaboration diagram for `schemas.ChatMessageBase`:



8.13.1 Detailed Description

Schema cơ sở cho tin nhắn chat.

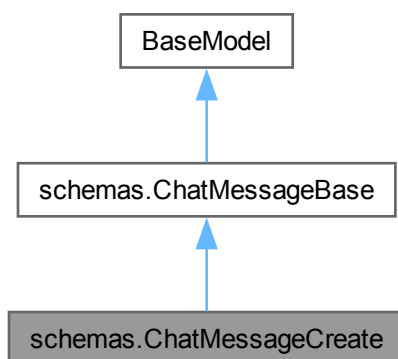
Definition at line [174](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

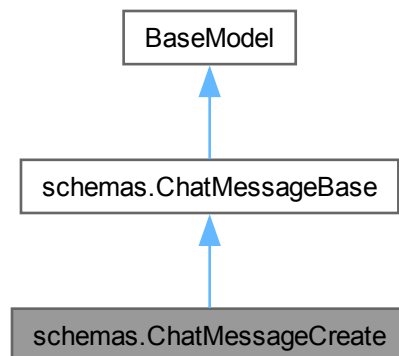
- [backend/schemas.py](#)

8.14 schemas.ChatMessageCreate Class Reference

Inheritance diagram for `schemas.ChatMessageCreate`:



Collaboration diagram for schemas.ChatMessageCreate:



8.14.1 Detailed Description

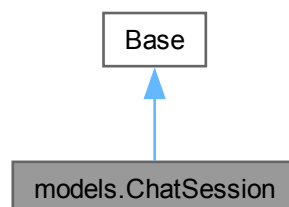
Definition at line 178 of file [schemas.py](#).

The documentation for this class was generated from the following file:

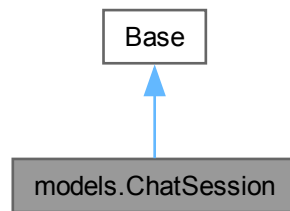
- [backend/schemas.py](#)

8.15 models.ChatSession Class Reference

Inheritance diagram for models.ChatSession:



Collaboration diagram for `models.ChatSession`:



Static Public Attributes

- `id` = `Column(Integer, primary__key=True, index=True)`
- `createdat` = `Column(DateTime, server__default=func.now())`
- `status` = `Column(String(255), nullable=False)`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `user` = `relationship("User", back_populates="chat_sessions")`
- `messages` = `relationship("ChatMessage", back_populates="session")`
- `notifications` = `relationship("Notification", back_populates="session")`

Static Private Attributes

- `str __tablename__ = "chatsession"`

8.15.1 Detailed Description

ORM model cho bảng `chatsession`, phiên trò chuyện với chatbot AI.

Definition at line 210 of file `models.py`.

8.15.2 Member Data Documentation

8.15.2.1 `__tablename__`

```
str models.ChatSession.__tablename__ = "chatsession" [static], [private]
```

Definition at line 212 of file `models.py`.

8.15.2.2 `createdat`

```
models.ChatSession.createdat = Column(DateTime, server__default=func.now()) [static]
```

Definition at line 214 of file `models.py`.

8.15.2.3 id

```
models.ChatSession.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 213 of file [models.py](#).

8.15.2.4 messages

```
models.ChatSession.messages = relationship("ChatMessage", back_populates="session") [static]
```

Definition at line 219 of file [models.py](#).

8.15.2.5 notifications

```
models.ChatSession.notifications = relationship("Notification", back_populates="session") [static]
```

Definition at line 220 of file [models.py](#).

8.15.2.6 status

```
models.ChatSession.status = Column(String(255), nullable=False) [static]
```

Definition at line 215 of file [models.py](#).

8.15.2.7 user

```
models.ChatSession.user = relationship("User", back_populates="chat_sessions") [static]
```

Definition at line 218 of file [models.py](#).

8.15.2.8 userid

```
models.ChatSession.userid = Column(Integer, ForeignKey("User.id")) [static]
```

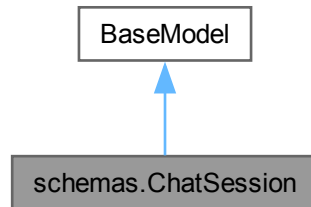
Definition at line 216 of file [models.py](#).

The documentation for this class was generated from the following file:

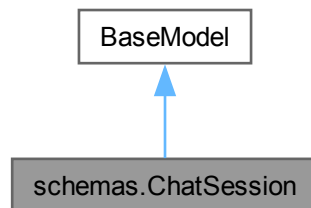
- [backend/models.py](#)

8.16 schemas.ChatSession Class Reference

Inheritance diagram for schemas.ChatSession:



Collaboration diagram for schemas.ChatSession:



Static Public Attributes

- list `messages` = []
- `model_config` = `ConfigDict(from_attributes=True)`

8.16.1 Detailed Description

Definition at line 192 of file `schemas.py`.

8.16.2 Member Data Documentation

8.16.2.1 messages

list `schemas.ChatSession.messages` = [] [static]

Definition at line 196 of file `schemas.py`.

8.16.2.2 model_config

`schemas.ChatSession.model_config = ConfigDict(from_attributes=True)` [static]

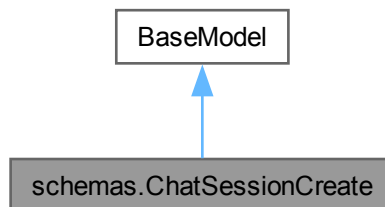
Definition at line 197 of file [schemas.py](#).

The documentation for this class was generated from the following file:

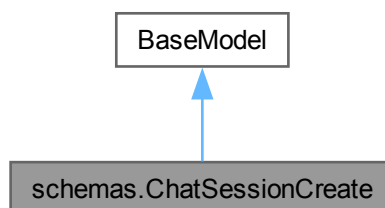
- [backend/schemas.py](#)

8.17 schemas.ChatSessionCreate Class Reference

Inheritance diagram for `schemas.ChatSessionCreate`:



Collaboration diagram for `schemas.ChatSessionCreate`:



8.17.1 Detailed Description

Schema tạo phiên trò chuyện chatbot mới.

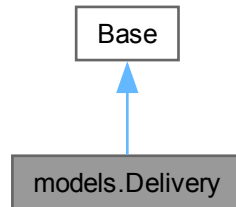
Definition at line 188 of file [schemas.py](#).

The documentation for this class was generated from the following file:

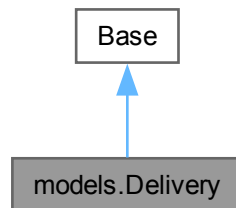
- [backend/schemas.py](#)

8.18 models.Delivery Class Reference

Inheritance diagram for models.Delivery:



Collaboration diagram for models.Delivery:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `status` = `Column(String(255), nullable=False)`
- `deliverytime` = `Column(Time)`
- `orderid` = `Column(Integer, ForeignKey("orders.id"))`
- `shipperid` = `Column(Integer, ForeignKey("shipper.id"))`
- `order` = `relationship("Orders", back_populates="delivery")`
- `shipper` = `relationship("Shipper", back_populates="deliveries")`

Static Private Attributes

- `str __tablename__ = "delivery"`

8.18.1 Detailed Description

ORM model cho bảng delivery, thông tin giao hàng.

Definition at line 165 of file `models.py`.

8.18.2 Member Data Documentation

8.18.2.1 `__tablename__`

```
str models.Delivery.__tablename__ = "delivery" [static], [private]
```

Definition at line 167 of file [models.py](#).

8.18.2.2 `deliverytime`

```
models.Delivery.deliverytime = Column(Time) [static]
```

Definition at line 170 of file [models.py](#).

8.18.2.3 `id`

```
models.Delivery.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 168 of file [models.py](#).

8.18.2.4 `order`

```
models.Delivery.order = relationship("Orders", back_populates="delivery") [static]
```

Definition at line 174 of file [models.py](#).

8.18.2.5 `orderid`

```
models.Delivery.orderid = Column(Integer, ForeignKey("orders.id")) [static]
```

Definition at line 171 of file [models.py](#).

8.18.2.6 `shipper`

```
models.Delivery.shipper = relationship("Shipper", back_populates="deliveries") [static]
```

Definition at line 175 of file [models.py](#).

8.18.2.7 `shipperid`

```
models.Delivery.shipperid = Column(Integer, ForeignKey("shipper.id")) [static]
```

Definition at line 172 of file [models.py](#).

8.18.2.8 status

```
models.Delivery.status = Column(String(255), nullable=False) [static]
```

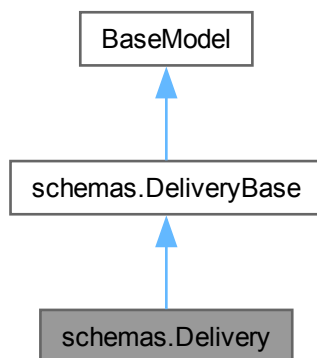
Definition at line 169 of file [models.py](#).

The documentation for this class was generated from the following file:

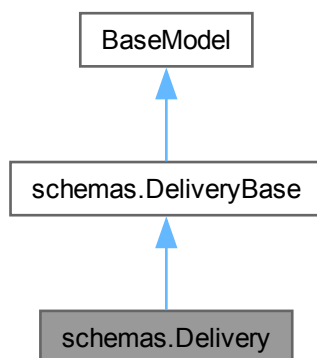
- [backend/models.py](#)

8.19 schemas.Delivery Class Reference

Inheritance diagram for schemas.Delivery:



Collaboration diagram for schemas.Delivery:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

Static Public Attributes inherited from [schemas.DeliveryBase](#)

- Optional `deliverytime` = None

8.19.1 Detailed Description

Definition at line 307 of file [schemas.py](#).

8.19.2 Member Data Documentation

8.19.2.1 `model_config`

```
int schemas.Delivery.model_config = ConfigDict(from__attributes=True) [static]
```

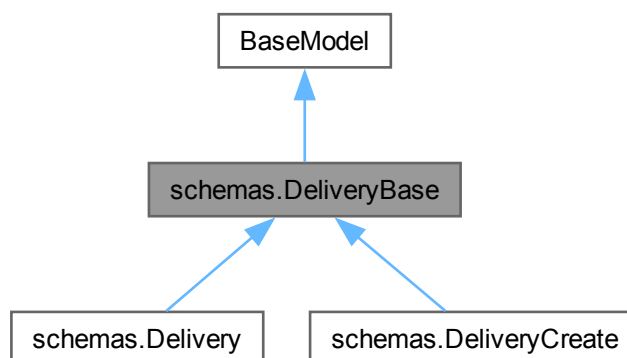
Definition at line 309 of file [schemas.py](#).

The documentation for this class was generated from the following file:

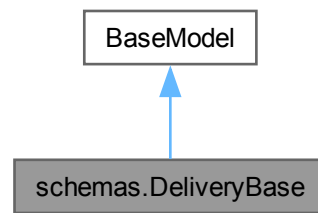
- backend/[schemas.py](#)

8.20 schemas.DeliveryBase Class Reference

Inheritance diagram for `schemas.DeliveryBase`:



Collaboration diagram for `schemas.DeliveryBase`:



Static Public Attributes

- Optional `deliverytime` = None

8.20.1 Detailed Description

Schema cơ sở cho thông tin giao hàng.

Definition at line 297 of file [schemas.py](#).

8.20.2 Member Data Documentation

8.20.2.1 deliverytime

Optional `schemas.DeliveryBase.deliverytime` = None [static]

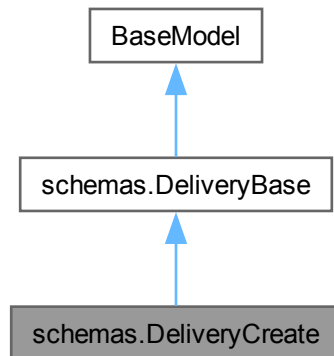
Definition at line 300 of file [schemas.py](#).

The documentation for this class was generated from the following file:

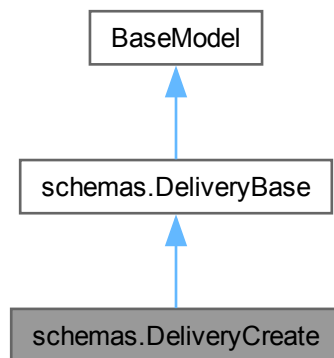
- [backend/schemas.py](#)

8.21 schemas.DeliveryCreate Class Reference

Inheritance diagram for schemas.DeliveryCreate:



Collaboration diagram for schemas.DeliveryCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.DeliveryBase](#)

- Optional [deliverytime](#) = None

8.21.1 Detailed Description

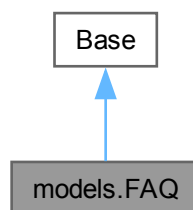
Definition at line 304 of file [schemas.py](#).

The documentation for this class was generated from the following file:

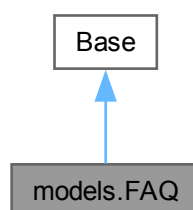
- [backend/schemas.py](#)

8.22 models.FAQ Class Reference

Inheritance diagram for models.FAQ:



Collaboration diagram for models.FAQ:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `question` = Column(String(255))
- `answer` = Column(Text)
- `isactive` = Column(Boolean, default=True)
- `user_faqs` = relationship("UserFAQ", back_populates="faq")

Static Private Attributes

- `str \_\_tablename\_\_ = "faq"`

8.22.1 Detailed Description

ORM model cho bảng faq, lưu trữ câu hỏi thường gặp.

Definition at line [21](#) of file [models.py](#).

8.22.2 Member Data Documentation

8.22.2.1 `__tablename__`

```
str models.FAQ.__tablename__ = "faq" [static], [private]
```

Definition at line [23](#) of file [models.py](#).

8.22.2.2 `answer`

```
models.FAQ.answer = Column(Text) [static]
```

Definition at line [26](#) of file [models.py](#).

8.22.2.3 `id`

```
models.FAQ.id = Column(Integer, primary__key=True, index=True) [static]
```

Definition at line [24](#) of file [models.py](#).

8.22.2.4 `isactive`

```
models.FAQ.isactive = Column(Boolean, default=True) [static]
```

Definition at line [27](#) of file [models.py](#).

8.22.2.5 `question`

```
models.FAQ.question = Column(String(255)) [static]
```

Definition at line [25](#) of file [models.py](#).

8.22.2.6 user_faqs

```
models.FAQ.user_faqs = relationship("UserFAQ", back_populates="faq") [static]
```

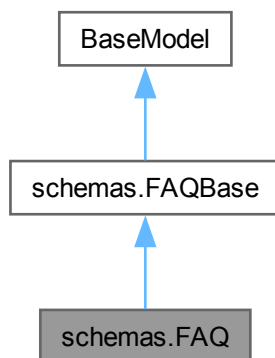
Definition at line 28 of file [models.py](#).

The documentation for this class was generated from the following file:

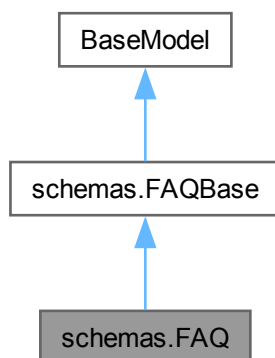
- backend/[models.py](#)

8.23 schemas.FAQ Class Reference

Inheritance diagram for schemas.FAQ:



Collaboration diagram for schemas.FAQ:



Static Public Attributes

- int `model_config` = `ConfigDict(from__attributes=True)`

Static Public Attributes inherited from `schemas.FAQBase`

- bool `isactive` = `True`

8.23.1 Detailed Description

Definition at line 241 of file `schemas.py`.

8.23.2 Member Data Documentation

8.23.2.1 `model_config`

```
int schemas.FAQ.model_config = ConfigDict(from__attributes=True) [static]
```

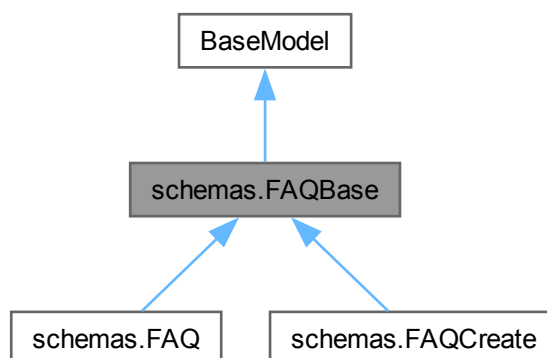
Definition at line 243 of file `schemas.py`.

The documentation for this class was generated from the following file:

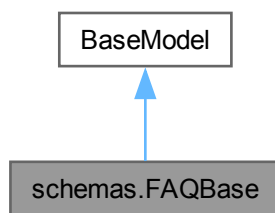
- `backend/schemas.py`

8.24 `schemas.FAQBase` Class Reference

Inheritance diagram for `schemas.FAQBase`:



Collaboration diagram for `schemas.FAQBase`:



Static Public Attributes

- bool `isactive` = True

8.24.1 Detailed Description

Schema cơ sở cho câu hỏi thường gặp.

Definition at line [232](#) of file [schemas.py](#).

8.24.2 Member Data Documentation

8.24.2.1 isactive

bool `schemas.FAQBase.isactive` = True [static]

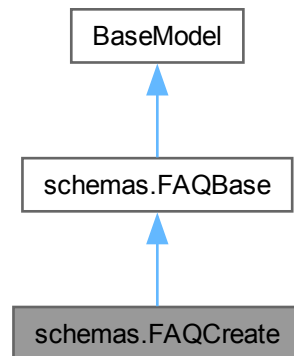
Definition at line [236](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

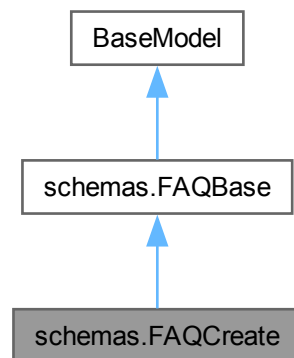
- [backend/schemas.py](#)

8.25 schemas.FAQCreate Class Reference

Inheritance diagram for schemas.FAQCreate:



Collaboration diagram for schemas.FAQCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.FAQBase](#)

- bool [isactive](#) = True

8.25.1 Detailed Description

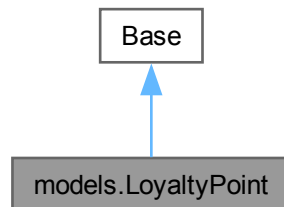
Definition at line 238 of file [schemas.py](#).

The documentation for this class was generated from the following file:

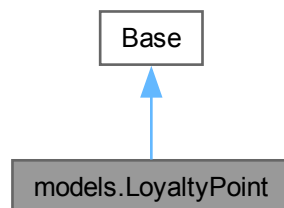
- [backend/schemas.py](#)

8.26 models.LoyaltyPoint Class Reference

Inheritance diagram for models.LoyaltyPoint:



Collaboration diagram for models.LoyaltyPoint:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `points` = `Column(Integer, default=0)`
- `updatedat` = `Column(Date)`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `menuitemid` = `Column(Integer, ForeignKey("menuitem.id"))`
- `user` = `relationship("User", back_populates="loyalty_points")`
- `menu_item` = `relationship("MenuItem")`

Static Private Attributes

- str `__tablename__` = "loyaltpoint"

8.26.1 Detailed Description

ORM model cho bảng loaltpoint, điểm tích lũy thành viên.

Definition at line 266 of file [models.py](#).

8.26.2 Member Data Documentation

8.26.2.1 `__tablename__`

str models.LoyaltyPoint.__tablename__ = "loyaltpoint" [static], [private]

Definition at line 268 of file [models.py](#).

8.26.2.2 `id`

models.LoyaltyPoint.id = Column(Integer, primary_key=True, index=True) [static]

Definition at line 269 of file [models.py](#).

8.26.2.3 `menu_item`

models.LoyaltyPoint.menu_item = relationship("MenuItem") [static]

Definition at line 276 of file [models.py](#).

8.26.2.4 `menuitemid`

models.LoyaltyPoint.menuitemid = Column(Integer, ForeignKey("menuitem.id")) [static]

Definition at line 273 of file [models.py](#).

8.26.2.5 `points`

models.LoyaltyPoint.points = Column(Integer, default=0) [static]

Definition at line 270 of file [models.py](#).

8.26.2.6 `updatedat`

models.LoyaltyPoint.updatedat = Column(Date) [static]

Definition at line 271 of file [models.py](#).

8.26.2.7 user

```
models.LoyaltyPoint.user = relationship("User", back_populates="loyalty_points") [static]
```

Definition at line 275 of file [models.py](#).

8.26.2.8 userid

```
models.LoyaltyPoint.userid = Column(Integer, ForeignKey("User.id")) [static]
```

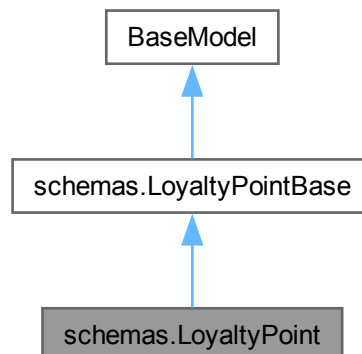
Definition at line 272 of file [models.py](#).

The documentation for this class was generated from the following file:

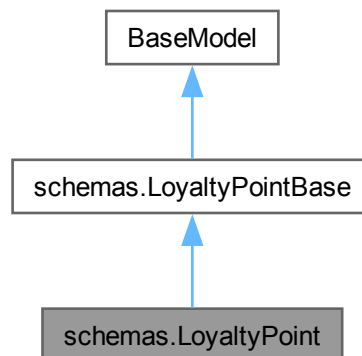
- [backend/models.py](#)

8.27 schemas.LoyaltyPoint Class Reference

Inheritance diagram for schemas.LoyaltyPoint:



Collaboration diagram for schemas.LoyaltyPoint:



Static Public Attributes

- date `model_config` = `ConfigDict(from_attributes=True)`

Static Public Attributes inherited from [schemas.LoyaltyPointBase](#)

- int `points` = 0

8.27.1 Detailed Description

Definition at line 391 of file [schemas.py](#).

8.27.2 Member Data Documentation

8.27.2.1 model_config

date `schemas.LoyaltyPoint.model_config` = `ConfigDict(from_attributes=True)` [static]

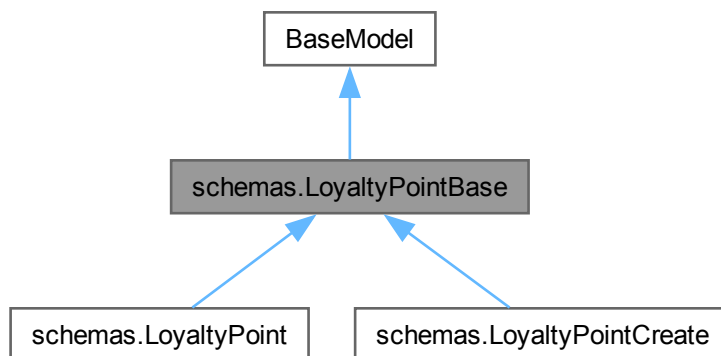
Definition at line 394 of file [schemas.py](#).

The documentation for this class was generated from the following file:

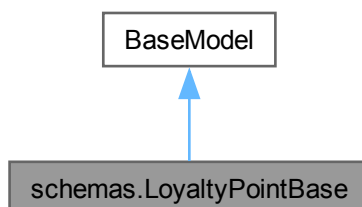
- backend/[schemas.py](#)

8.28 schemas.LoyaltyPointBase Class Reference

Inheritance diagram for schemas.LoyaltyPointBase:



Collaboration diagram for schemas.LoyaltyPointBase:



Static Public Attributes

- int `points` = 0

8.28.1 Detailed Description

Schema cơ sở cho điểm tích lũy thành viên.

Definition at line 382 of file [schemas.py](#).

8.28.2 Member Data Documentation

8.28.2.1 points

`int schemas.LoyaltyPointBase.points = 0` [static]

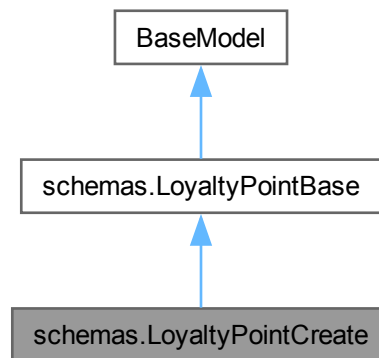
Definition at line 384 of file [schemas.py](#).

The documentation for this class was generated from the following file:

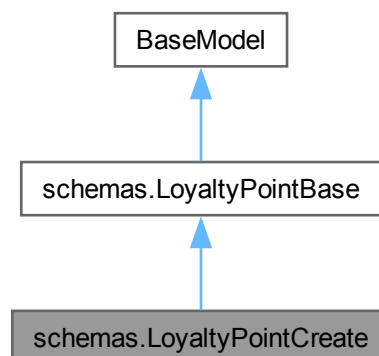
- [backend/schemas.py](#)

8.29 schemas.LoyaltyPointCreate Class Reference

Inheritance diagram for `schemas.LoyaltyPointCreate`:



Collaboration diagram for `schemas.LoyaltyPointCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.LoyaltyPointBase](#)

- int [points](#) = 0

8.29.1 Detailed Description

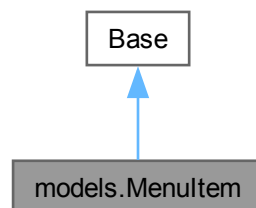
Definition at line 388 of file [schemas.py](#).

The documentation for this class was generated from the following file:

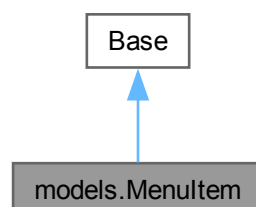
- backend/[schemas.py](#)

8.30 models.MenuItem Class Reference

Inheritance diagram for models.MenuItem:



Collaboration diagram for models.MenuItem:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `name` = `Column(String(255), nullable=False)`
- `image_url` = `Column(Text)`
- `price` = `Column(Integer)`
- `is_available` = `Column(Boolean, default=True)`
- `description` = `Column(Text)`
- `restaurantid` = `Column(Integer, ForeignKey("restaurant.id"))`
- `categoryid` = `Column(Integer, ForeignKey("category.id"))`
- `restaurant` = `relationship("Restaurant", back_populates="menu_items")`
- `category` = `relationship("Category", back_populates="menu_items")`
- `order_items` = `relationship("OrderItem", back_populates="menu_item")`
- `reviews` = `relationship("Review", back_populates="menu_item")`
- `social_shares` = `relationship("SocialShare", back_populates="menu_item")`

Static Private Attributes

- `str __tablename__ = "menuitem"`

8.30.1 Detailed Description

ORM model cho bảng menuitem, món ăn thuộc nhà hàng.

Definition at line 93 of file [models.py](#).

8.30.2 Member Data Documentation

8.30.2.1 `__tablename__`

```
str models.MenuItem.__tablename__ = "menuitem" [static], [private]
```

Definition at line 95 of file [models.py](#).

8.30.2.2 `category`

```
models.MenuItem.category = relationship("Category", back_populates="menu_items") [static]
```

Definition at line 106 of file [models.py](#).

8.30.2.3 `categoryid`

```
models.MenuItem.categoryid = Column(Integer, ForeignKey("category.id")) [static]
```

Definition at line 103 of file [models.py](#).

8.30.2.4 description

```
models.MenuItem.description = Column(Text) [static]
```

Definition at line 101 of file [models.py](#).

8.30.2.5 id

```
models.MenuItem.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 96 of file [models.py](#).

8.30.2.6 image_url

```
models.MenuItem.image_url = Column(Text) [static]
```

Definition at line 98 of file [models.py](#).

8.30.2.7 is_available

```
models.MenuItem.is_available = Column(Boolean, default=True) [static]
```

Definition at line 100 of file [models.py](#).

8.30.2.8 name

```
models.MenuItem.name = Column(String(255), nullable=False) [static]
```

Definition at line 97 of file [models.py](#).

8.30.2.9 order_items

```
models.MenuItem.order_items = relationship("OrderItem", back_populates="menu_item") [static]
```

Definition at line 107 of file [models.py](#).

8.30.2.10 price

```
models.MenuItem.price = Column(Integer) [static]
```

Definition at line 99 of file [models.py](#).

8.30.2.11 restaurant

```
models.MenuItem.restaurant = relationship("Restaurant", back_populates="menu_items") [static]
```

Definition at line 105 of file [models.py](#).

8.30.2.12 restaurantid

```
models.MenuItem.restaurantid = Column(Integer, ForeignKey("restaurant.id")) [static]
```

Definition at line 102 of file [models.py](#).

8.30.2.13 reviews

```
models.MenuItem.reviews = relationship("Review", back_populates="menu_item") [static]
```

Definition at line 108 of file [models.py](#).

8.30.2.14 social_shares

```
models.MenuItem.social_shares = relationship("SocialShare", back_populates="menu_item") [static]
```

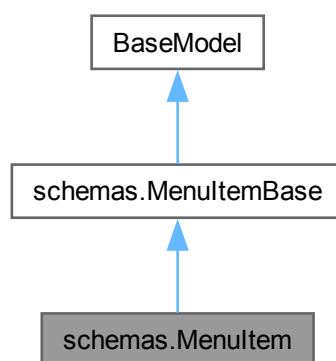
Definition at line 109 of file [models.py](#).

The documentation for this class was generated from the following file:

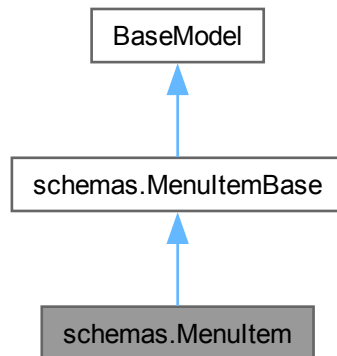
- [backend/models.py](#)

8.31 schemas.MenuItem Class Reference

Inheritance diagram for schemas.MenuItem:



Collaboration diagram for `schemas.MenuItem`:



Static Public Attributes

- int `model_config` = `ConfigDict(from_attributes=True)`

Static Public Attributes inherited from `schemas.MenuItemBase`

- Optional `image_url` = `None`
- bool `is_available` = `True`
- Optional `description` = `None`

8.31.1 Detailed Description

Definition at line 41 of file `schemas.py`.

8.31.2 Member Data Documentation

8.31.2.1 `model_config`

```
int schemas.MenuItem.model_config = ConfigDict(from_attributes=True) [static]
```

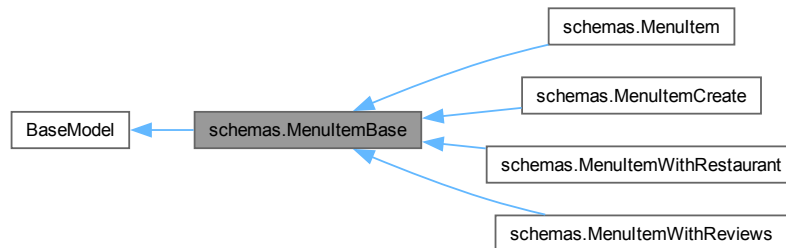
Definition at line 43 of file `schemas.py`.

The documentation for this class was generated from the following file:

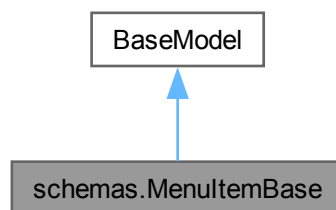
- backend/`schemas.py`

8.32 schemas.MenuItemBase Class Reference

Inheritance diagram for schemas.MenuItemBase:



Collaboration diagram for schemas.MenuItemBase:



Static Public Attributes

- Optional `image_url` = None
- bool `is_available` = True
- Optional `description` = None

8.32.1 Detailed Description

Schema cơ sở cho món ăn trong thực đơn.

Definition at line 28 of file `schemas.py`.

8.32.2 Member Data Documentation

8.32.2.1 description

Optional `schemas.MenuItemBase.description` = None [static]

Definition at line 34 of file `schemas.py`.

8.32.2.2 `image_url`

Optional `schemas.MenuItemBase.image_url = None` [static]

Definition at line 31 of file [schemas.py](#).

8.32.2.3 `is_available`

bool `schemas.MenuItemBase.is_available = True` [static]

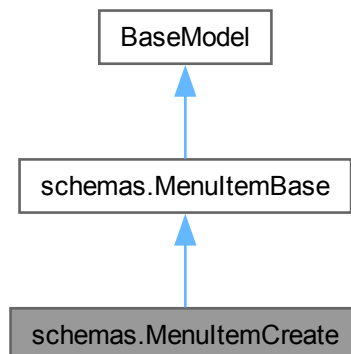
Definition at line 33 of file [schemas.py](#).

The documentation for this class was generated from the following file:

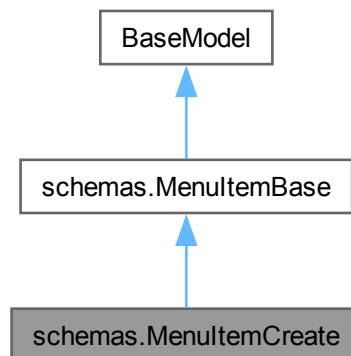
- [backend/schemas.py](#)

8.33 `schemas.MenuItemCreate` Class Reference

Inheritance diagram for `schemas.MenuItemCreate`:



Collaboration diagram for schemas.MenuItemCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.MenuItemBase](#)

- Optional [image_url](#) = None
- bool [is_available](#) = True
- Optional [description](#) = None

8.33.1 Detailed Description

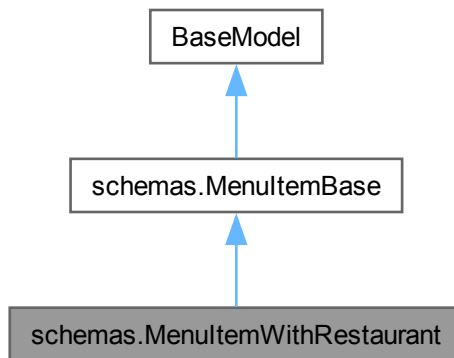
Definition at line 38 of file [schemas.py](#).

The documentation for this class was generated from the following file:

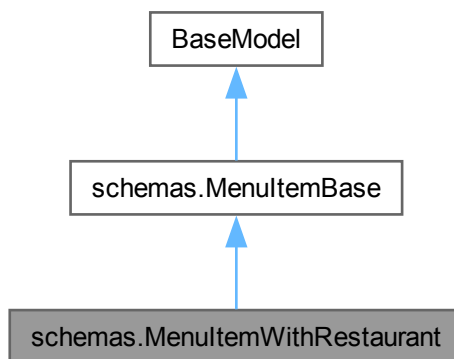
- backend/[schemas.py](#)

8.34 schemas.MenuItemWithRestaurant Class Reference

Inheritance diagram for schemas.MenuItemWithRestaurant:



Collaboration diagram for schemas.MenuItemWithRestaurant:



Static Public Attributes

- Optional [restaurant_name](#) = None
- [model_config](#) = ConfigDict(from_attributes=True)

Static Public Attributes inherited from [schemas.MenuItemBase](#)

- Optional [image_url](#) = None
- bool [is_available](#) = True
- Optional [description](#) = None

8.34.1 Detailed Description

Definition at line 45 of file [schemas.py](#).

8.34.2 Member Data Documentation

8.34.2.1 model_config

`schemas.MenuItemWithRestaurant.model_config = ConfigDict(from_attributes=True)` [static]

Definition at line 48 of file [schemas.py](#).

8.34.2.2 restaurant_name

Optional `schemas.MenuItemWithRestaurant.restaurant_name = None` [static]

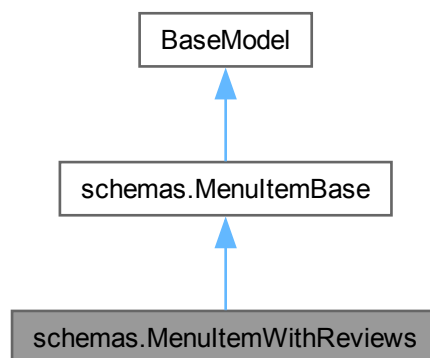
Definition at line 47 of file [schemas.py](#).

The documentation for this class was generated from the following file:

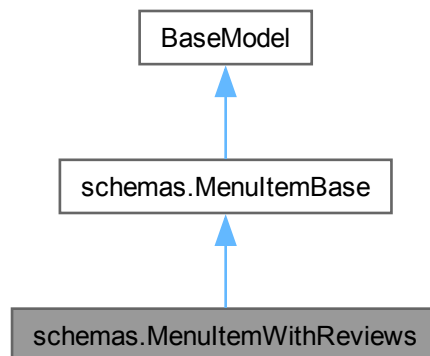
- [backend/schemas.py](#)

8.35 schemas.MenuItemWithReviews Class Reference

Inheritance diagram for `schemas.MenuItemWithReviews`:



Collaboration diagram for `schemas.MenuItemWithReviews`:



Static Public Attributes

- Optional `restaurant_name` = None
- list `reviews` = []
- float `avg_rating` = 0.0
- `model_config` = `ConfigDict(from_attributes=True)`

Static Public Attributes inherited from `schemas.MenuItemBase`

- Optional `image_url` = None
- bool `is_available` = True
- Optional `description` = None

8.35.1 Detailed Description

Schema món ăn kèm danh sách đánh giá và điểm trung bình.

Definition at line 337 of file `schemas.py`.

8.35.2 Member Data Documentation

8.35.2.1 `avg_rating`

`float schemas.MenuItemWithReviews.avg_rating = 0.0` [static]

Definition at line 342 of file `schemas.py`.

8.35.2.2 model_config

```
schemas.MenuItemWithReviews.model_config = ConfigDict(from_attributes=True) [static]
```

Definition at line 343 of file [schemas.py](#).

8.35.2.3 restaurant_name

```
Optional schemas.MenuItemWithReviews.restaurant_name = None [static]
```

Definition at line 340 of file [schemas.py](#).

8.35.2.4 reviews

```
list schemas.MenuItemWithReviews.reviews = [] [static]
```

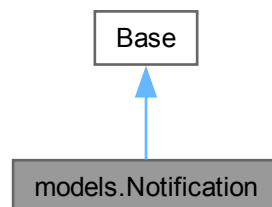
Definition at line 341 of file [schemas.py](#).

The documentation for this class was generated from the following file:

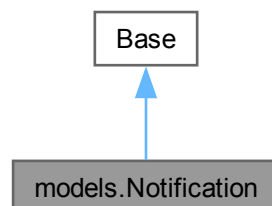
- [backend/schemas.py](#)

8.36 models.Notification Class Reference

Inheritance diagram for models.Notification:



Collaboration diagram for models.Notification:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `title` = `Column(String(255))`
- `type` = `Column(String(255))`
- `content` = `Column(Text)`
- `isread` = `Column(Boolean, default=False)`
- `createdat` = `Column(DateTime, server_default=func.now())`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `orderid` = `Column(Integer, ForeignKey("orders.id"), nullable=True)`
- `sessionid` = `Column(Integer, ForeignKey("chatsession.id"), nullable=True)`
- `user` = `relationship("User", back_populates="notifications")`
- `order` = `relationship("Orders", back_populates="notifications")`
- `session` = `relationship("ChatSession", back_populates="notifications")`

Static Private Attributes

- `str __tablename__ = "notification"`

8.36.1 Detailed Description

ORM model cho bảng notification, thông báo đẩy đến người dùng.

Definition at line 193 of file [models.py](#).

8.36.2 Member Data Documentation

8.36.2.1 `__tablename__`

`str models.Notification.__tablename__ = "notification" [static], [private]`

Definition at line 195 of file [models.py](#).

8.36.2.2 `content`

`models.Notification.content = Column(Text) [static]`

Definition at line 199 of file [models.py](#).

8.36.2.3 `createdat`

`models.Notification.createdat = Column(DateTime, server_default=func.now()) [static]`

Definition at line 201 of file [models.py](#).

8.36.2.4 id

```
models.Notification.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 196 of file [models.py](#).

8.36.2.5 isread

```
models.Notification.isread = Column(Boolean, default=False) [static]
```

Definition at line 200 of file [models.py](#).

8.36.2.6 order

```
models.Notification.order = relationship("Orders", back_populates="notifications") [static]
```

Definition at line 207 of file [models.py](#).

8.36.2.7 orderid

```
models.Notification.orderid = Column(Integer, ForeignKey("orders.id"), nullable=True) [static]
```

Definition at line 203 of file [models.py](#).

8.36.2.8 session

```
models.Notification.session = relationship("ChatSession", back_populates="notifications") [static]
```

Definition at line 208 of file [models.py](#).

8.36.2.9 sessionid

```
models.Notification.sessionid = Column(Integer, ForeignKey("chatsession.id"), nullable=True) [static]
```

Definition at line 204 of file [models.py](#).

8.36.2.10 title

```
models.Notification.title = Column(String(255)) [static]
```

Definition at line 197 of file [models.py](#).

8.36.2.11 type

```
models.Notification.type = Column(String(255)) [static]
```

Definition at line 198 of file [models.py](#).

8.36.2.12 user

```
models.Notification.user = relationship("User", back_populates="notifications") [static]
```

Definition at line 206 of file [models.py](#).

8.36.2.13 userid

```
models.Notification.userid = Column(Integer, ForeignKey("User.id")) [static]
```

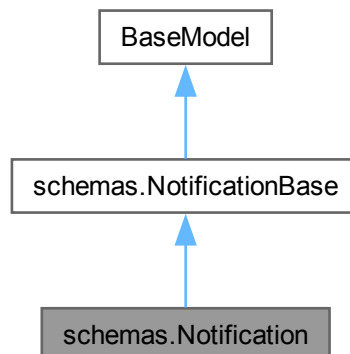
Definition at line 202 of file [models.py](#).

The documentation for this class was generated from the following file:

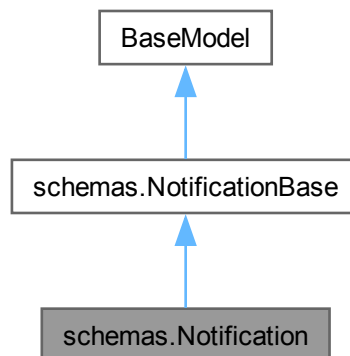
- [backend/models.py](#)

8.37 schemas.Notification Class Reference

Inheritance diagram for schemas.Notification:



Collaboration diagram for schemas.Notification:



Static Public Attributes

- datetime `model_config` = `ConfigDict(from_attributes=True)`

Static Public Attributes inherited from `schemas.NotificationBase`

- bool `isread` = `False`
- Optional `orderid` = `None`
- Optional `sessionid` = `None`

8.37.1 Detailed Description

Definition at line 360 of file `schemas.py`.

8.37.2 Member Data Documentation

8.37.2.1 `model_config`

datetime `schemas.Notification.model_config` = `ConfigDict(from_attributes=True)` [static]

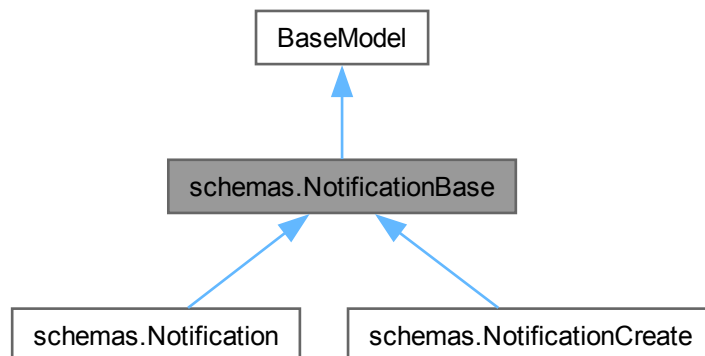
Definition at line 363 of file `schemas.py`.

The documentation for this class was generated from the following file:

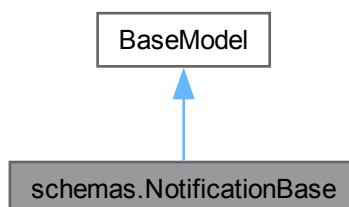
- `backend/schemas.py`

8.38 schemas.NotificationBase Class Reference

Inheritance diagram for schemas.NotificationBase:



Collaboration diagram for schemas.NotificationBase:



Static Public Attributes

- bool `isread` = False
- Optional `orderid` = None
- Optional `sessionid` = None

8.38.1 Detailed Description

Schema cơ sở cho thông báo đẩy.

Definition at line 347 of file `schemas.py`.

8.38.2 Member Data Documentation

8.38.2.1 isread

bool schemas.NotificationBase.isread = False [static]

Definition at line 352 of file [schemas.py](#).

8.38.2.2 orderid

Optional schemas.NotificationBase.orderid = None [static]

Definition at line 354 of file [schemas.py](#).

8.38.2.3 sessionid

Optional schemas.NotificationBase.sessionid = None [static]

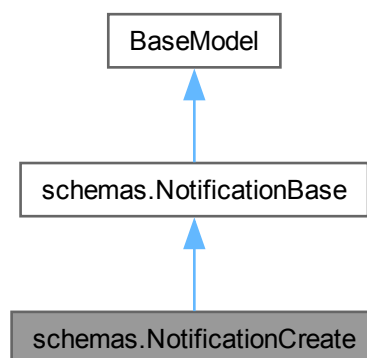
Definition at line 355 of file [schemas.py](#).

The documentation for this class was generated from the following file:

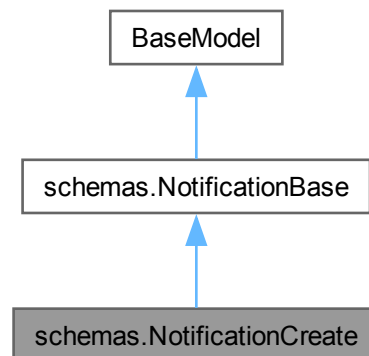
- backend/[schemas.py](#)

8.39 schemas.NotificationCreate Class Reference

Inheritance diagram for schemas.NotificationCreate:



Collaboration diagram for `schemas.NotificationCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.NotificationBase](#)

- bool [isread](#) = False
- Optional [orderid](#) = None
- Optional [sessionid](#) = None

8.39.1 Detailed Description

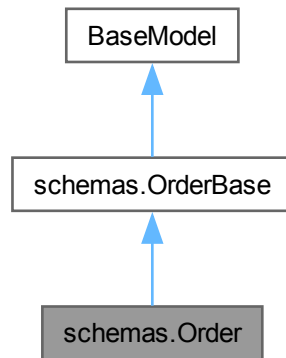
Definition at line [357](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

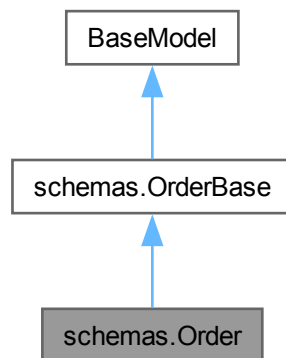
- [backend/schemas.py](#)

8.40 schemas.Order Class Reference

Inheritance diagram for schemas.Order:



Collaboration diagram for schemas.Order:



Static Public Attributes

- list `order_items` = []
- `model_config` = ConfigDict(from_attributes=True)

Static Public Attributes inherited from [schemas.OrderBase](#)

- Optional `preorderdate` = None
- Optional `preordertime` = None

8.40.1 Detailed Description

Definition at line 137 of file [schemas.py](#).

8.40.2 Member Data Documentation

8.40.2.1 `model_config`

`schemas.Order.model_config = ConfigDict(from_attributes=True)` [static]

Definition at line 141 of file [schemas.py](#).

8.40.2.2 `order_items`

`list schemas.Order.order_items = []` [static]

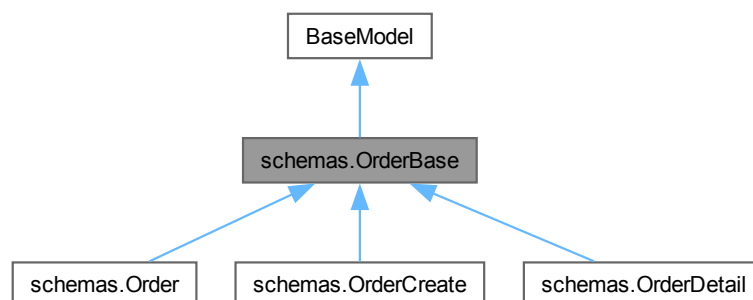
Definition at line 140 of file [schemas.py](#).

The documentation for this class was generated from the following file:

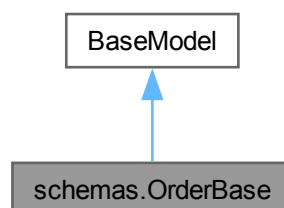
- [backend/schemas.py](#)

8.41 `schemas.OrderBase` Class Reference

Inheritance diagram for `schemas.OrderBase`:



Collaboration diagram for `schemas.OrderBase`:



Static Public Attributes

- Optional [preorderdate](#) = None
- Optional [preordertime](#) = None

8.41.1 Detailed Description

Schema cơ sở cho đơn hàng.

Definition at line [124](#) of file [schemas.py](#).

8.41.2 Member Data Documentation

8.41.2.1 preorderdate

Optional schemas.OrderBase.preorderdate = None [static]

Definition at line [127](#) of file [schemas.py](#).

8.41.2.2 preordertime

Optional schemas.OrderBase.preordertime = None [static]

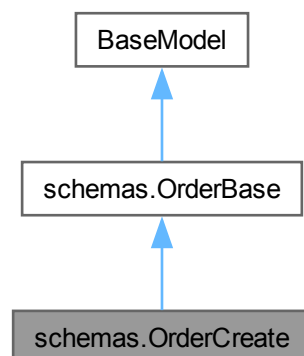
Definition at line [128](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

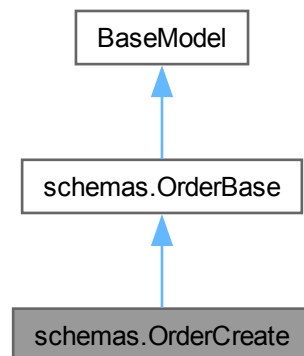
- backend/[schemas.py](#)

8.42 schemas.OrderCreate Class Reference

Inheritance diagram for schemas.OrderCreate:



Collaboration diagram for `schemas.OrderCreate`:



Static Public Attributes

- List `order_items` [`OrderItemCreate`]

Static Public Attributes inherited from `schemas.OrderBase`

- Optional `preorderdate` = None
- Optional `preordertime` = None

8.42.1 Detailed Description

Definition at line 134 of file `schemas.py`.

8.42.2 Member Data Documentation

8.42.2.1 `order_items`

List `schemas.OrderCreate.order_items` [`OrderItemCreate`] [static]

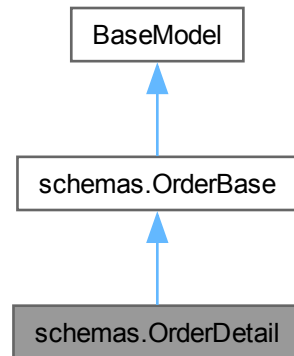
Definition at line 135 of file `schemas.py`.

The documentation for this class was generated from the following file:

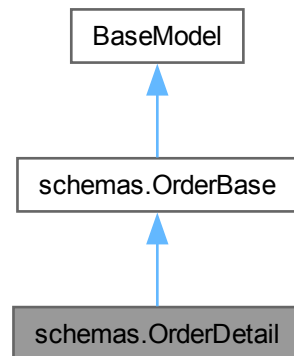
- backend/`schemas.py`

8.43 schemas.OrderDetail Class Reference

Inheritance diagram for schemas.OrderDetail:



Collaboration diagram for schemas.OrderDetail:



Static Public Attributes

- Optional [restaurant_name](#) = None
- Optional [address_detail](#) = None
- list [order_items](#) = []

Static Public Attributes inherited from [schemas.OrderBase](#)

- Optional [preorderdate](#) = None
- Optional [preordertime](#) = None

8.43.1 Detailed Description

Schema chi tiết đơn hàng đầy đủ kèm tên nhà hàng và địa chỉ.

Definition at line 152 of file [schemas.py](#).

8.43.2 Member Data Documentation

8.43.2.1 address__detail

Optional `schemas.OrderDetail.address__detail = None` [static]

Definition at line 157 of file [schemas.py](#).

8.43.2.2 order__items

list `schemas.OrderDetail.order__items = []` [static]

Definition at line 158 of file [schemas.py](#).

8.43.2.3 restaurant__name

Optional `schemas.OrderDetail.restaurant__name = None` [static]

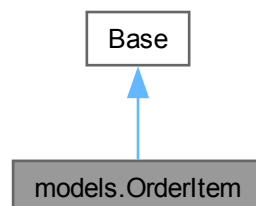
Definition at line 156 of file [schemas.py](#).

The documentation for this class was generated from the following file:

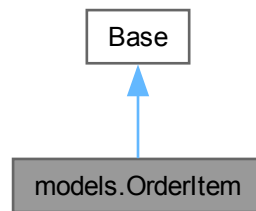
- [backend/schemas.py](#)

8.44 models.OrderItem Class Reference

Inheritance diagram for `models.OrderItem`:



Collaboration diagram for models.OrderItem:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `quantity` = `Column(Integer, nullable=False)`
- `price` = `Column(Integer, nullable=False)`
- `menuitemid` = `Column(Integer, ForeignKey("menuitem.id"))`
- `orderid` = `Column(Integer, ForeignKey("orders.id"))`
- `order` = `relationship("Orders", back_populates="order_items")`
- `menu_item` = `relationship("MenuItem", back_populates="order_items")`

Static Private Attributes

- `str __tablename__ = "orderitem"`

8.44.1 Detailed Description

ORM model cho bảng orderitem, chi tiết từng món trong đơn hàng.

Definition at line 144 of file [models.py](#).

8.44.2 Member Data Documentation

8.44.2.1 `__tablename__`

`str models.OrderItem.__tablename__ = "orderitem" [static], [private]`

Definition at line 146 of file [models.py](#).

8.44.2.2 `id`

`models.OrderItem.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line 147 of file [models.py](#).

8.44.2.3 menu_item

```
models.OrderItem.menu_item = relationship("MenuItem", back_populates="order_items") [static]
```

Definition at line 154 of file [models.py](#).

8.44.2.4 menuitemid

```
models.OrderItem.menuitemid = Column(Integer, ForeignKey("menuitem.id")) [static]
```

Definition at line 150 of file [models.py](#).

8.44.2.5 order

```
models.OrderItem.order = relationship("Orders", back_populates="order_items") [static]
```

Definition at line 153 of file [models.py](#).

8.44.2.6 orderid

```
models.OrderItem.orderid = Column(Integer, ForeignKey("orders.id")) [static]
```

Definition at line 151 of file [models.py](#).

8.44.2.7 price

```
models.OrderItem.price = Column(Integer, nullable=False) [static]
```

Definition at line 149 of file [models.py](#).

8.44.2.8 quantity

```
models.OrderItem.quantity = Column(Integer, nullable=False) [static]
```

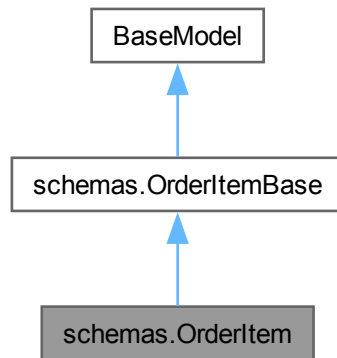
Definition at line 148 of file [models.py](#).

The documentation for this class was generated from the following file:

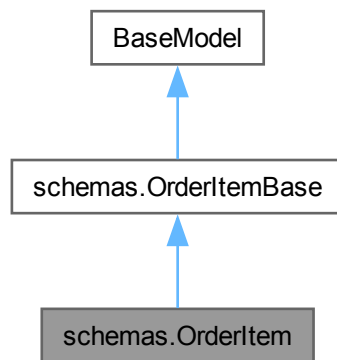
- backend/[models.py](#)

8.45 schemas.OrderItem Class Reference

Inheritance diagram for schemas.OrderItem:



Collaboration diagram for schemas.OrderItem:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

8.45.1 Detailed Description

Definition at line 120 of file [schemas.py](#).

8.45.2 Member Data Documentation

8.45.2.1 model_config

```
int schemas.OrderItem.model_config = ConfigDict(from_attributes=True) [static]
```

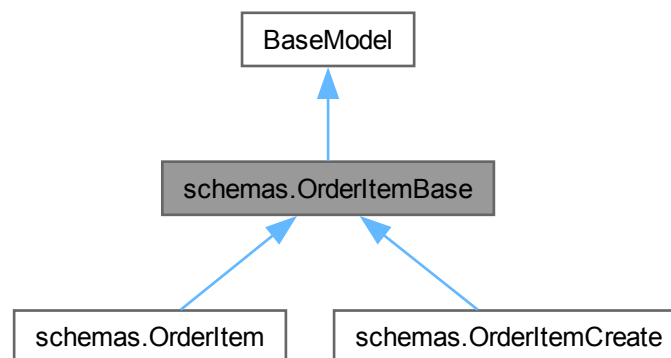
Definition at line 122 of file [schemas.py](#).

The documentation for this class was generated from the following file:

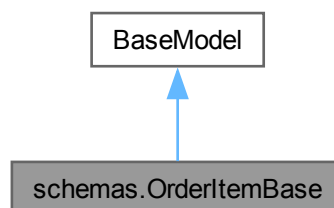
- [backend/schemas.py](#)

8.46 schemas.OrderItemBase Class Reference

Inheritance diagram for schemas.OrderItemBase:



Collaboration diagram for schemas.OrderItemBase:



8.46.1 Detailed Description

Schema cơ sở cho chi tiết món trong đơn hàng.

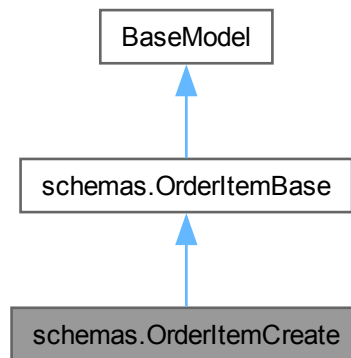
Definition at line 111 of file [schemas.py](#).

The documentation for this class was generated from the following file:

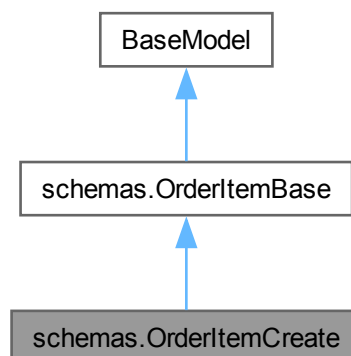
- [backend/schemas.py](#)

8.47 schemas.OrderItemCreate Class Reference

Inheritance diagram for schemas.OrderItemCreate:



Collaboration diagram for schemas.OrderItemCreate:



8.47.1 Detailed Description

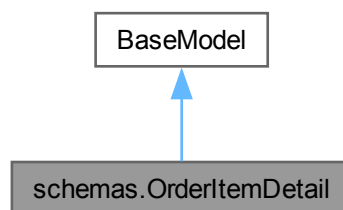
Definition at line 117 of file [schemas.py](#).

The documentation for this class was generated from the following file:

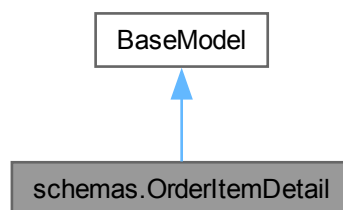
- [backend/schemas.py](#)

8.48 schemas.OrderItemDetail Class Reference

Inheritance diagram for schemas.OrderItemDetail:



Collaboration diagram for schemas.OrderItemDetail:



Static Public Attributes

- Optional `menuitem_name` = None
- Optional `image_url` = None

8.48.1 Detailed Description

Schema chi tiết món ăn trong đơn hàng kèm tên và ảnh.

Definition at line 143 of file [schemas.py](#).

8.48.2 Member Data Documentation

8.48.2.1 image_url

Optional schemas.OrderItemDetail.image_url = None [static]

Definition at line 150 of file [schemas.py](#).

8.48.2.2 menuitem_name

Optional schemas.OrderItemDetail.menuitem_name = None [static]

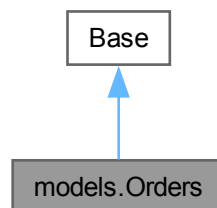
Definition at line 149 of file [schemas.py](#).

The documentation for this class was generated from the following file:

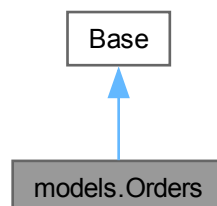
- [backend/schemas.py](#)

8.49 models.Orders Class Reference

Inheritance diagram for models.Orders:



Collaboration diagram for models.Orders:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `status` = `Column(String(255), nullable=False)`
- `createdat` = `Column(DateTime, server_default=func.now())`
- `preorderdate` = `Column(Date)`
- `preordertime` = `Column(Time)`
- `totalprice` = `Column(Integer)`
- `restaurantid` = `Column(Integer, ForeignKey("restaurant.id"))`
- `addressid` = `Column(Integer, ForeignKey("address.id"))`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `user` = `relationship("User", back_populates="orders")`
- `restaurant` = `relationship("Restaurant", back_populates="orders")`
- `address` = `relationship("Address", back_populates="orders")`
- `order_items` = `relationship("OrderItem", back_populates="order")`
- `payments` = `relationship("Payment", back_populates="order")`
- `delivery` = `relationship("Delivery", back_populates="order")`
- `used_promotions` = `relationship("UsedPromotion", back_populates="order")`
- `notifications` = `relationship("Notification", back_populates="order")`

Static Private Attributes

- `str __tablename__ = "orders"`

8.49.1 Detailed Description

ORM model cho bảng orders, đơn hàng đặt món.

Definition at line 122 of file [models.py](#).

8.49.2 Member Data Documentation

8.49.2.1 `__tablename__`

`str models.Orders.__tablename__ = "orders"` [static], [private]

Definition at line 124 of file [models.py](#).

8.49.2.2 `address`

`models.Orders.address = relationship("Address", back_populates="orders")` [static]

Definition at line 137 of file [models.py](#).

8.49.2.3 `addressid`

`models.Orders.addressid = Column(Integer, ForeignKey("address.id"))` [static]

Definition at line 132 of file [models.py](#).

8.49.2.4 createdat

```
models.Orders.createdat = Column(DateTime, server_default=func.now()) [static]
```

Definition at line 127 of file [models.py](#).

8.49.2.5 delivery

```
models.Orders.delivery = relationship("Delivery", back_populates="order") [static]
```

Definition at line 140 of file [models.py](#).

8.49.2.6 id

```
models.Orders.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 125 of file [models.py](#).

8.49.2.7 notifications

```
models.Orders.notifications = relationship("Notification", back_populates="order") [static]
```

Definition at line 142 of file [models.py](#).

8.49.2.8 order_items

```
models.Orders.order_items = relationship("OrderItem", back_populates="order") [static]
```

Definition at line 138 of file [models.py](#).

8.49.2.9 payments

```
models.Orders.payments = relationship("Payment", back_populates="order") [static]
```

Definition at line 139 of file [models.py](#).

8.49.2.10 preorderdate

```
models.Orders.preorderdate = Column(Date) [static]
```

Definition at line 128 of file [models.py](#).

8.49.2.11 preordertime

```
models.Orders.preordertime = Column(Time) [static]
```

Definition at line 129 of file [models.py](#).

8.49.2.12 restaurant

```
models.Orders.restaurant = relationship("Restaurant", back_populates="orders") [static]
```

Definition at line 136 of file [models.py](#).

8.49.2.13 restaurantid

```
models.Orders.restaurantid = Column(Integer, ForeignKey("restaurant.id")) [static]
```

Definition at line 131 of file [models.py](#).

8.49.2.14 status

```
models.Orders.status = Column(String(255), nullable=False) [static]
```

Definition at line 126 of file [models.py](#).

8.49.2.15 totalprice

```
models.Orders.totalprice = Column(Integer) [static]
```

Definition at line 130 of file [models.py](#).

8.49.2.16 used_promotions

```
models.Orders.used_promotions = relationship("UsedPromotion", back_populates="order") [static]
```

Definition at line 141 of file [models.py](#).

8.49.2.17 user

```
models.Orders.user = relationship("User", back_populates="orders") [static]
```

Definition at line 135 of file [models.py](#).

8.49.2.18 userid

```
models.Orders.userid = Column(Integer, ForeignKey("User.id")) [static]
```

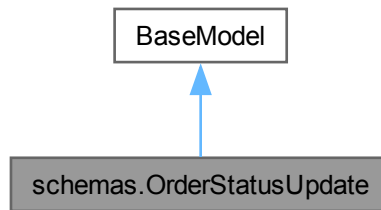
Definition at line 133 of file [models.py](#).

The documentation for this class was generated from the following file:

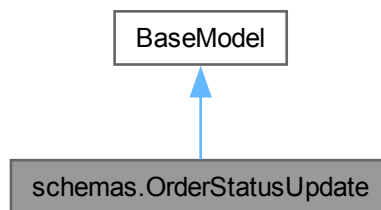
- [backend/models.py](#)

8.50 schemas.OrderStatusUpdate Class Reference

Inheritance diagram for schemas.OrderStatusUpdate:



Collaboration diagram for schemas.OrderStatusUpdate:



8.50.1 Detailed Description

Schema cập nhật trạng thái đơn hàng.

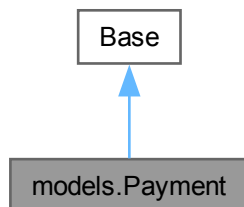
Definition at line 160 of file [schemas.py](#).

The documentation for this class was generated from the following file:

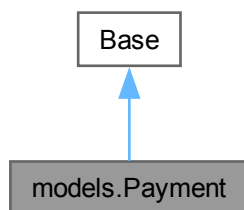
- [backend/schemas.py](#)

8.51 models.Payment Class Reference

Inheritance diagram for models.Payment:



Collaboration diagram for models.Payment:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `status` = Column(String(255), nullable=False)
- `method` = Column(String(255))
- `orderid` = Column(Integer, ForeignKey("orders.id"))
- `order` = relationship("Orders", back_populates="payments")

Static Private Attributes

- `str __tablename__ = "payment"`

8.51.1 Detailed Description

ORM model cho bảng payment, thông tin thanh toán đơn hàng.

Definition at line 156 of file `models.py`.

8.51.2 Member Data Documentation

8.51.2.1 `__tablename__`

```
str models.Payment.__tablename__ = "payment" [static], [private]
```

Definition at line 158 of file [models.py](#).

8.51.2.2 `id`

```
models.Payment.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 159 of file [models.py](#).

8.51.2.3 `method`

```
models.Payment.method = Column(String(255)) [static]
```

Definition at line 161 of file [models.py](#).

8.51.2.4 `order`

```
models.Payment.order = relationship("Orders", back_populates="payments") [static]
```

Definition at line 163 of file [models.py](#).

8.51.2.5 `orderid`

```
models.Payment.orderid = Column(Integer, ForeignKey("orders.id")) [static]
```

Definition at line 162 of file [models.py](#).

8.51.2.6 `status`

```
models.Payment.status = Column(String(255), nullable=False) [static]
```

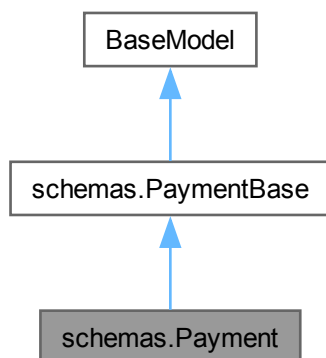
Definition at line 160 of file [models.py](#).

The documentation for this class was generated from the following file:

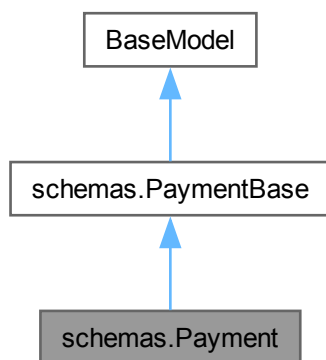
- [backend/models.py](#)

8.52 schemas.Payment Class Reference

Inheritance diagram for schemas.Payment:



Collaboration diagram for schemas.Payment:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

8.52.1 Detailed Description

Definition at line 291 of file [schemas.py](#).

8.52.2 Member Data Documentation

8.52.2.1 model_config

```
int schemas.Payment.model_config = ConfigDict(from_attributes=True) [static]
```

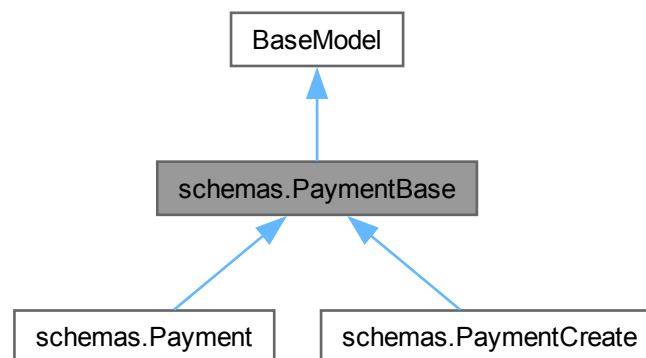
Definition at line 293 of file [schemas.py](#).

The documentation for this class was generated from the following file:

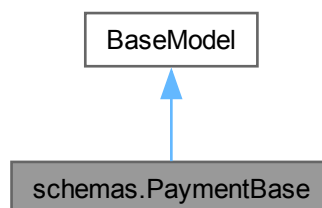
- [backend/schemas.py](#)

8.53 schemas.PaymentBase Class Reference

Inheritance diagram for schemas.PaymentBase:



Collaboration diagram for schemas.PaymentBase:



8.53.1 Detailed Description

Schema cơ sở cho thông tin thanh toán.

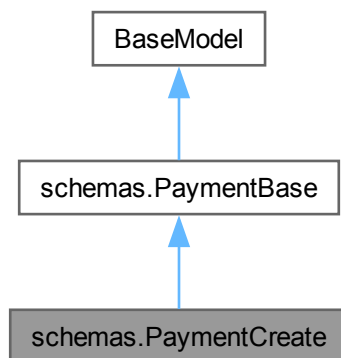
Definition at line 282 of file [schemas.py](#).

The documentation for this class was generated from the following file:

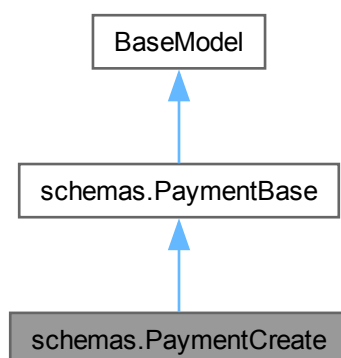
- [backend/schemas.py](#)

8.54 schemas.PaymentCreate Class Reference

Inheritance diagram for schemas.PaymentCreate:



Collaboration diagram for schemas.PaymentCreate:



8.54.1 Detailed Description

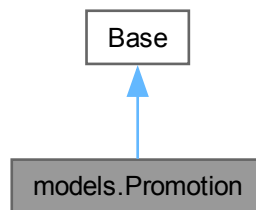
Definition at line 288 of file [schemas.py](#).

The documentation for this class was generated from the following file:

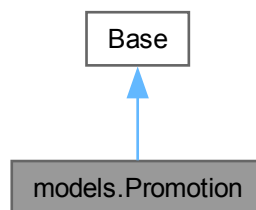
- [backend/schemas.py](#)

8.55 models.Promotion Class Reference

Inheritance diagram for models.Promotion:



Collaboration diagram for models.Promotion:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `code` = Column(String(255))
- `discounttype` = Column(String(255))
- `discountvalue` = Column(Integer)
- `expiredate` = Column(Date)
- `minordervalue` = Column(Integer)
- `status` = Column(String(255), nullable=False)
- `used_promotions` = relationship("UsedPromotion", back_populates="promotion")

Static Private Attributes

- `str \_\_tablename\_\_ = "promotion"`

8.55.1 Detailed Description

ORM model cho bảng promotion, mã khuyến mãi và giảm giá.

Definition at line 62 of file [models.py](#).

8.55.2 Member Data Documentation

8.55.2.1 `__tablename__`

`str models.Promotion.\_\_tablename\_\_ = "promotion" [static], [private]`

Definition at line 64 of file [models.py](#).

8.55.2.2 `code`

`models.Promotion.code = Column(String(255)) [static]`

Definition at line 66 of file [models.py](#).

8.55.2.3 `discounttype`

`models.Promotion.discounttype = Column(String(255)) [static]`

Definition at line 67 of file [models.py](#).

8.55.2.4 `discountvalue`

`models.Promotion.discountvalue = Column(Integer) [static]`

Definition at line 68 of file [models.py](#).

8.55.2.5 `expiredate`

`models.Promotion.expiredate = Column(Date) [static]`

Definition at line 69 of file [models.py](#).

8.55.2.6 `id`

`models.Promotion.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line 65 of file [models.py](#).

8.55.2.7 minordervalue

```
models.Promotion.minordervalue = Column(Integer) [static]
```

Definition at line 70 of file [models.py](#).

8.55.2.8 status

```
models.Promotion.status = Column(String(255), nullable=False) [static]
```

Definition at line 71 of file [models.py](#).

8.55.2.9 used_promotions

```
models.Promotion.used_promotions = relationship("UsedPromotion", back_populates="promotion") [static]
```

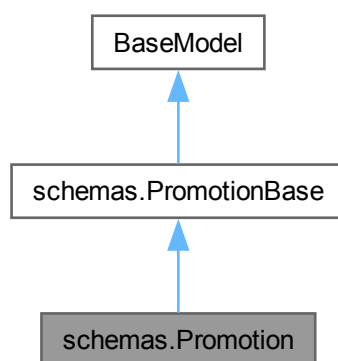
Definition at line 72 of file [models.py](#).

The documentation for this class was generated from the following file:

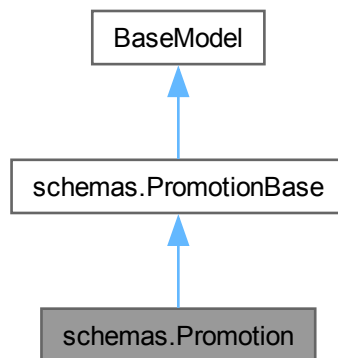
- backend/[models.py](#)

8.56 schemas.Promotion Class Reference

Inheritance diagram for schemas.Promotion:



Collaboration diagram for `schemas.Promotion`:



Static Public Attributes

- `int model_config = ConfigDict(from_attributes=True)`

8.56.1 Detailed Description

Definition at line 276 of file [schemas.py](#).

8.56.2 Member Data Documentation

8.56.2.1 `model_config`

`int schemas.Promotion.model_config = ConfigDict(from_attributes=True)` [static]

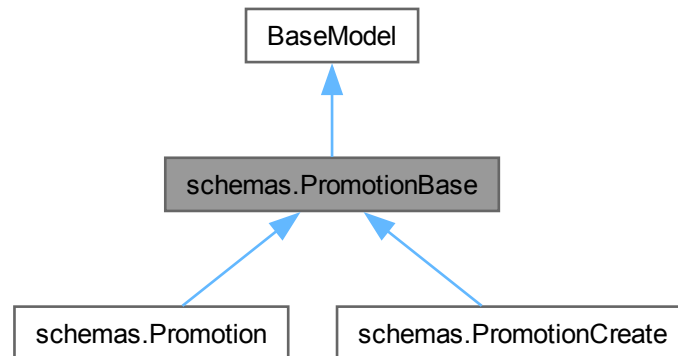
Definition at line 278 of file [schemas.py](#).

The documentation for this class was generated from the following file:

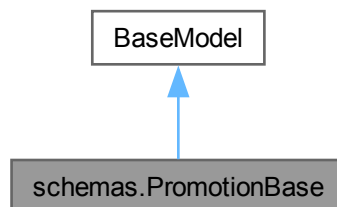
- [backend/schemas.py](#)

8.57 schemas.PromotionBase Class Reference

Inheritance diagram for schemas.PromotionBase:



Collaboration diagram for schemas.PromotionBase:



8.57.1 Detailed Description

Schema cơ sở cho mã khuyến mãi.

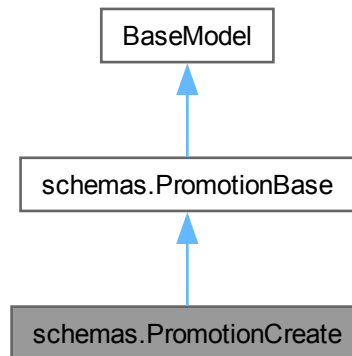
Definition at line 264 of file [schemas.py](#).

The documentation for this class was generated from the following file:

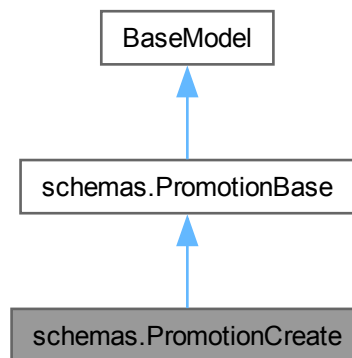
- [backend/schemas.py](#)

8.58 schemas.PromotionCreate Class Reference

Inheritance diagram for schemas.PromotionCreate:



Collaboration diagram for schemas.PromotionCreate:



8.58.1 Detailed Description

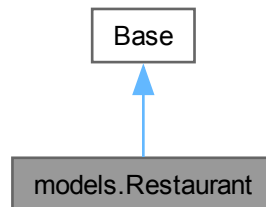
Definition at line [273](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

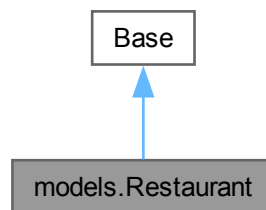
- [backend/schemas.py](#)

8.59 models.Restaurant Class Reference

Inheritance diagram for models.Restaurant:



Collaboration diagram for models.Restaurant:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `name` = Column(String(255), nullable=False)
- `image_url` = Column(Text)
- `address` = Column(String(255))
- `rating` = Column(Integer)
- `open_time` = Column(Time)
- `close_time` = Column(Time)
- `phone_number` = Column(String(255), nullable=False)
- `status` = Column(String(255), nullable=False)
- `description` = Column(Text)
- `menu_items` = relationship("MenuItem", back_populates="restaurant")
- `orders` = relationship("Orders", back_populates="restaurant")
- `reviews` = relationship("Review", back_populates="restaurant")
- `social_shares` = relationship("SocialShare", back_populates="restaurant")

Static Private Attributes

- `str \_\_tablename\_\_ = "restaurant"`

8.59.1 Detailed Description

ORM model cho bảng restaurant, thông tin nhà hàng.

Definition at line [74](#) of file [models.py](#).

8.59.2 Member Data Documentation

8.59.2.1 `__tablename__`

`str models.Restaurant.\_\_tablename\_\_ = "restaurant" [static], [private]`

Definition at line [76](#) of file [models.py](#).

8.59.2.2 `address`

`models.Restaurant.address = Column(String(255)) [static]`

Definition at line [80](#) of file [models.py](#).

8.59.2.3 `close_time`

`models.Restaurant.close_time = Column(Time) [static]`

Definition at line [83](#) of file [models.py](#).

8.59.2.4 `description`

`models.Restaurant.description = Column(Text) [static]`

Definition at line [86](#) of file [models.py](#).

8.59.2.5 `id`

`models.Restaurant.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line [77](#) of file [models.py](#).

8.59.2.6 `image_url`

`models.Restaurant.image_url = Column(Text) [static]`

Definition at line [79](#) of file [models.py](#).

8.59.2.7 menu_items

```
models.Restaurant.menu_items = relationship("MenuItem", back_populates="restaurant") [static]
```

Definition at line 88 of file [models.py](#).

8.59.2.8 name

```
models.Restaurant.name = Column(String(255), nullable=False) [static]
```

Definition at line 78 of file [models.py](#).

8.59.2.9 open_time

```
models.Restaurant.open_time = Column(Time) [static]
```

Definition at line 82 of file [models.py](#).

8.59.2.10 orders

```
models.Restaurant.orders = relationship("Orders", back_populates="restaurant") [static]
```

Definition at line 89 of file [models.py](#).

8.59.2.11 phone_number

```
models.Restaurant.phone_number = Column(String(255), nullable=False) [static]
```

Definition at line 84 of file [models.py](#).

8.59.2.12 rating

```
models.Restaurant.rating = Column(Integer) [static]
```

Definition at line 81 of file [models.py](#).

8.59.2.13 reviews

```
models.Restaurant.reviews = relationship("Review", back_populates="restaurant") [static]
```

Definition at line 90 of file [models.py](#).

8.59.2.14 social_shares

```
models.Restaurant.social_shares = relationship("SocialShare", back_populates="restaurant") [static]
```

Definition at line 91 of file [models.py](#).

8.59.2.15 status

```
models.Restaurant.status = Column(String(255), nullable=False) [static]
```

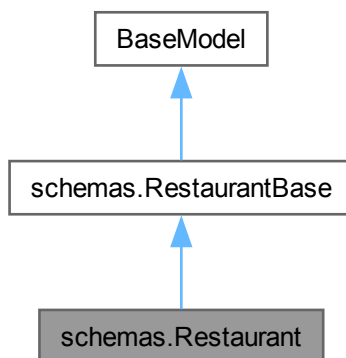
Definition at line 85 of file [models.py](#).

The documentation for this class was generated from the following file:

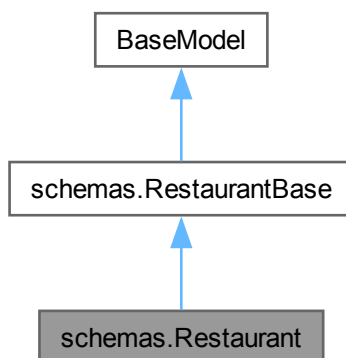
- [backend/models.py](#)

8.60 schemas.Restaurant Class Reference

Inheritance diagram for schemas.Restaurant:



Collaboration diagram for schemas.Restaurant:



Static Public Attributes

- list [menu_items](#) = []
- [model_config](#) = ConfigDict(from_attributes=True)

Static Public Attributes inherited from [schemas.RestaurantBase](#)

- Optional [image_url](#) = None
- Optional [address](#) = None
- Optional [rating](#) = None
- Optional [open_time](#) = None
- Optional [close_time](#) = None
- Optional [description](#) = None

8.60.1 Detailed Description

Definition at line 65 of file [schemas.py](#).

8.60.2 Member Data Documentation

8.60.2.1 menu_items

```
list schemas.Restaurant.menu_items = [] [static]
```

Definition at line 67 of file [schemas.py](#).

8.60.2.2 model_config

```
schemas.Restaurant.model_config = ConfigDict(from_attributes=True) [static]
```

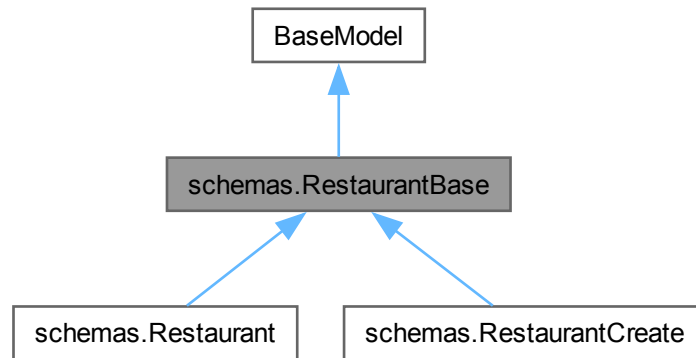
Definition at line 68 of file [schemas.py](#).

The documentation for this class was generated from the following file:

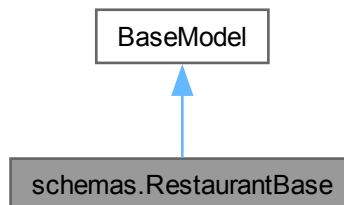
- backend/[schemas.py](#)

8.61 schemas.RestaurantBase Class Reference

Inheritance diagram for schemas.RestaurantBase:



Collaboration diagram for schemas.RestaurantBase:



Static Public Attributes

- Optional `image_url` = None
- Optional `address` = None
- Optional `rating` = None
- Optional `open_time` = None
- Optional `close_time` = None
- Optional `description` = None

8.61.1 Detailed Description

Schema cơ sở cho nhà hàng.

Definition at line 50 of file `schemas.py`.

8.61.2 Member Data Documentation

8.61.2.1 address

Optional schemas.RestaurantBase.address = None [static]

Definition at line 54 of file [schemas.py](#).

8.61.2.2 close_time

Optional schemas.RestaurantBase.close_time = None [static]

Definition at line 57 of file [schemas.py](#).

8.61.2.3 description

Optional schemas.RestaurantBase.description = None [static]

Definition at line 60 of file [schemas.py](#).

8.61.2.4 image_url

Optional schemas.RestaurantBase.image_url = None [static]

Definition at line 53 of file [schemas.py](#).

8.61.2.5 open_time

Optional schemas.RestaurantBase.open_time = None [static]

Definition at line 56 of file [schemas.py](#).

8.61.2.6 rating

Optional schemas.RestaurantBase.rating = None [static]

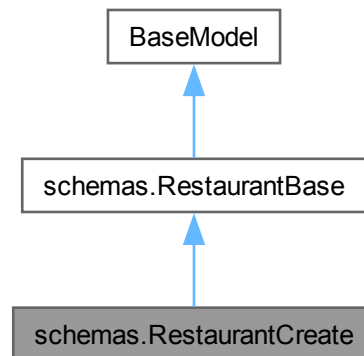
Definition at line 55 of file [schemas.py](#).

The documentation for this class was generated from the following file:

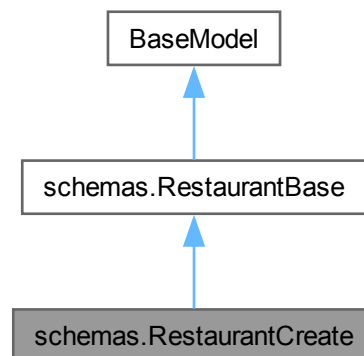
- [backend/schemas.py](#)

8.62 schemas.RestaurantCreate Class Reference

Inheritance diagram for schemas.RestaurantCreate:



Collaboration diagram for schemas.RestaurantCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.RestaurantBase](#)

- Optional [image_url](#) = None
- Optional [address](#) = None
- Optional [rating](#) = None
- Optional [open_time](#) = None
- Optional [close_time](#) = None
- Optional [description](#) = None

8.62.1 Detailed Description

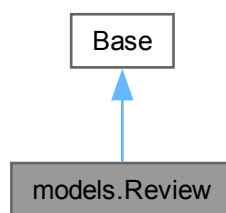
Definition at line 62 of file [schemas.py](#).

The documentation for this class was generated from the following file:

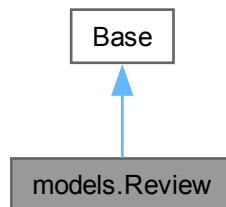
- [backend/schemas.py](#)

8.63 models.Review Class Reference

Inheritance diagram for models.Review:



Collaboration diagram for models.Review:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `rating` = `Column(Integer)`
- `comment` = `Column(Text)`
- `menuitemid` = `Column(Integer, ForeignKey("menuitem.id"))`
- `restaurantid` = `Column(Integer, ForeignKey("restaurant.id"))`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `orderid` = `Column(Integer, ForeignKey("orders.id"))`
- `user` = `relationship("User", back_populates="reviews")`
- `menu_item` = `relationship("MenuItem", back_populates="reviews")`
- `restaurant` = `relationship("Restaurant", back_populates="reviews")`
- `order` = `relationship("Orders")`

Static Private Attributes

- `str \_\_tablename\_\_ = "review"`

8.63.1 Detailed Description

ORM model cho bảng review, đánh giá của người dùng về món ăn và nhà hàng.

Definition at line [177](#) of file [models.py](#).

8.63.2 Member Data Documentation

8.63.2.1 `__tablename__`

`str models.Review.\_\_tablename\_\_ = "review" [static], [private]`

Definition at line [179](#) of file [models.py](#).

8.63.2.2 `comment`

`models.Review.comment = Column(Text) [static]`

Definition at line [182](#) of file [models.py](#).

8.63.2.3 `id`

`models.Review.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line [180](#) of file [models.py](#).

8.63.2.4 `menu_item`

`models.Review.menu_item = relationship("MenuItem", back_populates="reviews") [static]`

Definition at line [189](#) of file [models.py](#).

8.63.2.5 `menuitemid`

`models.Review.menuitemid = Column(Integer, ForeignKey("menuitem.id")) [static]`

Definition at line [183](#) of file [models.py](#).

8.63.2.6 `order`

`models.Review.order = relationship("Orders") [static]`

Definition at line [191](#) of file [models.py](#).

8.63.2.7 orderid

```
models.Review.orderid = Column(Integer, ForeignKey("orders.id")) [static]
```

Definition at line 186 of file [models.py](#).

8.63.2.8 rating

```
models.Review.rating = Column(Integer) [static]
```

Definition at line 181 of file [models.py](#).

8.63.2.9 restaurant

```
models.Review.restaurant = relationship("Restaurant", back_populates="reviews") [static]
```

Definition at line 190 of file [models.py](#).

8.63.2.10 restaurantid

```
models.Review.restaurantid = Column(Integer, ForeignKey("restaurant.id")) [static]
```

Definition at line 184 of file [models.py](#).

8.63.2.11 user

```
models.Review.user = relationship("User", back_populates="reviews") [static]
```

Definition at line 188 of file [models.py](#).

8.63.2.12 userid

```
models.Review.userid = Column(Integer, ForeignKey("User.id")) [static]
```

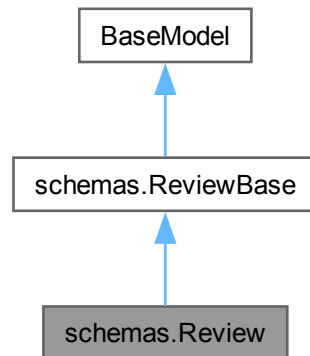
Definition at line 185 of file [models.py](#).

The documentation for this class was generated from the following file:

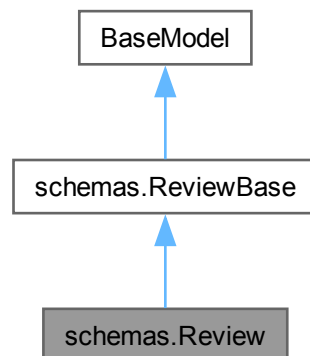
- [backend/models.py](#)

8.64 schemas.Review Class Reference

Inheritance diagram for schemas.Review:



Collaboration diagram for schemas.Review:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

Static Public Attributes inherited from [schemas.ReviewBase](#)

- Optional `comment` = None
- Optional `menuitemid` = None
- Optional `restaurantid` = None

8.64.1 Detailed Description

Definition at line 325 of file [schemas.py](#).

8.64.2 Member Data Documentation

8.64.2.1 model_config

```
int schemas.Review.model_config = ConfigDict(from_attributes=True) [static]
```

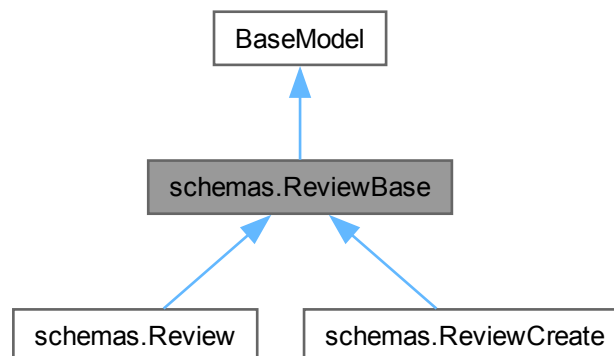
Definition at line 327 of file [schemas.py](#).

The documentation for this class was generated from the following file:

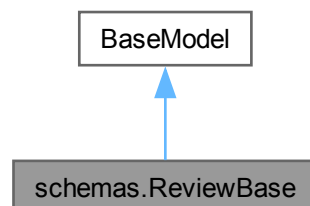
- [backend/schemas.py](#)

8.65 schemas.ReviewBase Class Reference

Inheritance diagram for schemas.ReviewBase:



Collaboration diagram for schemas.ReviewBase:



Static Public Attributes

- Optional `comment` = None
- Optional `menuitemid` = None
- Optional `restaurantid` = None

8.65.1 Detailed Description

Schema cơ sở cho đánh giá của người dùng.

Definition at line [313](#) of file [schemas.py](#).

8.65.2 Member Data Documentation

8.65.2.1 `comment`

Optional `schemas.ReviewBase.comment` = None [static]

Definition at line [316](#) of file [schemas.py](#).

8.65.2.2 `menuitemid`

Optional `schemas.ReviewBase.menuitemid` = None [static]

Definition at line [319](#) of file [schemas.py](#).

8.65.2.3 `restaurantid`

Optional `schemas.ReviewBase.restaurantid` = None [static]

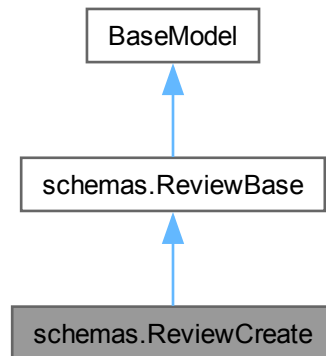
Definition at line [320](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

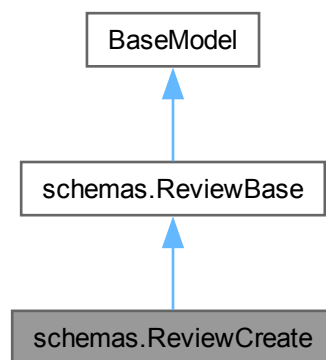
- [backend/schemas.py](#)

8.66 schemas.ReviewCreate Class Reference

Inheritance diagram for schemas.ReviewCreate:



Collaboration diagram for schemas.ReviewCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.ReviewBase](#)

- Optional [comment](#) = None
- Optional [menuitemid](#) = None
- Optional [restaurantid](#) = None

8.66.1 Detailed Description

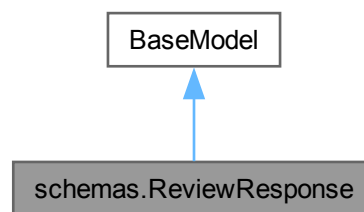
Definition at line 322 of file [schemas.py](#).

The documentation for this class was generated from the following file:

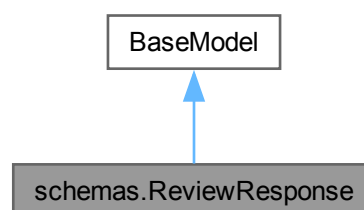
- [backend/schemas.py](#)

8.67 schemas.ReviewResponse Class Reference

Inheritance diagram for schemas.ReviewResponse:



Collaboration diagram for schemas.ReviewResponse:



Static Public Attributes

- Optional [comment](#) = None
- Optional [user_name](#) = None
- [model_config](#) = ConfigDict(from_attributes=True)

8.67.1 Detailed Description

Definition at line 329 of file [schemas.py](#).

8.67.2 Member Data Documentation

8.67.2.1 comment

Optional `schemas.ReviewResponse.comment = None` [static]

Definition at line 332 of file [schemas.py](#).

8.67.2.2 model_config

`schemas.ReviewResponse.model_config = ConfigDict(from_attributes=True)` [static]

Definition at line 335 of file [schemas.py](#).

8.67.2.3 user_name

Optional `schemas.ReviewResponse.user_name = None` [static]

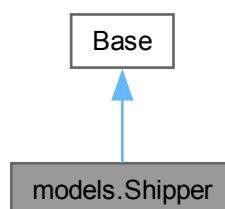
Definition at line 334 of file [schemas.py](#).

The documentation for this class was generated from the following file:

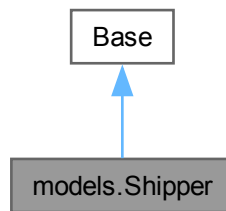
- [backend/schemas.py](#)

8.68 models.Shipper Class Reference

Inheritance diagram for `models.Shipper`:



Collaboration diagram for models.Shipper:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `name` = `Column(String(255), nullable=False)`
- `phone` = `Column(String(255), nullable=False)`
- `rating` = `Column(Integer)`
- `description` = `Column(Text)`
- `status` = `Column(String(255), nullable=False)`
- `deliveries` = `relationship("Delivery", back_populates="shipper")`

Static Private Attributes

- `str __tablename__ = "shipper"`

8.68.1 Detailed Description

ORM model cho bảng shipper, quản lý thông tin người giao hàng.

Definition at line 51 of file [models.py](#).

8.68.2 Member Data Documentation

8.68.2.1 `__tablename__`

```
str models.Shipper.__tablename__ = "shipper" [static], [private]
```

Definition at line 53 of file [models.py](#).

8.68.2.2 `deliveries`

```
models.Shipper.deliveries = relationship("Delivery", back_populates="shipper") [static]
```

Definition at line 60 of file [models.py](#).

8.68.2.3 description

```
models.Shipper.description = Column(Text) [static]
```

Definition at line 58 of file [models.py](#).

8.68.2.4 id

```
models.Shipper.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 54 of file [models.py](#).

8.68.2.5 name

```
models.Shipper.name = Column(String(255), nullable=False) [static]
```

Definition at line 55 of file [models.py](#).

8.68.2.6 phone

```
models.Shipper.phone = Column(String(255), nullable=False) [static]
```

Definition at line 56 of file [models.py](#).

8.68.2.7 rating

```
models.Shipper.rating = Column(Integer) [static]
```

Definition at line 57 of file [models.py](#).

8.68.2.8 status

```
models.Shipper.status = Column(String(255), nullable=False) [static]
```

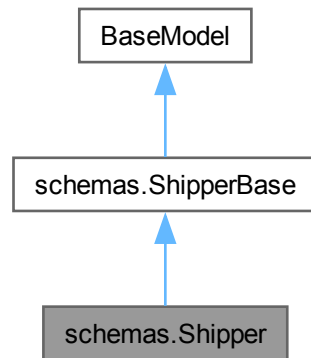
Definition at line 59 of file [models.py](#).

The documentation for this class was generated from the following file:

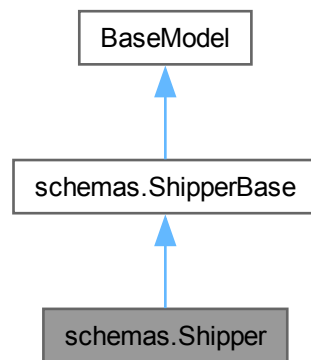
- backend/[models.py](#)

8.69 schemas.Shipper Class Reference

Inheritance diagram for schemas.Shipper:



Collaboration diagram for schemas.Shipper:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

Static Public Attributes inherited from [schemas.ShipperBase](#)

- Optional `rating` = None
- Optional `description` = None

8.69.1 Detailed Description

Definition at line 258 of file [schemas.py](#).

8.69.2 Member Data Documentation

8.69.2.1 model_config

```
int schemas.Shipper.model_config = ConfigDict(from_attributes=True) [static]
```

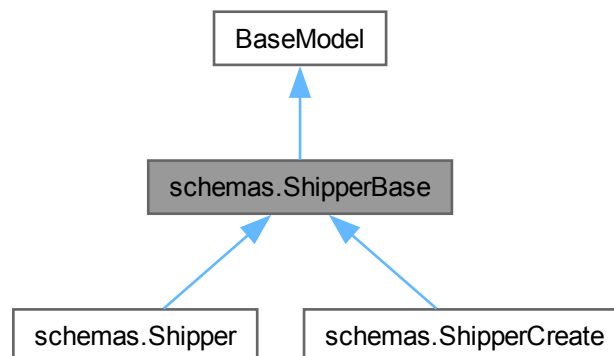
Definition at line 260 of file [schemas.py](#).

The documentation for this class was generated from the following file:

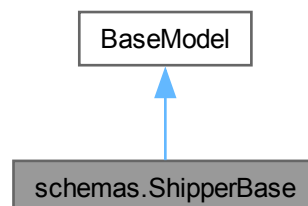
- [backend/schemas.py](#)

8.70 schemas.ShipperBase Class Reference

Inheritance diagram for schemas.ShipperBase:



Collaboration diagram for schemas.ShipperBase:



Static Public Attributes

- Optional `rating` = None
- Optional `description` = None

8.70.1 Detailed Description

Schema cơ sở cho người giao hàng.

Definition at line 247 of file [schemas.py](#).

8.70.2 Member Data Documentation

8.70.2.1 `description`

Optional `schemas.ShipperBase.description` = None [static]

Definition at line 252 of file [schemas.py](#).

8.70.2.2 `rating`

Optional `schemas.ShipperBase.rating` = None [static]

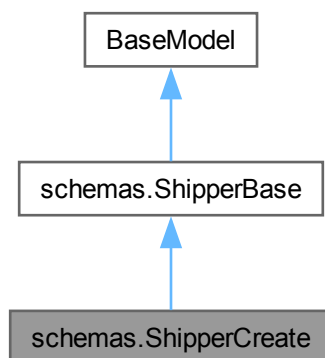
Definition at line 251 of file [schemas.py](#).

The documentation for this class was generated from the following file:

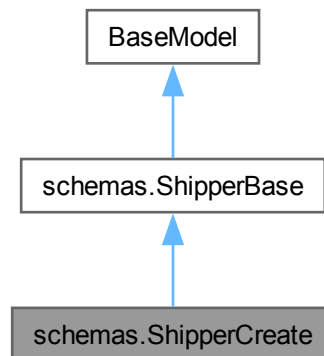
- [backend/schemas.py](#)

8.71 `schemas.ShipperCreate` Class Reference

Inheritance diagram for `schemas.ShipperCreate`:



Collaboration diagram for schemas.ShipperCreate:



Additional Inherited Members

Static Public Attributes inherited from [schemas.ShipperBase](#)

- Optional [rating](#) = None
- Optional [description](#) = None

8.71.1 Detailed Description

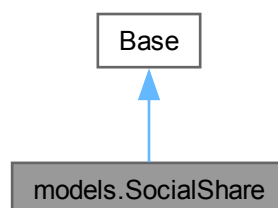
Definition at line [255](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

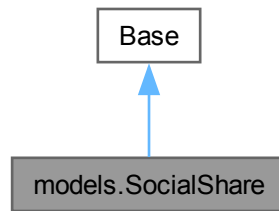
- [backend/schemas.py](#)

8.72 models.SocialShare Class Reference

Inheritance diagram for models.SocialShare:



Collaboration diagram for models.SocialShare:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `sharetype` = `Column(String(255))`
- `platform` = `Column(String(255))`
- `context` = `Column(Text)`
- `createdat` = `Column(Date)`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `restaurantid` = `Column(Integer, ForeignKey("restaurant.id"))`
- `menuitemid` = `Column(Integer, ForeignKey("menuitem.id"))`
- `user` = `relationship("User", back_populates="social_shares")`
- `restaurant` = `relationship("Restaurant", back_populates="social_shares")`
- `menu_item` = `relationship("MenuItem", back_populates="social_shares")`

Static Private Attributes

- `str __tablename__ = "socialshare"`

8.72.1 Detailed Description

ORM model cho bảng socialshare, lượt chia sẻ mạng xã hội.

Definition at line 278 of file `models.py`.

8.72.2 Member Data Documentation

8.72.2.1 `__tablename__`

`str models.SocialShare.__tablename__ = "socialshare"` [static], [private]

Definition at line 280 of file `models.py`.

8.72.2.2 context

```
models.SocialShare.context = Column(Text) [static]
```

Definition at line 284 of file [models.py](#).

8.72.2.3 createdat

```
models.SocialShare.createdat = Column(Date) [static]
```

Definition at line 285 of file [models.py](#).

8.72.2.4 id

```
models.SocialShare.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 281 of file [models.py](#).

8.72.2.5 menu_item

```
models.SocialShare.menu_item = relationship("MenuItem", back_populates="social_shares") [static]
```

Definition at line 292 of file [models.py](#).

8.72.2.6 menuitemid

```
models.SocialShare.menuitemid = Column(Integer, ForeignKey("menuitem.id")) [static]
```

Definition at line 288 of file [models.py](#).

8.72.2.7 platform

```
models.SocialShare.platform = Column(String(255)) [static]
```

Definition at line 283 of file [models.py](#).

8.72.2.8 restaurant

```
models.SocialShare.restaurant = relationship("Restaurant", back_populates="social_shares") [static]
```

Definition at line 291 of file [models.py](#).

8.72.2.9 restaurantid

```
models.SocialShare.restaurantid = Column(Integer, ForeignKey("restaurant.id")) [static]
```

Definition at line 287 of file [models.py](#).

8.72.2.10 sharetype

```
models.SocialShare.sharetype = Column(String(255)) [static]
```

Definition at line 282 of file [models.py](#).

8.72.2.11 user

```
models.SocialShare.user = relationship("User", back_populates="social_shares") [static]
```

Definition at line 290 of file [models.py](#).

8.72.2.12 userid

```
models.SocialShare.userid = Column(Integer, ForeignKey("User.id")) [static]
```

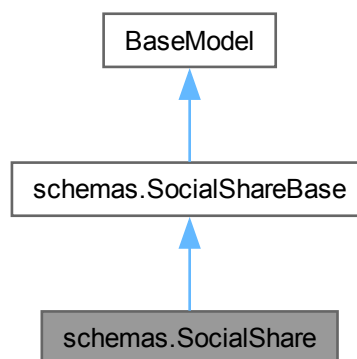
Definition at line 286 of file [models.py](#).

The documentation for this class was generated from the following file:

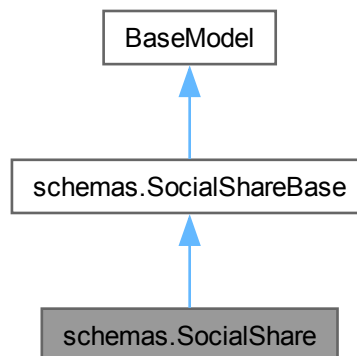
- [backend/models.py](#)

8.73 schemas.SocialShare Class Reference

Inheritance diagram for schemas.SocialShare:



Collaboration diagram for schemas.SocialShare:



Static Public Attributes

- date `model_config` = `ConfigDict(from__attributes=True)`

Static Public Attributes inherited from [schemas.SocialShareBase](#)

- Optional `context` = `None`

8.73.1 Detailed Description

Definition at line 410 of file [schemas.py](#).

8.73.2 Member Data Documentation

8.73.2.1 model_config

date `schemas.SocialShare.model_config` = `ConfigDict(from__attributes=True)` [static]

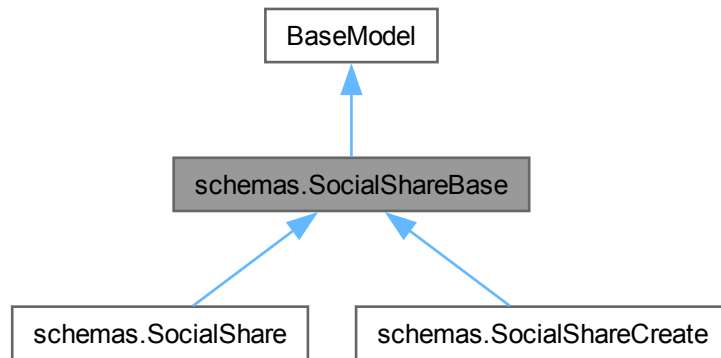
Definition at line 413 of file [schemas.py](#).

The documentation for this class was generated from the following file:

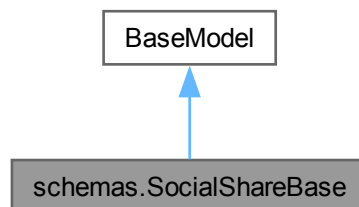
- [backend/schemas.py](#)

8.74 schemas.SocialShareBase Class Reference

Inheritance diagram for schemas.SocialShareBase:



Collaboration diagram for schemas.SocialShareBase:



Static Public Attributes

- Optional `context` = None

8.74.1 Detailed Description

Schema cơ sở cho lượt chia sẻ mạng xã hội.

Definition at line 398 of file [schemas.py](#).

8.74.2 Member Data Documentation

8.74.2.1 context

Optional `schemas.SocialShareBase.context = None` [static]

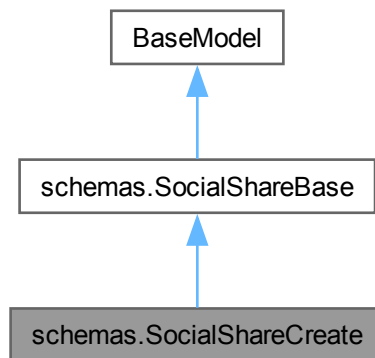
Definition at line 402 of file [schemas.py](#).

The documentation for this class was generated from the following file:

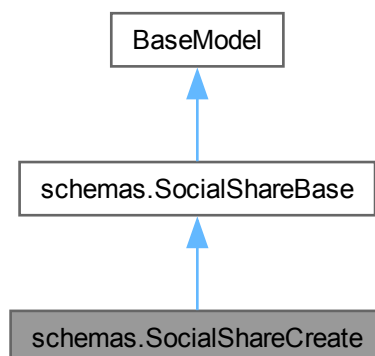
- [backend/schemas.py](#)

8.75 schemas.SocialShareCreate Class Reference

Inheritance diagram for `schemas.SocialShareCreate`:



Collaboration diagram for `schemas.SocialShareCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.SocialShareBase](#)

- Optional [context](#) = None

8.75.1 Detailed Description

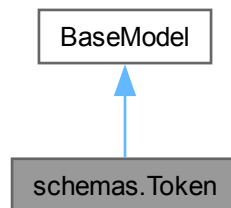
Definition at line [407](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

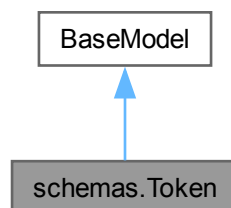
- [backend/schemas.py](#)

8.76 schemas.Token Class Reference

Inheritance diagram for schemas.Token:



Collaboration diagram for schemas.Token:



8.76.1 Detailed Description

Schema phản hồi JWT token khi đăng nhập thành công.

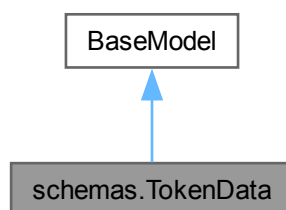
Definition at line 85 of file [schemas.py](#).

The documentation for this class was generated from the following file:

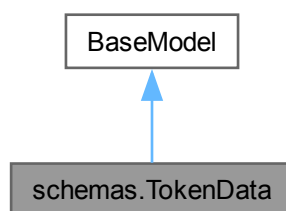
- [backend/schemas.py](#)

8.77 schemas.TokenData Class Reference

Inheritance diagram for schemas.TokenData:



Collaboration diagram for schemas.TokenData:



Static Public Attributes

- Optional [email](#) = None

8.77.1 Detailed Description

Schema dữ liệu giải mã từ JWT token.

Definition at line 90 of file [schemas.py](#).

8.77.2 Member Data Documentation

8.77.2.1 email

Optional `schemas.TokenData.email = None` [static]

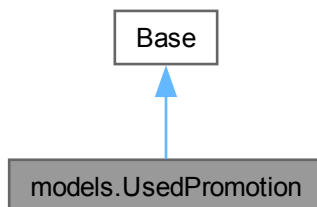
Definition at line 92 of file [schemas.py](#).

The documentation for this class was generated from the following file:

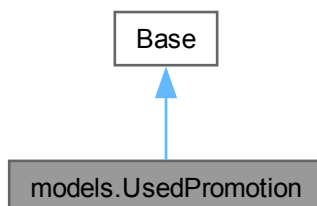
- [backend/schemas.py](#)

8.78 models.UsedPromotion Class Reference

Inheritance diagram for `models.UsedPromotion`:



Collaboration diagram for `models.UsedPromotion`:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `usedat` = `Column(Date)`
- `promotionid` = `Column(Integer, ForeignKey("promotion.id"))`
- `orderid` = `Column(Integer, ForeignKey("orders.id"))`
- `promotion` = `relationship("Promotion", back_populates="used_promotions")`
- `order` = `relationship("Orders", back_populates="used_promotions")`

Static Private Attributes

- `str __tablename__ = "usedpromotion"`

8.78.1 Detailed Description

ORM model cho bảng usedpromotion, mã khuyến mãi đã sử dụng.

Definition at line 255 of file [models.py](#).

8.78.2 Member Data Documentation

8.78.2.1 `__tablename__`

`str models.UsedPromotion.__tablename__ = "usedpromotion" [static], [private]`

Definition at line 257 of file [models.py](#).

8.78.2.2 `id`

`models.UsedPromotion.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line 258 of file [models.py](#).

8.78.2.3 `order`

`models.UsedPromotion.order = relationship("Orders", back_populates="used_promotions") [static]`

Definition at line 264 of file [models.py](#).

8.78.2.4 `orderid`

`models.UsedPromotion.orderid = Column(Integer, ForeignKey("orders.id")) [static]`

Definition at line 261 of file [models.py](#).

8.78.2.5 promotion

```
models.UsedPromotion.promotion = relationship("Promotion", back_populates="used_promotions") [static]
```

Definition at line 263 of file [models.py](#).

8.78.2.6 promotionid

```
models.UsedPromotion.promotionid = Column(Integer, ForeignKey("promotion.id")) [static]
```

Definition at line 260 of file [models.py](#).

8.78.2.7 usedat

```
models.UsedPromotion.usedat = Column(Date) [static]
```

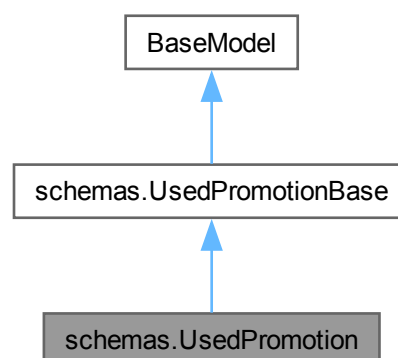
Definition at line 259 of file [models.py](#).

The documentation for this class was generated from the following file:

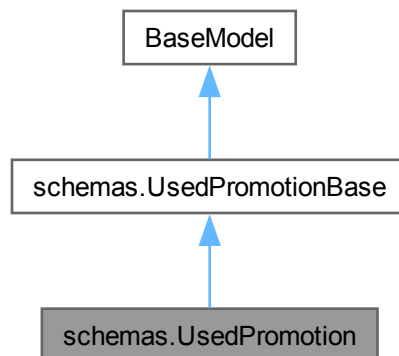
- [backend/models.py](#)

8.79 schemas.UsedPromotion Class Reference

Inheritance diagram for schemas.UsedPromotion:



Collaboration diagram for schemas.UsedPromotion:



Static Public Attributes

- int `model_config` = `ConfigDict(from_attributes=True)`

8.79.1 Detailed Description

Definition at line 376 of file [schemas.py](#).

8.79.2 Member Data Documentation

8.79.2.1 `model_config`

```
int schemas.UsedPromotion.model_config = ConfigDict(from_attributes=True) [static]
```

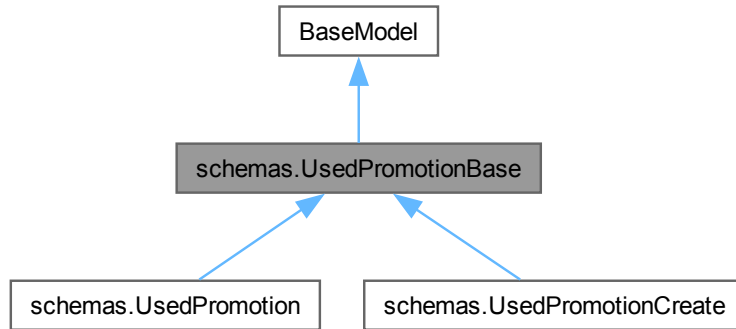
Definition at line 378 of file [schemas.py](#).

The documentation for this class was generated from the following file:

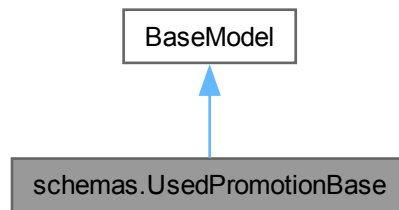
- [backend/schemas.py](#)

8.80 schemas.UsedPromotionBase Class Reference

Inheritance diagram for schemas.UsedPromotionBase:



Collaboration diagram for schemas.UsedPromotionBase:



8.80.1 Detailed Description

Schema cơ sở cho mã khuyến mãi đã sử dụng.

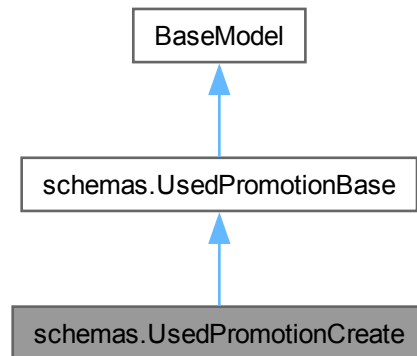
Definition at line 367 of file [schemas.py](#).

The documentation for this class was generated from the following file:

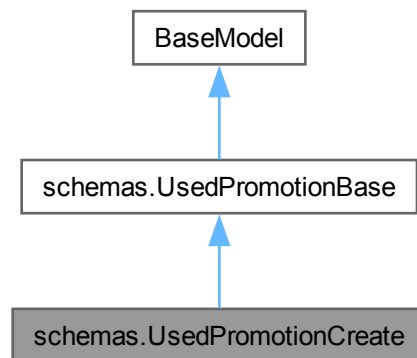
- [backend/schemas.py](#)

8.81 schemas.UsedPromotionCreate Class Reference

Inheritance diagram for schemas.UsedPromotionCreate:



Collaboration diagram for schemas.UsedPromotionCreate:



8.81.1 Detailed Description

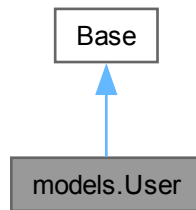
Definition at line [373](#) of file [schemas.py](#).

The documentation for this class was generated from the following file:

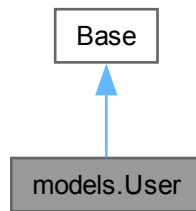
- [backend/schemas.py](#)

8.82 models.User Class Reference

Inheritance diagram for models.User:



Collaboration diagram for models.User:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `name` = `Column(String(255), nullable=False)`
- `username` = `Column(String(255), nullable=False)`
- `email` = `Column(String(255), nullable=False, unique=True)`
- `password` = `Column(String(255), nullable=False)`
- `phone` = `Column(String(255), nullable=False)`
- `role` = `Column(String(255), nullable=False)`
- `addresses` = `relationship("Address", back_populates="user")`
- `orders` = `relationship("Orders", back_populates="user")`
- `reviews` = `relationship("Review", back_populates="user")`
- `user_devices` = `relationship("UserDevice", back_populates="user")`
- `user_faqs` = `relationship("UserFAQ", back_populates="user")`
- `chat_sessions` = `relationship("ChatSession", back_populates="user")`
- `notifications` = `relationship("Notification", back_populates="user")`
- `loyalty_points` = `relationship("LoyaltyPoint", back_populates="user")`
- `social_shares` = `relationship("SocialShare", back_populates="user")`

Static Private Attributes

- `str \_\_tablename\_\_ = "User"`

8.82.1 Detailed Description

ORM model cho bảng User, lưu trữ thông tin tài khoản người dùng.

Definition at line [30](#) of file [models.py](#).

8.82.2 Member Data Documentation

8.82.2.1 `__tablename__`

```
str models.User.__tablename__ = "User" [static], [private]
```

Definition at line [32](#) of file [models.py](#).

8.82.2.2 `addresses`

```
models.User.addresses = relationship("Address", back_populates="user") [static]
```

Definition at line [41](#) of file [models.py](#).

8.82.2.3 `chat_sessions`

```
models.User.chat_sessions = relationship("ChatSession", back_populates="user") [static]
```

Definition at line [46](#) of file [models.py](#).

8.82.2.4 `email`

```
models.User.email = Column(String(255), nullable=False, unique=True) [static]
```

Definition at line [36](#) of file [models.py](#).

8.82.2.5 `id`

```
models.User.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line [33](#) of file [models.py](#).

8.82.2.6 `loyalty_points`

```
models.User.loyalty_points = relationship("LoyaltyPoint", back_populates="user") [static]
```

Definition at line [48](#) of file [models.py](#).

8.82.2.7 name

```
models.User.name = Column(String(255), nullable=False) [static]
```

Definition at line 34 of file [models.py](#).

8.82.2.8 notifications

```
models.User.notifications = relationship("Notification", back_populates="user") [static]
```

Definition at line 47 of file [models.py](#).

8.82.2.9 orders

```
models.User.orders = relationship("Orders", back_populates="user") [static]
```

Definition at line 42 of file [models.py](#).

8.82.2.10 password

```
models.User.password = Column(String(255), nullable=False) [static]
```

Definition at line 37 of file [models.py](#).

8.82.2.11 phone

```
models.User.phone = Column(String(255), nullable=False) [static]
```

Definition at line 38 of file [models.py](#).

8.82.2.12 reviews

```
models.User.reviews = relationship("Review", back_populates="user") [static]
```

Definition at line 43 of file [models.py](#).

8.82.2.13 role

```
models.User.role = Column(String(255), nullable=False) [static]
```

Definition at line 39 of file [models.py](#).

8.82.2.14 social_shares

```
models.User.social_shares = relationship("SocialShare", back_populates="user") [static]
```

Definition at line 49 of file [models.py](#).

8.82.2.15 user__devices

```
models.User.user__devices = relationship("UserDevice", back_populates="user") [static]
```

Definition at line 44 of file [models.py](#).

8.82.2.16 user__faqs

```
models.User.user__faqs = relationship("UserFAQ", back_populates="user") [static]
```

Definition at line 45 of file [models.py](#).

8.82.2.17 username

```
models.User.username = Column(String(255), nullable=False) [static]
```

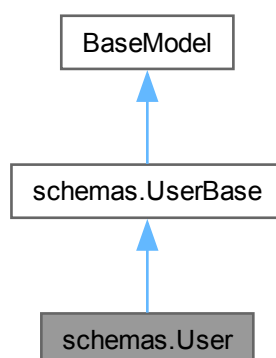
Definition at line 35 of file [models.py](#).

The documentation for this class was generated from the following file:

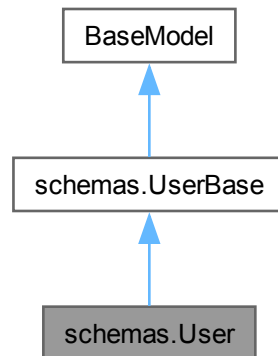
- backend/[models.py](#)

8.83 schemas.User Class Reference

Inheritance diagram for schemas.User:



Collaboration diagram for `schemas.User`:



Static Public Attributes

- `int model_config = ConfigDict(from_attributes=True)`

8.83.1 Detailed Description

Definition at line 81 of file [schemas.py](#).

8.83.2 Member Data Documentation

8.83.2.1 `model_config`

`int schemas.User.model_config = ConfigDict(from_attributes=True)` [static]

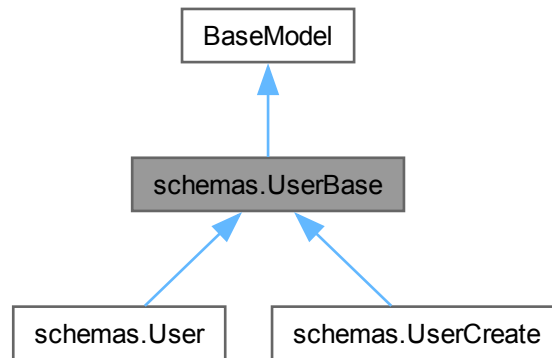
Definition at line 83 of file [schemas.py](#).

The documentation for this class was generated from the following file:

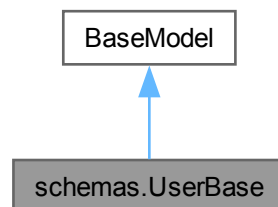
- [backend/schemas.py](#)

8.84 schemas.UserBase Class Reference

Inheritance diagram for schemas.UserBase:



Collaboration diagram for schemas.UserBase:



8.84.1 Detailed Description

Schema cơ sở cho người dùng.

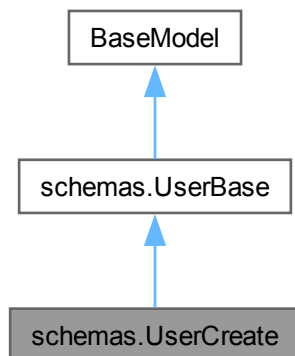
Definition at line 70 of file [schemas.py](#).

The documentation for this class was generated from the following file:

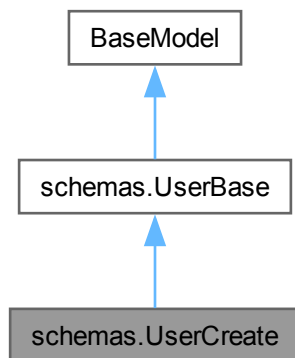
- [backend/schemas.py](#)

8.85 schemas.UserCreate Class Reference

Inheritance diagram for schemas.UserCreate:



Collaboration diagram for schemas.UserCreate:



8.85.1 Detailed Description

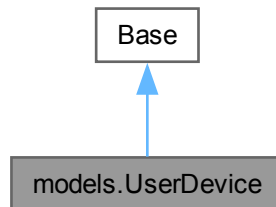
Definition at line 78 of file [schemas.py](#).

The documentation for this class was generated from the following file:

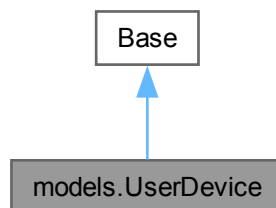
- [backend/schemas.py](#)

8.86 models.UserDevice Class Reference

Inheritance diagram for models.UserDevice:



Collaboration diagram for models.UserDevice:



Static Public Attributes

- `id` = `Column(Integer, primary_key=True, index=True)`
- `device_token` = `Column(String(255), nullable=False)`
- `device_type` = `Column(String(255))`
- `last_active` = `Column(DateTime, server_default=func.now())`
- `userid` = `Column(Integer, ForeignKey("User.id"))`
- `user` = `relationship("User", back_populates="user_devices")`

Static Private Attributes

- `str __tablename__ = "userdevice"`

8.86.1 Detailed Description

ORM model cho bảng userdevice, FCM token thiết bị của người dùng.

Definition at line 233 of file `models.py`.

8.86.2 Member Data Documentation

8.86.2.1 `__tablename__`

`str models.UserDevice.__tablename__ = "userdevice" [static], [private]`

Definition at line 235 of file [models.py](#).

8.86.2.2 `device_token`

`models.UserDevice.device_token = Column(String(255), nullable=False) [static]`

Definition at line 237 of file [models.py](#).

8.86.2.3 `device_type`

`models.UserDevice.device_type = Column(String(255)) [static]`

Definition at line 238 of file [models.py](#).

8.86.2.4 `id`

`models.UserDevice.id = Column(Integer, primary_key=True, index=True) [static]`

Definition at line 236 of file [models.py](#).

8.86.2.5 `last_active`

`models.UserDevice.last_active = Column(DateTime, server_default=func.now()) [static]`

Definition at line 239 of file [models.py](#).

8.86.2.6 `user`

`models.UserDevice.user = relationship("User", back_populates="user_devices") [static]`

Definition at line 242 of file [models.py](#).

8.86.2.7 `userid`

`models.UserDevice.userid = Column(Integer, ForeignKey("User.id")) [static]`

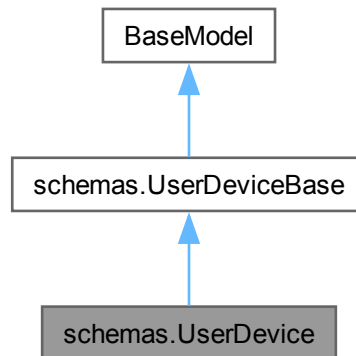
Definition at line 240 of file [models.py](#).

The documentation for this class was generated from the following file:

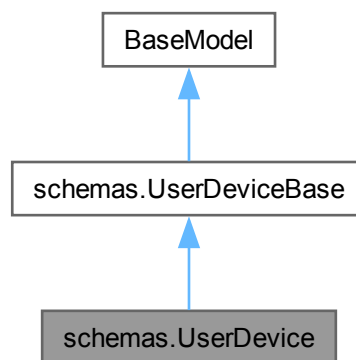
- [backend/models.py](#)

8.87 schemas.UserDevice Class Reference

Inheritance diagram for schemas.UserDevice:



Collaboration diagram for schemas.UserDevice:



Static Public Attributes

- int `model_config` = ConfigDict(from__attributes=True)

Static Public Attributes inherited from [schemas.UserDeviceBase](#)

- Optional `device_type` = None

8.87.1 Detailed Description

Definition at line 209 of file [schemas.py](#).

8.87.2 Member Data Documentation

8.87.2.1 model_config

```
int schemas.UserDevice.model_config = ConfigDict(from_attributes=True) [static]
```

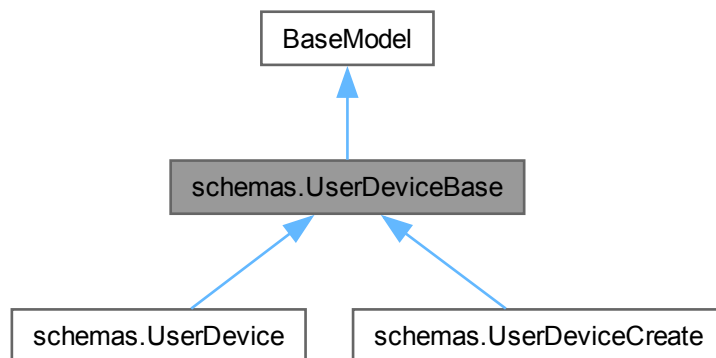
Definition at line 213 of file [schemas.py](#).

The documentation for this class was generated from the following file:

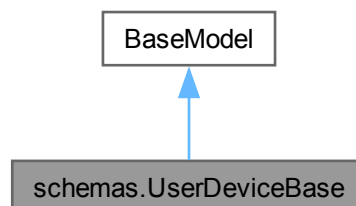
- [backend/schemas.py](#)

8.88 schemas.UserDeviceBase Class Reference

Inheritance diagram for schemas.UserDeviceBase:



Collaboration diagram for schemas.UserDeviceBase:



Static Public Attributes

- Optional `device_type` = None

8.88.1 Detailed Description

Schema cơ sở cho thiết bị người dùng (FCM token).

Definition at line 201 of file [schemas.py](#).

8.88.2 Member Data Documentation

8.88.2.1 device_type

Optional schemas.UserDeviceBase.device_type = None [static]

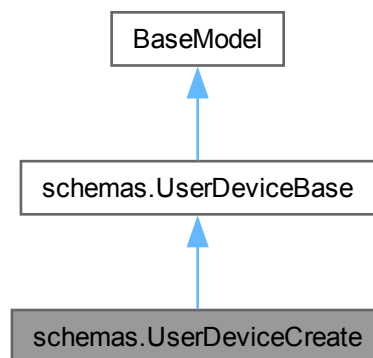
Definition at line 204 of file [schemas.py](#).

The documentation for this class was generated from the following file:

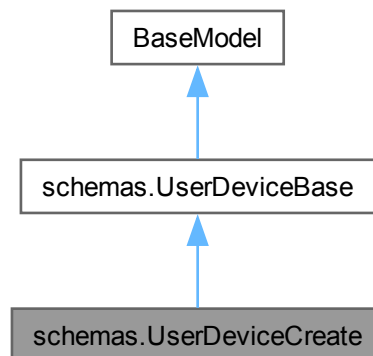
- [backend/schemas.py](#)

8.89 schemas.UserDeviceCreate Class Reference

Inheritance diagram for schemas.UserDeviceCreate:



Collaboration diagram for `schemas.UserDeviceCreate`:



Additional Inherited Members

Static Public Attributes inherited from [schemas.UserDeviceBase](#)

- Optional `device_type` = None

8.89.1 Detailed Description

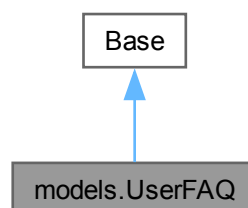
Definition at line 206 of file [schemas.py](#).

The documentation for this class was generated from the following file:

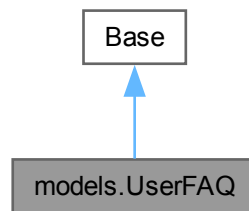
- [backend/schemas.py](#)

8.90 models.UserFAQ Class Reference

Inheritance diagram for `models.UserFAQ`:



Collaboration diagram for models.UserFAQ:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `viewedat` = Column(DateTime, server_default=func.now())
- `userid` = Column(Integer, ForeignKey("User.id"))
- `faqid` = Column(Integer, ForeignKey("faq.id"))
- `user` = relationship("User", back_populates="user_faqs")
- `faq` = relationship("FAQ", back_populates="user_faqs")

Static Private Attributes

- `str __tablename__ = "userfaq"`

8.90.1 Detailed Description

ORM model cho bảng userfaq, lịch sử xem FAQ của người dùng.

Definition at line 244 of file [models.py](#).

8.90.2 Member Data Documentation

8.90.2.1 `__tablename__`

`str models.UserFAQ.__tablename__ = "userfaq"` [static], [private]

Definition at line 246 of file [models.py](#).

8.90.2.2 `faq`

`models.UserFAQ.faq = relationship("FAQ", back_populates="user_faqs")` [static]

Definition at line 253 of file [models.py](#).

8.90.2.3 faqid

```
models.UserFAQ.faqid = Column(Integer, ForeignKey("faq.id")) [static]
```

Definition at line 250 of file [models.py](#).

8.90.2.4 id

```
models.UserFAQ.id = Column(Integer, primary_key=True, index=True) [static]
```

Definition at line 247 of file [models.py](#).

8.90.2.5 user

```
models.UserFAQ.user = relationship("User", back_populates="user_faqs") [static]
```

Definition at line 252 of file [models.py](#).

8.90.2.6 userid

```
models.UserFAQ.userid = Column(Integer, ForeignKey("User.id")) [static]
```

Definition at line 249 of file [models.py](#).

8.90.2.7 viewedat

```
models.UserFAQ.viewedat = Column(DateTime, server_default=func.now()) [static]
```

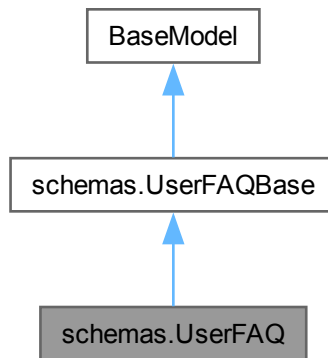
Definition at line 248 of file [models.py](#).

The documentation for this class was generated from the following file:

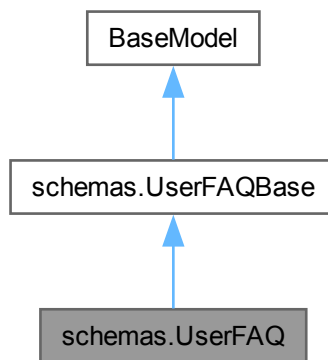
- [backend/models.py](#)

8.91 schemas.UserFAQ Class Reference

Inheritance diagram for schemas.UserFAQ:



Collaboration diagram for schemas.UserFAQ:



Static Public Attributes

- datetime `model_config` = ConfigDict(from_attributes=True)

8.91.1 Detailed Description

Definition at line 225 of file `schemas.py`.

8.91.2 Member Data Documentation

8.91.2.1 model_config

`datetime schemas.UserFAQ.model_config = ConfigDict(from_attributes=True) [static]`

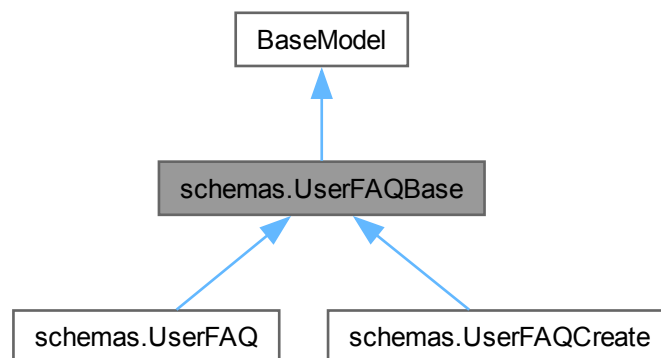
Definition at line 228 of file [schemas.py](#).

The documentation for this class was generated from the following file:

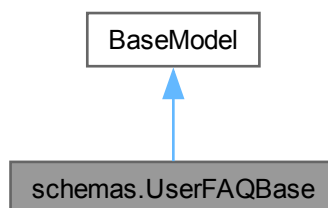
- [backend/schemas.py](#)

8.92 schemas.UserFAQBase Class Reference

Inheritance diagram for `schemas.UserFAQBase`:



Collaboration diagram for `schemas.UserFAQBase`:



8.92.1 Detailed Description

Schema cơ sở cho lịch sử xem FAQ của người dùng.

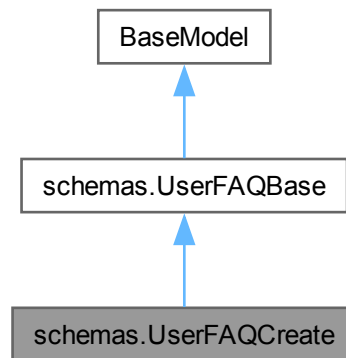
Definition at line 217 of file [schemas.py](#).

The documentation for this class was generated from the following file:

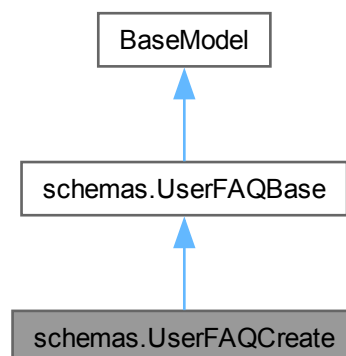
- [backend/schemas.py](#)

8.93 schemas.UserFAQCreate Class Reference

Inheritance diagram for schemas.UserFAQCreate:



Collaboration diagram for schemas.UserFAQCreate:



8.93.1 Detailed Description

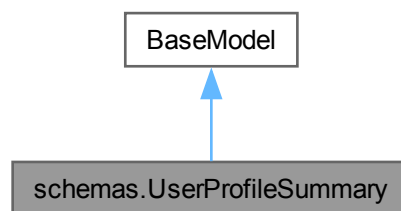
Definition at line 222 of file [schemas.py](#).

The documentation for this class was generated from the following file:

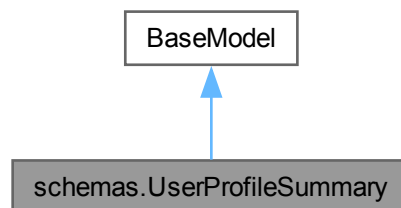
- [backend/schemas.py](#)

8.94 schemas.UserProfileSummary Class Reference

Inheritance diagram for schemas.UserProfileSummary:



Collaboration diagram for schemas.UserProfileSummary:



8.94.1 Detailed Description

Schema tóm tắt hồ sơ người dùng: điểm, đơn hàng, tổng chi tiêu.

Definition at line 164 of file [schemas.py](#).

The documentation for this class was generated from the following file:

- [backend/schemas.py](#)

Chapter 9

File Documentation

9.1 app/app/src/main/java/com/example/myapp/adapters/AddressAdapter.kt File Reference

Packages

- package [AddressAdapter](#)
- package [AddressAdapter.AddressViewHolder](#)

9.2 AddressAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.adapters
00002
00007
00008 import android.view.LayoutInflater
00009 import android.view.View
00010 import android.view.ViewGroup
00011 import android.widget.TextView
00012 import androidx.recyclerview.widget.RecyclerView
00013 import com.example.myapp.R
00014 import com.example.myapp.screens.api.Address
00015
00024 class AddressAdapter(
00025     private val addresses: List<Address>,
00026     private val onEditClick: (Address) -> Unit
00027 ) : RecyclerView.Adapter<AddressAdapter.AddressViewHolder>() {
00028
00036     class AddressViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
00037         val tvPhone: TextView = itemView.findViewById(R.id.tvPhone)
00038         val tvAddress: TextView = itemView.findViewById(R.id.tvAddress)
00039         val tvEdit: TextView = itemView.findViewById(R.id.tvEdit)
00040     }
00041
00049     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AddressViewHolder {
00050         val view = LayoutInflater.from(parent.context)
00051             .inflate(R.layout.item_address, parent, false)
00052         return AddressViewHolder(view)
00053     }
00054
00062     override fun onBindViewHolder(holder: AddressViewHolder, position: Int) {
00063         val address = addresses[position]
00064         holder.tvPhone.text = "SDT: ${address.phone ?: "Chưa có số điện thoại"}"
00065         holder.tvAddress.text = "Địa chỉ: ${address.detail}"
00066         holder.tvEdit.setOnClickListener {
00067             onEditClick(address)
00068         }
00069     }
00070
00076     override fun getItemCount(): Int = addresses.size
00077 }
```

9.3 app/app/src/main/java/com/example/myapp/adapters/ChatMessageAdapter.kt File Reference

Packages

- package [ChatMessageAdapter](#)
- package [ChatMessageAdapter.MessageViewHolder](#)

9.4 ChatMessageAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapplication.adapters
00002
00007
00008 import android.view.LayoutInflater
00009 import android.view.View
00010 import android.view.ViewGroup
00011 import android.widget.ImageView
00012 import android.widget.TextView
00013 import androidx.recyclerview.widget.RecyclerView
00014 import com.example.myapplication.R
00015
00023 data class ChatMessage(
00024     val message: String,
00025     val isUser: Boolean,
00026     val timestamp: String = ""
00027 )
00028
00035 class ChatMessageAdapter(private val messages: MutableList<ChatMessage>) :
00036     RecyclerView.Adapter<ChatMessageAdapter.MessageViewHolder>() {
00037
00039     companion object {
00040         private const val VIEW_TYPE_USER = 1
00041         private const val VIEW_TYPE_BOT = 2
00042     }
00043
00050     class MessageViewHolder(view: View) : RecyclerView.ViewHolder(view) {
00051         val tvMessage: TextView = view.findViewById(R.id.tvMessage)
00052         val ivAvatar: ImageView = view.findViewById(R.id.ivAvatar)
00053     }
00054
00061     override fun getItemViewType(position: Int): Int {
00062         return if (messages[position].isUser) VIEW_TYPE_USER else VIEW_TYPE_BOT
00063     }
00064
00072     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MessageViewHolder {
00073         val layoutId = if (viewType == VIEW_TYPE_USER) {
00074             R.layout.item_chat_user
00075         } else {
00076             R.layout.item_chat_bot
00077         }
00078         val view = LayoutInflater.from(parent.context).inflate(layoutId, parent, false)
00079         return MessageViewHolder(view)
00080     }
00081
00088     override fun onBindViewHolder(holder: MessageViewHolder, position: Int) {
00089         val message = messages[position]
00090         holder.tvMessage.text = message.message
00091     }
00092
00098     override fun getItemCount() = messages.size
00099
00106     fun addMessage(message: ChatMessage) {
00107         messages.add(message)
00108         notifyItemInserted(messages.size - 1)
00109     }
00110 }
```

9.5 app/app/src/main/java/com/example/myapp/adapters/FAQAdapter.kt File Reference

Packages

- package [FAQAdapter](#)

- package [FAQAdapter.FAQViewHolder](#)

9.6 FAQAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens.adapters
00002
00007
00008 import android.view.LayoutInflater
00009 import android.view.View
00010 import android.view.ViewGroup
00011 import android.widget.TextView
00012 import androidx.recyclerview.widget.RecyclerView
00013 import com.example.myapp.R
00014 import com.example.myapp.screens.api.FAQ
00015
00022 class FAQAdapter(private val faqs: List<FAQ>) :
00023     RecyclerView.Adapter<FAQAdapter.FAQViewHolder>() {
00024
00031     class FAQViewHolder(view: View) : RecyclerView.ViewHolder(view) {
00032         val tvQuestion: TextView = view.findViewById(R.id.tvQuestion)
00033         val tvAnswer: TextView = view.findViewById(R.id.tvAnswer)
00034     }
00035
00043     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): FAQViewHolder {
00044         val view = LayoutInflater.from(parent.context)
00045             .inflate(R.layout.item_faq, parent, false)
00046         return FAQViewHolder(view)
00047     }
00048
00056     override fun onBindViewHolder(holder: FAQViewHolder, position: Int) {
00057         val faq = faqs[position]
00058         holder.tvQuestion.text = faq.question
00059         holder.tvAnswer.text = faq.answer
00060
00061         // Toggle answer visibility when clicking on question
00062         holder.itemView.setOnClickListener {
00063             if (holder.tvAnswer.visibility == View.VISIBLE) {
00064                 holder.tvAnswer.visibility = View.GONE
00065             } else {
00066                 holder.tvAnswer.visibility = View.VISIBLE
00067             }
00068         }
00069     }
00070
00076     override fun getItemCount() = faqs.size
00077 }
```

9.7 app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapter.kt File Reference

Packages

- package [MenuItemAdapter](#)
- package [MenuItemAdapter.MenuItemViewHolder](#)

9.8 MenuItemAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.adapters
00002
00007
00008 import android.content.Context
00009 import android.content.Intent
00010 import android.view.LayoutInflater
00011 import android.view.View
```

```

00012 import android.view.ViewGroup
00013 import android.widget.ImageView
00014 import android.widget.RatingBar
00015 import android.widget.TextView
00016 import androidx.recyclerview.widget.RecyclerView
00017 import com.example.myapplication.R
00018 import com.example.myapplication.screens.api.MenuItem
00019 import com.example.myapplication.screens.food_detail
00020 import com.squareup.picasso.Picasso
00021
00022 class MenuItemAdapter(
00023     private val context: Context,
00024     private val menuItems: List<MenuItem>
00025 ) : RecyclerView.Adapter<MenuItemAdapter.MenuItemViewHolder>() {
00026
00027     class MenuItemViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
00028         val imgFood: ImageView = itemView.findViewById(R.id.imgFood)
00029         val tvFoodName: TextView = itemView.findViewById(R.id.tvFoodName)
00030         val tvFoodPrice: TextView = itemView.findViewById(R.id.tvFoodPrice)
00031         val tvRestaurantName: TextView = itemView.findViewById(R.id.tvRestaurantName)
00032         val ratingBar: RatingBar = itemView.findViewById(R.id.ratingBar)
00033         val tvRating: TextView = itemView.findViewById(R.id.tvRating)
00034         val tvDescription: TextView = itemView.findViewById(R.id.tvDescription)
00035         val foodTitleCard: View = itemView.findViewById(R.id.foodTitleCard)
00036     }
00037
00038     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MenuItemViewHolder {
00039         val view = LayoutInflater.from(context).inflate(R.layout.item_menu, parent, false)
00040         return MenuItemViewHolder(view)
00041     }
00042
00043     override fun onBindViewHolder(holder: MenuItemViewHolder, position: Int) {
00044         val menuItem = menuItems[position]
00045
00046         // Load menu item image
00047         if (!menuItem.image_url.isNullOrEmpty()) {
00048             Picasso.get()
00049                 .load(menuItem.image_url)
00050                 .placeholder(R.drawable.placeholder_loading)
00051                 .error(R.drawable.pngwing)
00052                 .into(holder.imgFood)
00053         }
00054
00055         // Set menu item name
00056         holder.tvFoodName.text = menuItem.name
00057
00058         // Set menu item price
00059         holder.tvFoodPrice.text = "${menuItem.price} VNĐ"
00060
00061         // Set restaurant name
00062         holder.tvRestaurantName.text = menuItem.restaurant_name ?: ""
00063
00064         // Set rating
00065         val avgRating = menuItem.avg_rating ?: 0f
00066         holder.ratingBar.rating = avgRating
00067         holder.tvRating.text = String.format("%.1f", avgRating)
00068
00069         // Set description
00070         holder.tvDescription.text = menuItem.description ?: ""
00071
00072         // Set click listener on foodTitleCard to navigate to food_detail
00073         holder.foodTitleCard.setOnClickListener {
00074             val intent = Intent(context, food_detail::class.java).apply {
00075                 putExtra("food_id", menuItem.id) // Pass food_id
00076                 putExtra("food_name", menuItem.name)
00077                 putExtra("food_price", menuItem.price)
00078                 putExtra("food_description", menuItem.description ?: "")
00079                 putExtra("food_image_url", menuItem.image_url ?: "")
00080                 putExtra("restaurant_name", menuItem.restaurant_name ?: "")
00081             }
00082             context.startActivity(intent)
00083         }
00084     }
00085
00086     override fun getItemCount(): Int = menuItems.size
00087 }

```


9.9 app/app/src/main/java/com/example/myapp/adapters/MenuItemAdapterSmall.kt File Reference

Packages

- package [MenuItemAdapterSmall](#)
- package [MenuItemAdapterSmall.MenuItemViewHolder](#)

9.10 MenuItemAdapterSmall.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.adapters
00002
00007
00008 import android.content.Context
00009 import android.content.Intent
00010 import android.view.LayoutInflater
00011 import android.view.View
00012 import android.view.ViewGroup
00013 import android.widget.ImageView
00014 import android.widget.TextView
00015 import androidx.recyclerview.widget.RecyclerView
00016 import com.example.myapp.R
00017 import com.example.myapp.screens.api.MenuItem
00018 import com.example.myapp.screens.food_detail
00019 import com.squareup.picasso.Picasso
00020
00029 class MenuItemAdapterSmall(
00030     private val context: Context,
00031     private val menuItems: List<MenuItem>
00032 ) : RecyclerView.Adapter<MenuItemAdapterSmall.MenuItemViewHolder>() {
00033
00042     class MenuItemViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
00043         val imgFood: ImageView = itemView.findViewById(R.id.imgFood)
00044         val tvFoodName: TextView = itemView.findViewById(R.id.tvFoodName)
00045         val tvFoodPrice: TextView = itemView.findViewById(R.id.tvFoodPrice)
00046         val foodTitleCard: View = itemView.findViewById(R.id.foodTitleCard)
00047     }
00048
00056     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MenuItemViewHolder {
00057         val view = LayoutInflater.from(context).inflate(R.layout.item_menu_small, parent, false)
00058         return MenuItemViewHolder(view)
00059     }
00060
00069     override fun onBindViewHolder(holder: MenuItemViewHolder, position: Int) {
00070         val menuItem = menuItems[position]
00071
00072         // Load menu item image
00073         if (!menuItem.image_url.isNullOrEmpty()) {
00074             Picasso.get()
00075                 .load(menuItem.image_url)
00076                 .placeholder(R.drawable.placeholder_loading)
00077                 .error(R.drawable.pngwing)
00078                 .into(holder.imgFood)
00079         }
00080
00081         // Set menu item name
00082         holder.tvFoodName.text = menuItem.name
00083
00084         // Set menu item price
00085         holder.tvFoodPrice.text = "${menuItem.price} VNĐ"
00086
00087         // Set click listener on foodTitleCard to navigate to food_detail
00088         holder.foodTitleCard.setOnClickListener {
00089             val intent = Intent(context, food_detail::class.java).apply {
00090                 putExtra("food_id", menuItem.id) // Pass food_id
00091                 putExtra("food_name", menuItem.name)
00092                 putExtra("food_price", menuItem.price)
00093                 putExtra("food_description", menuItem.description ?: "")
00094                 putExtra("food_image_url", menuItem.image_url ?: "")
00095                 putExtra("restaurant_name", menuItem.restaurant_name ?: "")
00096             }
00097             context.startActivity(intent)
00098         }
00099     }
00100
00106     override fun getItemCount(): Int = menuItems.size
00107 }

```

9.11 app/app/src/main/java/com/example/myapp/adapters/MyFirebaseMessagingService.kt File Reference

Packages

- package [FirebaseMessagingService](#)

Functions

- override fun [FirebaseMessagingService.onMessageReceived](#) (remoteMessage:RemoteMessage)
- fun [FirebaseMessagingService.showNotification](#) (title:String, body:String)

9.12 MyFirebaseMessagingService.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens
00002
00003
00004 import android.app.NotificationChannel
00005 import android.app.NotificationManager
00006 import android.content.Context
00007 import android.os.Build
00008 import android.util.Log
00009 import androidx.core.app.NotificationCompat
00010 import com.example.myapp.R
00011 import com.example.myapp.screens.api.FcmTokenRegistrar
00012 import com.google.firebase.messaging.FirebaseMessagingService
00013 import com.google.firebase.messaging.RemoteMessage
00014
00015 class MyFirebaseMessagingService : FirebaseMessagingService() {
00016
00017     // Hàm này chạy khi có thông báo bay tới
00018     override fun onMessageReceived(remoteMessage: RemoteMessage) {
00019         super.onMessageReceived(remoteMessage)
00020
00021         // 1. Lấy dữ liệu tiêu đề và nội dung
00022         val title = remoteMessage.notification?.title ?: "Thông báo từ Trung"
00023         val body = remoteMessage.notification?.body ?: "Nội dung trống"
00024
00025         showNotification(title, body)
00026     }
00027
00028     private fun showNotification(title: String, body: String) {
00029         val channelId = "food_delivery_channel"
00030         val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
00031
00032         // 2. Tạo Kênh (Channel) cho Android đời mới
00033         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
00034             val channel = NotificationChannel(
00035                 channelId,
00036                 "Thông báo đơn hàng",
00037                 NotificationManager.IMPORTANCE_HIGH
00038             ).apply {
00039                 description = "Kênh nhận thông báo đặt đồ ăn"
00040             }
00041             notificationManager.createNotificationChannel(channel)
00042         }
00043
00044         // 3. Xây dựng giao diện thông báo
00045         val notification = NotificationCompat.Builder(this, channelId)
00046             .setSmallIcon(R.drawable.pngwing) // Dùng icon pngwing bạn đã có
00047             .setContentTitle(title)
00048             .setContentText(body)
00049             .setPriority(NotificationCompat.PRIORITY_HIGH)
00050             .setAutoCancel(true)
00051             .build()
00052
00053         // 4. Hiển thị lên màn hình
00054         notificationManager.notify(System.currentTimeMillis().toInt(), notification)
00055     }
00056
00057     // Hàm này chạy khi Firebase cấp Token mới (dùng để lưu vào Database)
00058     override fun onNewToken(token: String) {
00059         Log.d("FCM_TOKEN", "Token mới nè: $token")
00060         FcmTokenRegistrar.cacheToken(this, token)
00061         FcmTokenRegistrar.syncPendingToken(this, "onNewToken")
00062     }
00063 }
```

9.13 app/app/src/main/java/com/example/myapp/adapters/↵ NotificationAdapter.kt File Reference

Packages

- package [NotificationAdapter](#)
- package [NotificationAdapter.NotificationViewHolder](#)

9.14 NotificationAdapter.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.adapters
00002
00007
00008 import android.view.LayoutInflater
00009 import android.view.View
00010 import android.view.ViewGroup
00011 import android.widget.TextView
00012 import androidx.recyclerview.widget.RecyclerView
00013 import com.example.myapp.R
00014 import com.example.myapp.screens.api.Notification
00015
00023 class NotificationAdapter(
00024     private val notifications: List<Notification>,
00025     private val onItemClick: (Notification) -> Unit
00026 ) : RecyclerView.Adapter<NotificationAdapter.NotificationViewHolder>() {
00027
00037     class NotificationViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
00038         val tvTitle: TextView = itemView.findViewById(R.id.tvNotificationTitle)
00039         val tvContent: TextView = itemView.findViewById(R.id.tvNotificationContent)
00040         val tvTime: TextView = itemView.findViewById(R.id.tvNotificationTime)
00041         val unreadIndicator: View = itemView.findViewById(R.id.unreadIndicator)
00042         val container: View = itemView.findViewById(R.id.notificationContainer)
00043     }
00044
00052     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): NotificationViewHolder {
00053         val view = LayoutInflater.from(parent.context)
00054             .inflate(R.layout.item_notification, parent, false)
00055         return NotificationViewHolder(view)
00056     }
00057
00065     override fun onBindViewHolder(holder: NotificationViewHolder, position: Int) {
00066         val notification = notifications[position]
00067
00068         holder.tvTitle.text = notification.title
00069         holder.tvContent.text = notification.content
00070         holder.tvTime.text = formatTime(notification.createdat)
00071
00072         // Hiển thị indicator nếu chưa đọc
00073         if (!notification.isread) {
00074             holder.unreadIndicator.visibility = View.VISIBLE
00075             holder.container.setBackgroundColor(android.graphics.Color.parseColor("#FFF5F5F5"))
00076         } else {
00077             holder.unreadIndicator.visibility = View.GONE
00078             holder.container.setBackgroundColor(android.graphics.Color.WHITE)
00079         }
00080
00081         holder.itemView.setOnClickListener {
00082             onItemClick(notification)
00083         }
00084     }
00085
00091     override fun getItemCount(): Int = notifications.size
00092
00100     private fun formatTime(dateTime: String): String {
00101         // Chuyển đổi định dạng thời gian từ API sang định dạng hiển thị
00102         return try {
00103             val inputFormat = java.text.SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", java.util.Locale.getDefault())
00104             val outputFormat = java.text.SimpleDateFormat("dd/MM/yyyy HH:mm", java.util.Locale.getDefault())
00105             val date = inputFormat.parse(dateTime)
00106             outputFormat.format(date ?: java.util.Date())
00107         } catch (e: Exception) {
00108             dateTime
00109         }
00110     }
00111 }

```

9.15 app/app/src/main/java/com/example/myapp/adapters/↵ RestaurantAdapter.kt File Reference

Packages

- package [RestaurantAdapter](#)
- package [RestaurantAdapter.RestaurantViewHolder](#)

9.16 RestaurantAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.adapters
00002
00007
00008 import android.content.Context
00009 import android.content.Intent
00010 import android.view.LayoutInflater
00011 import android.view.View
00012 import android.view.ViewGroup
00013 import android.widget.ImageView
00014 import android.widget.TextView
00015 import androidx.recyclerview.widget.RecyclerView
00016 import com.example.myapp.R
00017 import com.example.myapp.screens.api.Restaurant
00018 import com.example.myapp.screens.restaurant_profile
00019 import com.squareup.picasso.Picasso
00020
00029 class RestaurantAdapter(
00030     private val context: Context,
00031     private val restaurants: List<Restaurant>
00032 ) : RecyclerView.Adapter<RestaurantAdapter.RestaurantViewHolder>() {
00033
00043     class RestaurantViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
00044         val imgFood: ImageView = itemView.findViewById(R.id.imgFood)
00045         val tvRestaurantName: TextView = itemView.findViewById(R.id.tvRestaurantName)
00046         val tvRating: TextView = itemView.findViewById(R.id.tvRating)
00047         val tvOpeningHours: TextView = itemView.findViewById(R.id.tvOpeningHours)
00048         val foodTitleCard: View = itemView.findViewById(R.id.foodTitleCard)
00049     }
00050
00058     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): RestaurantViewHolder {
00059         val view = LayoutInflater.from(context).inflate(R.layout.item_restaurant, parent, false)
00060         return RestaurantViewHolder(view)
00061     }
00062
00071     override fun onBindViewHolder(holder: RestaurantViewHolder, position: Int) {
00072         val restaurant = restaurants[position]
00073
00074         // Load restaurant image
00075         if (!restaurant.image_url.isNullOrEmpty()) {
00076             Picasso.get()
00077                 .load(restaurant.image_url)
00078                 .placeholder(R.drawable.placeholder_loading)
00079                 .error(R.drawable.pngwing)
00080                 .into(holder.imgFood)
00081         }
00082
00083         // Set restaurant name
00084         holder.tvRestaurantName.text = restaurant.name
00085
00086         // Set restaurant rating
00087         holder.tvRating.text = restaurant.rating?.toString() ?: "N/A"
00088
00089         // Set opening hours
00090         val openTime = restaurant.open_time ?: "N/A"
00091         val closeTime = restaurant.close_time ?: "N/A"
00092         holder.tvOpeningHours.text = "Giờ mở cửa: $openTime - $closeTime"
00093
00094         // Set click listener for foodTitleCard
00095         holder.foodTitleCard.setOnClickListener {
00096             val intent = Intent(context, restaurant_profile::class.java)
00097             intent.putExtra("RESTAURANT_ID", restaurant.id)
00098             context.startActivity(intent)
00099         }
00100     }
00101
00107     override fun getItemCount(): Int = restaurants.size
00108 }
```

9.17 app/app/src/main/java/com/example/myapp/api/ApiService.kt File Reference

Packages

- package [ApiService](#)

9.18 ApiService.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens.api
00002
00003
00004
00005 import retrofit2.Call
00006 import retrofit2.http.Body
00007 import retrofit2.http.Field
00008 import retrofit2.http.FormUrlEncoded
00009 import retrofit2.http.GET
00010 import retrofit2.http.POST
00011 import retrofit2.http.Path
00012
00013 data class LoginRequest(
00014     val username: String,
00015     val password: String
00016 )
00017
00018 data class RegisterRequest(
00019     val name: String,
00020     val username: String,
00021     val email: String,
00022     val phone: String,
00023     val role: String,
00024     val password: String
00025 )
00026
00027 data class LoginResponse(
00028     val access_token: String,
00029     val token_type: String
00030 )
00031
00032 data class RegisterResponse(
00033     val name: String,
00034     val email: String,
00035     val phone: String,
00036     val role: String,
00037     val id: Int
00038 )
00039
00040 data class MenuItem(
00041     val id: Int,
00042     val name: String,
00043     val image_url: String?,
00044     val price: Int,
00045     val is_available: Boolean,
00046     val description: String?,
00047     val restaurantid: Int,
00048     val categoryid: Int,
00049     val restaurant_name: String? = null,
00050     val reviews: List<ReviewDetail>? = null,
00051     val avg_rating: Float? = null
00052 )
00053
00054 data class ReviewDetail(
00055     val id: Int,
00056     val rating: Int,
00057     val comment: String?,
00058     val userid: Int,
00059     val user_name: String?
00060 )
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
```

```
00079 data class Restaurant(  
00080     val id: Int,  
00081     val name: String,  
00082     val image_url: String?,  
00083     val address: String?,  
00084     val rating: Int?,  
00085     val open_time: String?,  
00086     val close_time: String?,  
00087     val phone_number: String,  
00088     val status: String,  
00089     val description: String?,  
00090     val menu_items: List<MenuItem>  
00091 )  
00092  
00093  
00095 data class Notification(  
00096     val id: Int,  
00097     val title: String,  
00098     val type: String,  
00099     val content: String,  
00100     val isread: Boolean,  
00101     val createdat: String,  
00102     val userid: Int,  
00103     val orderid: Int?,  
00104     val sessionid: Int?  
00105 )  
00106  
00107  
00109 data class NotificationCreate(  
00110     val title: String,  
00111     val type: String,  
00112     val content: String,  
00113     val isread: Boolean = false,  
00114     val userid: Int,  
00115     val orderid: Int? = null,  
00116     val sessionid: Int? = null  
00117 )  
00118  
00119  
00121 data class FAQ(  
00122     val id: Int,  
00123     val question: String,  
00124     val answer: String,  
00125     val isactive: Boolean  
00126 )  
00127  
00128  
00130 data class ChatMessageRequest(  
00131     val message: String  
00132 )  
00133  
00134  
00136 data class ChatMessageResponse(  
00137     val message: String,  
00138     val id: Int,  
00139     val senderrole: String,  
00140     val sentat: String,  
00141     val sessionid: Int  
00142 )  
00143  
00144  
00146 data class Address(  
00147     val id: Int,  
00148     val detail: String,  
00149     val phone: String?,  
00150     val userid: Int  
00151 )  
00152  
00153  
00155 data class AddressCreateRequest(  
00156     val detail: String,  
00157     val phone: String?,  
00158     val userid: Int  
00159 )  
00160  
00161  
00163 data class AddressUpdateRequest(  
00164     val detail: String?,  
00165     val phone: String?  
00166 )  
00167  
00168  
00170 data class VNPayResponse(  
00171     val payment_url: String  
00172 )  
00173  
00175 data class DeviceTokenResponse(  

```

```
00176     val message: String
00177 )
00178
00180 data class UserProfileSummary(
00181     val user_id: Int,
00182     val user_name: String,
00183     val points: Int,
00184     val delivered_orders: Int,
00185     val total_spent: Int
00186 )
00187
00189 data class OrderItemRequest(
00190     val quantity: Int,
00191     val price: Int,
00192     val menuitemid: Int
00193 )
00194
00196 data class OrderCreateRequest(
00197     val status: String,
00198     val preorderdate: String? = null,
00199     val preordertime: String? = null,
00200     val totalprice: Int,
00201     val restaurantid: Int,
00202     val addressid: Int,
00203     val userid: Int,
00204     val order_items: List<OrderItemRequest>
00205 )
00206
00208 data class OrderResponse(
00209     val id: Int,
00210     val status: String,
00211     val createdat: String,
00212     val preorderdate: String?,
00213     val preordertime: String?,
00214     val totalprice: Int,
00215     val restaurantid: Int,
00216     val addressid: Int,
00217     val userid: Int
00218 )
00219
00221 data class OrderItemDetailResponse(
00222     val id: Int,
00223     val quantity: Int,
00224     val price: Int,
00225     val menuitemid: Int,
00226     val menuitem_name: String?,
00227     val image_url: String? = null
00228 )
00229
00231 data class OrderDetailResponse(
00232     val id: Int,
00233     val status: String,
00234     val createdat: String,
00235     val preorderdate: String?,
00236     val preordertime: String?,
00237     val totalprice: Int,
00238     val restaurantid: Int,
00239     val addressid: Int,
00240     val userid: Int,
00241     val restaurant_name: String?,
00242     val address_detail: String?,
00243     val order_items: List<OrderItemDetailResponse>
00244 )
00245
00247 data class OrderStatusUpdateRequest(
00248     val status: String
00249 )
00250
00252 data class ReviewCreateRequest(
00253     val rating: Int,
00254     val comment: String? = null,
00255     val orderid: Int,
00256     val userid: Int,
00257     val menuitemid: Int? = null,
00258     val restaurantid: Int? = null
00259 )
00260
00262 data class ReviewResponse(
00263     val id: Int,
00264     val rating: Int,
00265     val comment: String?,
00266     val orderid: Int,
00267     val menuitemid: Int?,
00268     val restaurantid: Int?,
00269     val userid: Int
00270 )
00271
```

```

00273 data class PromotionResponse(
00274     val id: Int,
00275     val code: String,
00276     val discounttype: String,
00277     val discountvalue: Int,
00278     val expiredate: String,
00279     val minordervalue: Int,
00280     val status: String
00281 )
00282
00283
00285 interface ApiService {
00293     @FormUrlEncoded
00294     @POST("token")
00295     fun login(
00296         @Field("username") username: String,
00297         @Field("password") password: String,
00298         @Field("grant_type") grantType: String = "password"
00299     ): Call<LoginResponse>
00300
00301
00302
00303
00309     @POST("users/")
00310     fun register(@Body request: RegisterRequest): Call<RegisterResponse>
00311
00312
00317     @GET("users/me/")
00318     fun getCurrentUser(): Call<RegisterResponse>
00319
00325     @GET("users/{user_id}/profile-summary")
00326     fun getProfileSummary(@Path("user_id") userId: Int): Call<UserProfileSummary>
00327
00328
00333     @GET("restaurants/")
00334     fun getRestaurants(): Call<List<Restaurant>»
00335
00336
00342     @GET("restaurants/search/")
00343     fun searchRestaurantsByName(@retrofit2.http.Query("name") name: String): Call<List<Restaurant>»
00344
00345
00351     @GET("restaurants/{restaurant_id}/")
00352     fun getRestaurant(@Path("restaurant_id") restaurantId: Int): Call<Restaurant>
00353
00354
00360     @GET("users/{user_id}/notifications/")
00361     fun getUserNotifications(@Path("user_id") userId: Int): Call<List<Notification>»
00362
00363
00369     @POST("notifications/")
00370     fun createNotification(@Body notification: NotificationCreate): Call<Notification>
00371
00372
00378     @retrofit2.http.PUT("notifications/{notification_id}/read")
00379     fun markNotificationAsRead(@Path("notification_id") notificationId: Int): Call<Void>
00380
00381
00386     @GET("menu-items/")
00387     fun getAllMenuItems(): Call<List<MenuItem>»
00388
00394     @GET("menu-items/{menu_item_id}/")
00395     fun getMenuItemDetail(@Path("menu_item_id") menuItemId: Int): Call<MenuItem>
00396
00402     @GET("menu-items/search/")
00403     fun searchMenuItemsByName(@retrofit2.http.Query("name") name: String): Call<List<MenuItem>»
00404
00409     @GET("promotions/")
00410     fun getPromotions(): Call<List<PromotionResponse>»
00411
00412
00418     @GET("menu-items/category/search/")
00419     fun searchMenuItemsByCategory(@retrofit2.http.Query("category_name") categoryName: String):
00420     Call<List<MenuItem>»
00421
00426     @GET("faqs/")
00427     fun getFAQs(): Call<List<FAQ>»
00428
00429
00436     @POST("chat/")
00437     fun sendMessage(@Body request: ChatMessageRequest, @retrofit2.http.Query("session_id") sessionId: Int? = null):
00438     Call<ChatMessageResponse>
00439
00445     @GET("users/{user_id}/addresses/")
00446     fun getUserAddresses(@Path("user_id") userId: Int): Call<List<Address>»

```



```

00447
00448
00454     @POST("addresses/")
00455     fun createAddress(@Body address: AddressCreateRequest): Call<Address>
00456
00457
00458
00459
00466     // ... trong interface ApiService
00467     @GET("create-payment") // Thay bằng đường dẫn API thật của bạn
00468     fun getVNPayUrl(
00469         @retrofit2.http.Query("amount") amount: Int,
00470         @retrofit2.http.Query("order_id") orderId: String
00471     ): Call<VNPayResponse> // Giả sử backend trả về JSON {"url": "..."}
00472
00473
00480     @retrofit2.http.PUT("addresses/{address_id}")
00481     fun updateAddress(@Path("address_id") addressId: Int, @Body address: AddressUpdateRequest): Call<Address>
00482
00488     @POST("orders/")
00489     fun createOrder(@Body request: OrderCreateRequest): Call<OrderResponse>
00490
00496     @GET("users/{user_id}/orders/")
00497     fun getUserOrders(@Path("user_id") userId: Int): Call<List<OrderResponse>
00498
00504     @GET("orders/{order_id}/detail")
00505     fun getOrderDetail(@Path("order_id") orderId: Int): Call<OrderDetailResponse>
00506
00513     @FormUrlEncoded
00514     @POST("devices/token")
00515     fun updateDeviceToken(
00516         @Field("token") token: String,
00517         @Field("device_type") deviceType: String = "android"
00518     ): Call<DeviceTokenResponse>
00519
00526     @retrofit2.http.PUT("orders/{order_id}/status")
00527     fun updateOrderStatus(
00528         @Path("order_id") orderId: Int,
00529         @Body request: OrderStatusUpdateRequest
00530     ): Call<OrderResponse>
00531
00537     @POST("reviews/")
00538     fun createReview(@Body request: ReviewCreateRequest): Call<ReviewResponse>
00539
00540
00541
00542
00543 }
00544

```

9.19 app/app/src/main/java/com/example/myapp/api/FcmTokenRegistrar.kt File Reference

Packages

- package [FcmTokenRegistrar](#)

9.20 FcmTokenRegistrar.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens.api
00002
00008
00009 import android.content.Context
00010 import android.util.Log
00011 import com.google.firebase.messaging.FirebaseMessaging
00012 import retrofit2.Call
00013 import retrofit2.Callback
00014 import retrofit2.Response
00015
00017 object FcmTokenRegistrar {
00019     private const val TAG = "FCM_TOKEN"
00021     private const val PREF_NAME = "user_prefs"

```

```

00023 private const val KEY_PENDING_FCM_TOKEN = "pending_fcm_token"
00025 private const val KEY_SYNCED_FCM_TOKEN = "synced_fcm_token"
00026
00032 fun cacheToken(context: Context, token: String) {
00033     context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
00034         .edit()
00035         .putString(KEY_PENDING_FCM_TOKEN, token)
00036         .apply()
00037 }
00038
00045 fun syncCurrentToken(context: Context, source: String) {
00046     val sharedPref = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
00047     val accessToken = sharedPref.getString("access_token", null)
00048
00049     if (accessToken.isNullOrBlank()) {
00050         FirebaseMessaging.getInstance().token
00051             .addOnSuccessListener { token ->
00052                 cacheToken(context, token)
00053                 Log.d(TAG, "Cached FCM token from $source while waiting for login")
00054             }
00055             .addOnFailureListener { error ->
00056                 Log.w(TAG, "Failed to get FCM token from $source", error)
00057             }
00058     }
00059     return
00060 }
00061 syncPendingToken(context, source)
00062 }
00063
00070 fun syncPendingToken(context: Context, source: String) {
00071     val sharedPref = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
00072     val accessToken = sharedPref.getString("access_token", null)
00073
00074     if (accessToken.isNullOrBlank()) {
00075         Log.d(TAG, "Skip syncing token from $source because access token is missing")
00076         return
00077     }
00078
00079     val pendingToken = sharedPref.getString(KEY_PENDING_FCM_TOKEN, null)
00080     if (pendingToken.isNullOrBlank()) {
00081         FirebaseMessaging.getInstance().token
00082             .addOnSuccessListener { token ->
00083                 cacheToken(context, token)
00084                 pushToken(context, token, source)
00085             }
00086             .addOnFailureListener { error ->
00087                 Log.w(TAG, "Failed to get FCM token from $source", error)
00088             }
00089     }
00090     return
00091 }
00092 if (pendingToken == sharedPref.getString(KEY_SYNCED_FCM_TOKEN, null)) {
00093     Log.d(TAG, "FCM token already synced from $source")
00094     return
00095 }
00096
00097 pushToken(context, pendingToken, source)
00098 }
00099
00106 private fun pushToken(context: Context, token: String, source: String) {
00107     RetrofitClient.apiService.updateDeviceToken(token, "android")
00108         .enqueue(object : Callback<DeviceTokenResponse> {
00109             override fun onResponse(
00110                 call: Call<DeviceTokenResponse>,
00111                 response: Response<DeviceTokenResponse>
00112             ) {
00113                 if (response.isSuccessful) {
00114                     context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
00115                         .edit()
00116                         .putString(KEY_SYNCED_FCM_TOKEN, token)
00117                         .apply()
00118                     Log.d(TAG, "Synced FCM token from $source")
00119                 } else {
00120                     Log.w(TAG, "Failed to sync FCM token from $source: ${response.code()} ${response.message()}")
00121                 }
00122             }
00123         })
00124     override fun onFailure(call: Call<DeviceTokenResponse>, t: Throwable) {
00125         Log.w(TAG, "Failed to sync FCM token from $source", t)
00126     }
00127 }
00128 }
00129 }

```

9.21 app/app/src/main/java/com/example/myapp/api/NetworkConfig.kt File Reference

Packages

- package [NetworkConfig](#)

9.22 NetworkConfig.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens.api
00002
00008
00010 object NetworkConfig {
00011     // Cấu hình URL cho backend API
00012     // Thay đổi giá trị này tùy theo môi trường sử dụng
00013
00014
00016     // Cho Android Emulator (máy ảo)
00017     private const val EMULATOR_URL = "http://10.0.2.2:8000/"
00018
00019
00021     // Cho thiết bị thật - thay IP bằng địa chỉ IP của máy tính chạy backend
00022     // Cách lấy IP: Mở CMD -> gõ ipconfig -> tìm "IPv4 Address"
00023     private const val DEVICE_URL = "http://192.168.1.100:8000/" // Thay IP này
00024
00025
00027     // Chọn URL phù hợp:
00028     // - true: dùng cho emulator
00029     // - false: dùng cho thiết bị thật
00030     private const val USE_EMULATOR = true
00031
00032
00034     val BASE_URL: String
00035         get() = if (USE_EMULATOR) EMULATOR_URL else DEVICE_URL
00036
00037
00043     // Helper function để lấy IP máy tính (chạy trên máy tính)
00044     fun getLocalIpAddress(): String {
00045         try {
00046             val interfaces = java.net.NetworkInterface.getNetworkInterfaces()
00047             while (interfaces.hasMoreElements()) {
00048                 val networkInterface = interfaces.nextElement()
00049                 val addresses = networkInterface.inetAddresses
00050                 while (addresses.hasMoreElements()) {
00051                     val address = addresses.nextElement()
00052                     if (!address.isLoopbackAddress && address is java.net.Inet4Address) {
00053                         return address.hostAddress ?: "unknown"
00054                     }
00055                 }
00056             }
00057         } catch (e: Exception) {
00058             e.printStackTrace()
00059         }
00060         return "unknown"
00061     }
00062 }
00063
```

9.23 app/app/src/main/java/com/example/myapp/api/RetrofitClient.kt File Reference

Packages

- package [RetrofitClient](#)

9.24 RetrofitClient.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapplication.screens.api
00002
00008
00009 import android.content.Context
00010 import okhttp3.Interceptor
00011 import okhttp3.OkHttpClient
00012 import retrofit2.Retrofit
00013 import retrofit2.converter.gson.GsonConverterFactory
00014 import java.util.concurrent.TimeUnit
00015
00017 object RetrofitClient {
00019     // Sử dụng NetworkConfig để lấy BASE_URL
00020     // Thay đổi USE_EMULATOR trong NetworkConfig.kt tùy theo môi trường
00021     private val BASE_URL = NetworkConfig.BASE_URL
00023     private var appContext: Context? = null
00024
00030     fun init(context: Context) {
00031         appContext = context.applicationContext
00032     }
00033
00035     private val authInterceptor = Interceptor { chain ->
00036         val request = chain.request()
00037         val sharedPref = appContext?.getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00038         val token = sharedPref?.getString("access_token", null)
00039
00040         if (token != null) {
00041             val newRequest = request.newBuilder()
00042                 .header("Authorization", "Bearer $token")
00043                 .build()
00044             chain.proceed(newRequest)
00045         } else {
00046             chain.proceed(request)
00047         }
00048     }
00049
00051     private val okHttpClient = OkHttpClient.Builder()
00052         .addInterceptor(authInterceptor)
00053         .connectTimeout(30, TimeUnit.SECONDS)
00054         .readTimeout(60, TimeUnit.SECONDS)
00055         .writeTimeout(30, TimeUnit.SECONDS)
00056         .build()
00057
00059     private val retrofit: Retrofit by lazy {
00060         Retrofit.Builder()
00061             .baseUrl(BASE_URL)
00062             .client(okHttpClient)
00063             .addConverterFactory(GsonConverterFactory.create())
00064             .build()
00065     }
00066
00068     val apiService: ApiService by lazy {
00069         retrofit.create(ApiService::class.java)
00070     }
00071 }
```

9.25 app/app/src/main/java/com/example/myapp/screens/activity__ cart.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- lateinit var [AppCompatActivity.rvCart](#)

9.26 activity_cart.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.widget.CheckBox
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import androidx.appcompat.app.AppCompatActivity
00018 import androidx.appcompat.widget.AppCompatButton
00019 import androidx.recyclerview.widget.LinearLayoutManager
00020 import androidx.recyclerview.widget.RecyclerView
00021 import com.example.myapp.R
00022 import java.text.NumberFormat
00023 import java.util.Locale
00024
00034 class activity_cart : AppCompatActivity() {
00036     private lateinit var rvCart: RecyclerView
00038     private lateinit var tvTotalItems: TextView
00040     private lateinit var tvTotalPrice: TextView
00042     private lateinit var cbSelectAll: CheckBox
00044     private lateinit var btnCheckout: AppCompatButton
00046     private lateinit var adapter: CartAdapter
00047
00057     override fun onCreate(savedInstanceState: Bundle?) {
00058         super.onCreate(savedInstanceState)
00059         setContentView(R.layout.activity_cart)
00060
00061         // Ảnh xạ View
00062         rvCart = findViewById(R.id.rvCart)
00063         tvTotalItems = findViewById(R.id.tvTotalItems)
00064         tvTotalPrice = findViewById(R.id.tvTotalPrice)
00065         cbSelectAll = findViewById(R.id.cbSelectAll)
00066         btnCheckout = findViewById(R.id.btnCheckout)
00067         val btnBack: ImageView = findViewById(R.id.btnBack)
00068
00069         // Nút quay lại
00070         btnBack.setOnClickListener { finish() }
00071
00072         // CHUYỂN SANG MÀN HÌNH ORDER
00073         btnCheckout.setOnClickListener {
00074             val intent = Intent(this, order::class.java)
00075             startActivity(intent)
00076         }
00077
00078         // Thiết lập Adapter sử dụng dữ liệu từ class cart
00079         adapter = CartAdapter(
00080             cart.cartList,
00081             onUpdate = { updateSummary() },
00082             onDelete = { position ->
00083                 cart.cartList.removeAt(position)
00084                 adapter.notifyItemRemoved(position)
00085                 adapter.notifyItemRangeChanged(position, cart.cartList.size)
00086                 updateSummary()
00087             }
00088         )
00089         rvCart.layoutManager = LinearLayoutManager(this)
00090         rvCart.adapter = adapter
00091
00092         // Chọn tất cả
00093         cbSelectAll.setOnCheckedChangeListener { _, isChecked ->
00094             cart.cartList.forEach { it.isSelected = isChecked }
00095             adapter.notifyDataSetChanged()
00096             updateSummary()
00097         }
00098
00099         updateSummary()
00100     }
00101
00109     private fun updateSummary() {
00110         val selectedItems = cart.cartList.filter { it.isSelected }
00111         val totalQty = selectedItems.sumOf { it.qty }
00112         val totalPrice = selectedItems.sumOf { it.qty * it.price }
00113
00114         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00115         tvTotalItems.text = totalQty.toString()
00116         tvTotalPrice.text = fmt.format(totalPrice) + "đ"
00117
00118         // LUÔN BẬT NÚT ĐỂ TEST
00119         btnCheckout.isEnabled = true
00120         btnCheckout.alpha = 1.0f
00121     }

```

```
00122 }
```

9.27 app/app/src/main/java/com/example/myapp/screens/add_shipping_address.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- lateinit var [AppCompatActivity.edtPhone](#)

9.28 add_shipping_address.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapplication
00002
00009
00010 import android.content.Context
00011 import android.content.Intent
00012 import android.os.Bundle
00013 import android.widget.Button
00014 import android.widget.EditText
00015 import android.widget.ImageView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import com.example.myapplication.R
00019 import com.example.myapplication.screens.api.Address
00020 import com.example.myapplication.screens.api.AddressCreateRequest
00021 import com.example.myapplication.screens.api.ApiService
00022 import retrofit2.Call
00023 import retrofit2.Callback
00024 import retrofit2.Response
00025 import retrofit2.Retrofit
00026 import retrofit2.converter.gson.GsonConverterFactory
00027
00035 class add_shipping_address : AppCompatActivity() {
00037     private lateinit var edtPhone: EditText
00039     private lateinit var edtAddress: EditText
00041     private lateinit var btnAdd: Button
00042
00051     override fun onCreate(savedInstanceState: Bundle?) {
00052         super.onCreate(savedInstanceState)
00053         setContentView(R.layout.add_shipping_address)
00054
00055         // Initialize views
00056         edtPhone = findViewById(R.id.edtPhone)
00057         edtAddress = findViewById(R.id.edtAddress)
00058         btnAdd = findViewById(R.id.btnAdd)
00059
00060         // Click btnBack
00061         val btnBack: ImageView = findViewById(R.id.btn_back)
00062         btnBack.setOnClickListener {
00063             finish() // quay lại màn hình trước
00064         }
00065
00066         // 2. Thiết lập sự kiện click cho nút "Thêm"
00067         btnAdd.setOnClickListener {
00068             val phone = edtPhone.text.toString().trim()
00069             val address = edtAddress.text.toString().trim()
00070
00071             // Validate input
00072             if (phone.isEmpty()) {
00073                 edtPhone.error = "Vui lòng nhập số điện thoại"
00074                 edtPhone.requestFocus()
00075                 return@setOnClickListener
00076             }
00077
```

```

00078         if (address.isEmpty()) {
00079             edtAddress.error = "Vui lòng nhập địa chỉ"
00080             edtAddress.requestFocus()
00081             return@setOnClickListener
00082         }
00083
00084         // Lấy userId từ SharedPreferences
00085         val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00086         val userId = sharedPref.getInt("user_id", 1)
00087
00088         // Gọi API tạo địa chỉ
00089         createAddress(phone, address, userId)
00090     }
00091
00092     // click icProfile
00093     val icProfile: ImageView = findViewById(R.id.icProfile)
00094     icProfile.setOnClickListener {
00095         val intent = Intent(this, profile::class.java)
00096         startActivity(intent)
00097     }
00098
00099     val icHome: ImageView = findViewById(R.id.icHome)
00100     icHome.setOnClickListener {
00101         val intent = Intent(this, home::class.java)
00102         startActivity(intent)
00103     }
00104     val btnCart: ImageView = findViewById(R.id.icCart)
00105     btnCart.setOnClickListener {
00106         val intent = Intent(this, cart::class.java)
00107         startActivity(intent)
00108     }
00109 }
00110
00121 private fun createAddress(phone: String, address: String, userId: Int) {
00122     val retrofit = Retrofit.Builder()
00123         .baseUrl("http://10.0.2.2:8000/") // Android emulator localhost
00124         .addConverterFactory(GsonConverterFactory.create())
00125         .build()
00126
00127     val apiService = retrofit.create(ApiService::class.java)
00128
00129     val addressRequest = AddressCreateRequest(
00130         detail = address,
00131         phone = phone,
00132         userid = userId
00133     )
00134
00135     apiService.createAddress(addressRequest).enqueue(object : Callback<Address> {
00136         override fun onResponse(call: Call<Address>, response: Response<Address>) {
00137             if (response.isSuccessful) {
00138                 // Thành công - chuyển sang màn hình thông báo
00139                 Toast.makeText(this@add_shipping_address, "Thêm địa chỉ thành công!", Toast.LENGTH_SHORT).show()
00140                 val intent = Intent(this@add_shipping_address, shipping_address_added_successfully::class.java)
00141                 startActivity(intent)
00142                 finish()
00143             } else {
00144                 Toast.makeText(this@add_shipping_address, "Không thể thêm địa chỉ", Toast.LENGTH_SHORT).show()
00145             }
00146         }
00147
00148         override fun onFailure(call: Call<Address>, t: Throwable) {
00149             Toast.makeText(this@add_shipping_address, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00150         }
00151     })
00152 }
00153 }

```

9.29 app/app/src/main/java/com/example/myapp/screens/cart.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var Vui lòng chọn ít nhất Toast.LENGTH_SHORT. [AppCompatActivity.show](#) () return @setOnClickListener } [startActivity](#)(Intent(this

Variables

- lateinit var Vui lòng chọn ít nhất [AppCompatActivity.món](#)

9.30 cart.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.view.LayoutInflater
00015 import android.view.View
00016 import android.view.ViewGroup
00017 import android.widget.CheckBox
00018 import android.widget.ImageView
00019 import android.widget.TextView
00020 import android.widget.Toast
00021 import androidx.appcompat.app.AppCompatActivity
00022 import androidx.appcompat.widget.AppCompatButton
00023 import androidx.recyclerview.widget.LinearLayoutManager
00024 import androidx.recyclerview.widget.RecyclerView
00025 import com.example.myapp.R
00026 import com.squareup.picasso.Picasso
00027 import java.text.NumberFormat
00028 import java.util.Locale
00029
00030 import java.io.Serializable
00031
00042 data class CartItem(
00043     val id: Int,
00044     val name: String,
00045     val price: Int,
00046     var qty: Int,
00047     val imageUrl: String? = null,
00048     var isSelected: Boolean = false
00049 ) : Serializable
00050
00061 class cart : AppCompatActivity() {
00063     private lateinit var rvCart: RecyclerView
00065     private lateinit var tvTotalItems: TextView
00067     private lateinit var tvTotalPrice: TextView
00069     private lateinit var cbSelectAll: CheckBox
00071     private lateinit var btnCheckout: AppCompatButton
00073     private lateinit var adapter: CartAdapter
00074
00075     companion object {
00077         val cartList = mutableListOf<CartItem>()
00078     }
00079
00089     override fun onCreate(savedInstanceState: Bundle?) {
00090         super.onCreate(savedInstanceState)
00091         setContentView(R.layout.activity_cart)
00092
00093         rvCart = findViewById(R.id.rvCart)
00094         tvTotalItems = findViewById(R.id.tvTotalItems)
00095         tvTotalPrice = findViewById(R.id.tvTotalPrice)
00096         cbSelectAll = findViewById(R.id.cbSelectAll)
00097         btnCheckout = findViewById(R.id.btnCheckout)
00098         findViewById<ImageView>(R.id.btnBack).setOnClickListener { finish() }
00099
00100         // CLICK THANH TOÁN SANG ORDER
00101         btnCheckout.setOnClickListener {
00102             if (cartList.none { it.isSelected }) {
00103                 Toast.makeText(this, "Vui lòng chọn ít nhất 1 món", Toast.LENGTH_SHORT).show()
00104                 return@setOnClickListener
00105             }
00106             startActivity(Intent(this, order::class.java))
00107         }
00108
00109         adapter = CartAdapter(cartList, { updateSummary() }, { position ->
00110             cartList.removeAt(position)
00111             adapter.notifyItemRemoved(position)
00112             adapter.notifyItemRangeChanged(position, cartList.size)
00113             updateSummary()
00114         })
00115         rvCart.layoutManager = LinearLayoutManager(this)
00116         rvCart.adapter = adapter
00117

```



```

00118         cbSelectAll.setOnCheckedChangeListener { _, isChecked ->
00119             cartList.forEach { it.isSelected = isChecked }
00120             adapter.notifyDataSetChanged()
00121             updateSummary()
00122         }
00123         updateSummary()
00124     }
00125
00132     private fun updateSummary() {
00133         val selectedItems = cartList.filter { it.isSelected }
00134         val totalQty = selectedItems.sumOf { it.qty }
00135         val totalPrice = selectedItems.sumOf { it.qty * it.price }
00136         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00137         tvTotalItems.text = totalQty.toString()
00138         tvTotalPrice.text = fmt.format(totalPrice) + "đ"
00139         btnCheckout.isEnabled = selectedItems.isNotEmpty()
00140         btnCheckout.alpha = if (selectedItems.isNotEmpty()) 1f else 0.5f
00141     }
00142 }
00143
00151 class CartAdapter(
00152     private val items: MutableList<CartItem>,
00153     private val onUpdate: () -> Unit,
00154     private val onDelete: (Int) -> Unit
00155 ) : RecyclerView.Adapter<CartAdapter.ViewHolder>() {
00161     class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
00162         val imgFood: ImageView = view.findViewById(R.id.imgFood)
00163         val tvName: TextView = view.findViewById(R.id.tvName)
00164         val tvPrice: TextView = view.findViewById(R.id.tvPrice)
00165         val tvQty: TextView = view.findViewById(R.id.tvQty)
00166         val btnPlus: TextView = view.findViewById(R.id.btnPlus)
00167         val btnMinus: TextView = view.findViewById(R.id.btnMinus)
00168         val cbSelect: CheckBox = view.findViewById(R.id.cbSelect)
00169         val btnDelete: ImageView = view.findViewById(R.id.btnDelete)
00170     }
00171
00179     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
00180         val view = LayoutInflater.from(parent.context).inflate(R.layout.item_cart, parent, false)
00181         return ViewHolder(view)
00182     }
00183
00193     override fun onBindViewHolder(holder: ViewHolder, position: Int) {
00194         val item = items[position]
00195         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00196         holder.tvName.text = item.name
00197         holder.tvPrice.text = fmt.format(item.price) + "đ"
00198         holder.tvQty.text = item.qty.toString()
00199         holder.cbSelect.isChecked = item.isSelected
00200         if (!item.imageUrl.isNullOrEmpty()) Picasso.get().load(item.imageUrl).into(holder.imgFood)
00201         holder.btnPlus.setOnClickListener { item.qty++; notifyItemChanged(holder.adapterPosition); onUpdate() }
00202         holder.btnMinus.setOnClickListener { if (item.qty > 1) { item.qty--; notifyItemChanged(holder.adapterPosition);
onUpdate() } }
00203         holder.cbSelect.setOnClickListener { item.isSelected = holder.cbSelect.isChecked; onUpdate() }
00204         holder.btnDelete.setOnClickListener { onDelete(holder.adapterPosition) }
00205     }
00211     override fun getItemCount() = items.size
00212 }

```

9.31 app/app/src/main/java/com/example/myapp/screens/CartItemAdapter.kt File Reference ↩

Packages

- package [CartItemAdapter](#)
- package [CartItemAdapter.CartItemViewHolder](#)

Functions

- class [CartItemAdapter.CartItemViewHolder.CartItemViewHolder](#) (itemView:android.view.View)

9.32 CartItemAdapter.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapplication.screens
00002
00009
00010 import android.view.LayoutInflater
00011 import android.view.ViewGroup
00012 import android.widget.ImageView
00013 import android.widget.TextView
00014 import androidx.recyclerview.widget.RecyclerView
00015 import com.example.myapplication.R
00016 import com.squareup.picasso.Picasso
00017 import java.text.NumberFormat
00018 import java.util.Locale
00019
00028 class CartItemAdapter(private val items: List<CartItem>) :
    RecyclerView.Adapter<CartItemAdapter.CartItemViewHolder>() {
00029
00035     class CartItemViewHolder(itemView: android.view.View) : RecyclerView.ViewHolder(itemView) {
00037         private val imgCartItem: ImageView = itemView.findViewById(R.id.imgCartItem)
00039         private val tvCartItemName: TextView = itemView.findViewById(R.id.tvCartItemName)
00041         private val tvCartItemQty: TextView = itemView.findViewById(R.id.tvCartItemQty)
00043         private val tvCartItemPrice: TextView = itemView.findViewById(R.id.tvCartItemPrice)
00044
00052         fun bind(cartItem: CartItem) {
00053             tvCartItemName.text = cartItem.name
00054             tvCartItemQty.text = "SL: ${cartItem.qty}"
00055
00056             val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00057             val totalPrice = cartItem.qty * cartItem.price
00058             tvCartItemPrice.text = "${fmt.format(totalPrice)}đ"
00059
00060             if (!cartItem.imageUrl.isNullOrEmpty()) {
00061                 Picasso.get()
00062                     .load(cartItem.imageUrl)
00063                     .placeholder(R.drawable.placeholder_loading)
00064                     .error(R.drawable.pngwing)
00065                     .into(imgCartItem)
00066             } else {
00067                 imgCartItem.setImageResource(R.drawable.pngwing)
00068             }
00069         }
00070     }
00071
00079     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): CartItemViewHolder {
00080         val itemView = LayoutInflater.from(parent.context)
00081             .inflate(R.layout.order_cart_item, parent, false)
00082         return CartItemViewHolder(itemView)
00083     }
00084
00091     override fun onBindViewHolder(holder: CartItemViewHolder, position: Int) {
00092         holder.bind(items[position])
00093     }
00094
00100     override fun getItemCount(): Int = items.size
00101 }
```

9.33 app/app/src/main/java/com/example/myapp/screens/chatbot.kt

File Reference

Packages

- package [AppCompatActivity](#)

Variables

- lateinit var [AppCompatActivity.recyclerView](#)
- lateinit var mình là trợ lý ảo của ứng dụng đặt món. Mình có thể giúp gì cho đơn hàng của bạn hôm nay [AppCompatActivity.nay](#)

9.34 chatbot.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.os.Bundle
00014 import android.widget.Toast
00015 import androidx.appcompat.app.AppCompatActivity
00016 import android.view.View
00017 import android.widget.EditText
00018 import android.widget.ImageView
00019 import androidx.recyclerview.widget.LinearLayoutManager
00020 import androidx.recyclerview.widget.RecyclerView
00021 import com.example.myapp.R
00022 import com.example.myapp.screens.adapters.ChatMessage
00023 import com.example.myapp.screens.adapters.ChatMessageAdapter
00024 import com.example.myapp.screens.api.RetrofitClient
00025 import com.example.myapp.screens.api.ChatMessageRequest
00026 import com.example.myapp.screens.api.ChatMessageResponse
00027 import retrofit2.Call
00028 import retrofit2.Callback
00029 import retrofit2.Response
00030
00043 class chatbot : AppCompatActivity() {
00044     private lateinit var recyclerView: RecyclerView
00045     private lateinit var adapter: ChatMessageAdapter
00046     private lateinit var edtMessage: EditText
00047     private var sessionId: Int? = null
00048
00056     override fun onCreate(savedInstanceState: Bundle?) {
00057         super.onCreate(savedInstanceState)
00058         setContentView(R.layout.chatbot)
00059
00060         // click btnBack
00061         val btnBack: ImageView = findViewById(R.id.btn_back)
00062         btnBack.setOnClickListener {
00063             finish() // quay lại màn hình trước (start)
00064         }
00065
00066         // Setup RecyclerView
00067         recyclerView = findViewById(R.id.chatRecyclerView)
00068         recyclerView.layoutManager = LinearLayoutManager(this).apply {
00069             stackFromEnd = true // Tin nhắn mới hiện ở dưới
00070         }
00071
00072         // Khởi tạo adapter với tin nhắn chào mừng
00073         val initialMessages = mutableListOf<ChatMessage>()
00074         ChatMessage(
00075             message = "Chào bạn, mình là trợ lý ảo của ứng dụng đặt món. Mình có thể giúp gì cho đơn hàng của bạn hôm nay?",
00076             isUser = false
00077         )
00078     )
00079     adapter = ChatMessageAdapter(initialMessages)
00080     recyclerView.adapter = adapter
00081
00082     // Setup input và send button
00083     edtMessage = findViewById(R.id.edtMessage)
00084     val btnSend: ImageView = findViewById(R.id.btnSend)
00085     btnSend.setOnClickListener {
00086         sendMessage()
00087     }
00088 }
00089
00098 private fun sendMessage() {
00099     val messageText = edtMessage.text.toString().trim()
00100     if (messageText.isEmpty()) {
00101         return
00102     }
00103
00104     // Thêm tin nhắn người dùng vào RecyclerView
00105     adapter.addMessage(ChatMessage(message = messageText, isUser = true))
00106     recyclerView.scrollToPosition(adapter.itemCount - 1)
00107
00108     // Xóa input
00109     edtMessage.text.clear()
00110
00111     // Gọi API chat
00112     val request = ChatMessageRequest(message = messageText)
00113     RetrofitClient.apiService.sendMessage(request, sessionId).enqueue(object : Callback<ChatMessageResponse> {
00114         override fun onResponse(call: Call<ChatMessageResponse>, response: Response<ChatMessageResponse>) {
00115             if (response.isSuccessful) {
00116                 val chatResponse = response.body()
00117                 chatResponse?.let {

```

```

00118         // Lưu session_id cho các tin nhắn tiếp theo
00119         if (sessionId == null) {
00120             sessionId = it.sessionid
00121         }
00122         // Thêm tin nhắn bot vào RecyclerView
00123         adapter.addMessage(ChatMessage(message = it.message, isUser = false))
00124         recyclerView.scrollToPosition(adapter.itemCount - 1)
00125     }
00126 } else {
00127     Toast.makeText(this@chatbot,
00128         "Lỗi: ${response.code()}", Toast.LENGTH_SHORT).show()
00129 }
00130 }
00131
00132 override fun onFailure(call: Call<ChatMessageResponse>, t: Throwable) {
00133     Toast.makeText(this@chatbot,
00134         "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00135 }
00136 })
00137 }
00138 }

```

9.35 app/app/src/main/java/com/example/myapp/screens/customer__support.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()

9.36 customer_support.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.widget.Toast
00015 import androidx.appcompat.app.AppCompatActivity
00016 import android.view.View
00017 import android.widget.ImageView
00018 import androidx.recyclerview.widget.LinearLayoutManager
00019 import androidx.recyclerview.widget.RecyclerView
00020 import com.example.myapp.R
00021 import com.example.myapp.screens.adapters.FAQAdapter
00022 import com.example.myapp.screens.api.RetrofitClient
00023 import com.example.myapp.screens.api.FAQ
00024 import retrofit2.Call
00025 import retrofit2.Callback
00026 import retrofit2.Response
00027
00037 class customer_support : AppCompatActivity() {
00038     private lateinit var recyclerView: RecyclerView
00039     private lateinit var adapter: FAQAdapter
00040
00047     override fun onCreate(savedInstanceState: Bundle?) {
00048         super.onCreate(savedInstanceState)
00049         setContentView(R.layout.customer_support)
00050
00051         // click btnBack
00052         val btnBack: ImageView = findViewById(R.id.btn_back)
00053         btnBack.setOnClickListener {
00054             finish() // quay lại màn hình trước (start)
00055         }
00056

```

```

00057 // Setup RecyclerView
00058 recyclerView = findViewById(R.id.faqRecyclerView)
00059 recyclerView.layoutManager = LinearLayoutManager(this)
00060
00061 // Load FAQs from API
00062 loadFAQs()
00063
00064 // click btn_chatbot
00065 val btnchatbot: View = findViewById(R.id.btnChatbot)
00066 btnchatbot.setOnClickListener {
00067     val intent = Intent(this, chatbot::class.java)
00068     startActivity(intent)
00069 }
00070 }
00071
00072 private fun loadFAQs() {
00073     RetrofitClient.apiService.getFAQs().enqueue(object : Callback<List<FAQ> {
00074         override fun onResponse(call: Call<List<FAQ>, response: Response<List<FAQ>) {
00075             if (response.isSuccessful) {
00076                 val faqs = response.body() ?: emptyList()
00077                 adapter = FAQAdapter(faqs)
00078                 recyclerView.adapter = adapter
00079             } else {
00080                 Toast.makeText(this@customer_support,
00081                     "Lỗi tải FAQ: ${response.code()}", Toast.LENGTH_SHORT).show()
00082             }
00083         }
00084     })
00085
00086     override fun onFailure(call: Call<List<FAQ>, t: Throwable) {
00087         Toast.makeText(this@customer_support,
00088             "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00089     }
00090 }
00091
00092 override fun onFailure(call: Call<List<FAQ>, t: Throwable) {
00093     Toast.makeText(this@customer_support,
00094         "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00095 }
00096 }
00097 }

```

9.37 app/app/src/main/java/com/example/myapp/screens/delivered_order_details kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- var [AppCompatActivity](#).if (orderId > 0)
- lateinit var chatbot::class.java [AppCompatActivity](#).startActivity (intent) } } private fun loadFAQs()
- fun [AppCompatActivity](#).bindOrder (order:OrderDetailResponse)
- fun [AppCompatActivity](#).reorderCurrentOrder ()

Variables

- var [AppCompatActivity](#).orderId

9.38 delivered_order_details kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00003
00004 import android.content.Intent
00005 import android.os.Bundle
00006 import android.widget.Button
00007 import android.widget.ImageView

```

```

00016 import android.widget.TextView
00017 import android.widget.Toast
00018 import androidx.appcompat.app.AppCompatActivity
00019 import com.example.myapplication.R
00020 import com.example.myapplication.screens.api.OrderDetailResponse
00021 import com.example.myapplication.screens.api.OrderCreateRequest
00022 import com.example.myapplication.screens.api.OrderItemRequest
00023 import com.example.myapplication.screens.api.OrderResponse
00024 import com.example.myapplication.screens.api.RetrofitClient
00025 import retrofit2.Call
00026 import retrofit2.Callback
00027 import retrofit2.Response
00028 import java.text.NumberFormat
00029 import java.text.SimpleDateFormat
00030 import java.util.Date
00031 import java.util.Locale
00032
00033
00044 class delivered_order_details : AppCompatActivity() {
00045     private var orderId: Int = -1
00046     private var loadedOrderDetail: OrderDetailResponse? = null
00047
00054     override fun onCreate(savedInstanceState: Bundle?) {
00055         super.onCreate(savedInstanceState)
00056         setContentView(R.layout.delivered_order_details)
00057
00058         orderId = intent.getIntExtra("order_id", -1)
00059         if (orderId > 0) {
00060             loadOrderDetail(orderId)
00061         }
00062
00063
00064         val btnBack: ImageView = findViewById(R.id.btn_back)
00065         btnBack.setOnClickListener {
00066             finish() // quay lại màn hình trước
00067         }
00068         // Tìm nút "Đặt lại" theo ID trong XML
00069         val btnReorder: Button = findViewById(R.id.btnReorder)
00070
00071
00072         // Thiết lập sự kiện click
00073         btnReorder.setOnClickListener {
00074             reorderCurrentOrder()
00075         }
00076         // click icProfile
00077         val icProfile: ImageView = findViewById(R.id.icProfile)
00078         icProfile.setOnClickListener {
00079             val intent = Intent(this, profile::class.java)
00080             startActivity(intent)
00081         }
00082
00083
00084         val icHome: ImageView = findViewById(R.id.icHome)
00085         icHome.setOnClickListener {
00086             val intent = Intent(this, home::class.java)
00087             startActivity(intent)
00088         }
00089         val btnCart: ImageView = findViewById(R.id.icCart)
00090         btnCart.setOnClickListener {
00091             val intent = Intent(this, cart::class.java)
00092             startActivity(intent)
00093         }
00094     }
00095
00104     private fun loadOrderDetail(orderId: Int) {
00105         RetrofitClient.apiService.getOrderDetail(orderId).enqueue(object : Callback<OrderDetailResponse> {
00106             override fun onResponse(call: Call<OrderDetailResponse>, response: Response<OrderDetailResponse>) {
00107                 if (!response.isSuccessful || response.body() == null) {
00108                     Toast.makeText(this@delivered_order_details, "Không tải được chi tiết đơn",
00109                         Toast.LENGTH_SHORT).show()
00110                     return
00111                 }
00112                 bindOrder(response.body()!!)
00113             }
00114             override fun onFailure(call: Call<OrderDetailResponse>, t: Throwable) {
00115                 Toast.makeText(this@delivered_order_details, "Lỗi kết nối", Toast.LENGTH_SHORT).show()
00116             }
00117         })
00118     }
00119
00128     private fun bindOrder(order: OrderDetailResponse) {
00129         loadedOrderDetail = order
00130         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00131         val title = findViewById<TextView>(R.id.tvTitle)
00132         val content = findViewById<TextView>(R.id.tvOrderContent)
00133

```

```

00134     val totalQuantity = order.order_items.sumOf { it.quantity }
00135     val itemSummary = if (order.order_items.isEmpty()) {
00136         "(Không có chi tiết món)"
00137     } else {
00138         order.order_items.joinToString("\n") {
00139             val itemName = it.menuitem_name ?: "Món #${it.menuitemid}"
00140             if (it.image_url.isNullOrBlank()) {
00141                 "- $itemName x${it.quantity}"
00142             } else {
00143                 "- $itemName x${it.quantity}"
00144             }
00145         }
00146     }
00147
00148     title.text = "Chi tiết đơn hàng #${order.id} ngày ${formatDate(order.createdat)}"
00149     content.text = "$itemSummary\nTổng số món: $totalQuantity\nĐịa chỉ: ${order.address_detail ?:"
N/A"}\nPhương thức thanh toán: Thanh toán online/cash\nVoucher: Không có\nThành tiền:
${fmt.format(order.totalprice)}đ"
00150 }
00151
00159 private fun reorderCurrentOrder() {
00160     val sourceOrder = loadedOrderDetail
00161     if (sourceOrder == null) {
00162         Toast.makeText(this, "Đơn hàng chưa tải xong", Toast.LENGTH_SHORT).show()
00163         return
00164     }
00165
00166     // Thay vì gọi API tạo đơn ngay, ta chuyển sang màn hình Thanh toán (Order)
00167     // Màn hình Order đã có sẵn logic nhận "reorder_order_id" để tự tải lại các món cũ
00168     val intent = Intent(this, order::class.java)
00169     intent.putExtra("reorder_order_id", sourceOrder.id)
00170     startActivity(intent)
00171 }
00172
00179 private fun formatDate(raw: String): String {
00180     return try {
00181         val parser = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", Locale.US)
00182         val parsed: Date = parser.parse(raw) ?: return raw
00183         SimpleDateFormat("dd/MM/yyyy", Locale("vi", "VN")).format(parsed)
00184     } catch (e: Exception) {
00185         raw.take(10)
00186     }
00187 }
00188 }
00189

```

9.39 app/app/src/main/java/com/example/myapp/screens/detail_support.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState: Bundle?)

9.40 detail_support.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import androidx.appcompat.app.AppCompatActivity
00015 import android.view.View
00016 import android.widget.ImageView
00017 import com.example.myapp.R

```

```

00018
00025 class detail_support : AppCompatActivity() {
00031     override fun onCreate(savedInstanceState: Bundle?) {
00032         super.onCreate(savedInstanceState)
00033         setContentView(R.layout.detailsupport)
00034
00035         //      click btnBack
00036         val btnBack: ImageView = findViewById(R.id.btn_back)
00037         btnBack.setOnClickListener {
00038             finish() // quay lại màn hình trước (start)
00039         }
00040
00041         //      click btn_chatbot
00042         val btnchatbot: View = findViewById(R.id.btnChatbot)
00043         btnchatbot.setOnClickListener {
00044             val intent = Intent(this, chatbot::class.java)
00045             startActivity(intent)
00046         }
00047     }
00048 }

```

9.41 app/app/src/main/java/com/example/myapp/screens/discouts.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- var [AppCompatActivity.selectedPromotion](#)

9.42 discouts.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import android.widget.Toast
00018 import androidx.appcompat.app.AppCompatActivity
00019 import com.example.myapp.R
00020 import com.example.myapp.screens.api.PromotionResponse
00021 import com.example.myapp.screens.api.RetrofitClient
00022 import retrofit2.Call
00023 import retrofit2.Callback
00024 import retrofit2.Response
00025
00038 class discouts : AppCompatActivity() {
00039     private var selectedPromotion: PromotionResponse? = null
00040
00048     override fun onCreate(savedInstanceState: Bundle?) {
00049         super.onCreate(savedInstanceState)
00050         setContentView(R.layout.discouts)
00051
00052         val icBack = findViewById<ImageView>(R.id.icBack)
00053         val icHome = findViewById<ImageView>(R.id.icHome)
00054         val icProfile = findViewById<ImageView>(R.id.icProfile)
00055         val icCart = findViewById<ImageView>(R.id.icCart)
00056         val tvVoucherDesc1 = findViewById<TextView>(R.id.tvVoucherDesc1)
00057         val tvSelect1 = findViewById<TextView>(R.id.tvSelect1)
00058
00059         icBack.setOnClickListener { finish() }
00060         icHome.setOnClickListener { startActivity(Intent(this, home::class.java)) }
00061         icProfile.setOnClickListener { startActivity(Intent(this, profile::class.java)) }
00062         icCart.setOnClickListener { startActivity(Intent(this, cart::class.java)) }

```



```

00063
00064     tvVoucherDesc1.text = "Đang tải voucher..."
00065     tvSelect1.isEnabled = false
00066     loadPromotions(tvVoucherDesc1, tvSelect1)
00067
00068     tvSelect1.setOnClickListener {
00069         val promotion = selectedPromotion ?: return@setOnClickListener
00070         val resultIntent = Intent()
00071         resultIntent.putExtra("promotion_id", promotion.id)
00072         resultIntent.putExtra("promotion_code", promotion.code)
00073         resultIntent.putExtra("discount_type", promotion.discounttype)
00074         resultIntent.putExtra("discount_value", promotion.discountvalue)
00075         setResult(RESULT_OK, resultIntent)
00076         finish()
00077     }
00078 }
00079
00090 private fun loadPromotions(tvVoucherDesc1: TextView, tvSelect1: TextView) {
00091     RetrofitClient.apiService.getPromotions().enqueue(object : Callback<List<PromotionResponse>» {
00092         override fun onResponse(call: Call<List<PromotionResponse>», response: Response<List<PromotionResponse>») {
00093             if (!response.isSuccessful || response.body().isEmpty()) {
00094                 tvVoucherDesc1.text = "Hiện chưa có voucher khả dụng"
00095                 tvSelect1.isEnabled = false
00096                 return
00097             }
00098
00099             val promotions = response.body()!!
00100             selectedPromotion = promotions.firstOrNull { it.status.equals("active", ignoreCase = true) } ?:
promotions.first()
00101             val promotion = selectedPromotion!!
00102             val discountText = if (promotion.discounttype.equals("percent", ignoreCase = true)) {
00103                 "${promotion.discountvalue}%"
00104             } else {
00105                 "${promotion.discountvalue}đ"
00106             }
00107
00108             tvVoucherDesc1.text = "${promotion.code}: Giảm $discountText - ĐH tối thiểu ${promotion.minordervalue}đ"
00109             tvSelect1.isEnabled = true
00110         }
00111
00112         override fun onFailure(call: Call<List<PromotionResponse>», t: Throwable) {
00113             tvVoucherDesc1.text = "Không tải được voucher"
00114             tvSelect1.isEnabled = false
00115             Toast.makeText(this@discouts, "Lỗi kết nối khi tải voucher", Toast.LENGTH_SHORT).show()
00116         }
00117     })
00118 }
00119 }

```

9.43 app/app/src/main/java/com/example/myapp/screens/edit_shipping_address.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var [AppCompatActivity.if](#) (addressId== -1)
- fun [AppCompatActivity.updateAddress](#) (phone:String, address:String)

9.44 edit_shipping_address.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00009
00010 import android.content.Context
00011 import android.content.Intent

```

```

00012 import android.os.Bundle
00013 import android.widget.Button
00014 import android.widget.EditText
00015 import android.widget.ImageView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import com.example.myapplication.R
00019 import com.example.myapplication.screens.api.Address
00020 import com.example.myapplication.screens.api.AddressUpdateRequest
00021 import com.example.myapplication.screens.api.ApiService
00022 import retrofit2.Call
00023 import retrofit2.Callback
00024 import retrofit2.Response
00025 import retrofit2.Retrofit
00026 import retrofit2.converter.gson.GsonConverterFactory
00027
00035 class edit_shipping_address : AppCompatActivity() {
00037     private lateinit var edtPhone: EditText
00039     private lateinit var edtAddress: EditText
00041     private lateinit var btnEdit: Button
00043     private var addressId: Int = -1
00044
00053     override fun onCreate(savedInstanceState: Bundle?) {
00054         super.onCreate(savedInstanceState)
00055         setContentView(R.layout.edit_shipping_address)
00056
00057         // Initialize views
00058         edtPhone = findViewById(R.id.edtPhone)
00059         edtAddress = findViewById(R.id.edtAddress)
00060         btnEdit = findViewById(R.id.btnAdd)
00061
00062         // Get address_id from intent
00063         addressId = intent.getIntExtra("address_id", -1)
00064         if (addressId == -1) {
00065             Toast.makeText(this, "Không tìm thấy địa chỉ", Toast.LENGTH_SHORT).show()
00066             finish()
00067             return
00068         }
00069
00070         // Load current address data
00071         loadAddressData()
00072
00073         val btnBack: ImageView = findViewById(R.id.btn_back)
00074         btnBack.setOnClickListener {
00075             finish() // quay lại màn hình trước
00076         }
00077
00078         val btnCart: ImageView = findViewById(R.id.icCart)
00079         btnCart.setOnClickListener {
00080             val intent = Intent(this, cart::class.java)
00081             startActivity(intent)
00082         }
00083         // Set up click listener for edit button
00084         btnEdit.setOnClickListener {
00085             val phone = edtPhone.text.toString().trim()
00086             val address = edtAddress.text.toString().trim()
00087
00088             // Validate input
00089             if (phone.isEmpty()) {
00090                 edtPhone.error = "Vui lòng nhập số điện thoại"
00091                 edtPhone.requestFocus()
00092                 return@setOnClickListener
00093             }
00094
00095             if (address.isEmpty()) {
00096                 edtAddress.error = "Vui lòng nhập địa chỉ"
00097                 edtAddress.requestFocus()
00098                 return@setOnClickListener
00099             }
00100
00101             // Call API to update address
00102             updateAddress(phone, address)
00103         }
00104
00105         // click icProfile
00106         val icProfile: ImageView = findViewById(R.id.icProfile)
00107         icProfile.setOnClickListener {
00108             val intent = Intent(this, profile::class.java)
00109             startActivity(intent)
00110         }
00111
00112         val icHome: ImageView = findViewById(R.id.icHome)
00113         icHome.setOnClickListener {
00114             val intent = Intent(this, home::class.java)
00115             startActivity(intent)
00116         }
00117     }

```

```

00118
00125 private fun loadAddressData() {
00126     val retrofit = Retrofit.Builder()
00127         .baseUrl("http://10.0.2.2:8000/") // Android emulator localhost
00128         .addConverterFactory(GsonConverterFactory.create())
00129         .build()
00130
00131     val apiService = retrofit.create(ApiService::class.java)
00132
00133     // Get userId from SharedPreferences
00134     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00135     val userId = sharedPref.getInt("user_id", 1)
00136
00137     apiService.getUserAddresses(userId).enqueue(object : Callback<List<Address>> {
00138         override fun onResponse(call: Call<List<Address>>, response: Response<List<Address>>) {
00139             if (response.isSuccessful) {
00140                 val addresses = response.body() ?: emptyList()
00141                 val address = addresses.find { it.id == addressId }
00142                 if (address != null) {
00143                     edtPhone.setText(address.phone ?: "")
00144                     edtAddress.setText(address.detail)
00145                 } else {
00146                     Toast.makeText(this@edit_shipping_address, "Không tìm thấy địa chỉ", Toast.LENGTH_SHORT).show()
00147                     finish()
00148                 }
00149             } else {
00150                 Toast.makeText(this@edit_shipping_address, "Không thể tải thông tin địa chỉ",
00151                     Toast.LENGTH_SHORT).show()
00152             }
00153
00154             override fun onFailure(call: Call<List<Address>>, t: Throwable) {
00155                 Toast.makeText(this@edit_shipping_address, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00156             }
00157         })
00158     }
00159
00169 private fun updateAddress(phone: String, address: String) {
00170     val retrofit = Retrofit.Builder()
00171         .baseUrl("http://10.0.2.2:8000/") // Android emulator localhost
00172         .addConverterFactory(GsonConverterFactory.create())
00173         .build()
00174
00175     val apiService = retrofit.create(ApiService::class.java)
00176
00177     val addressUpdateRequest = AddressUpdateRequest(
00178         detail = address,
00179         phone = phone
00180     )
00181
00182     apiService.updateAddress(addressId, addressUpdateRequest).enqueue(object : Callback<Address> {
00183         override fun onResponse(call: Call<Address>, response: Response<Address>) {
00184             if (response.isSuccessful) {
00185                 // Thành công - chuyển sang màn hình thông báo
00186                 Toast.makeText(this@edit_shipping_address, "Cập nhật địa chỉ thành công!",
00187                     Toast.LENGTH_SHORT).show()
00188                 val intent = Intent(this@edit_shipping_address, shipping_address_fixed_successfully::class.java)
00189                 startActivity(intent)
00190                 finish()
00191             } else {
00192                 Toast.makeText(this@edit_shipping_address, "Không thể cập nhật địa chỉ",
00193                     Toast.LENGTH_SHORT).show()
00194             }
00195
00196             override fun onFailure(call: Call<Address>, t: Throwable) {
00197                 Toast.makeText(this@edit_shipping_address, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00198             }
00199         })
00200     }

```

9.45 app/app/src/main/java/com/example/myapp/screens/food_detail.kt File Reference

Packages

- package [AppCompatActivity](#)

9.46 food_detail.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.content.Intent
00014 import android.net.Uri
00015 import android.os.Bundle
00016 import android.widget.ImageView
00017 import android.widget.LinearLayout
00018 import android.widget.TextView
00019 import android.widget.Toast
00020 import androidx.appcompat.app.AppCompatActivity
00021 import androidx.appcompat.widget.AppCompatButton
00022 import com.example.myapp.R
00023 import com.example.myapp.screens.api.RetrofitClient
00024 import com.example.myapp.screens.api.MenuItem
00025 import com.example.myapp.screens.api.ReviewDetail
00026 import com.squareup.picasso.Picasso
00027 import retrofit2.Call
00028 import retrofit2.Callback
00029 import retrofit2.Response
00030 import java.text.NumberFormat
00031 import java.util.Locale
00032
00049 class food_detail : AppCompatActivity() {
00050     private var qty = 1
00051     private var price = 0
00052     private var foodId = -1
00053     private var foodName = ""
00054     private var foodImageUrl = ""
00055     private var foodDescription = ""
00056     private var reviews: List<ReviewDetail>? = null
00057     private var avgRating: Float = 0f
00058
00066     override fun onCreate(savedInstanceState: Bundle?) {
00067         super.onCreate(savedInstanceState)
00068         setContentView(R.layout.activity_food_detail)
00069
00070         // Handle Deep Link
00071         val data: Uri? = intent.data
00072         if (data != null && data.host == "yourapp.com" && data.path?.startsWith("/food") == true) {
00073             val idStr = data.getQueryParameter("id")
00074             foodId = idStr?.toIntOrNull() ?: -1
00075             if (foodId != -1) {
00076                 fetchFoodDetails(foodId)
00077             }
00078         } else {
00079             // Get data from normal intent
00080             foodId = intent.getIntExtra("food_id", -1)
00081             foodName = intent.getStringExtra("food_name") ?: ""
00082             price = intent.getIntExtra("food_price", 0)
00083             foodDescription = intent.getStringExtra("food_description") ?: ""
00084             foodImageUrl = intent.getStringExtra("food_image_url") ?: ""
00085
00086             // Fetch full data with reviews
00087             if (foodId != -1) {
00088                 fetchFoodDetailsWithReviews(foodId)
00089             } else {
00090                 setupUI()
00091             }
00092         }
00093     }
00094
00103     private fun fetchFoodDetails(id: Int) {
00104         RetrofitClient.apiService.getMenuItemDetail(id).enqueue(object : Callback<MenuItem> {
00105             override fun onResponse(call: Call<MenuItem>, response: Response<MenuItem>) {
00106                 if (response.isSuccessful) {
00107                     val item = response.body()
00108                     if (item != null) {
00109                         foodId = item.id
00110                         foodName = item.name
00111                         price = item.price
00112                         foodDescription = item.description ?: ""
00113                         foodImageUrl = item.image_url ?: ""
00114                         reviews = item.reviews ?: emptyList()
00115                         avgRating = item.avg_rating ?: 0f
00116                         setupUI()
00117                     } else {
00118                         Toast.makeText(this@food_detail, "Không tìm thấy món ăn", Toast.LENGTH_SHORT).show()
00119                     }
00120                 }
00121             }
00122             override fun onFailure(call: Call<MenuItem>, t: Throwable) {

```

```

00123         Toast.makeText(this@food_detail, "Lỗi kết nối", Toast.LENGTH_SHORT).show()
00124     }
00125 })
00126 }
00127
00136 private fun fetchFoodDetailsWithReviews(id: Int) {
00137     RetrofitClient.apiService.getMenuItemDetail(id).enqueue(object : Callback<MenuItem> {
00138         override fun onResponse(call: Call<MenuItem>, response: Response<MenuItem>) {
00139             if (response.isSuccessful) {
00140                 val item = response.body()
00141                 if (item != null) {
00142                     foodId = item.id
00143                     foodName = item.name
00144                     price = item.price
00145                     foodDescription = item.description ?: ""
00146                     foodImageUrl = item.image_url ?: ""
00147                     reviews = item.reviews ?: emptyList()
00148                     avgRating = item.avg_rating ?: 0f
00149                     setupUI()
00150                 } else {
00151                     setupUI()
00152                 }
00153             }
00154         }
00155         override fun onFailure(call: Call<MenuItem>, t: Throwable) {
00156             // Fallback to setupUI with existing data
00157             setupUI()
00158         }
00159     })
00160 }
00161
00169 private fun setupUI() {
00170     val btnshare: ImageView = findViewById(R.id.btn_share)
00171     btnshare.setOnClickListener {
00172         val intent = Intent(this, share::class.java).apply {
00173             putExtra("food_id", foodId)
00174             putExtra("food_name", foodName)
00175             putExtra("food_price", price)
00176             putExtra("food_description", foodDescription)
00177             putExtra("food_image_url", foodImageUrl)
00178         }
00179         startActivity(intent)
00180     }
00181
00182     val tvQty: TextView = findViewById(R.id.tvQty)
00183     val tvTotal: TextView = findViewById(R.id.tvTotalValue)
00184     val tvFoodName: TextView = findViewById(R.id.tvFoodName)
00185     val tvUnitPrice: TextView = findViewById(R.id.tvUnitPrice)
00186     val tvDescription: TextView = findViewById(R.id.tvDescription)
00187     val imgFood: ImageView = findViewById(R.id.imgFood)
00188     val btnDecrease: TextView = findViewById(R.id.btnDecrease)
00189     val btnIncrease: TextView = findViewById(R.id.btnIncrease)
00190     val btnAddCart: AppCompatButton = findViewById(R.id.btnAddCart)
00191     val btnPreOrder: AppCompatButton = findViewById(R.id.btnPreOrder)
00192     val btnBack: ImageView = findViewById(R.id.btnBack)
00193
00194     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00195
00196     tvFoodName.text = foodName
00197     tvUnitPrice.text = fmt.format(price) + "đ"
00198     tvDescription.text = foodDescription
00199
00200     if (foodImageUrl.isNotEmpty()) {
00201         Picasso.get().load(foodImageUrl).placeholder(R.drawable.placeholder_loading).error(R.drawable.pngwing).into(imgFood)
00202     }
00203
00204     // Hiển thị đánh giá trung bình bằng RatingBar (hỗ trợ sao lẻ 4.3, 4.5...)
00205     val ratingBar: android.widget.RatingBar? = try { findViewById(R.id.ratingBar) } catch (e: Exception) { null }
00206     val tvRating: TextView? = try { findViewById(R.id.tvRating) } catch (e: Exception) { null }
00207
00208     if (avgRating > 0) {
00209         ratingBar?.visibility = android.view.View.VISIBLE
00210         ratingBar?.rating = avgRating
00211         tvRating?.text = String.format("%.1f", avgRating)
00212         tvRating?.visibility = android.view.View.VISIBLE
00213     } else {
00214         ratingBar?.visibility = android.view.View.GONE
00215         tvRating?.text = "Chưa có đánh giá"
00216     }
00217
00218     // Hiển thị reviews nếu có
00219     displayReviews()
00220
00221     fun refresh() {
00222         tvQty.text = qty.toString()
00223         val totalPrice = qty * price

```

```

00224         tvTotal.text = fmt.format(totalPrice) + "d"
00225     }
00226
00227     btnDecrease.setOnClickListener { if (qty > 1) { qty--; refresh() } }
00228     btnIncrease.setOnClickListener { qty++; refresh() }
00229     btnAddCart.setOnClickListener { addToCart() }
00230
00231     btnPreOrder.setOnClickListener {
00232         val intent = Intent(this, pre_order::class.java).apply {
00233             putExtra("food_id", foodId)
00234             putExtra("food_name", foodName)
00235             putExtra("food_price", price)
00236             putExtra("food_image_url", foodImageUrl)
00237         }
00238         startActivity(intent)
00239     }
00240
00241     btnBack.setOnClickListener { finish() }
00242     refresh()
00243 }
00244
00252 private fun displayReviews() {
00253     try {
00254         val reviewsContainer: LinearLayout? = findViewById(R.id.reviewsContainer)
00255         if (reviewsContainer == null || reviews.isNullOrEmpty()) return
00256
00257         reviewsContainer.removeAllViews()
00258
00259         reviews!!.take(10).forEach { review -> // Hiển thị tối đa 10 reviews cho phong phú
00260             val reviewView = android.widget.LinearLayout(this).apply {
00261                 val params = LinearLayout.LayoutParams(
00262                     LinearLayout.LayoutParams.MATCH_PARENT,
00263                     LinearLayout.LayoutParams.WRAP_CONTENT
00264                 )
00265                 params.setMargins(0, 0, 0, 16) // Thêm khoảng cách dưới mỗi review
00266                 layoutParams = params
00267
00268                 orientation = LinearLayout.VERTICAL
00269                 setPadding(24, 20, 24, 20) // Tăng padding bên trong cho thoáng
00270                 setBackgroundResource(R.drawable.white) // Sử dụng background trắng của app
00271                 elevation = 2f // Thêm chút bóng đổ cho sang trọng
00272             }
00273
00274             val reviewHeader = TextView(this).apply {
00275                 text = "${review.user_name ?: "Người dùng ẩn danh"}"
00276                 textSize = 14f
00277                 setTextColor(android.graphics.Color.BLACK)
00278                 setTypeface(null, android.graphics.Typeface.BOLD)
00279             }
00280
00281             val ratingBar = TextView(this).apply {
00282                 text = " ".repeat(review.rating)
00283                 textSize = 12f
00284                 layoutParams = LinearLayout.LayoutParams(
00285                     LinearLayout.LayoutParams.WRAP_CONTENT,
00286                     LinearLayout.LayoutParams.WRAP_CONTENT
00287                 ).apply { topMargin = 4 }
00288             }
00289
00290             val reviewComment = TextView(this).apply {
00291                 text = review.comment ?: "Không có nội dung nhận xét."
00292                 textSize = 14f
00293                 setTextColor(android.graphics.Color.DKGRAY)
00294                 layoutParams = LinearLayout.LayoutParams(
00295                     LinearLayout.LayoutParams.MATCH_PARENT,
00296                     LinearLayout.LayoutParams.WRAP_CONTENT
00297                 ).apply { topMargin = 8 }
00298             }
00299
00300             reviewView.addView(reviewHeader)
00301             reviewView.addView(ratingBar)
00302             reviewView.addView(reviewComment)
00303             reviewsContainer.addView(reviewView)
00304         }
00305     } catch (e: Exception) {
00306         // Nếu container không tồn tại, bỏ qua
00307     }
00308 }
00309
00317 private fun addToCart() {
00318     if (foodId == -1) {
00319         Toast.makeText(this, "Lỗi dữ liệu món ăn", Toast.LENGTH_SHORT).show()
00320         return
00321     }
00322     val existingItem = cart.cartList.find { it.id == foodId }
00323     if (existingItem != null) {
00324         existingItem.qty += qty

```

```

00325     } else {
00326         cart.cartList.add(CartItem(id = foodId, name = foodName, price = price, qty = qty, imageUrl = foodImageUrl,
isSelected = true))
00327     }
00328     Toast.makeText(this, "Đã thêm vào giỏ hàng", Toast.LENGTH_SHORT).show()
00329     finish()
00330 }
00331 }

```

9.47 app/app/src/main/java/com/example/myapp/screens/home.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()
- fun [AppCompatActivity.searchMenuItems](#) (query:String)
- fun [AppCompatActivity.searchByCategory](#) (query:String)
- fun [AppCompatActivity.displayMenuItems](#) ()

Variables

- var [AppCompatActivity.menuItems](#)

9.48 home.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.os.Handler
00015 import android.os.Looper
00016 import android.text.Editable
00017 import android.text.TextWatcher
00018 import android.view.View
00019 import android.view.inputmethod.InputMethodManager
00020 import android.widget.EditText
00021 import android.widget.ImageView
00022 import android.widget.TextView
00023 import android.widget.Toast
00024 import androidx.appcompat.app.AppCompatActivity
00025 import androidx.recyclerview.widget.LinearLayoutManager
00026 import androidx.recyclerview.widget.RecyclerView
00027 import com.example.myapp.R
00028 import com.example.myapp.adapters.MenuItemAdapter
00029 import com.example.myapp.screens.api.MenuItem
00030 import com.example.myapp.screens.api.RetrofitClient
00031 import com.example.myapp.screens.profile
00032 import retrofit2.Call
00033 import retrofit2.Callback
00034 import retrofit2.Response
00035
00044 class home : AppCompatActivity() {
00046     private var menuItems: List<MenuItem> = emptyList()
00048     private lateinit var rvMenuItems: RecyclerView
00050     private lateinit var menuItemAdapter: MenuItemAdapter
00052     private lateinit var edtSearch: EditText
00054     private var searchJob: Call<List<MenuItem>>? = null
00056     private val searchHandler = Handler(Looper.getMainLooper())

```

```

00058     private var searchRunnable: Runnable? = null
00060     private val SEARCH_DELAY = 500L // 500ms delay for debouncing
00061
00071     override fun onCreate(savedInstanceState: Bundle?) {
00072         super.onCreate(savedInstanceState)
00073         setContentView(R.layout.home)
00074
00075         // click btnProfile
00076         val btnProfile: ImageView = findViewById(R.id.icProfile)
00077         btnProfile.setOnClickListener {
00078             val intent = Intent(this, profile::class.java)
00079             startActivity(intent)
00080         }
00081
00082         // click textDanhSachNhaHang
00083         val tvDanhSachNhaHang: TextView = findViewById(R.id.tvDanhSachNhaHang)
00084         tvDanhSachNhaHang.setOnClickListener {
00085             val intent = Intent(this, list_restaurant::class.java)
00086             startActivity(intent)
00087         }
00088
00089         // click btnNotification
00090         val btnNotification: ImageView = findViewById(R.id.imgNotification)
00091         btnNotification.setOnClickListener {
00092             val intent = Intent(this, notification::class.java)
00093             startActivity(intent)
00094         }
00095
00096         val btnCart: ImageView = findViewById(R.id.icCart)
00097         btnCart.setOnClickListener {
00098             val intent = Intent(this, cart::class.java)
00099             startActivity(intent)
00100         }
00101
00102         // Setup RecyclerView
00103         rvMenuItems = findViewById(R.id.rvMenuItems)
00104         rvMenuItems.layoutManager = LinearLayoutManager(this, LinearLayoutManager.HORIZONTAL, false)
00105
00106         // Setup Search - search as user types with debouncing
00107         edtSearch = findViewById(R.id.edtSearch)
00108         edtSearch.addTextChangedListener(object : TextWatcher {
00109             override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
00110
00111             override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
00112
00113             override fun afterTextChanged(s: Editable?) {
00114                 // Cancel any pending search
00115                 searchRunnable?.let { searchHandler.removeCallbacks(it) }
00116
00117                 // Create new search runnable with delay
00118                 searchRunnable = Runnable {
00119                     val query = s?.toString()?.trim() ?: ""
00120                     if (query.isEmpty()) {
00121                         fetchMenuItems()
00122                     } else {
00123                         searchMenuItems(query)
00124                     }
00125                 }
00126
00127                 // Post the search with delay
00128                 searchHandler.postDelayed(searchRunnable!!, SEARCH_DELAY)
00129             }
00130         })
00131
00132         // Fetch menu items from API
00133         fetchMenuItems()
00134     }
00135
00143     private fun fetchMenuItems() {
00144         // Cancel any ongoing search
00145         searchJob?.cancel()
00146
00147         RetrofitClient.apiService.getAllMenuItems().enqueue(object : Callback<List<MenuItem>> {
00148             override fun onResponse(call: Call<List<MenuItem>>, response: Response<List<MenuItem>>) {
00149                 if (response.isSuccessful) {
00150                     menuItems = response.body() ?: emptyList()
00151                     displayMenuItems()
00152                 } else {
00153                     Toast.makeText(this@home, "Lỗi khi tải danh sách món ăn", Toast.LENGTH_SHORT).show()
00154                 }
00155             }
00156
00157             override fun onFailure(call: Call<List<MenuItem>>, t: Throwable) {
00158                 if (!call.isCanceled) {
00159                     Toast.makeText(this@home, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00160                 }
00161             }
00162         })

```



```

00162     })
00163 }
00164
00173 private fun searchMenuItems(query: String) {
00174     // Cancel any ongoing search
00175     searchJob?.cancel()
00176
00177     // First search by name
00178     searchJob = RetrofitClient.apiService.searchMenuItemsByName(query)
00179     searchJob?.enqueue(object : Callback<List<MenuItem>» {
00180         override fun onResponse(call: Call<List<MenuItem>», response: Response<List<MenuItem>») {
00181             if (response.isSuccessful) {
00182                 val results = response.body() ?: emptyList()
00183                 if (results.isNotEmpty()) {
00184                     menuItems = results
00185                     displayMenuItems()
00186                 } else {
00187                     // If no results by name, search by category
00188                     searchByCategory(query)
00189                 }
00190             } else {
00191                 Toast.makeText(this@home, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00192             }
00193         }
00194     })
00195
00196     override fun onFailure(call: Call<List<MenuItem>», t: Throwable) {
00197         if (!call.isCanceled) {
00198             Toast.makeText(this@home, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00199         }
00200     }
00201 })
00202 }
00211 private fun searchByCategory(query: String) {
00212     searchJob = RetrofitClient.apiService.searchMenuItemsByCategory(query)
00213     searchJob?.enqueue(object : Callback<List<MenuItem>» {
00214         override fun onResponse(call: Call<List<MenuItem>», response: Response<List<MenuItem>») {
00215             if (response.isSuccessful) {
00216                 val results = response.body() ?: emptyList()
00217                 menuItems = results
00218                 displayMenuItems()
00219                 if (results.isEmpty()) {
00220                     Toast.makeText(this@home, "Không tìm thấy món ăn", Toast.LENGTH_SHORT).show()
00221                 }
00222             } else {
00223                 Toast.makeText(this@home, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00224             }
00225         }
00226     })
00227
00228     override fun onFailure(call: Call<List<MenuItem>», t: Throwable) {
00229         if (!call.isCanceled) {
00230             Toast.makeText(this@home, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00231         }
00232     }
00233 })
00234 }
00241 private fun displayMenuItems() {
00242     menuItemAdapter = MenuItemAdapter(this, menuItems)
00243     rvMenuItems.adapter = menuItemAdapter
00244 }
00245 }

```

9.49 app/app/src/main/java/com/example/myapp/screens/list_restaurant.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()
- fun [AppCompatActivity.searchRestaurants](#) (query:String)
- fun [AppCompatActivity.displayRestaurants](#) ()

Variables

- var [AppCompatActivity.restaurants](#)

9.50 list_restaurant.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.os.Handler
00016 import android.os.Looper
00017 import android.text.Editable
00018 import android.text.TextWatcher
00019 import android.view.inputmethod.InputMethodManager
00020 import android.widget.EditText
00021 import android.widget.ImageView
00022 import android.widget.TextView
00023 import android.widget.Toast
00024 import androidx.appcompat.app.AppCompatActivity
00025 import androidx.recyclerview.widget.LinearLayoutManager
00026 import androidx.recyclerview.widget.RecyclerView
00027 import com.example.myapp.R
00028 import com.example.myapp.adapters.RestaurantAdapter
00029 import com.example.myapp.screens.api.RetrofitClient
00030 import com.example.myapp.screens.api.Restaurant
00031 import com.example.myapp.screens.profile
00032 import retrofit2.Call
00033 import retrofit2.Callback
00034 import retrofit2.Response
00035
00051 class list_restaurant: AppCompatActivity() {
00052     private var restaurants: List<Restaurant> = emptyList()
00053     private lateinit var rvRestaurants: RecyclerView
00054     private lateinit var restaurantAdapter: RestaurantAdapter
00055     private lateinit var edtSearch: EditText
00056     private var searchJob: Call<List<Restaurant>>? = null
00057     private val searchHandler = Handler(Looper.getMainLooper())
00058     private var searchRunnable: Runnable? = null
00059     private val SEARCH_DELAY = 500L // 500ms delay for debouncing
00060
00068     override fun onCreate(savedInstanceState: Bundle?) {
00069         super.onCreate(savedInstanceState)
00070         setContentView(R.layout.list_restaurant)
00071
00072         // click icProfile
00073         val icProfile: ImageView = findViewById(R.id.icProfile)
00074         icProfile.setOnClickListener {
00075             val intent = Intent(this, profile::class.java)
00076             startActivity(intent)
00077         }
00078         // click textMonAn
00079         val tvMonAn: TextView = findViewById(R.id.tvMonAn)
00080         tvMonAn.setOnClickListener {
00081             val intent = Intent(this, home::class.java)
00082             startActivity(intent)
00083         }
00084         // click icHome
00085         val icHome: ImageView = findViewById(R.id.icHome)
00086         icHome.setOnClickListener {
00087             val intent = Intent(this, home::class.java)
00088             startActivity(intent)
00089         }
00090
00091         val btnCart: ImageView = findViewById(R.id.icCart)
00092         btnCart.setOnClickListener {
00093             val intent = Intent(this, cart::class.java)
00094             startActivity(intent)
00095         }
00096
00097         // click imgNotification
00098         val imgNotification: ImageView = findViewById(R.id.imgNotification)
00099         imgNotification.setOnClickListener {
00100             val intent = Intent(this, notification::class.java)
00101             startActivity(intent)
00102         }
00103
00104         // Setup RecyclerView

```

```

00105     rvRestaurants = findViewById(R.id.rvRestaurants)
00106     rvRestaurants.layoutManager = LinearLayoutManager(this, LinearLayoutManager.HORIZONTAL, false)
00107
00108     // Setup Search - search as user types with debouncing
00109     edtSearch = findViewById(R.id.edtSearch)
00110     edtSearch.addTextChangedListener(object : TextWatcher {
00111         override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
00112
00113         override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
00114
00115         override fun afterTextChanged(s: Editable?) {
00116             // Cancel any pending search
00117             searchRunnable?.let { searchHandler.removeCallbacks(it) }
00118
00119             // Create new search runnable with delay
00120             searchRunnable = Runnable {
00121                 val query = s?.toString()?.trim() ?: ""
00122                 if (query.isEmpty()) {
00123                     fetchRestaurants()
00124                 } else {
00125                     searchRestaurants(query)
00126                 }
00127             }
00128
00129             // Post the search with delay
00130             searchHandler.postDelayed(searchRunnable!!, SEARCH_DELAY)
00131         }
00132     })
00133
00134     // Fetch restaurants from API
00135     fetchRestaurants()
00136 }
00137
00144 private fun fetchRestaurants() {
00145     // Cancel any ongoing search
00146     searchJob?.cancel()
00147
00148     RetrofitClient.apiService.getRestaurants().enqueue(object : Callback<List<Restaurant>» {
00149         override fun onResponse(call: Call<List<Restaurant>», response: Response<List<Restaurant>») {
00150             if (response.isSuccessful) {
00151                 restaurants = response.body() ?: emptyList()
00152                 displayRestaurants()
00153             } else {
00154                 Toast.makeText(this@list_restaurant, "Lỗi khi tải danh sách nhà hàng", Toast.LENGTH_SHORT).show()
00155             }
00156         }
00157
00158         override fun onFailure(call: Call<List<Restaurant>», t: Throwable) {
00159             if (!call.isCanceled) {
00160                 Toast.makeText(this@list_restaurant, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00161             }
00162         }
00163     })
00164 }
00165
00174 private fun searchRestaurants(query: String) {
00175     // Cancel any ongoing search
00176     searchJob?.cancel()
00177
00178     searchJob = RetrofitClient.apiService.searchRestaurantsByName(query)
00179     searchJob?.enqueue(object : Callback<List<Restaurant>» {
00180         override fun onResponse(call: Call<List<Restaurant>», response: Response<List<Restaurant>») {
00181             if (response.isSuccessful) {
00182                 val results = response.body() ?: emptyList()
00183                 restaurants = results
00184                 displayRestaurants()
00185                 if (results.isEmpty()) {
00186                     Toast.makeText(this@list_restaurant, "Không tìm thấy nhà hàng", Toast.LENGTH_SHORT).show()
00187                 }
00188             } else {
00189                 Toast.makeText(this@list_restaurant, "Lỗi khi tìm kiếm", Toast.LENGTH_SHORT).show()
00190             }
00191         }
00192
00193         override fun onFailure(call: Call<List<Restaurant>», t: Throwable) {
00194             if (!call.isCanceled) {
00195                 Toast.makeText(this@list_restaurant, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00196             }
00197         }
00198     })
00199 }
00200
00206 private fun displayRestaurants() {
00207     restaurantAdapter = RestaurantAdapter(this, restaurants)
00208     rvRestaurants.adapter = restaurantAdapter
00209 }
00210 }

```

9.51 app/app/src/main/java/com/example/myapp/screens/↵ notification.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- lateinit var [AppCompatActivity.rvNotifications](#)

9.52 notification.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapplication
00002
00011
00012 import android.content.Context
00013 import android.os.Bundle
00014 import android.view.View
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import androidx.appcompat.app.AppCompatActivity
00018 import androidx.recyclerview.widget.LinearLayoutManager
00019 import androidx.recyclerview.widget.RecyclerView
00020 import com.example.myapplication.R
00021 import com.example.myapplication.adapters.NotificationAdapter
00022 import com.example.myapplication.screens.api.Notification
00023 import com.example.myapplication.screens.api.RetrofitClient
00024 import retrofit2.Call
00025 import retrofit2.Callback
00026 import retrofit2.Response
00027
00037 class notification : AppCompatActivity() {
00038
00040     private lateinit var rvNotifications: RecyclerView
00042     private lateinit var tvEmpty: TextView
00044     private lateinit var adapter: NotificationAdapter
00046     private val notifications = mutableListOf<Notification>()
00047
00056     override fun onCreate(savedInstanceState: Bundle?) {
00057         super.onCreate(savedInstanceState)
00058         setContentView(R.layout.notification)
00059
00060         // Khởi tạo views
00061         rvNotifications = findViewById(R.id.rvNotifications)
00062         tvEmpty = findViewById(R.id.tvEmpty)
00063
00064         // Setup RecyclerView
00065         adapter = NotificationAdapter(notifications) { notification ->
00066             // Xử lý khi click vào thông báo
00067             markAsRead(notification)
00068         }
00069         rvNotifications.layoutManager = LinearLayoutManager(this)
00070         rvNotifications.adapter = adapter
00071
00072         // Click btnBack
00073         val btnBack: ImageView = findViewById(R.id.btn_back)
00074         btnBack.setOnClickListener {
00075             finish() // quay lại màn hình trước
00076         }
00077
00078         // Load thông báo
00079         loadNotifications()
00080     }
00081
00089     private fun loadNotifications() {
00090         // Lấy userId từ SharedPreferences
00091         val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00092         val userId = sharedPref.getInt("user_id", 1) // Mặc định là 1 nếu không tìm thấy
00093
00094         android.util.Log.d("Notification", "Loading notifications for userId: $userId")
00095         android.util.Log.d("Notification", "Base URL: ${RetrofitClient.apiService.javaClass.name}")

```

```

00096
00097 RetrofitClient.apiService.getUserNotifications(userId).enqueue(object : Callback<List<Notification> > {
00098     override fun onResponse(call: Call<List<Notification>, response: Response<List<Notification>) {
00099         android.util.Log.d("Notification", "Response code: ${response.code()}")
00100         android.util.Log.d("Notification", "Response body: ${response.body()}")
00101
00102         if (response.isSuccessful) {
00103             val notificationList = response.body() ?: emptyList()
00104             android.util.Log.d("Notification", "Notifications count: ${notificationList.size}")
00105
00106             notifications.clear()
00107             notifications.addAll(notificationList)
00108             adapter.notifyDataSetChanged()
00109
00110             // Hiển thị thông báo trống nếu không có thông báo
00111             if (notifications.isEmpty()) {
00112                 tvEmpty.text = "Không có thông báo"
00113                 tvEmpty.visibility = View.VISIBLE
00114                 rvNotifications.visibility = View.GONE
00115             } else {
00116                 tvEmpty.visibility = View.GONE
00117                 rvNotifications.visibility = View.VISIBLE
00118             }
00119         } else {
00120             android.util.Log.e("Notification", "Error: ${response.code()} - ${response.message()}")
00121             tvEmpty.text = "Lỗi tải thông báo: ${response.code()}"
00122             tvEmpty.visibility = View.VISIBLE
00123             rvNotifications.visibility = View.GONE
00124         }
00125     }
00126
00127     override fun onFailure(call: Call<List<Notification>, t: Throwable) {
00128         android.util.Log.e("Notification", "Failure: ${t.message}", t)
00129         tvEmpty.text = "Lỗi kết nối: ${t.message}"
00130         tvEmpty.visibility = View.VISIBLE
00131         rvNotifications.visibility = View.GONE
00132     }
00133 })
00134 }
00135
00145 private fun markAsRead(notification: Notification) {
00146     if (!notification.isread) {
00147         RetrofitClient.apiService.markNotificationAsRead(notification.id).enqueue(object : Callback<Void> {
00148             override fun onResponse(call: Call<Void>, response: Response<Void>) {
00149                 if (response.isSuccessful) {
00150                     // Cập nhật trạng thái trong danh sách
00151                     val index = notifications.indexOfFirst { it.id == notification.id }
00152                     if (index != -1) {
00153                         notifications[index] = notification.copy(isread = true)
00154                         adapter.notifyItemChanged(index)
00155                     }
00156                 }
00157             }
00158         })
00159         override fun onFailure(call: Call<Void>, t: Throwable) {
00160             // Xử lý lỗi nếu cần
00161         }
00162     })
00163 }
00164 }
00165 }

```

9.53 app/app/src/main/java/com/example/myapp/screens/order.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- var [AppCompatActivity.currentAddress](#)

9.54 order.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapplication.screens
00002
00011
00012 import android.content.Context
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import androidx.activity.result.contract.ActivityResultContracts
00016 import android.widget.ImageView
00017 import android.widget.TextView
00018 import android.widget.Toast
00019 import android.view.View
00020 import android.view.LayoutInflater
00021 import androidx.appcompat.app.AppCompatActivity
00022 import com.example.myapplication.R
00023 import com.example.myapplication.screens.api.Address
00024 import com.example.myapplication.screens.api.MenuItem
00025 import com.example.myapplication.screens.api.OrderCreateRequest
00026 import com.example.myapplication.screens.api.OrderItemRequest
00027 import com.example.myapplication.screens.api.OrderResponse
00028 import com.example.myapplication.screens.api.RetrofitClient
00029 import com.squareup.picasso.Picasso
00030 import retrofit2.Call
00031 import retrofit2.Callback
00032 import retrofit2.Response
00033 import java.text.NumberFormat
00034 import java.util.Locale
00035
00047 class order : AppCompatActivity() {
00049     private var currentAddress: Address? = null
00051     private var restaurantId: Int = 1
00053     private val selectedItems = mutableListOf<CartItem>()
00055     private var reorderSourceOrderId: Int? = null
00057     private var baseTotalPrice: Int = 0
00059     private var selectedPromotionCode: String? = null
00061     private var selectedDiscountType: String? = null
00063     private var selectedDiscountValue: Int = 0
00065     private var selectedPaymentMethod: String = "cod"
00067     private var selectedPaymentMethodLabel: String = "Thanh toán khi nhận hàng"
00068
00070     private val voucherPickerLauncher = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) {
00071         result ->
00072             if (result.resultCode != RESULT_OK || result.data == null) return@registerForActivityResult
00073             selectedPromotionCode = result.data?.getStringExtra("promotion_code")
00074             selectedDiscountType = result.data?.getStringExtra("discount_type")
00075             selectedDiscountValue = result.data?.getIntExtra("discount_value", 0) ?: 0
00076             bindVoucherAndTotal()
00077         }
00078
00080     private val paymentMethodPickerLauncher =
00081         registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
00082             if (result.resultCode != RESULT_OK || result.data == null) return@registerForActivityResult
00083             selectedPaymentMethod = result.data?.getStringExtra("payment_method")?: selectedPaymentMethod
00084             selectedPaymentMethodLabel = result.data?.getStringExtra("payment_method_label")?:
00085                 selectedPaymentMethodLabel
00086             bindPaymentMethod()
00087         }
00097     override fun onCreate(savedInstanceState: Bundle?) {
00098         super.onCreate(savedInstanceState)
00099         setContentView(R.layout.order)
00100
00101         reorderSourceOrderId = intent.getIntExtra("reorder_order_id", -1).takeIf { it > 0 }
00102
00103         if (reorderSourceOrderId != null) {
00104             bindCartPreview()
00105             bindPaymentMethod()
00106             loadReorderSourceOrder(reorderSourceOrderId!!)
00107         } else {
00108             selectedItems.clear()
00109
00110             // THÊM: Kiểm tra xem có nhận danh sách món ăn từ Intent (ví dụ từ Đặt trước) không
00111             val passedItems = intent.getSerializableExtra("selected_items") as? ArrayList<CartItem>
00112             if (passedItems != null) {
00113                 selectedItems.addAll(passedItems)
00114             } else {
00115                 // Nếu không có thì mới lấy từ giỏ hàng thường
00116                 selectedItems.addAll(cart.cartList.filter { it.isSelected })
00117                 if (selectedItems.isEmpty()) {
00118                     selectedItems.addAll(cart.cartList)
00119                 }
00120             }
00121         }
00122     }
00123 }

```

```

00120     }
00121
00122     bindCartPreview()
00123     bindPaymentMethod()
00124     resolveRestaurantId()
00125 }
00126 loadUserAddress()
00127
00128 val cardVoucher = findViewById<View>(R.id.cardVoucher)
00129 val cardPaymentMethod = findViewById<View>(R.id.cardPaymentMethod)
00130 val lblAddress = findViewById<TextView>(R.id.lblAddress)
00131 val valAddress = findViewById<TextView>(R.id.valAddress)
00132 val btnCancel = findViewById<View>(R.id.btnCancel)
00133 val btnOrder = findViewById<View>(R.id.btnOrder)
00134 val btnBack = findViewById<ImageView>(R.id.icBack)
00135 val icHome = findViewById<ImageView>(R.id.icHome)
00136 val icProfile = findViewById<ImageView>(R.id.icProfile)
00137 val icCart = findViewById<ImageView>(R.id.icCart)
00138
00139 cardVoucher.setOnClickListener {
00140     voucherPickerLauncher.launch(Intent(this, discounts::class.java))
00141 }
00142
00143 cardPaymentMethod.setOnClickListener {
00144     val paymentIntent = Intent(this, payment_methods::class.java)
00145     val discountAmount = calculateDiscountAmount(baseTotalPrice)
00146     val payableTotal = (baseTotalPrice - discountAmount).coerceAtLeast(0)
00147     paymentIntent.putExtra("order_total", payableTotal)
00148     paymentIntent.putExtra("select_mode", true)
00149     paymentIntent.putExtra("selected_payment_method", selectedPaymentMethod)
00150     paymentMethodPickerLauncher.launch(paymentIntent)
00151 }
00152
00153 lblAddress.setOnClickListener {
00154     startActivity(Intent(this, savedeliveryaddress::class.java))
00155 }
00156
00157 valAddress.setOnClickListener {
00158     startActivity(Intent(this, savedeliveryaddress::class.java))
00159 }
00160
00161 btnCancel.setOnClickListener {
00162     finish()
00163 }
00164
00165 btnOrder.setOnClickListener {
00166     createOrderFromCart()
00167 }
00168
00169 btnBack.setOnClickListener { finish() }
00170 icHome.setOnClickListener { startActivity(Intent(this, home::class.java)) }
00171 icProfile.setOnClickListener { startActivity(Intent(this, profile::class.java)) }
00172 icCart.setOnClickListener { startActivity(Intent(this, cart::class.java)) }
00173
00174 }
00175
00182 private fun bindCartPreview() {
00183     val tvTotal = findViewById<TextView>(R.id.valTotal)
00184     val tvRestaurantName = findViewById<TextView>(R.id.tvRestaurantName)
00185     val tvAddress = findViewById<TextView>(R.id.valAddress)
00186     val cartItemsContainer = findViewById<android.widget.LinearLayout>(R.id.cartItemsContainer)
00187
00188     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00189     baseTotalPrice = selectedItems.sumOf { it.qty * it.price }
00190
00191     cartItemsContainer.removeAllViews()
00192
00193     if (selectedItems.isEmpty()) {
00194         val emptyView = android.widget.TextView(this).apply {
00195             text = "Giỏ hàng trống"
00196             textSize = 16f
00197             setTextColor(android.graphics.Color.GRAY)
00198             gravity = android.view.Gravity.CENTER
00199             setPadding(16, 32, 16, 32)
00200         }
00201         cartItemsContainer.addView(emptyView)
00202         tvTotal.text = "0đ"
00203         tvRestaurantName.text = "Giỏ hàng"
00204         tvAddress.text = "Chưa có địa chỉ"
00205         bindVoucherAndTotal()
00206         return
00207     }
00208
00209     // Display ALL items
00210     for (item in selectedItems) {
00211         val itemView = android.view.LayoutInflater.from(this)
00212             .inflate(R.layout.order_cart_item, null, false)

```

```

00213
00214     val imgCartItem = itemView.findViewById<ImageView>(R.id.imgCartItem)
00215     val tvCartItemName = itemView.findViewById<TextView>(R.id.tvCartItemName)
00216     val tvCartItemQty = itemView.findViewById<TextView>(R.id.tvCartItemQty)
00217     val tvCartItemUnitPrice = itemView.findViewById<TextView>(R.id.tvCartItemUnitPrice)
00218     val tvCartItemPrice = itemView.findViewById<TextView>(R.id.tvCartItemPrice)
00219
00220     tvCartItemName.text = item.name
00221     tvCartItemQty.text = "Số lượng: ${item.qty}"
00222     tvCartItemUnitPrice.text = "Đơn giá: ${fmt.format(item.price)}đ"
00223     tvCartItemPrice.text = "Thành tiền: ${fmt.format(item.qty * item.price)}đ"
00224
00225     if (item.imageUrl.isNullOrEmpty()) {
00226         Picasso.get()
00227             .load(item.imageUrl)
00228             .placeholder(R.drawable.placeholder_loading)
00229             .error(R.drawable.pngwing)
00230             .into(imgCartItem)
00231     } else {
00232         imgCartItem.setImageResource(R.drawable.pngwing)
00233     }
00234
00235     itemView.layoutParams = android.widget.LinearLayout.LayoutParams(
00236         android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
00237         android.widget.LinearLayout.LayoutParams.WRAP_CONTENT
00238     ).apply {
00239         bottomMargin = 12
00240     }
00241
00242     cartItemsContainer.addView(itemView)
00243 }
00244
00245 tvRestaurantName.text = "Quán hàng"
00246 tvAddress.text = "Đang tải..."
00247 tvTotal.text = "${fmt.format(baseTotalPrice)}đ"
00248 bindVoucherAndTotal()
00249 }
00250
00251 private fun bindVoucherAndTotal() {
00252     val voucherValueText = findViewById<TextView>(R.id.tvVoucherValue)
00253     val tvTotal = findViewById<TextView>(R.id.valTotal)
00254     val discountAmount = calculateDiscountAmount(baseTotalPrice)
00255     val payableTotal = (baseTotalPrice - discountAmount).coerceAtLeast(0)
00256     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00257
00258     voucherValueText.text = if (!selectedPromotionCode.isNullOrEmpty() && selectedDiscountValue > 0) {
00259         "$selectedPromotionCode (-${fmt.format(discountAmount)}đ)"
00260     } else {
00261         "Chọn >"
00262     }
00263
00264     tvTotal.text = "${fmt.format(payableTotal)}đ"
00265 }
00266
00267 private fun bindPaymentMethod() {
00268     findViewById<TextView>(R.id.tvPaymentMethodValue).text = selectedPaymentMethodLabel
00269 }
00270
00271 private fun calculateDiscountAmount(total: Int): Int {
00272     if (selectedDiscountValue <= 0) return 0
00273     return if ((selectedDiscountType?.equals("percent", ignoreCase = true)) {
00274         ((total * selectedDiscountValue) / 100.0).toInt()
00275     } else {
00276         selectedDiscountValue
00277     })
00278 }
00279
00280 private fun loadUserAddress() {
00281     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00282     val userId = sharedPref.getInt("user_id", -1)
00283     if (userId <= 0) {
00284         findViewById<TextView>(R.id.valAddress).text = "Chưa đăng nhập"
00285         return
00286     }
00287
00288     RetrofitClient.apiService.getUserAddresses(userId).enqueue(object : Callback<List<Address>> {
00289         override fun onResponse(call: Call<List<Address>>, response: Response<List<Address>>) {
00290             if (!response.isSuccessful || response.body().isNullOrEmpty()) {
00291                 findViewById<TextView>(R.id.valAddress).text = "Chưa có địa chỉ"
00292                 return
00293             }
00294             currentAddress = response.body()!!.first()
00295             findViewById<TextView>(R.id.valAddress).text = currentAddress!!.detail
00296         }
00297     })
00298
00299     override fun onFailure(call: Call<List<Address>>, t: Throwable) {
00300         findViewById<TextView>(R.id.valAddress).text = "Không tải được địa chỉ"
00301     }
00302 }

```



```

00321     }
00322     })
00323 }
00324
00331 private fun resolveRestaurantId() {
00332     if (reorderSourceOrderId != null) return
00333     val menuIds = selectedItems.map { it.id }.toSet()
00334     if (menuIds.isEmpty()) return
00335
00336     RetrofitClient.apiService.getAllMenuItems().enqueue(object : Callback<List<MenuItem>» {
00337         override fun onResponse(call: Call<List<MenuItem>», response: Response<List<MenuItem>») {
00338             if (!response.isSuccessful || response.body().isEmptyOrNull()) return
00339             val firstMatched = response.body()!!.firstOrNull { it.id in menuIds }
00340             if (firstMatched != null) {
00341                 restaurantId = firstMatched.restaurantid
00342                 findViewById<TextView>(R.id.tvRestaurantName).text = firstMatched.restaurant_name ?: "Nhà hàng"
00343                 #${firstMatched.restaurantid}"
00344             }
00345
00346             override fun onFailure(call: Call<List<MenuItem>», t: Throwable) {
00347             }
00348         })
00349     }
00350
00359 private fun loadReorderSourceOrder(orderId: Int) {
00360     RetrofitClient.apiService.getOrderDetail(orderId).enqueue(object :
00361     Callback<com.example.myapp.screens.api.OrderDetailResponse> {
00362         override fun onResponse(call: Call<com.example.myapp.screens.api.OrderDetailResponse>, response:
00363         Response<com.example.myapp.screens.api.OrderDetailResponse>) {
00364             if (!response.isSuccessful || response.body() == null) {
00365                 Toast.makeText(this@order, "Không tải được đơn để đặt lại", Toast.LENGTH_SHORT).show()
00366                 return
00367             }
00368             val sourceOrder = response.body()!!
00369             selectedItems.clear()
00370             selectedItems.addAll(
00371                 sourceOrder.order_items.map { item ->
00372                     CartItem(
00373                         id = item.menuitemid,
00374                         name = item.menuitem_name ?: "Món #${item.menuitemid}",
00375                         price = item.price,
00376                         qty = item.quantity,
00377                         imageUrl = item.image_url
00378                     )
00379                 }
00380             )
00381             restaurantId = sourceOrder.restaurantid
00382             findViewById<TextView>(R.id.tvRestaurantName).text = sourceOrder.restaurant_name ?: "Nhà hàng"
00383             #${sourceOrder.restaurantid}"
00384             bindCartPreview()
00385
00386             override fun onFailure(call: Call<com.example.myapp.screens.api.OrderDetailResponse>, t: Throwable) {
00387                 Toast.makeText(this@order, "Lỗi mạng khi tải đơn đặt lại", Toast.LENGTH_SHORT).show()
00388             }
00389         })
00390     }
00391
00399 private fun createOrderFromCart() {
00400     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00401     val userId = sharedPref.getInt("user_id", -1)
00402
00403     if (userId <= 0) {
00404         Toast.makeText(this, "Bạn cần đăng nhập lại", Toast.LENGTH_SHORT).show()
00405         return
00406     }
00407
00408     val address = currentAddress
00409     if (address == null) {
00410         Toast.makeText(this, "Vui lòng thêm địa chỉ giao hàng", Toast.LENGTH_SHORT).show()
00411         startActivity(Intent(this, savedeliveryaddress::class.java))
00412         return
00413     }
00414
00415     if (selectedItems.isEmpty()) {
00416         Toast.makeText(this, "Giỏ hàng đang trống", Toast.LENGTH_SHORT).show()
00417         return
00418     }
00419
00420     val totalPrice = selectedItems.sumOf { it.qty * it.price }
00421     val orderItems = selectedItems.map {
00422         OrderItemRequest(quantity = it.qty, price = it.price, menuitemid = it.id)
00423     }
00424
00425     val request = OrderCreateRequest(

```

```

00426         status = "pending",
00427         totalprice = totalPrice,
00428         restaurantid = restaurantId,
00429         addressid = address.id,
00430         userid = userId,
00431         order_items = orderItems
00432     )
00433
00434     RetrofitClient.apiService.createOrder(request).enqueue(object : Callback<OrderResponse> {
00435         override fun onResponse(call: Call<OrderResponse>, response: Response<OrderResponse>) {
00436             if (!response.isSuccessful || response.body() == null) {
00437                 Toast.makeText(this@order, "Không thể tạo đơn hàng", Toast.LENGTH_SHORT).show()
00438                 return
00439             }
00440
00441             val createdOrder = response.body()!!
00442             cart.cartList.removeAll(selectedItems.toSet())
00443             selectedItems.clear()
00444             val discountAmount = calculateDiscountAmount(totalPrice)
00445             val payableTotal = (totalPrice - discountAmount).coerceAtLeast(0)
00446
00447             if (selectedPaymentMethod.equals("online", ignoreCase = true)) {
00448                 val intent = Intent(this@order, payment_methods::class.java)
00449                 intent.putExtra("order_id", createdOrder.id)
00450                 intent.putExtra("order_total", payableTotal)
00451                 intent.putExtra("select_mode", false)
00452                 intent.putExtra("auto_pay_now", true)
00453                 startActivity(intent)
00454             } else {
00455                 val intent = Intent(this@order, payment_successful::class.java)
00456                 intent.putExtra("order_id", createdOrder.id)
00457                 startActivity(intent)
00458             }
00459         }
00460
00461         override fun onFailure(call: Call<OrderResponse>, t: Throwable) {
00462             Toast.makeText(this@order, "Lỗi mạng khi tạo đơn", Toast.LENGTH_SHORT).show()
00463         }
00464     })
00465 }
00466 }
00467 }

```

9.55 app/app/src/main/java/com/example/myapp/screens/order_history.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()
- fun [AppCompatActivity.bindTopTwoOrders](#) (orders:List< OrderResponse >)
- fun [AppCompatActivity.openOrderDetail](#) (orderId:Int?)

Variables

- var [AppCompatActivity.firstOrderId](#)

9.56 order_history.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Context
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import android.widget.Toast
00018 import androidx.appcompat.app.AppCompatActivity
00019 import com.example.myapp.R
00020 import com.example.myapp.screens.api.OrderResponse
00021 import com.example.myapp.screens.api.RetrofitClient
00022 import retrofit2.Call
00023 import retrofit2.Callback
00024 import retrofit2.Response
00025 import java.text.SimpleDateFormat
00026 import java.util.Date
00027 import java.util.Locale
00028
00038 class order_history : AppCompatActivity() {
00040     private var firstOrderId: Int? = null
00042     private var secondOrderId: Int? = null
00043
00052     override fun onCreate(savedInstanceState: Bundle?) {
00053         super.onCreate(savedInstanceState)
00054         setContentView(R.layout.order_history)
00055
00056         loadOrders()
00057
00058
00059         val btnBack: ImageView = findViewById(R.id.btn_back)
00060         btnBack.setOnClickListener {
00061             finish() // quay lại màn hình trước
00062         }
00063
00064         val btnCart: ImageView = findViewById(R.id.icCart)
00065         btnCart.setOnClickListener {
00066             val intent = Intent(this, cart::class.java)
00067             startActivity(intent)
00068         }
00069         val tvViewDetails1: TextView = findViewById(R.id.tvViewDetails1)
00070         tvViewDetails1.setOnClickListener {
00071             openOrderDetail(firstOrderId)
00072         }
00073
00074         val tvViewDetails2: TextView = findViewById(R.id.tvViewDetails2)
00075         tvViewDetails2.setOnClickListener {
00076             openOrderDetail(secondOrderId)
00077         }
00078         // click icProfile
00079         val icProfile: ImageView = findViewById(R.id.icProfile)
00080         icProfile.setOnClickListener {
00081             val intent = Intent(this, profile::class.java)
00082             startActivity(intent)
00083         }
00084
00085         val icHome: ImageView = findViewById(R.id.icHome)
00086         icHome.setOnClickListener {
00087             val intent = Intent(this, home::class.java)
00088             startActivity(intent)
00089         }
00090     }
00091
00098     private fun loadOrders() {
00099         val userId = resolveUserIdForHistory()
00100
00101         RetrofitClient.apiService.getUserOrders(userId).enqueue(object : Callback<List<OrderResponse> {
00102             override fun onResponse(call: Call<List<OrderResponse>, response: Response<List<OrderResponse>) {
00103                 if (!response.isSuccessful || response.body().isEmpty()) {
00104                     bindTopTwoOrders(emptyList())
00105                     return
00106                 }
00107                 val sorted = response.body()!!
00108                     .filter { it.status == "completed" }
00109                     .sortedByDescending { it.createdAt }
00110                 bindTopTwoOrders(sorted)
00111             }
00112         })
00113
00114         override fun onFailure(call: Call<List<OrderResponse>, t: Throwable) {
00115             Toast.makeText(this@order_history, "Không tải được lịch sử đơn", Toast.LENGTH_SHORT).show()
00116         }
00117     }

```

```

00116     })
00117   }
00118
00128   private fun bindTopTwoOrders(orders: List<OrderResponse>) {
00129       val tvDate1: TextView = findViewById(R.id.tvDate1)
00130       val tvDate2: TextView = findViewById(R.id.tvDate2)
00131       val tvViewDetails1: TextView = findViewById(R.id.tvViewDetails1)
00132       val tvViewDetails2: TextView = findViewById(R.id.tvViewDetails2)
00133
00134       val first = orders.getOrNull(0)
00135       val second = orders.getOrNull(1)
00136
00137       firstOrderId = first?.id
00138       secondOrderId = second?.id
00139
00140       tvDate1.text = first?.let { "${formatDate(it.createdat)} - #${it.id} - ${it.status}" } ?: "Chưa có đơn hàng"
00141       tvDate2.text = second?.let { "${formatDate(it.createdat)} - #${it.id} - ${it.status}" } ?: "Chưa có đơn hàng"
00142
00143       tvViewDetails1.alpha = if (first != null) 1f else 0.4f
00144       tvViewDetails2.alpha = if (second != null) 1f else 0.4f
00145   }
00146
00152   private fun openOrderDetail(orderId: Int?) {
00153       if (orderId == null) {
00154           Toast.makeText(this, "Không có đơn để xem", Toast.LENGTH_SHORT).show()
00155           return
00156       }
00157       val intent = Intent(this, delivered_order_details::class.java)
00158       intent.putExtra("order_id", orderId)
00159       startActivity(intent)
00160   }
00161
00170   private fun resolveUserIdForHistory(): Int {
00171       val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00172       val userId = sharedPref.getInt("user_id", -1)
00173       if (userId > 0) return userId
00174
00175       val userName = sharedPref.getString("user_name", "") ?: ""
00176       if (userName.contains("Trung", ignoreCase = true) || userName.isBlank()) {
00177           return 1
00178       }
00179       return 1
00180   }
00181
00188   private fun formatDate(raw: String): String {
00189       return try {
00190           val parser = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", Locale.US)
00191           val parsed: Date = parser.parse(raw) ?: return raw
00192           SimpleDateFormat("dd/MM/yyyy", Locale("vi", "VN")).format(parsed)
00193       } catch (e: Exception) {
00194           raw.take(10)
00195       }
00196   }
00197 }

```

9.57 app/app/src/main/java/com/example/myapp/screens/OrderTracking.kt File Reference

Packages

- package [AppCompatActivity](#)

9.58 OrderTracking.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Context
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import androidx.appcompat.app.AppCompatActivity
00016 import android.widget.ImageView

```

```

00017 import android.widget.TextView
00018 import android.widget.Toast
00019 import android.view.LayoutInflater
00020 import com.example.myapp.R
00021 import com.example.myapp.screens.api.OrderDetailResponse
00022 import com.example.myapp.screens.api.OrderResponse
00023 import com.example.myapp.screens.api.RetrofitClient
00024 import com.example.myapp.screens.api.UserProfileSummary
00025 import retrofit2.Call
00026 import retrofit2.Callback
00027 import retrofit2.Response
00028 import java.text.NumberFormat
00029 import java.util.Locale
00030
00042 class OrderTracking : AppCompatActivity() {
00044     private var allPendingOrders = listOf<OrderDetailResponse>()
00046     private var allCompletedOrders = listOf<OrderDetailResponse>()
00048     private var displayedPendingCount = 0
00050     private var displayedCompletedCount = 0
00052     private val itemsPerPage = 5
00053
00055     private lateinit var ordersContainer: android.widget.LinearLayout
00057     private lateinit var btnLoadMore: android.widget.Button
00059     private lateinit var scrollViewOrders: android.widget.ScrollView
00061     private var isAutoLoading = false
00062
00071     override fun onCreate(savedInstanceState: Bundle?) {
00072         super.onCreate(savedInstanceState)
00073         setContentView(R.layout.order_tracking)
00074
00075         ordersContainer = findViewById(R.id.ordersContainer)
00076         btnLoadMore = findViewById(R.id.btnLoadMore)
00077         scrollViewOrders = findViewById(R.id.scrollViewOrders)
00078
00079         loadPointsSummary()
00080         loadAllOrders()
00081
00082         // Auto-load more orders when scrolling to the bottom
00083         scrollViewOrders.setOnScrollChangeListener { v, __, scrollY, __, __ ->
00084             val scrollView = v as android.widget.ScrollView
00085             val child = scrollView.getChildAt(0)
00086             if (child != null) {
00087                 // Check if we reached the bottom (with 200px buffer)
00088                 if (scrollY + scrollView.height >= child.height - 200) {
00089                     if (!isAutoLoading) {
00090                         isAutoLoading = true
00091                         loadMoreOrders()
00092                         // Reset flag after a delay to prevent multiple triggers
00093                         scrollView.postDelayed({ isAutoLoading = false }, 1000)
00094                     }
00095                 }
00096             }
00097         }
00098
00099         // Back Button - Return to previous screen
00100         val btnBack: ImageView = findViewById(R.id.btn_back)
00101         btnBack.setOnClickListener {
00102             finish()
00103         }
00104
00105         // Bottom Navigation - Home
00106         val icHome: ImageView = findViewById(R.id.icHome)
00107         icHome.setOnClickListener {
00108             startActivity(Intent(this, home::class.java))
00109         }
00110
00111         // Bottom Navigation - Wishlist
00112         val icHeart: ImageView = findViewById(R.id.icHeart)
00113         icHeart.setOnClickListener {
00114             android.util.Log.d("OrderTracking", "Wishlist clicked")
00115         }
00116
00117         // Bottom Navigation - Cart
00118         val icCart: ImageView = findViewById(R.id.icCart)
00119         icCart.setOnClickListener {
00120             startActivity(Intent(this, activity_cart::class.java))
00121         }
00122
00123         // Bottom Navigation - Profile
00124         val icProfile: ImageView = findViewById(R.id.icProfile)
00125         icProfile.setOnClickListener {
00126             startActivity(Intent(this, profile::class.java))
00127         }
00128
00129         // Load More Button
00130         btnLoadMore.setOnClickListener {
00131             loadMoreOrders()

```

```

00132     }
00133 }
00134
00141 private fun loadPointsSummary() {
00142     val userId = resolveUserIdForTracking()
00143     val pointsView = findViewById<TextView>(R.id.tvPointsSummary)
00144     RetrofitClient.apiService.getProfileSummary(userId).enqueue(object : Callback<UserProfileSummary> {
00145         override fun onResponse(call: Call<UserProfileSummary>, response: Response<UserProfileSummary>) {
00146             if (response.isSuccessful) {
00147                 pointsView.text = "Điểm tích lũy: ${response.body()?.points ?: 0}"
00148             }
00149         }
00150     })
00151     override fun onFailure(call: Call<UserProfileSummary>, t: Throwable) {
00152         pointsView.text = "Điểm tích lũy: 0"
00153     }
00154 }
00155 }
00156
00163 private fun loadAllOrders() {
00164     val userId = resolveUserIdForTracking()
00165     RetrofitClient.apiService.getUserOrders(userId).enqueue(object : Callback<List<OrderResponse>> {
00166         override fun onResponse(call: Call<List<OrderResponse>>, response: Response<List<OrderResponse>>) {
00167             if (!response.isSuccessful || response.body().isEmpty()) {
00168                 Toast.makeText(this@OrderTracking, "Không có đơn hàng", Toast.LENGTH_SHORT).show()
00169                 btnLoadMore.visibility = android.view.View.GONE
00170                 return
00171             }
00172
00173             val allOrders = response.body()!!.sortedByDescending { it.createdat }
00174
00175             // Tách riêng pending và completed
00176             val pendingOrderIds = allOrders.filter { !isCompletedStatus(it.status) }.map { it.id }
00177             val completedOrderIds = allOrders.filter { isCompletedStatus(it.status) }.map { it.id }
00178
00179             // Load chi tiết từng order
00180             loadOrdersDetail(pendingOrderIds, completedOrderIds)
00181         }
00182     })
00183     override fun onFailure(call: Call<List<OrderResponse>>, t: Throwable) {
00184         Toast.makeText(this@OrderTracking, "Không tải được đơn hàng", Toast.LENGTH_SHORT).show()
00185         btnLoadMore.visibility = android.view.View.GONE
00186     }
00187 }
00188 }
00189
00199 private fun loadOrdersDetail(pendingOrderIds: List<Int>, completedOrderIds: List<Int>) {
00200     val pending = mutableListOf<OrderDetailResponse>()
00201     val completed = mutableListOf<OrderDetailResponse>()
00202     var completed_count = 0
00203     val total_count = pendingOrderIds.size + completedOrderIds.size
00204
00205     // Load pending orders
00206     for (orderId in pendingOrderIds) {
00207         RetrofitClient.apiService.getOrderDetail(orderId).enqueue(object : Callback<OrderDetailResponse> {
00208             override fun onResponse(call: Call<OrderDetailResponse>, response: Response<OrderDetailResponse>) {
00209                 if (response.isSuccessful && response.body() != null) {
00210                     pending.add(response.body()!!)
00211                 }
00212                 completed_count++
00213                 if (completed_count == total_count) {
00214                     bindAllOrders(pending, completed)
00215                 }
00216             }
00217         })
00218         override fun onFailure(call: Call<OrderDetailResponse>, t: Throwable) {
00219             completed_count++
00220             if (completed_count == total_count) {
00221                 bindAllOrders(pending, completed)
00222             }
00223         }
00224     })
00225 }
00226
00227 // Load completed orders
00228 for (orderId in completedOrderIds) {
00229     RetrofitClient.apiService.getOrderDetail(orderId).enqueue(object : Callback<OrderDetailResponse> {
00230         override fun onResponse(call: Call<OrderDetailResponse>, response: Response<OrderDetailResponse>) {
00231             if (response.isSuccessful && response.body() != null) {
00232                 completed.add(response.body()!!)
00233             }
00234             completed_count++
00235             if (completed_count == total_count) {
00236                 bindAllOrders(pending, completed)
00237             }
00238         }
00239     })

```

```

00240         override fun onFailure(call: Call<OrderDetailResponse>, t: Throwable) {
00241             completed_count++
00242             if (completed_count == total_count) {
00243                 bindAllOrders(pending, completed)
00244             }
00245         }
00246     })
00247 }
00248
00249 // Handle empty case
00250 if (total_count == 0) {
00251     bindAllOrders(emptyList(), emptyList())
00252 }
00253 }
00254
00261 private fun bindAllOrders(pending: List<OrderDetailResponse>, completed: List<OrderDetailResponse>) {
00262     allPendingOrders = pending
00263     allCompletedOrders = completed
00264     displayedPendingCount = 0
00265     displayedCompletedCount = 0
00266
00267     ordersContainer.removeAllViews()
00268
00269     // Display initial orders
00270     displayNextOrders()
00271 }
00272
00280 private fun displayNextOrders() {
00281     val inflater = LayoutInflater.from(this)
00282
00283     // Pending orders section
00284     val pendingToDisplay = allPendingOrders.drop(displayedPendingCount).take(itemsPerPage)
00285     if (pendingToDisplay.isNotEmpty()) {
00286         if (displayedPendingCount == 0) {
00287             val labelView = android.widget.TextView(this).apply {
00288                 text = "Chưa giao"
00289                 setTextColor(android.graphics.Color.parseColor("#d32f2f"))
00290                 textSize = 14f
00291                 set Typeface(null, android.graphics.Typeface.BOLD)
00292                 setPadding(10, 20, 0, 16)
00293             }
00294             ordersContainer.addView(labelView)
00295         }
00296         pendingToDisplay.forEach { order ->
00297             addOrderCard(ordersContainer, order, order.status, inflater)
00298         }
00299         displayedPendingCount += pendingToDisplay.size
00300     }
00301
00302     // Completed orders section
00303     val completedToDisplay = allCompletedOrders.drop(displayedCompletedCount).take(itemsPerPage)
00304     if (completedToDisplay.isNotEmpty()) {
00305         if (displayedCompletedCount == 0) {
00306             val labelView = android.widget.TextView(this).apply {
00307                 text = "Đã giao"
00308                 setTextColor(android.graphics.Color.parseColor("#34a853"))
00309                 textSize = 14f
00310                 set Typeface(null, android.graphics.Typeface.BOLD)
00311                 setPadding(10, 40, 0, 16)
00312             }
00313             ordersContainer.addView(labelView)
00314         }
00315         completedToDisplay.forEach { order ->
00316             addOrderCard(ordersContainer, order, order.status, inflater)
00317         }
00318         displayedCompletedCount += completedToDisplay.size
00319     }
00320
00321     // Update Load More button visibility
00322     val hasMore = displayedPendingCount < allPendingOrders.size || displayedCompletedCount <
allCompletedOrders.size
00323     btnLoadMore.visibility = if (hasMore) android.view.View.VISIBLE else android.view.View.GONE
00324 }
00325
00329 private fun loadMoreOrders() {
00330     displayNextOrders()
00331 }
00332
00344 private fun addOrderCard(container: android.widget.LinearLayout, order: OrderDetailResponse, status: String, inflater:
LayoutInflater) {
00345     val isCompleted = isCompletedStatus(status)
00346     val statusColor = if (isCompleted) android.graphics.Color.parseColor("#34a853") else
android.graphics.Color.parseColor("#d32f2f")
00347     val statusText = if (isCompleted) "Đã giao" else "Chưa giao"
00348
00349     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00350     val cardView = androidx.cardview.widget.CardView(this).apply {

```

```

00351         layoutParams = android.widget.LinearLayout.LayoutParams(
00352             android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
00353             android.widget.LinearLayout.LayoutParams.WRAP_CONTENT
00354         ).apply {
00355             bottomMargin = 16
00356         }
00357         radius = 30f
00358         cardElevation = 4f
00359     }
00360
00361     val mainLayout = android.widget.LinearLayout(this).apply {
00362         layoutParams = android.view.ViewGroup.LayoutParams(
00363             android.view.ViewGroup.LayoutParams.MATCH_PARENT,
00364             android.view.ViewGroup.LayoutParams.WRAP_CONTENT
00365         )
00366         orientation = android.widget.LinearLayout.VERTICAL
00367         setBackgroundColor(android.graphics.Color.WHITE)
00368         setPadding(20, 20, 20, 20)
00369     }
00370
00371     // Details section
00372     val detailsLayout = android.widget.LinearLayout(this).apply {
00373         layoutParams = android.widget.LinearLayout.LayoutParams(
00374             android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
00375             android.widget.LinearLayout.LayoutParams.WRAP_CONTENT
00376         ).apply {
00377             bottomMargin = 12
00378         }
00379         orientation = android.widget.LinearLayout.VERTICAL
00380     }
00381
00382     // Dish names
00383     val totalQty = order.order_items.sumOf { it.quantity }
00384     val itemSummary = if (order.order_items.isEmpty()) {
00385         "Đơn hàng #${order.id}"
00386     } else {
00387         order.order_items.joinToString("\n") { item ->
00388             "• ${item.menuitem_name ?: "Món #${item.menuitemid}"} x${item.quantity}"
00389         }
00390     }
00391
00392     val dishNameView = android.widget.TextView(this).apply {
00393         text = itemSummary
00394         setTextColor(android.graphics.Color.BLACK)
00395         textSize = 14f
00396         setTypeface(null, android.graphics.Typeface.BOLD)
00397         setPadding(0, 0, 0, 8)
00398     }
00399     detailsLayout.addView(dishNameView)
00400
00401     // Quantity
00402     val qtyView = android.widget.TextView(this).apply {
00403         text = "SL: $totalQty"
00404         setTextColor(android.graphics.Color.BLACK)
00405         textSize = 14f
00406         setPadding(0, 0, 0, 8)
00407     }
00408     detailsLayout.addView(qtyView)
00409
00410     // Restaurant
00411     val restaurantView = android.widget.TextView(this).apply {
00412         text = order.restaurant_name ?: "Nhà hàng #${order.restaurantid}"
00413         setTextColor(android.graphics.Color.BLACK)
00414         textSize = 14f
00415     }
00416     detailsLayout.addView(restaurantView)
00417
00418     mainLayout.addView(detailsLayout)
00419
00420     // Status and Price Row
00421     val statusRow = android.widget.LinearLayout(this).apply {
00422         layoutParams = android.widget.LinearLayout.LayoutParams(
00423             android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
00424             android.widget.LinearLayout.LayoutParams.WRAP_CONTENT
00425         ).apply {
00426             bottomMargin = 12
00427         }
00428         orientation = android.widget.LinearLayout.HORIZONTAL
00429         gravity = android.view.Gravity.CENTER_VERTICAL
00430     }
00431
00432     val statusView = android.widget.TextView(this).apply {
00433         layoutParams = android.widget.LinearLayout.LayoutParams(
00434             0,
00435             android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
00436             1f
00437         )

```



```

00438         text = statusText
00439         setTextColor(statusColor)
00440         textSize = 14f
00441         setTypeface(null, android.graphics.Typeface.BOLD)
00442     }
00443     statusRow.addView(statusView)
00444
00445     val priceView = android.widget.TextView(this).apply {
00446         text = "${fmt.format(order.totalprice)}d"
00447         setTextColor(android.graphics.Color.BLACK)
00448         textSize = 16f
00449         setTypeface(null, android.graphics.Typeface.BOLD)
00450     }
00451     statusRow.addView(priceView)
00452
00453     mainLayout.addView(statusRow)
00454
00455     // Detail button
00456     val detailView = android.widget.TextView(this).apply {
00457         text = "Chi tiết"
00458         setTextColor(android.graphics.Color.BLACK)
00459         textSize = 14f
00460         setTypeface(null, android.graphics.Typeface.BOLD)
00461         isClickable = true
00462         isFocusable = true
00463         setOnClickListener {
00464             openOrderDetail(order.id, status, order.restaurant_name)
00465         }
00466     }
00467     mainLayout.addView(detailView)
00468
00469     cardView.addView(mainLayout)
00470
00471     // Also add click listener to card
00472     cardView.setOnClickListener {
00473         openOrderDetail(order.id, status, order.restaurant_name)
00474     }
00475
00476     container.addView(cardView)
00477 }
00478
00479 private fun resolveUserIdForTracking(): Int {
00480     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00481     val userId = sharedPref.getInt("user_id", -1)
00482     if (userId > 0) return userId
00483
00484     val userName = sharedPref.getString("user_name", "") ?: ""
00485     if (userName.contains("Trung", ignoreCase = true) || userName.isBlank()) {
00486         return 1
00487     }
00488     return 1
00489 }
00490
00491 private fun isCompletedStatus(status: String?): Boolean {
00492     val normalized = (status ?: "").lowercase(Locale.ROOT)
00493     return normalized == "completed" || normalized == "delivered" || normalized == "done"
00494 }
00495
00496 private fun openOrderDetail(orderId: Int?, status: String, restaurantName: String?) {
00497     if (orderId == null) {
00498         Toast.makeText(this, "Không có đơn để xem", Toast.LENGTH_SHORT).show()
00499         return
00500     }
00501
00502     val intent = Intent(this, OrderTrackingDetail::class.java)
00503     intent.putExtra("order_id", orderId)
00504     intent.putExtra("order_status", status)
00505     // Nếu không có tên quán, lấy tên tạm từ UI title (nếu có) hoặc dùng Mã đơn
00506     val titleText = restaurantName ?: "Đơn hàng #$orderId"
00507     intent.putExtra("order_name", titleText)
00508     startActivity(intent)
00509 }
00510
00511 }
00512
00513 }

```

9.59 app/app/src/main/java/com/example/myapp/screens/OrderTrackingDetail.kt File Reference

Packages

- package [AppCompatActivity](#)

9.60 OrderTrackingDetail.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.graphics.Color
00014 import android.graphics.PorterDuff
00015 import android.os.Bundle
00016 import android.view.View
00017 import androidx.appcompat.app.AppCompatActivity
00018 import android.widget.ImageView
00019 import android.widget.EditText
00020 import android.widget.TextView
00021 import android.widget.LinearLayout
00022 import androidx.appcompat.widget.AppCompatButton
00023 import com.example.myapp.R
00024 import com.example.myapp.screens.api.OrderDetailResponse
00025 import com.example.myapp.screens.api.ReviewCreateRequest
00026 import com.example.myapp.screens.api.RetrofitClient
00027 import retrofit2.Call
00028 import retrofit2.Callback
00029 import retrofit2.Response
00030
00041 class OrderTrackingDetail : AppCompatActivity() {
00043     private var selectedRating = 0
00044
00054     override fun onCreate(savedInstanceState: Bundle?) {
00055         super.onCreate(savedInstanceState)
00056         setContentView(R.layout.order_tracking_detail)
00057
00058         // Get order information from Intent
00059         val orderName = intent.getStringExtra("order_name") ?: "Chi tiết đơn hàng"
00060         val orderId = intent.getIntExtra("order_id", -1)
00061         val orderStatus = intent.getStringExtra("order_status") ?: ""
00062
00063         val tvTitle: TextView = findViewById(R.id.tvTitle)
00064         tvTitle.text = orderName
00065
00066         val timelineContainer: View = findViewById(R.id.timelineContainer)
00067         val ratingContainer: View = findViewById(R.id.starratingContainer)
00068         val etFeedbackView: View = findViewById(R.id.etFeedback)
00069         val btnRateAction: AppCompatButton = findViewById(R.id.btnRate)
00070
00071         val isCompleted = isCompletedStatus(orderStatus)
00072
00073         // Luôn hiển thị đầy đủ các thành phần như trong file thiết kế XML
00074         timelineContainer.visibility = View.VISIBLE
00075         ratingContainer.visibility = View.VISIBLE
00076         etFeedbackView.visibility = View.VISIBLE
00077         btnRateAction.visibility = View.VISIBLE
00078
00079         // Nếu đơn hàng chưa hoàn thành, có thể làm mờ hoặc vô hiệu hóa nút đánh giá (tùy chọn)
00080         if (!isCompleted) {
00081             btnRateAction.alpha = 0.5f
00082             btnRateAction.isEnabled = false
00083             btnRateAction.text = "Chờ giao hàng để đánh giá"
00084         }
00085
00086         if (orderId > 0) {
00087             loadOrderSummary(orderId)
00088         }
00089
00090         // Back Button - Return to previous screen
00091         val btnBack: ImageView = findViewById(R.id.btn_back)
00092         btnBack.setOnClickListener {
00093             finish()
00094         }
00095
00096         val stars = listOf(
00097             findViewById<ImageView>(R.id.star1),
00098             findViewById<ImageView>(R.id.star2),
00099             findViewById<ImageView>(R.id.star3),
00100             findViewById<ImageView>(R.id.star4),
00101             findViewById<ImageView>(R.id.star5)
00102         )
00103         updateStarColors(stars)
00104         stars.forEachIndexed { index, star ->
00105             star.setOnClickListener {
00106                 selectedRating = index + 1
00107                 updateStarColors(stars)
00108             }
00109         }
00110

```

```

00111 // Rate Button - Submit review/rating
00112 btnRateAction.setOnClickListener {
00113     val etFeedback: EditText = findViewById(R.id.etFeedback)
00114     val feedbackText = etFeedback.text.toString()
00115     val resolvedOrderId = intent.getIntExtra("order_id", -1)
00116     val userId = getSharedPreferences("user_prefs", MODE_PRIVATE).getInt("user_id", -1)
00117
00118     if (resolvedOrderId <= 0) {
00119         android.widget.Toast.makeText(
00120             this,
00121             "Thiếu mã đơn hàng để gửi đánh giá",
00122             android.widget.Toast.LENGTH_SHORT
00123         ).show()
00124         return@setOnClickListener
00125     }
00126
00127     if (selectedRating <= 0) {
00128         android.widget.Toast.makeText(
00129             this,
00130             "Vui lòng chọn số sao",
00131             android.widget.Toast.LENGTH_SHORT
00132         ).show()
00133         return@setOnClickListener
00134     }
00135
00136     if (feedbackText.isBlank()) {
00137         android.widget.Toast.makeText(
00138             this,
00139             "Vui lòng nhập phản hồi",
00140             android.widget.Toast.LENGTH_SHORT
00141         ).show()
00142         return@setOnClickListener
00143     }
00144
00145     if (userId <= 0) {
00146         android.widget.Toast.makeText(
00147             this,
00148             "Không tìm thấy thông tin người dùng",
00149             android.widget.Toast.LENGTH_SHORT
00150         ).show()
00151         return@setOnClickListener
00152     }
00153
00154     btnRateAction.isEnabled = false
00155     RetrofitClient.apiService.createReview(
00156         ReviewCreateRequest(
00157             rating = selectedRating,
00158             comment = feedbackText,
00159             orderid = resolvedOrderId,
00160             userid = userId
00161         )
00162     ).enqueue(object : Callback<com.example.myapp.screens.api.ReviewResponse> {
00163         override fun onResponse(
00164             call: Call<com.example.myapp.screens.api.ReviewResponse>,
00165             response: Response<com.example.myapp.screens.api.ReviewResponse>
00166         ) {
00167             btnRateAction.isEnabled = true
00168             if (!response.isSuccessful) {
00169                 android.widget.Toast.makeText(
00170                     this@OrderTrackingDetail,
00171                     "Gửi đánh giá thất bại",
00172                     android.widget.Toast.LENGTH_SHORT
00173                 ).show()
00174                 return
00175             }
00176
00177             android.widget.Toast.makeText(
00178                 this@OrderTrackingDetail,
00179                 "Cảm ơn bạn đã đánh giá!",
00180                 android.widget.Toast.LENGTH_SHORT
00181             ).show()
00182             finish()
00183         }
00184
00185         override fun onFailure(call: Call<com.example.myapp.screens.api.ReviewResponse>, t: Throwable) {
00186             btnRateAction.isEnabled = true
00187             android.widget.Toast.makeText(
00188                 this@OrderTrackingDetail,
00189                 "Không thể gửi đánh giá",
00190                 android.widget.Toast.LENGTH_SHORT
00191             ).show()
00192         }
00193     })
00194 }
00195
00196 // Bottom Navigation - Home
00197 val icHome: ImageView = findViewById(R.id.icHome)

```

```

00198         icHome.setOnClickListener {
00199             startActivity(Intent(this, home::class.java))
00200         }
00201
00202         // Bottom Navigation - Wishlist
00203         val icHeart: ImageView = findViewById(R.id.icHeart)
00204         icHeart.setOnClickListener {
00205             // TODO: Navigate to favorites/wishlist page
00206             android.util.Log.d("OrderTracking", "Wishlist clicked")
00207         }
00208
00209         // Bottom Navigation - Cart
00210         val icCart: ImageView = findViewById(R.id.icCart)
00211         icCart.setOnClickListener {
00212             startActivity(Intent(this, cart::class.java))
00213         }
00214
00215         // Bottom Navigation - Profile
00216         val icProfile: ImageView = findViewById(R.id.icProfile)
00217         icProfile.setOnClickListener {
00218             startActivity(Intent(this, profile::class.java))
00219         }
00220     }
00221
00222     private fun loadOrderSummary(orderId: Int) {
00223         val summaryView: TextView = findViewById(R.id.tvOrderSummary)
00224         RetrofitClient.apiService.getOrderDetail(orderId).enqueue(object : Callback<OrderDetailResponse> {
00225             override fun onResponse(call: Call<OrderDetailResponse>, response: Response<OrderDetailResponse>) {
00226                 if (!response.isSuccessful || response.body() == null) {
00227                     summaryView.text = "Không tải được chi tiết đơn hàng"
00228                     return
00229                 }
00230
00231                 val order = response.body()!!
00232                 val totalQuantity = order.order_items.sumOf { it.quantity }
00233                 val itemLines = order.order_items.joinToString(", ") { item ->
00234                     "${item.menuitem_name}: "Món #${item.menuitemid}" (x${item.quantity})"
00235                 }
00236                 summaryView.text = "Mã đơn: #${orderId} | Số món: $totalQuantity\n$itemLines"
00237
00238                 // Populate order details in timeline container
00239                 populateOrderDetails(order)
00240             }
00241
00242             override fun onFailure(call: Call<OrderDetailResponse>, t: Throwable) {
00243                 summaryView.text = "Không tải được chi tiết đơn hàng"
00244             }
00245         })
00246     }
00247
00248     private fun populateOrderDetails(order: OrderDetailResponse) {
00249         try {
00250             // Order ID
00251             findViewById<TextView>(R.id.tvOrderId).text = "#${orderId}"
00252
00253             // Total Price - Format with thousands separator
00254             val priceFormatted = String.format("%.d", order.totalprice).replace(",", ".")
00255             findViewById<TextView>(R.id.tvTotalPrice).text = "${priceFormatted}d"
00256
00257             // Restaurant Name
00258             findViewById<TextView>(R.id.tvRestaurantName).text = order.restaurant_name ?: "--"
00259
00260             // Order Status - Convert to Vietnamese
00261             val statusText = when (order.status.lowercase()) {
00262                 "pending" -> "Chờ xác nhận"
00263                 "confirmed" -> "Đã xác nhận"
00264                 "paid" -> "Đã thanh toán"
00265                 "delivering" -> "Đang giao"
00266                 "completed" -> "Đã giao"
00267                 "cancelled" -> "Đã hủy"
00268                 else -> order.status
00269             }
00270             findViewById<TextView>(R.id.tvOrderStatus).text = statusText
00271
00272             // Delivery Address
00273             findViewById<TextView>(R.id.tvDeliveryAddress).text = order.address_detail ?: "--"
00274
00275             // Order Date - Format the date
00276             val dateFormatted = formatOrderDate(order.createdat)
00277             findViewById<TextView>(R.id.tvOrderDate).text = dateFormatted
00278
00279             // Populate Items List
00280             populateItemsList(order.order_items)
00281         } catch (e: Exception) {
00282             android.util.Log.e("OrderTrackingDetail", "Error populating order details: ${e.message}")
00283         }
00284     }

```

```

00302     }
00303
00312     private fun populateItemsList(items: List<com.example.myapp.screens.api.OrderItemDetailResponse>) {
00313         val itemsListContainer = findViewById<LinearLayout>(R.id.itemsListContainer)
00314         itemsListContainer.removeAllViews()
00315
00316         for (item in items) {
00317             val itemLayout = LinearLayout(this)
00318             itemLayout.layoutParams = LinearLayout.LayoutParams(
00319                 LinearLayout.LayoutParams.MATCH_PARENT,
00320                 LinearLayout.LayoutParams.WRAP_CONTENT
00321             ).apply {
00322                 bottomMargin = 8
00323             }
00324             itemLayout.orientation = LinearLayout.HORIZONTAL
00325             itemLayout.gravity = android.view.Gravity.CENTER_VERTICAL
00326
00327             // Item Name and Quantity
00328             val itemNameView = TextView(this)
00329             itemNameView.layoutParams = LinearLayout.LayoutParams(
00330                 0,
00331                 LinearLayout.LayoutParams.WRAP_CONTENT,
00332                 If
00333             )
00334             itemNameView.text = "${item.menuitem_name ?: "Món #${item.menuitemid}"} (x${item.quantity})"
00335             itemNameView.textSize = 13f
00336             itemNameView.setTextColor(Color.parseColor("#1f1f1f"))
00337             itemNameView.typeface = android.graphics.Typeface.create(android.graphics.Typeface.SANS_SERIF,
00338                 android.graphics.Typeface.NORMAL)
00339
00340             // Item Price
00341             val itemPriceView = TextView(this)
00342             itemPriceView.layoutParams = LinearLayout.LayoutParams(
00343                 LinearLayout.LayoutParams.WRAP_CONTENT,
00344                 LinearLayout.LayoutParams.WRAP_CONTENT
00345             )
00346             val price = item.price * item.quantity
00347             val priceFormatted = String.format("%d", price).replace(",", " ")
00348             itemPriceView.text = "${priceFormatted}d"
00349             itemPriceView.textSize = 13f
00350             itemPriceView.setTextColor(Color.parseColor("#ff8001"))
00351             itemPriceView.typeface = android.graphics.Typeface.DEFAULT_BOLD
00352             itemPriceView.setPadding(16, 0, 0, 0)
00353
00354             itemLayout.addView(itemNameView)
00355             itemLayout.addView(itemPriceView)
00356             itemsListContainer.addView(itemLayout)
00357         }
00358
00365     private fun formatOrderDate(dateString: String?): String {
00366         if (dateString.isNullOrEmpty()) return "--"
00367         return try {
00368             // Expected format from backend: 2025-04-06T10:30:45
00369             val parts = dateString.split("T")
00370             if (parts.size >= 1) {
00371                 val dateParts = parts[0].split("-")
00372                 if (dateParts.size == 3) {
00373                     // Convert from 2025-04-06 to 06/04/2025
00374                     "${dateParts[2]}/${dateParts[1]}/${dateParts[0]}"
00375                 } else {
00376                     dateString
00377                 }
00378             } else {
00379                 dateString
00380             }
00381         } catch (e: Exception) {
00382             dateString
00383         }
00384     }
00385
00393     private fun updateStarColors(stars: List<ImageView>) {
00394         stars.forEachIndexed { index, star ->
00395             val color = if (index < selectedRating) "#FFC107" else "#CFCFCF"
00396             star.setColorFilter(Color.parseColor(color), PorterDuff.Mode.SRC_IN)
00397         }
00398     }
00399
00406     private fun isCompletedStatus(status: String?): Boolean {
00407         val normalized = (status ?: "").lowercase(java.util.Locale.ROOT)
00408         return normalized == "completed" || normalized == "delivered" || normalized == "done"
00409     }
00410 }

```

9.61 app/app/src/main/java/com/example/myapp/screens/payment_methods.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- var [AppCompatActivity.isRequestingVnPay](#)

9.62 payment_methods.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import android.widget.Toast
00018 import androidx.appcompat.app.AppCompatActivity
00019 import com.example.myapp.R
00020 import com.example.myapp.screens.api.OrderResponse
00021 import com.example.myapp.screens.api.OrderStatusUpdateRequest
00022 import com.example.myapp.screens.api.RetrofitClient
00023 import com.example.myapp.screens.api.VNPayResponse
00024 import retrofit2.Call
00025 import retrofit2.Callback
00026 import retrofit2.Response
00027
00028
00044 class payment_methods : AppCompatActivity() {
00045
00046     private var isRequestingVnPay: Boolean = false
00047
00048
00056     override fun onCreate(savedInstanceState: Bundle?) {
00057         super.onCreate(savedInstanceState)
00058         setContentView(R.layout.payment_methods)
00059
00060         val isSelectMode = intent.getBooleanExtra("select_mode", false)
00061         val autoPayNow = intent.getBooleanExtra("auto_pay_now", false)
00062
00063         val tvPayOnDelivery = findViewById<TextView>(R.id.tvPayOnDelivery)
00064         val tvPayNow = findViewById<TextView>(R.id.tvPayNow)
00065         val icBack = findViewById<ImageView>(R.id.icBack)
00066
00067         tvPayOnDelivery.setOnClickListener {
00068             if (isSelectMode) {
00069                 val resultIntent = Intent()
00070                 resultIntent.putExtra("payment_method", "cod")
00071                 resultIntent.putExtra("payment_method_label", "Thanh toán khi nhận hàng")
00072                 setResult(RESULT_OK, resultIntent)
00073                 finish()
00074             } else {
00075                 val orderId = intent.getIntExtra("order_id", -1)
00076                 if (orderId > 0) {
00077                     updateOrderStatusAndGoSuccess(orderId, "confirmed")
00078                 } else {
00079                     Toast.makeText(this, "Đã chọn thanh toán khi nhận hàng", Toast.LENGTH_SHORT).show()
00080                     startActivity(Intent(this, payment_successful::class.java))
00081                 }
00082             }
00083         }
00084
00085         tvPayNow.setOnClickListener {
00086
00087             if (isSelectMode) {
00088                 val resultIntent = Intent()
00089                 resultIntent.putExtra("payment_method", "online")
00090

```

```

00091         resultIntent.putExtra("payment_method_label", "Thanh toán ngay")
00092         setResult(RESULT_OK, resultIntent)
00093         finish()
00094         return@setOnClickListener
00095     }
00096
00097     startVnPayPayment()
00098 }
00099
00100     if (!isSelectedMode && autoPayNow) {
00101         startVnPayPayment()
00102     }
00103 }
00104
00105
00106     // nút quay lại
00107     icBack.setOnClickListener {
00108         finish()
00109     }
00110 }
00111
00112 private fun startVnPayPayment() {
00113     if (isRequestingVnPay) return
00114     isRequestingVnPay = true
00115
00116     val orderId = intent.getIntExtra("order_id", -1).let {
00117         if (it > 0) "DH$it" else "DH" + System.currentTimeMillis()
00118     }
00119     val amount = intent.getIntExtra("order_total", 70000)
00120
00121     RetrofitClient.apiService.getVNPayUrl(amount, orderId)
00122         .enqueue(object : Callback<VNPayResponse> {
00123             override fun onResponse(
00124                 call: Call<VNPayResponse>,
00125                 response: Response<VNPayResponse>
00126             ) {
00127                 isRequestingVnPay = false
00128                 if (response.isSuccessful && response.body() != null) {
00129                     val paymentUrl = response.body()!!.payment_url
00130                     val intent = Intent(this@payment_methods, VNPayActivity::class.java)
00131                     intent.putExtra("URL", paymentUrl)
00132                     intent.putExtra("order_id", this@payment_methods.intent.getIntExtra("order_id", -1))
00133                     startActivity(intent)
00134                 } else {
00135                     Toast.makeText(
00136                         this@payment_methods,
00137                         "Lấy link thanh toán thất bại",
00138                         Toast.LENGTH_SHORT
00139                     ).show()
00140                 }
00141             }
00142         })
00143
00144     override fun onFailure(call: Call<VNPayResponse>, t: Throwable) {
00145         isRequestingVnPay = false
00146         Toast.makeText(
00147             this@payment_methods,
00148             "Không kết nối được server",
00149             Toast.LENGTH_SHORT
00150         ).show()
00151     }
00152 }
00153
00154 private fun updateOrderStatusAndGoSuccess(orderId: Int, status: String) {
00155     RetrofitClient.apiService.updateOrderStatus(orderId, OrderStatusUpdateRequest(status))
00156         .enqueue(object : Callback<OrderResponse> {
00157             override fun onResponse(call: Call<OrderResponse>, response: Response<OrderResponse>) {
00158                 if (!response.isSuccessful) {
00159                     Toast.makeText(this@payment_methods, "Không cập nhật được trạng thái đơn",
00160                         Toast.LENGTH_SHORT).show()
00161                     return
00162                 }
00163                 val successIntent = Intent(this@payment_methods, payment_successful::class.java)
00164                 successIntent.putExtra("order_id", orderId)
00165                 startActivity(successIntent)
00166                 finish()
00167             }
00168         })
00169
00170     override fun onFailure(call: Call<OrderResponse>, t: Throwable) {
00171         Toast.makeText(this@payment_methods, "Lỗi mạng khi cập nhật đơn", Toast.LENGTH_SHORT).show()
00172     }
00173 }
00174 }
00175 }
00176 }
00177 }
00178 }
00179 }
00180 }
00181 }
00182 }
00183 }
00184 }
00185 }
00186 }
00187 }
00188 }
00189 }
00190 }

```

9.63 app/app/src/main/java/com/example/myapp/screens/payment_↵ successful.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState:Bundle?)
- fun [AppCompatActivity.updateOrderStatusToCODConfirmed](#) (orderId:Int)

9.64 payment_↵successful.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.widget.Button
00015 import android.widget.ImageView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import com.example.myapp.R
00019 import com.example.myapp.screens.api.OrderResponse
00020 import com.example.myapp.screens.api.OrderStatusUpdateRequest
00021 import com.example.myapp.screens.api.RetrofitClient
00022 import com.example.myapp.screens.home
00023 import retrofit2.Call
00024 import retrofit2.Callback
00025 import retrofit2.Response
00026
00034 class payment_successful : AppCompatActivity() {
00042     override fun onCreate(savedInstanceState: Bundle?) {
00043         super.onCreate(savedInstanceState)
00044         setContentView(R.layout.payment_successful)
00045
00046         val orderId = intent.getIntExtra("order_id", -1)
00047
00048         // Nếu là COD (payment từ order.kt), cập nhật status thành "confirmed" để gửi thông báo
00049         if (orderId > 0) {
00050             updateOrderStatusToCODConfirmed(orderId)
00051         }
00052
00053         // Tìm nút "Trở về trang chủ" theo ID btnReturnHome
00054         val btnReturnHome: Button = findViewById(R.id.btnReturnHome)
00055
00056         // Thiết lập sự kiện click
00057         btnReturnHome.setOnClickListener {
00058             // Chuyển về màn hình Home
00059             val intent = Intent(this, home::class.java)
00060             // Xóa hết các Activity cũ để khi nhấn Back ở Home không bị quay lại trang thành công
00061             intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00062             startActivity(intent)
00063             finish()
00064         }
00065         // click icProfile
00066         val icProfile: ImageView = findViewById(R.id.icProfile)
00067         icProfile.setOnClickListener {
00068             val intent = Intent(this, profile::class.java)
00069             startActivity(intent)
00070         }
00071
00072         val icHome: ImageView = findViewById(R.id.icHome)
00073         icHome.setOnClickListener {
00074             val intent = Intent(this, home::class.java)
00075             startActivity(intent)
00076         }
00077
00078         val btnCart: ImageView = findViewById(R.id.icCart)
00079         btnCart.setOnClickListener {
```



```

00080         val intent = Intent(this, cart::class.java)
00081         startActivity(intent)
00082     }
00083 }
00084
00094 private fun updateOrderStatusToCODConfirmed(orderId: Int) {
00095     RetrofitClient.apiService.updateOrderStatus(orderId, OrderStatusUpdateRequest("confirmed"))
00096     .enqueue(object : Callback<OrderResponse> {
00097         override fun onResponse(call: Call<OrderResponse>, response: Response<OrderResponse>) {
00098             if (response.isSuccessful) {
00099                 // Thông báo sẽ được gửi từ backend
00100             } else {
00101                 Toast.makeText(this@payment_successful, "Cập nhật đơn hàng thất bại",
00102                     Toast.LENGTH_SHORT).show()
00103             }
00104         }
00105         override fun onFailure(call: Call<OrderResponse>, t: Throwable) {
00106             Toast.makeText(this@payment_successful, "Lỗi mạng", Toast.LENGTH_SHORT).show()
00107         }
00108     })
00109 }
00110 }

```

9.65 app/app/src/main/java/com/example/myapp/screens/PointsDetail.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val activity_cart::class.java val profile::class.java fun [AppCompatActivity.loadOrders](#) ()
- fun [AppCompatActivity.bindOrderCard](#) (titleId:Int, pointsId:Int, order:OrderResponse?, fallback← TitlePrefix:String)
- fun [AppCompatActivity.resolveUserIdForPoints](#) ()

Variables

- var [firstOrderId](#) val [AppCompatActivity.pointsCard2](#)
- var [firstOrderId](#) val secondOrderId val [AppCompatActivity.pointsCard3](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val [AppCompatActivity.pointsCard4](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val [AppCompatActivity.icHome](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val [AppCompatActivity.icHeart](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val [AppCompatActivity.icCart](#)
- var [firstOrderId](#) val secondOrderId val thirdOrderId val fourthOrderId val home::class.java val Wish-list clicked val activity_cart::class.java val [AppCompatActivity.icProfile](#)

9.66 PointsDetail.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00009
00010 import android.content.Context
00011 import android.content.Intent
00012 import android.os.Bundle
00013 import androidx.appcompat.app.AppCompatActivity
00014 import android.widget.ImageView
00015 import android.widget.TextView
00016 import androidx.cardview.widget.CardView
00017 import android.widget.Toast
00018 import com.example.myapp.R
00019 import com.example.myapp.screens.api.OrderResponse
00020 import com.example.myapp.screens.api.RetrofitClient
00021 import retrofit2.Call
00022 import retrofit2.Callback
00023 import retrofit2.Response
00024
00033 class PointsDetail : AppCompatActivity() {
00035     private var firstOrderId: Int? = null
00037     private var secondOrderId: Int? = null
00039     private var thirdOrderId: Int? = null
00041     private var fourthOrderId: Int? = null
00042
00051     override fun onCreate(savedInstanceState: Bundle?) {
00052         super.onCreate(savedInstanceState)
00053         setContentView(R.layout.points_detail)
00054
00055         loadOrders()
00056
00057         // Back Button - Return to previous screen
00058         val btnBack: ImageView = findViewById(R.id.btn_back)
00059         btnBack.setOnClickListener {
00060             finish()
00061         }
00062
00063         // Points Card 1 - Click to view order detail
00064         val pointsCard1: CardView = findViewById(R.id.pointsCard1)
00065         val tvDishName1: TextView = findViewById(R.id.tvDishName1)
00066         pointsCard1.setOnClickListener {
00067             navigateToOrderDetail(tvDishName1.text.toString(), firstOrderId)
00068         }
00069
00070         // Points Card 2 - Click to view order detail
00071         val pointsCard2: CardView = findViewById(R.id.pointsCard2)
00072         val tvDishName2: TextView = findViewById(R.id.tvDishName2)
00073         pointsCard2.setOnClickListener {
00074             navigateToOrderDetail(tvDishName2.text.toString(), secondOrderId)
00075         }
00076
00077         // Points Card 3 - Click to view order detail
00078         val pointsCard3: CardView = findViewById(R.id.pointsCard3)
00079         val tvDishName3: TextView = findViewById(R.id.tvDishName3)
00080         pointsCard3.setOnClickListener {
00081             navigateToOrderDetail(tvDishName3.text.toString(), thirdOrderId)
00082         }
00083
00084         // Points Card 4 - Click to view order detail
00085         val pointsCard4: CardView = findViewById(R.id.pointsCard4)
00086         val tvDishName4: TextView = findViewById(R.id.tvDishName4)
00087         pointsCard4.setOnClickListener {
00088             navigateToOrderDetail(tvDishName4.text.toString(), fourthOrderId)
00089         }
00090
00091         // Bottom Navigation - Home
00092         val icHome: ImageView = findViewById(R.id.icHome)
00093         icHome.setOnClickListener {
00094             startActivity(Intent(this, home::class.java))
00095         }
00096
00097         // Bottom Navigation - Wishlist
00098         val icHeart: ImageView = findViewById(R.id.icHeart)
00099         icHeart.setOnClickListener {
00100             android.util.Log.d("PointsDetail", "Wishlist clicked")
00101         }
00102
00103         // Bottom Navigation - Cart
00104         val icCart: ImageView = findViewById(R.id.icCart)
00105         icCart.setOnClickListener {
00106             startActivity(Intent(this, activity_cart::class.java))
00107         }
00108

```

```

00109     // Bottom Navigation - Profile
00110     val icProfile: ImageView = findViewById(R.id.icProfile)
00111     icProfile.setOnClickListener {
00112         startActivity(Intent(this, profile::class.java))
00113     }
00114 }
00115
00122 private fun loadOrders() {
00123     val userId = resolveUserIdForPoints()
00124     RetrofitClient.apiService.getUserOrders(userId).enqueue(object : Callback<List<OrderResponse>» {
00125         override fun onResponse(call: Call<List<OrderResponse>», response: Response<List<OrderResponse>») {
00126             if (!response.isSuccessful || response.body().isNullOrEmpty()) {
00127                 Toast.makeText(this@PointsDetail, "Không có đơn hàng để đánh giá", Toast.LENGTH_SHORT).show()
00128                 return
00129             }
00130
00131             val completedOrders = response.body()!!
00132                 .filter { it.status == "completed" }
00133                 .sortedByDescending { it.createdat }
00134
00135             firstOrderId = completedOrders.getOrNull(0)?.id
00136             secondOrderId = completedOrders.getOrNull(1)?.id
00137             thirdOrderId = completedOrders.getOrNull(2)?.id
00138             fourthOrderId = completedOrders.getOrNull(3)?.id
00139
00140             bindOrderCard(R.id.tvDishName1, R.id.tvPoints1, completedOrders.getOrNull(0), "Đơn hàng #")
00141             bindOrderCard(R.id.tvDishName2, R.id.tvPoints2, completedOrders.getOrNull(1), "Đơn hàng #")
00142             bindOrderCard(R.id.tvDishName3, R.id.tvPoints3, completedOrders.getOrNull(2), "Đơn hàng #")
00143             bindOrderCard(R.id.tvDishName4, R.id.tvPoints4, completedOrders.getOrNull(3), "Đơn hàng #")
00144         }
00145
00146         override fun onFailure(call: Call<List<OrderResponse>», t: Throwable) {
00147             Toast.makeText(this@PointsDetail, "Không tải được đơn hàng", Toast.LENGTH_SHORT).show()
00148         }
00149     })
00150 }
00151
00163 private fun bindOrderCard(titleId: Int, pointsId: Int, order: OrderResponse?, fallbackTitlePrefix: String) {
00164     val titleView: TextView = findViewById(titleId)
00165     val pointsView: TextView = findViewById(pointsId)
00166
00167     if (order == null) {
00168         titleView.text = "Chưa có đơn hàng"
00169         pointsView.text = "0"
00170         return
00171     }
00172
00173     titleView.text = "$fallbackTitlePrefix${order.id}"
00174     pointsView.text = (order.totalprice / 1000).toString()
00175 }
00176
00184 private fun resolveUserIdForPoints(): Int {
00185     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00186     val userId = sharedPref.getInt("user_id", -1)
00187     if (userId > 0) return userId
00188
00189     val userName = sharedPref.getString("user_name", "") ?: ""
00190     if (userName.contains("Trung", ignoreCase = true) || userName.isBlank()) {
00191         return 1
00192     }
00193     return 1
00194 }
00195
00205 private fun navigateToOrderDetail(dishName: String, orderId: Int?) {
00206     if (orderId == null) {
00207         Toast.makeText(this, "Thiếu mã đơn hàng để đánh giá", Toast.LENGTH_SHORT).show()
00208         return
00209     }
00210     val intent = Intent(this, OrderTrackingDetail::class.java)
00211     intent.putExtra("order_name", dishName)
00212     intent.putExtra("order_id", orderId)
00213     startActivity(intent)
00214 }
00215 }

```

9.67 app/app/src/main/java/com/example/myapp/screens/pre_order.kt File Reference

Packages

- package [AppCompatActivity](#)

9.68 pre_order.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00009
00010 import android.app.DatePickerDialog
00011 import android.app.TimePickerDialog
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.widget.ImageView
00015 import android.widget.TextView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import androidx.appcompat.widget.AppCompatButton
00019 import com.example.myapp.R
00020 import com.squareup.picasso.Picasso
00021 import java.text.NumberFormat
00022 import java.util.*
00023
00033 class pre_order : AppCompatActivity() {
00035     private var qty = 1
00037     private var price = 0
00039     private var foodId = -1
00041     private var foodName = ""
00043     private var foodImageUrl = ""
00044
00046     private var selectedCalendar: Calendar = Calendar.getInstance()
00048     private var isSelected = false
00050     private var isTimeSelected = false
00051
00060     override fun onCreate(savedInstanceState: Bundle?) {
00061         super.onCreate(savedInstanceState)
00062         setContentView(R.layout.activity_pre_order)
00063
00064         foodId = intent.getIntExtra("food_id", -1)
00065         foodName = intent.getStringExtra("food_name") ?: ""
00066         price = intent.getIntExtra("food_price", 0)
00067         foodImageUrl = intent.getStringExtra("food_image_url") ?: ""
00068
00069         val tvFoodName: TextView = findViewById(R.id.tvFoodName)
00070         val tvUnitPrice: TextView = findViewById(R.id.tvUnitPrice)
00071         val tvQty: TextView = findViewById(R.id.tvQty)
00072         val tvTotalValue: TextView = findViewById(R.id.tvTotalValue)
00073         val imgFood: ImageView = findViewById(R.id.imgFood)
00074
00075         val btnSelectDate: TextView = findViewById(R.id.btnSelectDate)
00076         val btnSelectTime: TextView = findViewById(R.id.btnSelectTime)
00077         val tvSelectedDateTime: TextView = findViewById(R.id.tvSelectedDateTime)
00078
00079         val btnDecrease: TextView = findViewById(R.id.btnDecrease)
00080         val btnIncrease: TextView = findViewById(R.id.btnIncrease)
00081         val btnAddToCart: AppCompatButton = findViewById(R.id.btnAddToCart)
00082         val btnBack: ImageView = findViewById(R.id.btnBack)
00083
00084         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00085
00086         tvFoodName.text = foodName
00087         tvUnitPrice.text = fmt.format(price) + "đ"
00088
00089         if (foodImageUrl.isNotEmpty()) {
00090             Picasso.get().load(foodImageUrl).placeholder(R.drawable.placeholder_loading).error(R.drawable.pngwing).into(imgFood)
00091         }
00092
00093         fun updateUI() {
00094             tvQty.text = qty.toString()
00095             tvTotalValue.text = fmt.format(qty * price) + "đ"
00096
00097             if (isSelected && isTimeSelected) {
00098                 val day = selectedCalendar.get(Calendar.DAY_OF_MONTH)
00099                 val month = selectedCalendar.get(Calendar.MONTH) + 1
00100                 val year = selectedCalendar.get(Calendar.YEAR)
00101                 val hour = selectedCalendar.get(Calendar.HOUR_OF_DAY)
00102                 val minute = selectedCalendar.get(Calendar.MINUTE)
00103
00104                 val timeStr = String.format("%02d:%02d", hour, minute)
00105                 val dateStr = "$day/$month/$year"
00106
00107                 tvSelectedDateTime.text = "Giao lúc: $timeStr, $dateStr"
00108                 tvSelectedDateTime.setTextColor(resources.getColor(android.R.color.black))
00109             } else {
00110                 tvSelectedDateTime.text = "Chưa chọn thời điểm giao"
00111                 tvSelectedDateTime.setTextColor(resources.getColor(android.R.color.holo_red_dark))
00112             }
00113         }
00114     }
00115 }

```

```

00113     }
00114
00115     btnDecrease.setOnClickListener { if (qty > 1) { qty--; updateUI() } }
00116     btnIncrease.setOnClickListener { qty++; updateUI() }
00117
00118     btnSelectDate.setOnClickListener {
00119         val calendar = Calendar.getInstance()
00120         val datePickerDialog = DatePickerDialog(this, { _, year, month, dayOfMonth ->
00121             selectedCalendar.set(Calendar.YEAR, year)
00122             selectedCalendar.set(Calendar.MONTH, month)
00123             selectedCalendar.set(Calendar.DAY_OF_MONTH, dayOfMonth)
00124             isDateSelected = true
00125             isTimeSelected = false // Reset time when date changes to force re-validation
00126             updateUI()
00127         }, calendar.get(Calendar.YEAR), calendar.get(Calendar.MONTH), calendar.get(Calendar.DAY_OF_MONTH))
00128
00129         datePickerDialog.datePicker.minDate = calendar.timeInMillis
00130         datePickerDialog.show()
00131     }
00132
00133     btnSelectTime.setOnClickListener {
00134         if (!isDateSelected) {
00135             Toast.makeText(this, "Vui lòng chọn ngày trước", Toast.LENGTH_SHORT).show()
00136             return@setOnClickListener
00137         }
00138
00139         val calendar = Calendar.getInstance()
00140         TimePickerDialog(this, { _, hourOfDay, minute ->
00141             val tempCalendar = selectedCalendar.clone() as Calendar
00142             tempCalendar.set(Calendar.HOUR_OF_DAY, hourOfDay)
00143             tempCalendar.set(Calendar.MINUTE, minute)
00144
00145             val now = Calendar.getInstance()
00146             val minTime = Calendar.getInstance()
00147             minTime.add(Calendar.MINUTE, 30)
00148
00149             if (tempCalendar.before(minTime)) {
00150                 Toast.makeText(this, "Thời gian giao phải ít nhất sau 30 phút kể từ bây giờ", Toast.LENGTH_LONG).show()
00151             } else {
00152                 selectedCalendar.set(Calendar.HOUR_OF_DAY, hourOfDay)
00153                 selectedCalendar.set(Calendar.MINUTE, minute)
00154                 isTimeSelected = true
00155                 updateUI()
00156             }
00157         }, calendar.get(Calendar.HOUR_OF_DAY), calendar.get(Calendar.MINUTE), true).show()
00158     }
00159
00160     btnAddToCart.setOnClickListener {
00161         if (!isDateSelected || !isTimeSelected) {
00162             Toast.makeText(this, "Chưa chọn thời điểm giao", Toast.LENGTH_SHORT).show()
00163         } else {
00164             addToPreOrderCart()
00165         }
00166     }
00167
00168     btnBack.setOnClickListener { finish() }
00169     updateUI()
00170 }
00171
00172 private fun addToPreOrderCart() {
00173     val day = selectedCalendar.get(Calendar.DAY_OF_MONTH)
00174     val month = selectedCalendar.get(Calendar.MONTH) + 1
00175     val year = selectedCalendar.get(Calendar.YEAR)
00176     val hour = selectedCalendar.get(Calendar.HOUR_OF_DAY)
00177     val minute = selectedCalendar.get(Calendar.MINUTE)
00178
00179     val timeStr = String.format("%02d:%02d", hour, minute)
00180     val dateStr = "$day/$month/$year"
00181
00182     pre_order_cart.preOrderList.add(
00183         PreOrderItem(
00184             id = foodId,
00185             name = foodName,
00186             price = price,
00187             qty = qty,
00188             imageUrl = foodImageUrl,
00189             deliveryTime = "$timeStr, $dateStr"
00190         )
00191     )
00192     Toast.makeText(this, "Đã thêm vào giỏ hàng đặt trước", Toast.LENGTH_SHORT).show()
00193     val intent = Intent(this, pre_order_cart::class.java)
00194     startActivity(intent)
00195     finish()
00196 }
00197
00198 }
00199
00200 }
00201
00202 }
00203 }

```

9.69 app/app/src/main/java/com/example/myapp/screens/pre_order_↵ _cart.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()

Variables

- lateinit var [AppCompatActivity.rvPreOrderCart](#)

9.70 pre_order_cart.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens
00002
00009
00010 import android.content.Intent
00011 import android.os.Bundle
00012 import android.view.LayoutInflater
00013 import android.view.View
00014 import android.view.ViewGroup
00015 import android.widget.CheckBox
00016 import android.widget.ImageView
00017 import android.widget.TextView
00018 import androidx.appcompat.app.AppCompatActivity
00019 import androidx.appcompat.widget.AppCompatButton
00020 import androidx.recyclerview.widget.LinearLayoutManager
00021 import androidx.recyclerview.widget.RecyclerView
00022 import com.example.myapp.R
00023 import com.squareup.picasso.Picasso
00024 import java.text.NumberFormat
00025 import java.util.Locale
00026
00038 data class PreOrderItem(
00039     val id: Int,
00040     val name: String,
00041     val price: Int,
00042     var qty: Int,
00043     val imageUrl: String? = null,
00044     val deliveryTime: String,
00045     var isSelected: Boolean = false
00046 )
00047
00056 class pre_order_cart : AppCompatActivity() {
00058     private lateinit var rvPreOrderCart: RecyclerView
00060     private lateinit var tvTotalItems: TextView
00062     private lateinit var tvTotalPrice: TextView
00064     private lateinit var cbSelectAll: CheckBox
00066     private lateinit var btnCheckout: AppCompatButton
00068     private lateinit var adapter: PreOrderCartAdapter
00069
00070     companion object {
00072         val preOrderList = mutableListOf<PreOrderItem>()
00073     }
00074
00083     override fun onCreate(savedInstanceState: Bundle?) {
00084         super.onCreate(savedInstanceState)
00085         setContentView(R.layout.activity_pre_order_cart)
00086
00087         // Anh xa View
00088         rvPreOrderCart = findViewById(R.id.rvPreOrderCart)
00089         tvTotalItems = findViewById(R.id.tvTotalItems)
00090         tvTotalPrice = findViewById(R.id.tvTotalPrice)
00091         cbSelectAll = findViewById(R.id.cbSelectAll)
```

```

00092     btnCheckout = findViewById(R.id.btnCheckout)
00093     val btnBack: ImageView = findViewById(R.id.btnBack)
00094     val btnNormalCart: ImageView = findViewById(R.id.btnNormalCart)
00095
00096     // Nút quay lại
00097     btnBack.setOnClickListener { finish() }
00098
00099     // Quay lại giỏ hàng thường
00100     btnNormalCart.setOnClickListener {
00101         val intent = Intent(this, cart::class.java)
00102         startActivity(intent)
00103         finish()
00104     }
00105
00106     // --- SỰ KIỆN CLICK THANH TOÁN ĐẶT TRƯỚC ---
00107     btnCheckout.setOnClickListener {
00108         val selectedItems = preOrderList.filter { it.isSelected }
00109         if (selectedItems.isEmpty()) return@setOnClickListener
00110
00111         // Chuyển đổi PreOrderItem sang CartItem để tương thích với màn hình order
00112         val cartItems = ArrayList<CartItem>(selectedItems.map {
00113             CartItem(
00114                 id = it.id,
00115                 name = it.name,
00116                 price = it.price,
00117                 qty = it.qty,
00118                 imageUrl = it.imageUrl,
00119                 isSelected = true
00120             )
00121         })
00122
00123         // Chuyển sang màn hình order và gửi danh sách món ăn
00124         val intent = Intent(this, order::class.java)
00125         intent.putExtra("selected_items", cartItems)
00126         intent.putExtra("is_pre_order", true)
00127         startActivity(intent)
00128     }
00129
00130     // Thiết lập RecyclerView
00131     adapter = PreOrderCartAdapter(
00132         preOrderList,
00133         onUpdate = { updateSummary() },
00134         onDelete = { position ->
00135             preOrderList.removeAt(position)
00136             adapter.notifyItemRemoved(position)
00137             adapter.notifyItemRangeChanged(position, preOrderList.size)
00138             updateSummary()
00139         }
00140     )
00141     rvPreOrderCart.layoutManager = LinearLayoutManager(this)
00142     rvPreOrderCart.adapter = adapter
00143
00144     // Chọn tất cả
00145     cbSelectAll.setOnCheckedChangeListener { _, isChecked ->
00146         preOrderList.forEach { it.isSelected = isChecked }
00147         adapter.notifyDataSetChanged()
00148         updateSummary()
00149     }
00150
00151     updateSummary()
00152 }
00153
00154 private fun updateSummary() {
00155     val selectedItems = preOrderList.filter { it.isSelected }
00156     val totalQty = selectedItems.sumOf { it.qty }
00157     val totalPrice = selectedItems.sumOf { it.qty * it.price }
00158
00159     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00160     tvTotalItems.text = totalQty.toString()
00161     tvTotalPrice.text = fmt.format(totalPrice) + "đ"
00162
00163     // Chỉ cho phép bấm nếu có chọn món
00164     btnCheckout.isEnabled = selectedItems.isNotEmpty()
00165     btnCheckout.alpha = if (selectedItems.isNotEmpty()) 1.0f else 0.5f
00166 }
00167
00168 class PreOrderCartAdapter(
00169     private val items: MutableList<PreOrderItem>,
00170     private val onUpdate: () -> Unit,
00171     private val onDelete: (Int) -> Unit
00172 ) : RecyclerView.Adapter<PreOrderCartAdapter.ViewHolder>() {
00173
00174     class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
00175         val imgFood: ImageView = view.findViewById(R.id.imgFood)
00176         val tvName: TextView = view.findViewById(R.id.tvName)
00177         val tvPrice: TextView = view.findViewById(R.id.tvPrice)
00178     }

```

```

00204     val tvQty: TextView = view.findViewById(R.id.tvQty)
00206     val tvDeliveryTime: TextView = view.findViewById(R.id.tvDeliveryTime)
00208     val btnPlus: TextView = view.findViewById(R.id.btnPlus)
00210     val btnMinus: TextView = view.findViewById(R.id.btnMinus)
00212     val cbSelect: CheckBox = view.findViewById(R.id.cbSelect)
00214     val btnDelete: ImageView = view.findViewById(R.id.btnDelete)
00215 }
00216
00224 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
00225     val view = LayoutInflater.from(parent.context).inflate(R.layout.item_pre_order_cart, parent, false)
00226     return ViewHolder(view)
00227 }
00228
00238 override fun onBindViewHolder(holder: ViewHolder, position: Int) {
00239     val item = items[position]
00240     val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00241
00242     holder.tvName.text = item.name
00243     holder.tvPrice.text = fmt.format(item.price) + "đ"
00244     holder.tvQty.text = item.qty.toString()
00245     holder.tvDeliveryTime.text = "Giao lúc: " + item.deliveryTime
00246     holder.cbSelect.isChecked = item.isSelected
00247
00248     if (!item.imageUrl.isNullOrEmpty()) {
00249         Picasso.get().load(item.imageUrl)
00250             .placeholder(R.drawable.placeholder_loading)
00251             .error(R.drawable.pngwing)
00252             .into(holder.imgFood)
00253     } else {
00254         holder.imgFood.setImageResource(R.drawable.pngwing)
00255     }
00256
00257     holder.btnPlus.setOnClickListener {
00258         item.qty++
00259         notifyItemChanged(holder.adapterPosition)
00260         onUpdate()
00261     }
00262
00263     holder.btnMinus.setOnClickListener {
00264         if (item.qty > 1) {
00265             item.qty--
00266             notifyItemChanged(holder.adapterPosition)
00267             onUpdate()
00268         }
00269     }
00270
00271     holder.cbSelect.setOnClickListener {
00272         item.isSelected = holder.cbSelect.isChecked
00273         onUpdate()
00274     }
00275
00276     holder.btnDelete.setOnClickListener {
00277         onDelete(holder.adapterPosition)
00278     }
00279 }
00280
00286 override fun getItemCount() = items.size
00287 }

```

9.71 app/app/src/main/java/com/example/myapp/screens/profile.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState: Bundle?)

9.72 profile.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapplication.screens
00002
00011
00012 import android.content.Context
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.view.View
00016 import android.widget.ImageView
00017 import android.widget.LinearLayout
00018 import android.widget.TextView
00019 import androidx.appcompat.app.AppCompatActivity
00020 import androidx.core.content.edit
00021 import com.example.myapplication.R
00022 import com.example.myapplication.screens.api.RetrofitClient
00023 import com.example.myapplication.screens.api.UserProfileSummary
00024 import retrofit2.Call
00025 import retrofit2.Callback
00026 import retrofit2.Response
00027
00037 class profile : AppCompatActivity() {
00046     override fun onCreate(savedInstanceState: Bundle?) {
00047         super.onCreate(savedInstanceState)
00048         setContentView(R.layout.profile)
00049
00050         val tvToi: TextView = findViewById(R.id.tvToi)
00051         val tvPoints: TextView = findViewById(R.id.tvPoints)
00052         val imgCoin: ImageView = findViewById(R.id.imgCoin)
00053
00054         val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00055         val userName = sharedPref.getString("user_name", "Tôì")
00056         val userId = sharedPref.getInt("user_id", -1)
00057
00058         tvToi.text = userName
00059
00060         // Load points from API
00061         if (userId > 0) {
00062             RetrofitClient.apiService.getProfileSummary(userId).enqueue(object : Callback<UserProfileSummary> {
00063                 override fun onResponse(call: Call<UserProfileSummary>, response: Response<UserProfileSummary>) {
00064                     if (response.isSuccessful) {
00065                         response.body()?.let {
00066                             tvToi.text = it.user_name
00067                             tvPoints.text = it.points.toString()
00068                         }
00069                     }
00070                 }
00071             })
00072         }
00073     }
00074
00075     // Click points or coin to open order tracking
00076     val openTracking = View.OnClickListener {
00077         val intent = Intent(this, OrderTracking::class.java)
00078         startActivity(intent)
00079     }
00080     tvPoints.setOnClickListener(openTracking)
00081     imgCoin.setOnClickListener(openTracking)
00082
00083     // Navigation & Other buttons
00084     findViewById<LinearLayout>(R.id.btnlogout).setOnClickListener {
00085         getSharedPreferences("user", Context.MODE_PRIVATE).edit { clear() }
00086         val intent = Intent(this, signin::class.java).apply {
00087             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00088         }
00089         startActivity(intent)
00090         finish()
00091     }
00092
00093     findViewById<ImageView>(R.id.icHome).setOnClickListener {
00094         startActivity(Intent(this, home::class.java))
00095     }
00096
00097     findViewById<ImageView>(R.id.icCart).setOnClickListener {
00098         startActivity(Intent(this, cart::class.java))
00099     }
00100
00101     findViewById<View>(R.id.btnSupport).setOnClickListener {
00102         startActivity(Intent(this, customer_support::class.java))
00103     }
00104
00105     findViewById<LinearLayout>(R.id.btnAddress).setOnClickListener {
00106         startActivity(Intent(this, savedeliveryaddress::class.java))
00107     }

```

```

00108
00109     findViewById<LinearLayout>(R.id.btnHistory).setOnClickListener {
00110         startActivity(Intent(this, order_history::class.java))
00111     }
00112 }
00113 }

```

9.73 app/app/src/main/java/com/example/myapp/screens/restaurant_↵ _profile.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()

Variables

- var [AppCompatActivity.restaurant](#)

9.74 restaurant_profile.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.content.Intent
00014 import android.os.Bundle
00015 import android.widget.ImageView
00016 import android.widget.TextView
00017 import android.widget.Toast
00018 import androidx.appcompat.app.AppCompatActivity
00019 import androidx.recyclerview.widget.LinearLayoutManager
00020 import androidx.recyclerview.widget.RecyclerView
00021 import com.example.myapp.R
00022 import com.example.myapp.adapters.MenuItemAdapterSmall
00023 import com.example.myapp.screens.api.RetrofitClient
00024 import com.example.myapp.screens.api.Restaurant
00025 import com.example.myapp.screens.home
00026 import com.squareup.picasso.Picasso
00027 import retrofit2.Call
00028 import retrofit2.Callback
00029 import retrofit2.Response
00030
00042 class restaurant_profile: AppCompatActivity() {
00043     private var restaurant: Restaurant? = null
00044     private lateinit var rvMenuItems: RecyclerView
00045     private lateinit var menuItemAdapter: MenuItemAdapterSmall
00046
00054     override fun onCreate(savedInstanceState: Bundle?) {
00055         super.onCreate(savedInstanceState)
00056         setContentView(R.layout.restaurant_profile)
00057
00058         // click btnBack
00059         val btnBack: ImageView = findViewById(R.id.btn_back)
00060         btnBack.setOnClickListener {
00061             finish()
00062         }
00063
00064         // click icProfile
00065         val icProfile: ImageView = findViewById(R.id.icProfile)
00066         icProfile.setOnClickListener {
00067             val intent = Intent(this, profile::class.java)
00068             startActivity(intent)

```

```

00069     }
00070
00071     // click icHome
00072     val icHome: ImageView = findViewById(R.id.icHome)
00073     icHome.setOnClickListener {
00074         val intent = Intent(this, home::class.java)
00075         startActivity(intent)
00076     }
00077
00078     val btnCart: ImageView = findViewById(R.id.icCart)
00079     btnCart.setOnClickListener {
00080         val intent = Intent(this, cart::class.java)
00081         startActivity(intent)
00082     }
00083
00084     val btn_share: ImageView = findViewById(R.id.btn_share)
00085     btn_share.setOnClickListener {
00086         restaurant?.let { rest ->
00087             val intent = Intent(this, share_restaurant::class.java).apply {
00088                 putExtra("restaurant_name", rest.name)
00089                 putExtra("restaurant_address", rest.address)
00090                 putExtra("restaurant_rating", rest.rating ?: 0)
00091                 putExtra("restaurant_image_url", rest.image_url)
00092                 putExtra("restaurant_open_time", rest.open_time)
00093                 putExtra("restaurant_close_time", rest.close_time)
00094             }
00095             startActivity(intent)
00096         }
00097     }
00098
00099     // Setup RecyclerView
00100     rvMenuItems = findViewById(R.id.rvMenuItems)
00101     rvMenuItems.layoutManager = LinearLayoutManager(this, LinearLayoutManager.HORIZONTAL, false)
00102
00103     // Get restaurant ID from intent
00104     val restaurantId = intent.getIntExtra("RESTAURANT_ID", -1)
00105     if (restaurantId != -1) {
00106         fetchRestaurantDetails(restaurantId)
00107     } else {
00108         Toast.makeText(this, "Không tìm thấy nhà hàng", Toast.LENGTH_SHORT).show()
00109         finish()
00110     }
00111 }
00112
00121 private fun fetchRestaurantDetails(restaurantId: Int) {
00122     RetrofitClient.apiService.getRestaurant(restaurantId).enqueue(object : Callback<Restaurant> {
00123         override fun onResponse(call: Call<Restaurant>, response: Response<Restaurant>) {
00124             if (response.isSuccessful) {
00125                 restaurant = response.body()
00126                 displayRestaurantDetails()
00127             } else {
00128                 Toast.makeText(this@restaurant_profile, "Lỗi khi tải thông tin nhà hàng", Toast.LENGTH_SHORT).show()
00129             }
00130         }
00131     })
00132     override fun onFailure(call: Call<Restaurant>, t: Throwable) {
00133         Toast.makeText(this@restaurant_profile, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00134     }
00135 }
00136
00144 private fun displayRestaurantDetails() {
00145     restaurant?.let { restaurant ->
00146         // Display restaurant banner image
00147         val imgRestaurantBanner: ImageView = findViewById(R.id.imgRestaurantBanner)
00148         if (!restaurant.image_url.isNullOrEmpty()) {
00149             Picasso.get()
00150                 .load(restaurant.image_url)
00151                 .placeholder(R.drawable.placeholder_loading)
00152                 .error(R.drawable.pngwing)
00153                 .into(imgRestaurantBanner)
00154         }
00155
00156         // Display restaurant name
00157         val tvRestaurantName: TextView = findViewById(R.id.tvRestaurantName)
00158         tvRestaurantName.text = restaurant.name
00159
00160         // Display restaurant rating
00161         val tvRating: TextView = findViewById(R.id.tvRating)
00162         tvRating.text = restaurant.rating?.toString() ?: "N/A"
00163
00164         // Display restaurant address
00165         val tvAddress: TextView = findViewById(R.id.tvAddress)
00166         tvAddress.text = restaurant.address ?: "Chưa có địa chỉ"
00167
00168         // Display opening hours
00169         val tvOpeningHours: TextView = findViewById(R.id.tvOpeningHours)

```

```

00170         val openTime = restaurant.open_time ?: "N/A"
00171         val closeTime = restaurant.close_time ?: "N/A"
00172         tvOpeningHours.text = "Giờ mở cửa: $openTime - $closeTime"
00173
00174         // Display phone number
00175         val tvPhoneNumber: TextView = findViewById(R.id.tvPhoneNumber)
00176         tvPhoneNumber.text = "Số điện thoại: ${restaurant.phone_number}"
00177
00178         // Display description
00179         val tvDescription: TextView = findViewById(R.id.tvDescription)
00180         tvDescription.text = "Mô tả: ${restaurant.description ?: "Chưa có mô tả"}"
00181
00182         // Display menu items
00183         displayMenuItems(restaurant.menu_items)
00184     }
00185 }
00186
00195 private fun displayMenuItems(menuItems: List<com.example.myapplication.screens.api.MenuItem>) {
00196     val availableMenuItems = menuItems.filter { it.is_available }
00197     menuItemAdapter = MenuItemAdapterSmall(this, availableMenuItems)
00198     rvMenuItems.adapter = menuItemAdapter
00199 }
00200 }

```

9.75 app/app/src/main/java/com/example/myapp/screens/savedeliveryaddress.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- lateinit var chatbot::class.java [AppCompatActivity.startActivity](#) (intent) } } private fun loadFAQs()

Variables

- lateinit var [AppCompatActivity.rvAddresses](#)

9.76 savedeliveryaddress.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapplication.screens
00002
00009
00010 import android.content.Context
00011 import android.content.Intent
00012 import android.os.Bundle
00013 import android.widget.Button
00014 import android.widget.ImageView
00015 import android.widget.Toast
00016 import androidx.appcompat.app.AppCompatActivity
00017 import androidx.recyclerview.widget.LinearLayoutManager
00018 import androidx.recyclerview.widget.RecyclerView
00019 import com.example.myapplication.R
00020 import com.example.myapplication.adapters.AddressAdapter
00021 import com.example.myapplication.screens.api.Address
00022 import com.example.myapplication.screens.api.ApiService
00023 import retrofit2.Call
00024 import retrofit2.Callback
00025 import retrofit2.Response
00026 import retrofit2.Retrofit
00027 import retrofit2.converter.gson.GsonConverterFactory
00028
00036 class savedeliveryaddress : AppCompatActivity() {
00038     private lateinit var rvAddresses: RecyclerView

```

```

00040 private lateinit var addressAdapter: AddressAdapter
00042 private val addressList = mutableListOf<Address>()
00043
00052 override fun onCreate(savedInstanceState: Bundle?) {
00053     super.onCreate(savedInstanceState)
00054     setContentView(R.layout.savedeliveryaddress)
00055
00056     val btnBack: ImageView = findViewById(R.id.btn_back)
00057     btnBack.setOnClickListener {
00058         finish() // quay lại màn hình trước
00059     }
00060     // 1. Click icon home để về trang chủ
00061     val ic_Home: ImageView = findViewById(R.id.icHome)
00062     ic_Home.setOnClickListener {
00063         val intent = Intent(this, home::class.java)
00064         startActivity(intent)
00065     }
00066
00067     val btnCart: ImageView = findViewById(R.id.icCart)
00068     btnCart.setOnClickListener {
00069         val intent = Intent(this, cart::class.java)
00070         startActivity(intent)
00071     }
00072     // 2. Click vào nút "Thêm địa chỉ giao hàng" (Button) sang màn hình thêm
00073     val btnAddAddress: Button = findViewById(R.id.btnAddAddress)
00074     btnAddAddress.setOnClickListener {
00075         val intent = Intent(this, add_shipping_address::class.java)
00076         startActivity(intent)
00077     }
00078     // click icProfile
00079     val icProfile: ImageView = findViewById(R.id.icProfile)
00080     icProfile.setOnClickListener {
00081         val intent = Intent(this, profile::class.java)
00082         startActivity(intent)
00083     }
00084
00085     val icHome: ImageView = findViewById(R.id.icHome)
00086     icHome.setOnClickListener {
00087         val intent = Intent(this, home::class.java)
00088         startActivity(intent)
00089     }
00090
00091     // Setup RecyclerView
00092     rvAddresses = findViewById(R.id.rvAddresses)
00093     rvAddresses.layoutManager = LinearLayoutManager(this)
00094     addressAdapter = AddressAdapter(addressList) { address ->
00095         // Handle edit click
00096         val intent = Intent(this, edit_shipping_address::class.java)
00097         intent.putExtra("address_id", address.id)
00098         startActivity(intent)
00099     }
00100     rvAddresses.adapter = addressAdapter
00101
00102     // Fetch addresses from API
00103     fetchUserAddresses()
00104 }
00105
00112 private fun fetchUserAddresses() {
00113     // Lấy userId từ SharedPreferences
00114     val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00115     val userId = sharedPref.getInt("user_id", 1)
00116
00117     val retrofit = Retrofit.Builder()
00118         .baseUrl("http://10.0.2.2:8000/") // Android emulator localhost
00119         .addConverterFactory(GsonConverterFactory.create())
00120         .build()
00121
00122     val apiService = retrofit.create(ApiService::class.java)
00123
00124     apiService.getUserAddresses(userId).enqueue(object : Callback<List<Address>> {
00125         override fun onResponse(call: Call<List<Address>>, response: Response<List<Address>>) {
00126             if (response.isSuccessful) {
00127                 val addresses = response.body() ?: emptyList()
00128                 addressList.clear()
00129                 addressList.addAll(addresses)
00130                 addressAdapter.notifyDataSetChanged()
00131             } else {
00132                 Toast.makeText(this@savedeliveryaddress, "Không thể tải danh sách địa chỉ",
00133                     Toast.LENGTH_SHORT).show()
00134             }
00135         }
00136
00137         override fun onFailure(call: Call<List<Address>>, t: Throwable) {
00138             Toast.makeText(this@savedeliveryaddress, "Lỗi kết nối: ${t.message}", Toast.LENGTH_SHORT).show()
00139         }
00140     })
}

```

```
00141 }
```

9.77 app/app/src/main/java/com/example/myapp/screens/share.kt File Reference

Packages

- package [AppCompatActivity](#)

Variables

- var [AppCompatActivity.foodId](#)

9.78 share.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens
00002
00009
00010 import android.content.Intent
00011 import android.net.Uri
00012 import android.os.Bundle
00013 import android.widget.ImageView
00014 import android.widget.LinearLayout
00015 import android.widget.TextView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import com.example.myapp.R
00019 import com.squareup.picasso.Picasso
00020 import java.text.NumberFormat
00021 import java.util.Locale
00022
00030 class share : AppCompatActivity() {
00032     private var foodId: Int = -1
00034     private var foodName: String = ""
00036     private var foodDescription: String = ""
00037
00046     override fun onCreate(savedInstanceState: Bundle?) {
00047         super.onCreate(savedInstanceState)
00048         setContentView(R.layout.activity_share)
00049
00050         foodId = intent.getIntExtra("food_id", -1)
00051         foodName = intent.getStringExtra("food_name") ?: ""
00052         val foodPrice = intent.getIntExtra("food_price", 0)
00053         foodDescription = intent.getStringExtra("food_description") ?: ""
00054         val foodImageUrl = intent.getStringExtra("food_image_url") ?: ""
00055
00056         val tvFoodName: TextView = findViewById(R.id.tvFoodName)
00057         val tvFoodDescription: TextView = findViewById(R.id.tvFoodDescription)
00058         val tvFoodPrice: TextView = findViewById(R.id.tvFoodPrice)
00059         val imgFood: ImageView = findViewById(R.id.imgFood)
00060
00061         val fmt = NumberFormat.getNumberInstance(Locale("vi", "VN"))
00062
00063         tvFoodName.text = foodName
00064         tvFoodDescription.text = foodDescription
00065         tvFoodPrice.text = "Giá: " + fmt.format(foodPrice) + "đ"
00066
00067         if (foodImageUrl.isNotEmpty()) {
00068             Picasso.get()
00069                 .load(foodImageUrl)
00070                 .placeholder(R.drawable.placeholder_loading)
00071                 .error(R.drawable.pngwing)
00072                 .into(imgFood)
00073         }
00074
00075         findViewById<LinearLayout>(R.id.btnFacebook).setOnClickListener { shareToApp("com.facebook.katana") }
00076         findViewById<LinearLayout>(R.id.btnZalo).setOnClickListener { shareToApp("com.zing.zalo") }
00077         findViewById<LinearLayout>(R.id.btnTwitter).setOnClickListener { shareToApp("com.twitter.android") }
```

```

00078     findViewById<LinearLayout>(R.id.btnMessenger).setOnClickListener { shareToApp("com.facebook.orca") }
00079
00080     findViewById<ImageView>(R.id.btnBack).setOnClickListener { finish() }
00081 }
00082
00092 private fun shareToApp(packageName: String) {
00093     val shareLink = "https://yourapp.com/food?id=$foodId"
00094     val shareText = "Thử ngay món $foodName cực ngon tại MyApp!\n$n$foodDescription\nXem chi tiết tại: $shareLink"
00095
00096     val intent = Intent(Intent.ACTION_SEND).apply {
00097         type = "text/plain"
00098         putExtra(Intent.EXTRA_TEXT, shareText)
00099         setPackage(packageName)
00100     }
00101
00102     try {
00103         startActivity(intent)
00104     } catch (e: Exception) {
00105         // Fallback if app not installed
00106         val chooser = Intent.createChooser(Intent(Intent.ACTION_SEND).apply {
00107             type = "text/plain"
00108             putExtra(Intent.EXTRA_TEXT, shareText)
00109             }, "Chia sẻ qua")
00110         startActivity(chooser)
00111     }
00112 }
00113 }

```

9.79 app/app/src/main/java/com/example/myapp/screens/share_restaurant.kt File Reference ↗

Packages

- package [AppCompatActivity](#)

Variables

- var [AppCompatActivity.restaurantId](#)

9.80 share_restaurant.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00009
00010 import android.content.Intent
00011 import android.net.Uri
00012 import android.os.Bundle
00013 import android.widget.ImageView
00014 import android.widget.LinearLayout
00015 import android.widget.TextView
00016 import androidx.appcompat.app.AppCompatActivity
00017 import com.example.myapp.R
00018 import com.squareup.picasso.Picasso
00019
00027 class share_restaurant : AppCompatActivity() {
00029     private var restaurantId: Int = -1
00031     private var restaurantName: String = ""
00033     private var restaurantAddress: String = ""
00034
00044     override fun onCreate(savedInstanceState: Bundle?) {
00045         super.onCreate(savedInstanceState)
00046         setContentView(R.layout.activity_share_restaurant)
00047
00048         restaurantId = intent.getIntExtra("RESTAURANT_ID", -1)
00049         restaurantName = intent.getStringExtra("restaurant_name") ?: ""
00050         restaurantAddress = intent.getStringExtra("restaurant_address") ?: ""
00051         val rating = intent.getIntExtra("restaurant_rating", 0)
00052         val imageUrl = intent.getStringExtra("restaurant_image_url") ?: ""

```

```

00053     val openTime = intent.getStringExtra("restaurant_open_time") ?: ""
00054     val closeTime = intent.getStringExtra("restaurant_close_time") ?: ""
00055
00056     val tvRestName: TextView = findViewById(R.id.tvRestName)
00057     val tvRating: TextView = findViewById(R.id.tvRating)
00058     val tvAddress: TextView = findViewById(R.id.tvAddress)
00059     val tvOpenTime: TextView = findViewById(R.id.tvOpenTime)
00060     val imgRestaurant: ImageView = findViewById(R.id.imgRestaurant)
00061
00062     tvRestName.text = restaurantName
00063     tvRating.text = "$rating "
00064     tvAddress.text = restaurantAddress
00065     tvOpenTime.text = "Mở cửa: $openTime - $closeTime"
00066
00067     if (imageUrl.isNotEmpty()) {
00068         Picasso.get()
00069             .load(imageUrl)
00070             .placeholder(R.drawable.placeholder_loading)
00071             .error(R.drawable.pngwing)
00072             .into(imgRestaurant)
00073     }
00074
00075     findViewById<LinearLayout>(R.id.btnFacebook).setOnClickListener { shareToApp("com.facebook.katana") }
00076     findViewById<LinearLayout>(R.id.btnZalo).setOnClickListener { shareToApp("com.zing.zalo") }
00077     findViewById<LinearLayout>(R.id.btnTwitter).setOnClickListener { shareToApp("com.twitter.android") }
00078     findViewById<LinearLayout>(R.id.btnMessenger).setOnClickListener { shareToApp("com.facebook.orca") }
00079
00080     findViewById<ImageView>(R.id.btnBack).setOnClickListener { finish() }
00081 }
00082
00092 private fun shareToApp(packageName: String) {
00093     val shareLink = "https://yourapp.com/restaurant?id=$restaurantId"
00094     val shareText = "Khám phá ngay nhà hàng $restaurantName tại MyApp!\nĐịa chỉ: $restaurantAddress\nXem thêm
tại: $shareLink"
00095
00096     val intent = Intent(Intent.ACTION_SEND).apply {
00097         type = "text/plain"
00098         putExtra(Intent.EXTRA_TEXT, shareText)
00099         setPackage(packageName)
00100     }
00101
00102     try {
00103         startActivity(intent)
00104     } catch (e: Exception) {
00105         val chooser = Intent.createChooser(Intent(Intent.ACTION_SEND).apply {
00106             type = "text/plain"
00107             putExtra(Intent.EXTRA_TEXT, shareText)
00108         }, "Chia sẻ qua")
00109         startActivity(chooser)
00110     }
00111 }
00112 }

```

9.81 app/app/src/main/java/com/example/myapp/screens/shipping_↵ address_added_successfully.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState:Bundle?)

9.82 shipping_address_added_successfully.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002

```



```
00008
00009 import android.content.Intent
00010 import android.os.Bundle
00011 import android.widget.Button // Import Button
00012 import android.widget.ImageView
00013 import androidx.appcompat.app.AppCompatActivity
00014 import com.example.myapp.R
00015 import com.example.myapp.screens.home
00016
00024 class shipping_address_added_successfully : AppCompatActivity() {
00033     override fun onCreate(savedInstanceState: Bundle?) {
00034         super.onCreate(savedInstanceState)
00035         setContentView(R.layout.shipping_address_added_successfully)
00036
00037         // Tìm nút "Trở về trang chủ" theo ID btnReturnHome
00038         val btnReturnHome: Button = findViewById(R.id.btnReturnHome)
00039
00040         // Thiết lập sự kiện click
00041         btnReturnHome.setOnClickListener {
00042             // Quay về trang chủ (Home)
00043             val intent = Intent(this, home::class.java)
00044             // Xóa các trang trước đó để không bị quay lại trang thành công này khi nhấn Back
00045             intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00046             startActivity(intent)
00047             finish()
00048         }
00049         // click icProfile
00050         val icProfile: ImageView = findViewById(R.id.icProfile)
00051         icProfile.setOnClickListener {
00052             val intent = Intent(this, profile::class.java)
00053             startActivity(intent)
00054         }
00055
00056         val icHome: ImageView = findViewById(R.id.icHome)
00057         icHome.setOnClickListener {
00058             val intent = Intent(this, home::class.java)
00059             startActivity(intent)
00060         }
00061
00062         val btnCart: ImageView = findViewById(R.id.icCart)
00063         btnCart.setOnClickListener {
00064             val intent = Intent(this, cart::class.java)
00065             startActivity(intent)
00066         }
00067     }
00068 }
```

9.83 app/app/src/main/java/com/example/myapp/screens/shipping_address_fixed_successfully.kt File Reference ↩

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState:Bundle?)

9.84 shipping_address_fixed_successfully.kt

[Go to the documentation of this file.](#)

```
00001 package com.example.myapp.screens
00002
00008
00009 import android.content.Intent
00010 import android.os.Bundle
00011 import android.widget.Button // Import thêm Button
00012 import android.widget.ImageView
00013 import androidx.appcompat.app.AppCompatActivity
```

```

00014 import com.example.myapp.R
00015 import com.example.myapp.screens.home
00016
00024 class shipping_address_fixed_successfully : AppCompatActivity() {
00033     override fun onCreate(savedInstanceState: Bundle?) {
00034         super.onCreate(savedInstanceState)
00035         setContentView(R.layout.shipping_address_fixed_successfully)
00036
00037         // Tìm nút "Trở về trang chủ" bằng ID đã đặt trong XML
00038         val btnReturnHome: Button = findViewById(R.id.btnReturnHome)
00039
00040         // Cài đặt sự kiện click
00041         btnReturnHome.setOnClickListener {
00042             // Tạo Intent chuyển về trang Home
00043             val intent = Intent(this, home::class.java)
00044             // Xóa các Activity trước đó để không bị quay lại trang thông báo này
00045             intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
00046             startActivity(intent)
00047             finish()
00048         }
00049         // click icProfile
00050         val icProfile: ImageView = findViewById(R.id.icProfile)
00051         icProfile.setOnClickListener {
00052             val intent = Intent(this, profile::class.java)
00053             startActivity(intent)
00054         }
00055
00056         val icHome: ImageView = findViewById(R.id.icHome)
00057         icHome.setOnClickListener {
00058             val intent = Intent(this, home::class.java)
00059             startActivity(intent)
00060         }
00061         val btnCart: ImageView = findViewById(R.id.icCart)
00062         btnCart.setOnClickListener {
00063             val intent = Intent(this, cart::class.java)
00064             startActivity(intent)
00065         }
00066     }
00067 }

```

9.85 app/app/src/main/java/com/example/myapp/screens/signin.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState:Bundle?)

9.86 signin.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Context
00013 import android.content.Intent
00014 import android.os.Bundle
00015
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import android.view.View
00019 import android.widget.EditText
00020 import android.widget.ImageView
00021 import com.example.myapp.R
00022 import com.example.myapp.screens.api.LoginResponse
00023 import com.example.myapp.screens.api.FcmTokenRegistrar

```

```

00024 import com.example.myapp.screens.api.RetrofitClient
00025 import retrofit2.Call
00026 import retrofit2.Callback
00027 import retrofit2.Response
00028
00040 class signin : AppCompatActivity() {
00049     override fun onCreate(savedInstanceState: Bundle?) {
00050         super.onCreate(savedInstanceState)
00051         setContentView(R.layout.signin)
00052
00053         // click btnBack
00054         val btnBack: ImageView = findViewById(R.id.btn_back)
00055         btnBack.setOnClickListener {
00056             finish() // quay lại màn hình trước (start)
00057         }
00058
00059         // click btnSignIn
00060         val btnSignIn: View = findViewById(R.id.btn_signin)
00061         btnSignIn.setOnClickListener {
00062             val etUsername: EditText = findViewById(R.id.et_username)
00063             val etPassword: EditText = findViewById(R.id.et_password)
00064
00065             val username = etUsername.text.toString().trim()
00066             val password = etPassword.text.toString().trim()
00067
00068             if (username.isEmpty() || password.isEmpty()) {
00069                 Toast.makeText(this, "Vui lòng nhập đầy đủ thông tin", Toast.LENGTH_SHORT).show()
00070                 return@setOnClickListener
00071             }
00072
00073             // Gọi API đăng nhập
00074             RetrofitClient.apiService.login(username, password).enqueue(object : Callback<LoginResponse> {
00075                 override fun onResponse(call: Call<LoginResponse>, response: Response<LoginResponse>) {
00076                     if (response.isSuccessful) {
00077                         val loginResponse = response.body()
00078                         if (loginResponse != null) {
00079                             // Đăng nhập thành công
00080                             Toast.makeText(this@signin, "Đăng nhập thành công", Toast.LENGTH_SHORT).show()
00081
00082                             // Lưu access token vào SharedPreferences
00083                             val sharedPref = getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
00084                             with(sharedPref.edit()) {
00085                                 putString("access_token", loginResponse.access_token)
00086                                 apply()
00087                             }
00088
00089                             FcmTokenRegistrar.syncCurrentToken(this@signin, "signin")
00090
00091                             // Lấy thông tin user hiện tại
00092                             RetrofitClient.apiService.getCurrentUser().enqueue(object :
00093                                 Callback<com.example.myapp.screens.api.RegisterResponse> {
00094                                     override fun onResponse(call: Call<com.example.myapp.screens.api.RegisterResponse>, response:
00095                                         Response<com.example.myapp.screens.api.RegisterResponse>) {
00096                                         if (response.isSuccessful) {
00097                                             val user = response.body()
00098                                             if (user != null) {
00099                                                 // Lưu user ID và tên vào SharedPreferences
00100                                                 with(sharedPref.edit()) {
00101                                                     putInt("user_id", user.id)
00102                                                     putString("user_name", user.name)
00103                                                     apply()
00104                                                 }
00105                                             }
00106                                         }
00107                                     }
00108                                 // Chuyển đến màn hình chính
00109                                 val intent = Intent(this@signin, home::class.java)
00110                                 startActivity(intent)
00111                                 finish()
00112                             }
00113
00114                             override fun onFailure(call: Call<com.example.myapp.screens.api.RegisterResponse>, t: Throwable) {
00115                                 // Vẫn chuyển đến màn hình chính ngay cả khi lỗi
00116                                 val intent = Intent(this@signin, home::class.java)
00117                                 startActivity(intent)
00118                                 finish()
00119                             }
00120                         }
00121                     } else {
00122                         // Đăng nhập thất bại
00123                         Toast.makeText(this@signin, "Đăng nhập thất bại", Toast.LENGTH_SHORT).show()
00124                     }
00125                 } else {
00126                     // Lấy thông báo lỗi từ server
00127                     val errorBody = response.errorBody()?.string()
00128                     val errorMessage = if (errorBody != null) {
00129                         try {
00130                             val jsonObject = org.json.JSONObject(errorBody)

```

```

00128             if (jsonObject.has("detail")) {
00129                 jsonObject.getString("detail")
00130             } else {
00131                 "Sai tên đăng nhập hoặc mật khẩu"
00132             }
00133         } catch (e: Exception) {
00134             "Sai tên đăng nhập hoặc mật khẩu"
00135         }
00136     } else {
00137         "Sai tên đăng nhập hoặc mật khẩu"
00138     }
00139     Toast.makeText(this@signin, errorMessage, Toast.LENGTH_SHORT).show()
00140 }
00141 }
00142
00143 override fun onFailure(call: Call<LoginResponse>, t: Throwable) {
00144     Toast.makeText(this@signin, "Lỗi mạng: ${t.message}", Toast.LENGTH_SHORT).show()
00145 }
00146 })
00147 }
00148 }
00149 }

```

9.87 app/app/src/main/java/com/example/myapp/screens/signup.kt File Reference

Packages

- package [AppCompatActivity](#)

Functions

- override fun [AppCompatActivity.onCreate](#) (savedInstanceState:Bundle?)

9.88 signup.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.content.Intent
00013 import android.os.Bundle
00014 import android.widget.ImageView
00015 import android.widget.TextView
00016 import android.widget.Toast
00017 import androidx.appcompat.app.AppCompatActivity
00018 import android.view.View
00019 import android.widget.EditText
00020 import com.example.myapp.R
00021 import com.example.myapp.screens.api.RegisterRequest
00022 import com.example.myapp.screens.api.RegisterResponse
00023 import com.example.myapp.screens.api.RetrofitClient
00024 import retrofit2.Call
00025 import retrofit2.Callback
00026 import retrofit2.Response
00027
00038 class signup : AppCompatActivity() {
00048     override fun onCreate(savedInstanceState: Bundle?) {
00049         super.onCreate(savedInstanceState)
00050         setContentView(R.layout.signup)
00051
00052         // click btnBack
00053         val btnBack: ImageView = findViewById(R.id.btn_back)
00054         btnBack.setOnClickListener {
00055             finish()
00056         }
00057
00058         // click btnSignIn
00059         val tvLogin: TextView = findViewById(R.id.tvLogin)

```

```

00060     tvLogin.setOnClickListener {
00061         val intent = Intent(this, signin::class.java)
00062         startActivity(intent)
00063     }
00064
00065     // click btnSignUp
00066     val btnSignUp: View = findViewById(R.id.btn_signup)
00067     btnSignUp.setOnClickListener {
00068         val etUsername: EditText = findViewById(R.id.et_username)
00069         val etPassword: EditText = findViewById(R.id.et_password)
00070         val etRePassword: EditText = findViewById(R.id.et_re_password)
00071
00072         val username = etUsername.text.toString().trim()
00073         val password = etPassword.text.toString().trim()
00074         val rePassword = etRePassword.text.toString().trim()
00075
00076         if (username.isEmpty() || password.isEmpty() || rePassword.isEmpty()) {
00077             Toast.makeText(this, "Vui lòng nhập đầy đủ thông tin", Toast.LENGTH_SHORT).show()
00078             return@setOnClickListener
00079         }
00080
00081         if (password != rePassword) {
00082             Toast.makeText(this, "Mật khẩu không khớp", Toast.LENGTH_SHORT).show()
00083             return@setOnClickListener
00084         }
00085
00086         // Gọi API đăng ký
00087         val registerRequest = RegisterRequest(
00088             name = username,
00089             username = username,
00090             email = "$username@example.com",
00091             phone = "0000000000",
00092             role = "user",
00093             password = password
00094         )
00095         RetrofitClient.apiService.register(registerRequest).enqueue(object : Callback<RegisterResponse> {
00096             override fun onResponse(call: Call<RegisterResponse>, response: Response<RegisterResponse>) {
00097                 if (response.isSuccessful) {
00098                     val registerResponse = response.body()
00099                     if (registerResponse != null) {
00100                         // Đăng ký thành công
00101                         Toast.makeText(this@signup, "Đăng ký thành công", Toast.LENGTH_SHORT).show()
00102                         // Chuyển đến màn hình đăng nhập
00103                         val intent = Intent(this@signup, signin::class.java)
00104                         startActivity(intent)
00105                         finish()
00106                     } else {
00107                         // Đăng ký thất bại
00108                         Toast.makeText(this@signup, "Đăng ký thất bại", Toast.LENGTH_SHORT).show()
00109                     }
00110                 } else {
00111                     // Lấy thông báo lỗi từ server
00112                     val responseBody = response.errorBody()?.string()
00113                     val errorMessage = if (errorBody != null) {
00114                         try {
00115                             val jsonObject = org.json.JSONObject(errorBody)
00116                             if (jsonObject.has("detail")) {
00117                                 jsonObject.getString("detail")
00118                             } else {
00119                                 "Đăng ký thất bại"
00120                             }
00121                         } catch (e: Exception) {
00122                             "Đăng ký thất bại"
00123                         }
00124                     } else {
00125                         "Đăng ký thất bại"
00126                     }
00127                     Toast.makeText(this@signup, errorMessage, Toast.LENGTH_SHORT).show()
00128                 }
00129             }
00130         })
00131         override fun onFailure(call: Call<RegisterResponse>, t: Throwable) {
00132             Toast.makeText(this@signup, "Lỗi mạng: ${t.message}", Toast.LENGTH_SHORT).show()
00133         }
00134     })
00135 }
00136 }
00137 }

```

9.89 app/app/src/main/java/com/example/myapp/screens/start.kt File Reference

Packages

- package [AppCompatActivity](#)

9.90 start.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00011
00012 import android.Manifest
00013 import android.content.Intent
00014 import android.content.pm.PackageManager
00015 import android.os.Bundle
00016 import android.os.Build
00017 import androidx.appcompat.app.AppCompatActivity
00018 import android.view.View
00019 import com.example.myapp.screens.api.RetrofitClient
00020
00021
00022 // Thêm các bản import này
00023 import android.util.Log
00024 import androidx.activity.result.contract.ActivityResultContracts
00025 import androidx.core.content.ContextCompat.startActivity
00026 import androidx.core.content.ContextCompat
00027 import com.example.myapp.R
00028 import com.example.myapp.screens.api.FcmTokenRegistrar
00029
00039 class start : AppCompatActivity() {
00041     private val requestNotificationPermission =
00042         registerForActivityResult(ActivityResultContracts.RequestPermission()) { granted ->
00043             Log.d("FCM_DEBUG", "POST_NOTIFICATIONS granted: $granted")
00044         }
00045
00054     override fun onCreate(savedInstanceState: Bundle?) {
00055         super.onCreate(savedInstanceState)
00056         setContentView(R.layout.start)
00057
00058         RetrofitClient.init(this)
00059
00060         requestNotificationPermissionIfNeeded()
00061
00062         FcmTokenRegistrar.syncCurrentToken(this, "start")
00063
00064         val btnSignIn: View = findViewById(R.id.btnSignIn)
00065         btnSignIn.setOnClickListener {
00066             startActivity(Intent(this, signin::class.java))
00067         }
00068
00069         val btnSignUp: View = findViewById(R.id.btnSignUp)
00070         btnSignUp.setOnClickListener {
00071             startActivity(Intent(this, signup::class.java))
00072         }
00073     }
00074
00081     private fun requestNotificationPermissionIfNeeded() {
00082         if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU) return
00083
00084         val permissionGranted = ContextCompat.checkSelfPermission(
00085             this,
00086             Manifest.permission.POST_NOTIFICATIONS
00087         ) == PackageManager.PERMISSION_GRANTED
00088
00089         if (!permissionGranted) {
00090             requestNotificationPermission.launch(Manifest.permission.POST_NOTIFICATIONS)
00091         }
00092     }
00093 }

```

9.91 app/app/src/main/java/com/example/myapp/screens/VNPayActivity.kt File Reference

Packages

- package [AppCompatActivity](#)

9.92 VNPayActivity.kt

[Go to the documentation of this file.](#)

```

00001 package com.example.myapp.screens
00002
00012
00013 import android.annotation.SuppressLint
00014 import android.content.Intent
00015 import android.os.Bundle
00016 import android.webkit.WebView
00017 import android.webkit.WebViewClient
00018 import androidx.appcompat.app.AppCompatActivity
00019 import android.widget.Toast
00020 import com.example.myapp.screens.api.OrderResponse
00021 import com.example.myapp.screens.api.OrderStatusUpdateRequest
00022 import com.example.myapp.screens.api.RetrofitClient
00023 import retrofit2.Call
00024 import retrofit2.Callback
00025 import retrofit2.Response
00026
00027
00038 class VNPayActivity : AppCompatActivity() {
00039     private var isHandlingReturn = false
00040
00049     @SuppressLint("SetJavaScriptEnabled")
00050     override fun onCreate(savedInstanceState: Bundle?) {
00051         super.onCreate(savedInstanceState)
00052         val webView = WebView(this)
00053         setContentView(webView)
00054
00055
00056         val paymentUrl = intent.getStringExtra("URL") ?: ""
00057         val orderId = intent.getIntExtra("order_id", -1)
00058         webView.settings.javaScriptEnabled = true
00059
00060
00061         webView.webViewClient = object : WebViewClient() {
00062             override fun shouldOverrideUrlLoading(view: WebView?, url: String?): Boolean {
00063                 // Nếu thấy link vnpay_return từ server trả về thì đóng webview
00064                 if (url != null && url.contains("vnpay_return") && !isHandlingReturn) {
00065                     isHandlingReturn = true
00066                     if (url.contains("vnp_ResponseCode=00")) {
00067                         if (orderId > 0) {
00068                             RetrofitClient.apiService.updateOrderStatus(orderId, OrderStatusUpdateRequest("paid"))
00069                             .enqueue(object : Callback<OrderResponse> {
00070                                 override fun onResponse(call: Call<OrderResponse>, response: Response<OrderResponse>) {
00071                                     if (!response.isSuccessful) {
00072                                         Toast.makeText(this@VNPayActivity, "Không cập nhật được trạng thái đơn",
00073                                             Toast.LENGTH_SHORT).show()
00074                                         finish()
00075                                         return
00076                                     }
00077                                     val successIntent = Intent(this@VNPayActivity, payment_successful::class.java)
00078                                     successIntent.putExtra("order_id", orderId)
00079                                     startActivity(successIntent)
00080                                     finish()
00081                                 }
00082                             })
00083                         } else {
00084                             Toast.makeText(this@VNPayActivity, "Lỗi mạng khi cập nhật đơn",
00085                                 Toast.LENGTH_SHORT).show()
00086                             finish()
00087                         }
00088                     } else {
00089                         startActivity(Intent(this@VNPayActivity, payment_successful::class.java))
00090                         finish()
00091                     }
00092                 } else {

```

```

00092         finish()
00093     }
00094     return true
00095 }
00096 return false
00097 }
00098 }
00099 webView.loadUrl(paymentUrl)
00100 }
00101 }
00102
00103
00104

```

9.93 backend/auth.py File Reference

Namespaces

- namespace [auth](#)

Functions

- [auth.verify_password](#) (plain_password, hashed_password)
- [auth.get_password_hash](#) (password)
- [auth.create_access_token](#) (dict data, Optional[timedelta] expires_delta=None)
- [auth.get_current_user](#) (str token=Depends([oauth2_scheme](#)), Session db=Depends([database.get_db](#)))

Variables

- [auth.SECRET_KEY](#) = os.getenv("SECRET_KEY")
- [auth.ALGORITHM](#) = os.getenv("ALGORITHM", "HS256")
- [auth.ACCESS_TOKEN_EXPIRE_MINUTES](#) = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))
- [auth.pwd_context](#) = CryptContext(schemes=["bcrypt"], deprecated="auto")
- [auth.oauth2_scheme](#) = OAuth2PasswordBearer(tokenUrl="token")

9.94 auth.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: auth.py
00003
00004 Xác thực và phân quyền người dùng.
00005 Cung cấp JWT token, mã hóa mật khẩu bcrypt, và dependency lấy user hiện tại.
00006 """
00007
00008 import os
00009 from datetime import datetime, timedelta
00010 from typing import Optional
00011 from jose import JWTError, jwt
00012 from passlib.context import CryptContext
00013 from fastapi import Depends, HTTPException, status
00014 from fastapi.security import OAuth2PasswordBearer
00015 from sqlalchemy.orm import Session
00016 from dotenv import load_dotenv
00017
00018 import models, database
00019
00020 load_dotenv()
00021
00022 # Khóa bí mật để ký JWT token, lấy từ biến môi trường
00023 SECRET_KEY = os.getenv("SECRET_KEY")

```



```

00024 # Thuật toán mã hóa JWT, mặc định HS256
00025 ALGORITHM = os.getenv("ALGORITHM", "HS256")
00026 # Thời gian hết hạn token (phút), mặc định 30 phút
00027 ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))
00028
00029 # Context mã hóa mật khẩu bằng bcrypt
00030 pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
00031 # Scheme trích xuất Bearer token từ header Authorization
00032 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
00033
00034 def verify_password(plain_password, hashed_password):
00035     """Xác thực mật khẩu plaintext với mật khẩu đã hash bằng bcrypt."""
00036     return pwd_context.verify(plain_password, hashed_password)
00037
00038 def get_password_hash(password):
00039     """Hash mật khẩu plaintext bằng bcrypt."""
00040     return pwd_context.hash(password)
00041
00042 def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
00043     """Tạo JWT access token với payload và thời gian hết hạn."""
00044     to_encode = data.copy()
00045     if expires_delta:
00046         expire = datetime.utcnow() + expires_delta
00047     else:
00048         expire = datetime.utcnow() + timedelta(minutes=15)
00049     to_encode.update({"exp": expire})
00050     encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
00051     return encoded_jwt
00052
00053 async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(database.get_db)):
00054     """Dependency: giải mã JWT token và trả về user hiện tại. Raise 401 nếu token không hợp lệ."""
00055     credentials_exception = HTTPException(
00056         status_code=status.HTTP_401_UNAUTHORIZED,
00057         detail="Could not validate credentials",
00058         headers={"WWW-Authenticate": "Bearer"},
00059     )
00060     try:
00061         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
00062         email: str = payload.get("sub")
00063         if email is None:
00064             raise credentials_exception
00065     except JWTError:
00066         raise credentials_exception
00067     user = db.query(models.User).filter(models.User.email == email).first()
00068     if user is None:
00069         raise credentials_exception
00070     return user

```

9.95 backend/chatbot_service.py File Reference

Classes

- class [chatbot_service.ChatBotService](#)

Namespaces

- namespace [chatbot_service](#)

Variables

- [chatbot_service.GEMINI_API_KEY](#) = os.getenv("GEMINI_API_KEY")

9.96 chatbot_service.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: chatbot_service.py
00003
00004 Service chatbot sử dụng Google Gemini AI để trả lời câu hỏi của người dùng.
00005 Tự động lấy dữ liệu nhà hàng, món ăn và đơn hàng từ DB làm ngữ cảnh cho AI.
00006 """
00007
00008 import os
00009 import anyio
00010 from google import genai
00011 from google.genai import errors as genai_errors
00012 from sqlalchemy.orm import Session
00013 import models
00014 from dotenv import load_dotenv
00015
00016 load_dotenv()
00017
00018 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
00019
00020 class ChatBotService:
00021     """Service xử lý chatbot AI, kết nối Google Gemini và truy vấn database."""
00022
00023     def __init__(self, db: Session, user_id: int = None):
00024         self.db = db
00025         self.user_id = user_id
00026         # Khởi tạo Client bằng SDK chính thức
00027         if GEMINI_API_KEY:
00028             self.client = genai.Client(api_key=GEMINI_API_KEY)
00029         else:
00030             self.client = None
00031
00032     def get_app_context(self):
00033         """Lấy dữ liệu từ DB để làm ngữ cảnh cho AI."""
00034         try:
00035             restaurants = self.db.query(models.Restaurant).all()
00036             menu_items = self.db.query(models.Menuitem).all()
00037             # Lấy đơn hàng theo user_id nếu có, ngược lại lấy tất cả
00038             if self.user_id:
00039                 orders = self.db.query(models.Orders).filter(models.Orders.userid == self.user_id).all()
00040             else:
00041                 orders = self.db.query(models.Orders).all()
00042
00043             context = "Dưới đây là thông tin về ứng dụng Đặt Món:\n\n"
00044
00045             context += "NHÀ HÀNG:\n"
00046             for r in restaurants:
00047                 context += f"- {r.name}: {r.description} (Địa chỉ: {r.address}, Trạng thái: {r.status})\n"
00048
00049             context += "\nMÓN ĂN:\n"
00050             for m in menu_items:
00051                 res_name = next((r.name for r in restaurants if r.id == m.restaurantid), "Không rõ")
00052                 context += f"- {m.name}: Giá {m.price}đ, Mô tả: {m.description} (Tại nhà hàng: {res_name})\n"
00053
00054             context += "\nĐƠN HÀNG:\n"
00055             for o in orders:
00056                 user_name = "Không rõ"
00057                 try:
00058                     user = self.db.query(models.User).filter(models.User.id == o.userid).first()
00059                     if user:
00060                         user_name = user.name
00061                 except:
00062                     pass
00063                 context += f"- Đơn hàng #{o.id}: Trạng thái {o.status}, Tổng tiền {o.totalprice}đ, Khách hàng: {user_name}\n"
00064
00065             return context
00066         except Exception as e:
00067             print(f"Lỗi lấy dữ liệu DB: {e}")
00068             return "Hiện chưa có thông tin cụ thể."
00069
00070     def _call_gemini_sdk(self, prompt: str):
00071         """Hàm đồng bộ gọi AI thông qua SDK."""
00072         try:
00073             # Nâng cấp lên model thế hệ 3 mới nhất
00074             response = self.client.models.generate_content(
00075                 model='gemini-3-flash-preview',
00076                 contents=prompt
00077             )
00078             return response.text
00079         except genai_errors.ClientError as e:
00080             error_code = getattr(e, 'code', None)
00081             error_message = str(e)

```

```

00082
00083     # Xử lý lỗi 429 - Quota exceeded
00084     if error_code == 429 or 'RESOURCE_EXHAUSTED' in error_message:
00085         print(f"Lỗi 429 - Quota exceeded")
00086         raise Exception("QUOTA_EXCEEDED")
00087     else:
00088         print(f"Lỗi SDK Gemini: {e}")
00089         raise e
00090     except Exception as e:
00091         print(f"Lỗi SDK Gemini: {e}")
00092         raise e
00093
00094     async def get_response(self, user_message: str):
00095         """Nhận tin nhắn người dùng, xây dựng prompt và gọi Gemini AI trả lời."""
00096         if not self.client:
00097             return "Xin lỗi, ChatBot chưa được cấu hình API Key trong file .env."
00098
00099         app_context = self.get_app_context()
00100
00101         prompt = f"""
00102         Bạn là trợ lý ảo của ứng dụng 'Đặt Món' - Food Delivery Việt Nam.
00103
00104         {app_context}
00105
00106         HƯỚNG DẪN:
00107         1. Trả lời dựa trên dữ liệu nhà hàng, món ăn và đơn hàng được cung cấp ở trên.
00108         2. Thân thiện, ngắn gọn, trả lời bằng tiếng Việt.
00109         3. Nếu khách hàng hỏi về đơn hàng, hãy kiểm tra thông tin đơn hàng trong dữ liệu.
00110
00111         Câu hỏi khách hàng: {user_message}
00112         """
00113
00114         try:
00115             # Chạy trong thread pool để đảm bảo non-blocking cho FastAPI
00116             return await anyio.to_thread.run_sync(self._call_gemini_sdk, prompt)
00117         except Exception as e:
00118             error_message = str(e)
00119             if "QUOTA_EXCEEDED" in error_message:
00120                 return "Xin lỗi, hệ thống AI tạm thời quá tải do đã đạt giới hạn sử dụng miễn phí. Vui lòng thử lại sau 1-2 phút hoặc liên hệ admin để nâng cấp API."
00121             return "Rất tiếc, AI đang gặp một chút trục trặc. Bạn thử lại sau nhé!"

```

9.97 backend/check_db.py File Reference

Namespaces

- namespace [check_db](#)

Variables

- [check_db.conn](#) = `sqlite3.connect('test.db')`
- [check_db.cursor](#) = `conn.cursor()`
- [check_db.tables](#) = `cursor.fetchall()`
- [check_db.fcm](#) = `cursor.fetchall()`
- [check_db.notifs](#) = `cursor.fetchall()`

9.98 check_db.py

[Go to the documentation of this file.](#)

```

00001 import sqlite3
00002 conn = sqlite3.connect('test.db')
00003 cursor = conn.cursor()
00004 cursor.execute('SELECT name FROM sqlite_master WHERE type=\
00005 table\;')
00006 tables = cursor.fetchall()
00007 print('Tables:', tables)
00008 cursor.execute('SELECT * FROM userdevice WHERE userid=1;')
00009 fcm = cursor.fetchall()
00010 print('FCM:', fcm)
00011 cursor.execute('SELECT * FROM notification;')
00012 notifs = cursor.fetchall()
00013 print('Notifs:', notifs)
00014 conn.close()

```

9.99 backend/database.py File Reference

Namespaces

- namespace [database](#)

Functions

- [database.get_db\(\)](#)

Variables

- [database.SQLALCHEMY_DATABASE_URL](#) = `os.getenv("DATABASE_URL", "postgresql://postgres:password@localhost/food_delivery")`
- [database.engine](#) = `create_engine(SQLALCHEMY_DATABASE_URL)`
- [database.SessionLocal](#) = `sessionmaker(autocommit=False, autoflush=False, bind=engine)`
- [database.Base](#) = `declarative_base()`

9.100 database.py

[Go to the documentation of this file.](#)

```
00001 """
00002 Module: database.py
00003
00004 Cấu hình kết nối SQLAlchemy và session cho cơ sở dữ liệu PostgreSQL.
00005 Cung cấp engine, SessionLocal và hàm get_db() dependency cho FastAPI.
00006 """
00007
00008 from sqlalchemy import create_engine
00009 from sqlalchemy.orm import sessionmaker, declarative_base
00010 import os
00011 from dotenv import load_dotenv
00012
00013 load_dotenv()
00014
00015 # URL kết nối database, ưu tiên biến môi trường, mặc định là PostgreSQL local
00016 SQLALCHEMY_DATABASE_URL = os.getenv("DATABASE_URL",
00017                                     "postgresql://postgres:password@localhost/food_delivery")
00018
00019 # SQLAlchemy engine quản lý kết nối pool tới database
00020 engine = create_engine(SQLALCHEMY_DATABASE_URL)
00021
00022 # Session factory tạo các session kết nối database
00023 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
00024
00025 # Base class để khai báo các ORM model kế thừa
00026 Base = declarative_base()
00027
00028 def get_db():
00029     """FastAPI dependency: tạo session database cho mỗi request, tự động đóng sau khi hoàn tất."""
00030     db = SessionLocal()
00031     try:
00032         yield db
00033     finally:
00034         db.close()
```

9.101 backend/firebase_utils.py File Reference

Namespaces

- namespace [firebase_utils](#)

Functions

- `firebase_utils.init_firebase()`
- `firebase_utils._upload_image_sync` (file_content, filename)
- `firebase_utils.upload_image` (file_content, filename)

Variables

- `firebase_utils.FIREBASE_CREDENTIALS_PATH` = `os.getenv("FIREBASE_CREDENTIALS_PATH")`
- `firebase_utils.FIREBASE_STORAGE_BUCKET` = `os.getenv("FIREBASE_STORAGE_BUCKET")`

9.102 firebase_utils.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: firebase_utils.py
00003
00004 Khởi tạo Firebase Admin SDK và upload ảnh lên Firebase Storage.
00005 Hỗ trợ upload bất đồng bộ qua anyio để không chặn event loop.
00006 """
00007
00008 import firebase_admin
00009 from firebase_admin import credentials, storage
00010 import os
00011 from dotenv import load_dotenv
00012 import uuid
00013 import anyio
00014
00015 load_dotenv()
00016
00017 # Đường dẫn file credentials Firebase (JSON service account)
00018 FIREBASE_CREDENTIALS_PATH = os.getenv("FIREBASE_CREDENTIALS_PATH")
00019 # Tên bucket Firebase Storage để upload ảnh
00020 FIREBASE_STORAGE_BUCKET = os.getenv("FIREBASE_STORAGE_BUCKET")
00021
00022 def init_firebase():
00023     """Khởi tạo Firebase Admin SDK với credentials từ file service account."""
00024     if not FIREBASE_CREDENTIALS_PATH or not os.path.exists(FIREBASE_CREDENTIALS_PATH):
00025         print(f"Warning: Firebase credentials not found at {FIREBASE_CREDENTIALS_PATH}. Image upload will fail.")
00026         return False
00027
00028     try:
00029         cred = credentials.Certificate(FIREBASE_CREDENTIALS_PATH)
00030         firebase_admin.initialize_app(cred, {
00031             'storageBucket': FIREBASE_STORAGE_BUCKET
00032         })
00033         return True
00034     except Exception as e:
00035         print(f"Error initializing Firebase: {e}")
00036         return False
00037
00038 def _upload_image_sync(file_content, filename):
00039     """Upload ảnh đồng bộ lên Firebase Storage, trả về URL công khai."""
00040     if not firebase_admin._apps:
00041         if not init_firebase():
00042             return None
00043
00044     try:
00045         bucket = storage.bucket()
00046         ext = filename.split('.')[-1]
00047         unique_filename = f"images/{uuid.uuid4()}.{ext}"
00048         blob = bucket.blob(unique_filename)
00049         blob.upload_from_string(file_content, content_type=f"image/{ext}")
00050         blob.make_public()
00051         return blob.public_url
00052     except Exception as e:
00053         print(f"Error uploading image: {e}")
00054         return None
00055
00056 async def upload_image(file_content, filename):
00057     """Bản async của hàm upload ảnh, không gây nghẽn event loop."""
00058     return await anyio.to_thread.run_sync(_upload_image_sync, file_content, filename)

```

9.103 backend/main.py File Reference

Namespaces

- namespace [main](#)

Functions

- [main.read_root](#) ()
- [main.login_for_access_token](#) (OAuth2PasswordRequestForm form_data=Depends(), Session db=Depends(get_db))
- [main.create_user](#) (schemas.UserCreate user, Session db=Depends(get_db))
- [main.read_users_me](#) (models.User current_user=Depends(auth.get_current_user))
- [main.update_device_token](#) (str token=Form(...), str device_type=Form("android"), models.User current_user=Depends(auth.get_current_user), Session db=Depends(get_db))
- [main.read_users](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [main.create_address](#) (schemas.AddressCreate address, Session db=Depends(get_db))
- [main.read_user_addresses](#) (int user_id, Session db=Depends(get_db))
- [main.update_address](#) (int address_id, schemas.AddressUpdate address, Session db=Depends(get_db))
- [main.read_categories](#) (Session db=Depends(get_db))
- [main.read_promotions](#) (bool active_only=True, Session db=Depends(get_db))
- [main.chat_with_bot](#) (Gửi, tin, nhắn, đến, chatbot, AI, và, nhận, phản, hồi, schemas.ChatMessageCreate chat_input, Optional[int] session_id=None, models.User current_user=Depends(auth.get_current_user), Session db=Depends(get_db))
- [main.create_restaurant](#) (Tạo, nhà, hàng, mới, với, ảnh, upload, lên, Firebase, Storage, str name=Form(...), Optional[str] address=Form(None), str phone_number=Form(...), str status=Form(...), Optional[str] description=Form(None), Optional[UploadFile] image=File(None), Session db=Depends(get_db))
- [main.read_restaurants](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [main.search_restaurants_by_name](#) (str name, int skip=0, int limit=100, Session db=Depends(get_db))
- [main.read_restaurant](#) (int restaurant_id, Session db=Depends(get_db))
- [main.create_menu_item](#) (Tạo, món, ăn, mới, với, ảnh, upload, lên, Firebase, Storage, str name=Form(...), Decimal price=Form(...), Optional[str] description=Form(None), int restaurant_id=Form(...), int category_id=Form(...), Optional[UploadFile] image=File(None), Session db=Depends(get_db))
- [main.build_menuitem_with_reviews](#) (models.MenuItem item, Session db)
- [main.read_all_menu_items](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [main.read_restaurant_menu](#) (int restaurant_id, Session db=Depends(get_db))
- [main.search_menu_items_by_name](#) (str name, int skip=0, int limit=100, Session db=Depends(get_db))
- [main.search_menu_items_by_category_name](#) (str category_name, int skip=0, int limit=100, Session db=Depends(get_db))
- [main.read_menu_item_detail](#) (int menu_item_id, Session db=Depends(get_db))
- [main.vnpay_return](#) (Request request, Session db=Depends(get_db))
- [main.create_payment](#) (Tạo, URL, thanh, toán, VNPay, cho, đơn, hàng, str order_id, int amount, Request request)
- [main.create_order](#) (schemas.OrderCreate order, Session db=Depends(get_db))
- [main.read_user_orders](#) (int user_id, Session db=Depends(get_db))
- [main.read_order_detail](#) (int order_id, Session db=Depends(get_db))
- [main.update_order_status](#) (int order_id, schemas.OrderStatusUpdate payload, Session db=Depends(get_db))
- [main.create_review](#) (schemas.ReviewCreate review, Session db=Depends(get_db))
- [main.read_user_profile_summary](#) (int user_id, Session db=Depends(get_db))
- [main.read_faqs](#) (int skip=0, int limit=100, Session db=Depends(get_db))
- [main.read_user_notifications](#) (int user_id, Session db=Depends(get_db))
- [main.create_notification](#) (schemas.NotificationCreate notification, Session db=Depends(get_db))
- [main.mark_notification_as_read](#) (int notification_id, Session db=Depends(get_db))

Variables

- [main.bind](#)
- [main.app](#) = FastAPI(title="Đặt Món Food Delivery API")
- [main.allow_origins](#)
- [main.allow_credentials](#)
- [main.allow_methods](#)
- [main.allow_headers](#)
- [main.host](#)
- [main.port](#)

9.104 main.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: main.py
00003
00004 FastAPI application chính của hệ thống Đặt Món.
00005 Định nghĩa tất cả REST API endpoints: auth, user, restaurant, menu, order, payment, chatbot, notification.
00006 """
00007
00008 from fastapi import FastAPI, Depends, HTTPException, File, UploadFile, Form, status
00009 from fastapi.security import OAuth2PasswordRequestForm
00010 from fastapi.middleware.cors import CORSMiddleware
00011 from sqlalchemy.orm import Session
00012 from typing import List, Optional
00013 from decimal import Decimal
00014 from datetime import timedelta, datetime
00015
00016
00017 import models, schemas, auth, os
00018 from database import engine, get_db
00019 from firebase_utils import upload_image
00020 from chatbot_service import ChatBotService
00021 from notification_service import notify_user
00022
00023
00024 from payment_service import generate_vnpay_url
00025 from fastapi import Request
00026
00027
00028 # Create tables in the database (optional if using migrations like Alembic)
00029 models.Base.metadata.create_all(bind=engine)
00030
00031
00032 app = FastAPI(title="Đặt Món Food Delivery API")
00033
00034
00035 # CORS middleware - Cho phép Android app kết nối
00036 app.add_middleware(
00037     CORSMiddleware,
00038     allow_origins=["*"], # Cho phép tất cả origins (development)
00039     allow_credentials=True,
00040     allow_methods=["*"],
00041     allow_headers=["*"],
00042 )
00043
00044
00045 @app.get("/")
00046 def read_root():
00047     """Endpoint gốc, trả lời chào mừng API."""
00048     return {"message": "Welcome to Đặt Món Food Delivery API"}
00049
00050
00051 # --- AUTH ENDPOINTS ---
00052
00053
00054 @app.post("/token", response_model=schemas.Token)
00055 async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends(), db: Session =
    Depends(get_db)):
00056     """Đăng nhập bằng username/password, trả về JWT access token."""
00057     user = db.query(models.User).filter(models.User.username == form_data.username).first()
00058     if not user or not auth.verify_password(form_data.password, user.password):
00059         raise HTTPException(
00060             status_code=status.HTTP_401_UNAUTHORIZED,
00061             detail="Incorrect username or password",

```

```

00062         headers={"WWW-Authenticate": "Bearer"},
00063     )
00064     access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
00065     access_token = auth.create_access_token(
00066         data={"sub": user.email}, expires_delta=access_token_expires
00067     )
00068     return {"access_token": access_token, "token_type": "bearer"}
00069
00070
00071 # --- USER ENDPOINTS ---
00072
00073
00074 @app.post("/users/", response_model=schemas.User)
00075 def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
00076     """Đăng ký tài khoản người dùng mới."""
00077     # Check if user exists
00078     db_user = db.query(models.User).filter(models.User.email == user.email).first()
00079     if db_user:
00080         raise HTTPException(status_code=400, detail="Email already registered")
00081
00082     hashed_password = auth.get_password_hash(user.password)
00083     db_user = models.User(
00084         name=user.name,
00085         username=user.username,
00086         email=user.email,
00087         password=hashed_password,
00088         phone=user.phone,
00089         role=user.role
00090     )
00091     db.add(db_user)
00092     db.commit()
00093     db.refresh(db_user)
00094     return db_user
00095
00096
00097 @app.get("/users/me/", response_model=schemas.User)
00098 async def read_users_me(current_user: models.User = Depends(auth.get_current_user)):
00099     """Lấy thông tin người dùng hiện tại từ JWT token."""
00100     return current_user
00101
00102
00103 @app.post("/devices/token")
00104 async def update_device_token(
00105     token: str = Form(...),
00106     device_type: str = Form("android"),
00107     current_user: models.User = Depends(auth.get_current_user),
00108     db: Session = Depends(get_db)
00109 ):
00110     """Android App gọi API này để gửi FCM Token lên Server."""
00111     # Kiểm tra xem token này đã có chưa
00112     db_device = db.query(models.UserDevice).filter(models.UserDevice.device_token == token).first()
00113     if not db_device:
00114         db_device = models.UserDevice(device_token=token, device_type=device_type, userid=current_user.id)
00115         db.add(db_device)
00116     else:
00117         db_device.userid = current_user.id
00118         # Cập nhật thời gian hoạt động
00119
00120     db.commit()
00121     return {"message": "Device token updated successfully"}
00122
00123
00124 @app.get("/users/", response_model=List[schemas.User])
00125 def read_users(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00126     """Lấy danh sách tất cả người dùng (phân trang)."""
00127     users = db.query(models.User).offset(skip).limit(limit).all()
00128     return users
00129
00130
00131 # --- ADDRESS ENDPOINTS ---
00132
00133
00134 @app.post("/addresses/", response_model=schemas.Address)
00135 def create_address(address: schemas.AddressCreate, db: Session = Depends(get_db)):
00136     """Tạo địa chỉ giao hàng mới cho người dùng."""
00137     db_address = models.Address(detail=address.detail, phone=address.phone, userid=address.userid)
00138     db.add(db_address)
00139     db.commit()
00140     db.refresh(db_address)
00141     return db_address
00142
00143
00144 @app.get("/users/{user_id}/addresses/", response_model=List[schemas.Address])
00145 def read_user_addresses(user_id: int, db: Session = Depends(get_db)):
00146     """Lấy danh sách địa chỉ giao hàng của người dùng."""
00147     return db.query(models.Address).filter(models.Address.userid == user_id).all()
00148

```



```

00149
00150 @app.put("/addresses/{address_id}/", response_model=schemas.Address)
00151 def update_address(address_id: int, address: schemas.AddressUpdate, db: Session = Depends(get_db)):
00152     """Cập nhật địa chỉ giao hàng."""
00153     db_address = db.query(models.Address).filter(models.Address.id == address_id).first()
00154     if not db_address:
00155         raise HTTPException(status_code=404, detail="Address not found")
00156
00157     # Cập nhật các trường
00158     if address.detail is not None:
00159         db_address.detail = address.detail
00160     if address.phone is not None:
00161         db_address.phone = address.phone
00162
00163     db.commit()
00164     db.refresh(db_address)
00165     return db_address
00166
00167
00168 # --- CATEGORY ENDPOINTS ---
00169
00170
00171 @app.get("/categories/", response_model=List[schemas.Category])
00172 def read_categories(db: Session = Depends(get_db)):
00173     """Lấy danh sách tất cả danh mục món ăn."""
00174     return db.query(models.Category).all()
00175
00176
00177 @app.get("/promotions/", response_model=List[schemas.Promotion])
00178 def read_promotions(active_only: bool = True, db: Session = Depends(get_db)):
00179     """Lấy danh sách mã khuyến mãi, mặc định chỉ lấy mã đang active."""
00180     query = db.query(models.Promotion)
00181     if active_only:
00182         query = query.filter(models.Promotion.status.ilike("active"))
00183     return query.all()
00184
00185
00186 # --- CHAT ENDPOINTS ---
00187
00188
00189 @app.post("/chat/", response_model=schemas.ChatMessage)
00190 async def chat_with_bot(
00191     """Gửi tin nhắn đến chatbot AI và nhận phản hồi."""
00192     chat_input: schemas.ChatMessageCreate,
00193     session_id: Optional[int] = None,
00194     current_user: models.User = Depends(auth.get_current_user),
00195     db: Session = Depends(get_db)
00196 ):
00197     # 1. Tìm hoặc tạo ChatSession
00198     if session_id:
00199         session = db.query(models.ChatSession).filter(models.ChatSession.id == session_id).first()
00200         if not session:
00201             raise HTTPException(status_code=404, detail="Chat Session not found")
00202     else:
00203         session = models.ChatSession(status="active", userid=current_user.id)
00204         db.add(session)
00205         db.commit()
00206         db.refresh(session)
00207
00208
00209     # 2. Lưu tin nhắn người dùng
00210     user_msg = models.ChatMessage(senderrole="user", message=chat_input.message, sessionid=session.id)
00211     db.add(user_msg)
00212     db.commit() # Lưu trước để AI có thể đọc lịch sử nếu cần (trong tương lai)
00213
00214
00215     # 3. Gọi AI xử lý
00216     bot_service = ChatBotService(db, user_id=current_user.id)
00217     bot_response_text = await bot_service.get_response(chat_input.message)
00218
00219
00220     # 4. Lưu phản hồi của AI
00221     bot_msg = models.ChatMessage(senderrole="bot", message=bot_response_text, sessionid=session.id)
00222     db.add(bot_msg)
00223
00224     db.commit()
00225     db.refresh(bot_msg)
00226
00227     return bot_msg
00228
00229 # --- RESTAURANT ENDPOINTS ---
00230
00231 @app.post("/restaurants/", response_model=schemas.Restaurant)
00232 async def create_restaurant(
00233     """Tạo nhà hàng mới với ảnh upload lên Firebase Storage."""
00234     name: str = Form(...),
00235     address: Optional[str] = Form(None),

```

```

00236     phone_number: str = Form(...),
00237     status: str = Form(...),
00238     description: Optional[str] = Form(None),
00239     image: Optional[UploadFile] = File(None),
00240     db: Session = Depends(get_db)
00241 ):
00242     image_url = None
00243     if image:
00244         content = await image.read()
00245         image_url = await upload_image(content, image.filename)
00246
00247     db_restaurant = models.Restaurant(
00248         name=name,
00249         address=address,
00250         phone_number=phone_number,
00251         status=status,
00252         description=description,
00253         image_url=image_url
00254     )
00255     db.add(db_restaurant)
00256     db.commit()
00257     db.refresh(db_restaurant)
00258     return db_restaurant
00259
00260
00261 @app.get("/restaurants/", response_model=List[schemas.Restaurant])
00262 def read_restaurants(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00263     """Lấy danh sách tất cả nhà hàng (phân trang)."""
00264     restaurants = db.query(models.Restaurant).offset(skip).limit(limit).all()
00265     return restaurants
00266
00267
00268 @app.get("/restaurants/search/", response_model=List[schemas.Restaurant])
00269 def search_restaurants_by_name(name: str, skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00270     """Tìm kiếm nhà hàng theo tên (không phân biệt hoa thường)."""
00271     restaurants = db.query(models.Restaurant).filter(
00272         models.Restaurant.name.ilike(f"%{name}%")
00273     ).offset(skip).limit(limit).all()
00274     return restaurants
00275
00276
00277 @app.get("/restaurants/{restaurant_id}/", response_model=schemas.Restaurant)
00278 def read_restaurant(restaurant_id: int, db: Session = Depends(get_db)):
00279     """Lấy chi tiết nhà hàng theo ID."""
00280     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == restaurant_id).first()
00281     if not restaurant:
00282         raise HTTPException(status_code=404, detail="Restaurant not found")
00283     return restaurant
00284
00285
00286 # --- MENU ITEM ENDPOINTS ---
00287
00288
00289 @app.post("/menu-items/", response_model=schemas.MenuItem)
00290 async def create_menu_item(
00291     """Tạo món ăn mới với ảnh upload lên Firebase Storage."""
00292     name: str = Form(...),
00293     price: Decimal = Form(...),
00294     description: Optional[str] = Form(None),
00295     restaurantid: int = Form(...),
00296     categoryid: int = Form(...),
00297     image: Optional[UploadFile] = File(None),
00298     db: Session = Depends(get_db)
00299 ):
00300     image_url = None
00301     if image:
00302         content = await image.read()
00303         image_url = await upload_image(content, image.filename)
00304
00305     db_item = models.MenuItem(
00306         name=name,
00307         price=price,
00308         description=description,
00309         restaurantid=restaurantid,
00310         categoryid=categoryid,
00311         image_url=image_url
00312     )
00313     db.add(db_item)
00314     db.commit()
00315     db.refresh(db_item)
00316     return db_item
00317
00318
00319 def build_menuitem_with_reviews(item: models.MenuItem, db: Session):
00320     """Helper function to build MenuItem with reviews and avg_rating."""
00321     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == item.restaurantid).first()
00322     restaurant_name = restaurant.name if restaurant else ""

```

```

00323
00324 # Fetch reviews for this menu item
00325 reviews = db.query(models.Review).filter(models.Review.menuitemid == item.id).all()
00326 reviews_data = []
00327 total_rating = 0
00328
00329 for review in reviews:
00330     user = db.query(models.User).filter(models.User.id == review.userid).first()
00331     user_name = user.name if user else "Unknown"
00332     reviews_data.append({
00333         "id": review.id,
00334         "rating": review.rating,
00335         "comment": review.comment,
00336         "userid": review.userid,
00337         "user_name": user_name
00338     })
00339     total_rating += review.rating
00340
00341 avg_rating = total_rating / len(reviews) if reviews else 0.0
00342
00343 return {
00344     "id": item.id,
00345     "name": item.name,
00346     "image_url": item.image_url,
00347     "price": item.price,
00348     "is_available": item.is_available,
00349     "description": item.description,
00350     "restaurantid": item.restaurantid,
00351     "categoryid": item.categoryid,
00352     "restaurant_name": restaurant_name,
00353     "reviews": reviews_data,
00354     "avg_rating": avg_rating
00355 }
00356
00357 @app.get("/menu-items/", response_model=List[schemas.MenuItemWithReviews])
00358 def read_all_menu_items(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00359     """Lấy danh sách tất cả món ăn từ tất cả nhà hàng, bao gồm reviews."""
00360     menu_items = db.query(models.MenuItem).offset(skip).limit(limit).all()
00361
00362     result = []
00363     for item in menu_items:
00364         item_dict = build_menuitem_with_reviews(item, db)
00365         result.append(item_dict)
00366     return result
00367
00368
00369 @app.get("/restaurants/{restaurant_id}/menu/", response_model=List[schemas.MenuItem])
00370 def read_restaurant_menu(restaurant_id: int, db: Session = Depends(get_db)):
00371     """Lấy danh sách món ăn của một nhà hàng."""
00372     return db.query(models.MenuItem).filter(models.MenuItem.restaurantid == restaurant_id).all()
00373
00374
00375 @app.get("/menu-items/search/", response_model=List[schemas.MenuItemWithReviews])
00376 def search_menu_items_by_name(name: str, skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00377     """Tìm kiếm món ăn theo tên (không phân biệt hoa thường)."""
00378     menu_items = db.query(models.MenuItem).filter(
00379         models.MenuItem.name.ilike(f"%{name}%")
00380     ).offset(skip).limit(limit).all()
00381
00382     result = []
00383     for item in menu_items:
00384         item_dict = build_menuitem_with_reviews(item, db)
00385         result.append(item_dict)
00386     return result
00387
00388
00389 @app.get("/menu-items/category/search/", response_model=List[schemas.MenuItemWithReviews])
00390 def search_menu_items_by_category_name(category_name: str, skip: int = 0, limit: int = 100, db: Session =
00391     Depends(get_db)):
00392     """Tìm kiếm món ăn theo tên danh mục (không phân biệt hoa thường)."""
00393     # Tìm category theo tên
00394     categories = db.query(models.Category).filter(
00395         models.Category.name.ilike(f"%{category_name}%")
00396     ).all()
00397     category_ids = [cat.id for cat in categories]
00398
00399     if not category_ids:
00400         return []
00401
00402     menu_items = db.query(models.MenuItem).filter(
00403         models.MenuItem.categoryid.in_(category_ids)
00404     ).offset(skip).limit(limit).all()
00405
00406     result = []
00407     for item in menu_items:
00408         item_dict = build_menuitem_with_reviews(item, db)

```

```

00409         result.append(item_dict)
00410     return result
00411
00412
00413 @app.get("/menu-items/{menu_item_id}/", response_model=schemas.MenuItemWithReviews)
00414 def read_menu_item_detail(menu_item_id: int, db: Session = Depends(get_db)):
00415     """Lấy chi tiết món ăn bao gồm reviews."""
00416     item = db.query(models.MenuItem).filter(models.MenuItem.id == menu_item_id).first()
00417     if not item:
00418         raise HTTPException(status_code=404, detail="Menu item not found")
00419
00420     return build_menuitem_with_reviews(item, db)
00421 # --- VNPAY RETURN ---
00422 @app.get("/vnpay_return")
00423 async def vnpay_return(request: Request, db: Session = Depends(get_db)):
00424     """Callback từ VNPay sau khi thanh toán, cập nhật trạng thái đơn hàng."""
00425
00426
00427     params = dict(request.query_params)
00428
00429
00430     response_code = params.get("vnp_ResponseCode")
00431     order_ref = params.get("vnp_TxnRef") # Format: DH{order_id}
00432
00433     # Trích xuất order_id từ reference (VD: "DH123" -> 123)
00434     try:
00435         # Nếu là định dạng "DH{id}"
00436         if order_ref and order_ref.startswith("DH"):
00437             order_id = int(order_ref[2:])
00438         else:
00439             order_id = int(order_ref) if order_ref else 0
00440     except (ValueError, AttributeError):
00441         order_id = 0
00442
00443
00444     if response_code == "00":
00445         # Thanh toán thành công - cập nhật đơn hàng trong DB
00446         if order_id > 0:
00447             order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00448             if order:
00449                 order.status = "paid"
00450
00451                 # Tạo Payment record
00452                 payment_record = models.Payment(
00453                     status="success",
00454                     method="vnpay",
00455                     orderid=order.id
00456                 )
00457                 db.add(payment_record)
00458                 db.commit()
00459                 db.refresh(order)
00460
00461                 # Gửi notification
00462                 total_formatted = f"{order.totalprice:,.0f}".replace(".", ",")
00463                 notify_user(
00464                     db=db,
00465                     user_id=order.userid,
00466                     title="Đơn hàng được đặt thành công! ",
00467                     body=f"Đơn hàng #{order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ quán xác
nhận.",
00468                     order_id=order.id
00469                 )
00470
00471     return {
00472         "status": "success",
00473         "message": f"Thanh toán thành công cho đơn {order_ref}"
00474     }
00475 else:
00476     # Thanh toán thất bại - cập nhật Payment record
00477     if order_id > 0:
00478         payment_record = models.Payment(
00479             status="failed",
00480             method="vnpay",
00481             orderid=order_id
00482         )
00483         db.add(payment_record)
00484         db.commit()
00485
00486     return {
00487         "status": "failed",
00488         "message": "Thanh toán thất bại"
00489     }
00490
00491
00492 # --- PAYMENT ENDPOINTS ---
00493
00494

```

```

00495 @app.get("/create-payment")
00496 async def create_payment(
00497     """Tạo URL thanh toán VNPAY cho đơn hàng."""
00498     order_id: str,
00499     amount: int,
00500     request: Request
00501 ):
00502     ip_address = request.client.host
00503
00504     payment_url = generate_vnpay_url(
00505         order_id=str(order_id),
00506         amount=amount,
00507         ip_address=ip_address
00508     )
00509
00510
00511
00512     return {
00513         "payment_url": payment_url
00514     }
00515
00516
00517 # --- ORDER ENDPOINTS ---
00518
00519
00520 @app.post("/orders/", response_model=schemas.Order)
00521 def create_order(order: schemas.OrderCreate, db: Session = Depends(get_db)):
00522     """Tạo đơn hàng mới kèm danh sách món ăn và gửi thông báo."""
00523     db_order = models.Orders(
00524         status=order.status,
00525         preorderdate=order.preorderdate,
00526         preorderetime=order.preorderetime,
00527         totalprice=order.totalprice,
00528         restaurantid=order.restaurantid,
00529         addressid=order.addressid,
00530         userid=order.userid
00531     )
00532     db.add(db_order)
00533     db.commit()
00534     db.refresh(db_order)
00535
00536     for item in order.order_items:
00537         db_order_item = models.OrderItem(
00538             quantity=item.quantity,
00539             price=item.price,
00540             menuitemid=item.menuitemid,
00541             orderid=db_order.id
00542         )
00543         db.add(db_order_item)
00544
00545     db.commit()
00546     db.refresh(db_order)
00547
00548     # Nếu là COD (status="confirmed"), gửi thông báo ngay lúc tạo đơn hàng
00549     if db_order.status == "confirmed":
00550         try:
00551             total_formatted = f"{db_order.totalprice:.0f}".replace(".", "")
00552             notify_user(
00553                 db=db,
00554                 user_id=db_order.userid,
00555                 title="Đơn hàng được đặt thành công!",
00556                 body=f"Đơn hàng #{db_order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ quán xác
nhận.",
00557                 order_id=db_order.id
00558             )
00559         except Exception as e:
00560             print(f"Error sending notification: {e}")
00561
00562     return db_order
00563
00564
00565 @app.get("/users/{user_id}/orders/", response_model=List[schemas.Order])
00566 def read_user_orders(user_id: int, db: Session = Depends(get_db)):
00567     """Lấy danh sách đơn hàng của người dùng."""
00568     return db.query(models.Orders).filter(models.Orders.userid == user_id).all()
00569
00570
00571 @app.get("/orders/{order_id}/detail", response_model=schemas.OrderDetail)
00572 def read_order_detail(order_id: int, db: Session = Depends(get_db)):
00573     """Lấy chi tiết đơn hàng kèm tên nhà hàng, địa chỉ và danh sách món."""
00574     order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00575     if not order:
00576         raise HTTPException(status_code=404, detail="Order not found")
00577
00578     restaurant_name = order.restaurant.name if order.restaurant else None
00579     address_detail = order.address.detail if order.address else None
00580

```

```

00581     order_items = []
00582     for item in order.order_items:
00583         order_items.append({
00584             "id": item.id,
00585             "quantity": item.quantity,
00586             "price": item.price,
00587             "menuitemid": item.menuitemid,
00588             "menuitem_name": item.menu_item.name if item.menu_item else None,
00589             "image_url": item.menu_item.image_url if item.menu_item else None
00590         })
00591
00592     return {
00593         "id": order.id,
00594         "status": order.status,
00595         "preorderdate": order.preorderdate,
00596         "preordertime": order.preordertime,
00597         "totalprice": order.totalprice,
00598         "restaurantid": order.restaurantid,
00599         "addressid": order.addressid,
00600         "userid": order.userid,
00601         "createdat": order.createdat,
00602         "restaurant_name": restaurant_name,
00603         "address_detail": address_detail,
00604         "order_items": order_items
00605     }
00606
00607
00608 @app.put("/orders/{order_id}/status", response_model=schemas.Order)
00609 def update_order_status(order_id: int, payload: schemas.OrderStatusUpdate, db: Session = Depends(get_db)):
00610     """Cập nhật trạng thái đơn hàng và gửi thông báo tương ứng"""
00611     order = db.query(models.Orders).filter(models.Orders.id == order_id).first()
00612     if not order:
00613         raise HTTPException(status_code=404, detail="Order not found")
00614
00615     old_status = order.status
00616     order.status = payload.status
00617     db.commit()
00618     db.refresh(order)
00619
00620     print(f"DEBUG: Update order {order_id}: {old_status} -> {payload.status}")
00621
00622     # Tạo thông báo khi trạng thái đơn hàng thay đổi
00623     try:
00624         notification_title = ""
00625         notification_body = ""
00626
00627         if payload.status in ["paid", "confirmed"] and old_status not in ["paid", "confirmed"]:
00628             notification_title = "Đơn hàng được đặt thành công! "
00629             total_formatted = f"{order.totalprice:,.0f}".replace(".", "")
00630             notification_body = f"Đơn hàng #{order.id} đã được đặt thành công. Tổng tiền: {total_formatted}đ. Đang chờ  
quản xác nhận."
00631             print(f"DEBUG: Sending notification for order {order_id}")
00632         elif payload.status == "delivering":
00633             notification_title = "Đơn hàng đang giao "
00634             notification_body = f"Đơn hàng #{order.id} đang trên đường giao đến bạn"
00635         elif payload.status == "completed":
00636             notification_title = "Đơn hàng đã giao "
00637             notification_body = f"Đơn hàng #{order.id} đã giao thành công. Cảm ơn đã đặt hàng!"
00638         elif payload.status == "cancelled":
00639             notification_title = "Đơn hàng bị hủy "
00640             notification_body = f"Đơn hàng #{order.id} đã bị hủy"
00641
00642         # Nếu có thay đổi trạng thái, tạo thông báo
00643         if notification_title:
00644             print(f"DEBUG: Sending '{notification_title}' to user {order.userid}")
00645             notify_user(
00646                 db=db,
00647                 user_id=order.userid,
00648                 title=notification_title,
00649                 body=notification_body,
00650                 order_id=order.id
00651             )
00652             print(f"DEBUG: Notification sent successfully")
00653     except Exception as e:
00654         print(f"Lỗi khi tạo thông báo trạng thái: {str(e)}")
00655
00656     return order
00657
00658
00659 @app.post("/reviews/", response_model=schemas.Review)
00660 def create_review(review: schemas.ReviewCreate, db: Session = Depends(get_db)):
00661     """Tạo đánh giá cho đơn hàng và cập nhật điểm trung bình nhà hàng."""
00662     order = db.query(models.Orders).filter(models.Orders.id == review.orderid).first()
00663     if not order:
00664         raise HTTPException(status_code=404, detail="Order not found")
00665
00666     if order.userid != review.userid:

```

```

00667         raise HTTPException(status_code=403, detail="Review does not belong to this user")
00668
00669     order_item = db.query(models.OrderItem).filter(models.OrderItem.orderid == order.id).first()
00670     if not order_item:
00671         raise HTTPException(status_code=400, detail="Order has no items to review")
00672
00673     menuitemid = review.menuitemid or order_item.menuitemid
00674     restaurantid = review.restaurantid or order.restaurantid
00675
00676     db_review = models.Review(
00677         rating=review.rating,
00678         comment=review.comment,
00679         orderid=review.orderid,
00680         menuitemid=menuitemid,
00681         restaurantid=restaurantid,
00682         userid=review.userid,
00683     )
00684     db.add(db_review)
00685     db.commit()
00686
00687     # Tính lại điểm đánh giá trung bình của nhà hàng
00688     restaurant = db.query(models.Restaurant).filter(models.Restaurant.id == restaurantid).first()
00689     if restaurant:
00690         all_reviews = db.query(models.Review).filter(models.Review.restaurantid == restaurantid).all()
00691         if all_reviews:
00692             avg_rating = sum(r.rating for r in all_reviews) / len(all_reviews)
00693             restaurant.rating = int(round(avg_rating))
00694             db.add(restaurant)
00695             db.commit()
00696
00697     db.refresh(db_review)
00698     return db_review
00699
00700
00701 @app.get("/users/{user_id}/profile-summary", response_model=schemas.UserProfileSummary)
00702 def read_user_profile_summary(user_id: int, db: Session = Depends(get_db)):
00703     """Lấy tóm tắt hồ sơ người dùng: điểm, đơn hàng đã giao, tổng chi tiêu."""
00704     user = db.query(models.User).filter(models.User.id == user_id).first()
00705     if not user:
00706         raise HTTPException(status_code=404, detail="User not found")
00707
00708     orders = db.query(models.Orders).filter(models.Orders.userid == user_id).all()
00709     delivered_orders = [o for o in orders if (o.status or "").lower() in {"delivered", "completed", "done"}]
00710     total_spent = sum(o.totalprice or 0 for o in orders)
00711
00712     # Quy ước điểm đơn giản để Android hiển thị động thay cho số cứng.
00713     points = total_spent // 10000
00714
00715     return {
00716         "user_id": user.id,
00717         "user_name": user.name,
00718         "points": points,
00719         "delivered_orders": len(delivered_orders),
00720         "total_spent": total_spent
00721     }
00722
00723
00724 # --- FAQ ENDPOINTS ---
00725
00726
00727 @app.get("/faqs/", response_model=List[schemas.FAQ])
00728 def read_faqs(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
00729     """Lấy danh sách các câu hỏi thường gặp (FAQ)."""
00730     return db.query(models.FAQ).filter(models.FAQ.isactive == True).offset(skip).limit(limit).all()
00731
00732
00733 # --- NOTIFICATION ENDPOINTS ---
00734
00735
00736 @app.get("/users/{user_id}/notifications/", response_model=List[schemas.Notification])
00737 def read_user_notifications(user_id: int, db: Session = Depends(get_db)):
00738     """Lấy danh sách thông báo của người dùng."""
00739     return db.query(models.Notification).filter(models.Notification.userid ==
00740         user_id).order_by(models.Notification.createdat.desc()).all()
00741
00742 @app.post("/notifications/", response_model=schemas.Notification)
00743 def create_notification(notification: schemas.NotificationCreate, db: Session = Depends(get_db)):
00744     """Tạo thông báo mới và gửi push notification."""
00745     # Sử dụng notify_user để tạo thông báo - nó sẽ vừa lưu vào DB vừa gửi push
00746     notify_user(
00747         db=db,
00748         user_id=notification.userid,
00749         title=notification.title,
00750         body=notification.content,
00751         order_id=notification.orderid
00752     )

```

```

00753
00754 # Lấy thông báo vừa tạo để trả về
00755 db_notification = db.query(models.Notification).filter(
00756     models.Notification.userid == notification.userid,
00757     models.Notification.title == notification.title
00758 ).order_by(models.Notification.id.desc()).first()
00759
00760 return db_notification if db_notification else {
00761     "id": 0,
00762     "title": notification.title,
00763     "type": notification.type,
00764     "content": notification.content,
00765     "isread": notification.isread,
00766     "createdat": datetime.now(),
00767     "userid": notification.userid,
00768     "orderid": notification.orderid,
00769     "sessionid": notification.sessionid
00770 }
00771
00772
00773 @app.put("/notifications/{notification_id}/read")
00774 def mark_notification_as_read(notification_id: int, db: Session = Depends(get_db)):
00775     """Đánh dấu thông báo đã đọc."""
00776     db_notification = db.query(models.Notification).filter(models.Notification.id == notification_id).first()
00777     if not db_notification:
00778         raise HTTPException(status_code=404, detail="Notification not found")
00779
00780     db_notification.isread = True
00781     db.commit()
00782     return {"message": "Notification marked as read"}
00783
00784
00785 if __name__ == "__main__":
00786     import uvicorn
00787     uvicorn.run(app, host="0.0.0.0", port=8000)
00788
00789
00790

```

9.105 backend/models.py File Reference

Classes

- class [models.Category](#)
- class [models.FAQ](#)
- class [models.User](#)
- class [models.Shipper](#)
- class [models.Promotion](#)
- class [models.Restaurant](#)
- class [models.MenuItem](#)
- class [models.Address](#)
- class [models.Orders](#)
- class [models.OrderItem](#)
- class [models.Payment](#)
- class [models.Delivery](#)
- class [models.Review](#)
- class [models.Notification](#)
- class [models.ChatSession](#)
- class [models.ChatMessage](#)
- class [models.UserDevice](#)
- class [models.UserFAQ](#)
- class [models.UsedPromotion](#)
- class [models.LoyaltyPoint](#)
- class [models.SocialShare](#)

Namespaces

- namespace [models](#)

9.106 models.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: models.py
00003
00004 Định nghĩa tất cả SQLAlchemy ORM models cho cơ sở dữ liệu ứng dụng đặt món.
00005 Bao gồm 19 model: User, Restaurant, MenuItem, Orders, Payment, Delivery, Review, v.v.
00006 """
00007
00008 from sqlalchemy import Column, Integer, String, Boolean, Numeric, Text, Date, Time, ForeignKey, DateTime, func
00009 from sqlalchemy.orm import relationship
00010 from database import Base
00011
00012 class Category(Base):
00013     """ORM model cho bảng category, phân loại món ăn."""
00014     __tablename__ = "category"
00015     id = Column(Integer, primary_key=True, index=True)
00016     name = Column(String(255), nullable=False)
00017     type = Column(String(255))
00018     description = Column(Text)
00019     menu_items = relationship("MenuItem", back_populates="category")
00020
00021 class FAQ(Base):
00022     """ORM model cho bảng faq, lưu trữ câu hỏi thường gặp."""
00023     __tablename__ = "faq"
00024     id = Column(Integer, primary_key=True, index=True)
00025     question = Column(String(255))
00026     answer = Column(Text)
00027     isactive = Column(Boolean, default=True)
00028     user_faqs = relationship("UserFAQ", back_populates="faq")
00029
00030 class User(Base):
00031     """ORM model cho bảng User, lưu trữ thông tin tài khoản người dùng."""
00032     __tablename__ = "User" # Quoted in SQL as "User"
00033     id = Column(Integer, primary_key=True, index=True)
00034     name = Column(String(255), nullable=False)
00035     username = Column(String(255), nullable=False)
00036     email = Column(String(255), nullable=False, unique=True)
00037     password = Column(String(255), nullable=False)
00038     phone = Column(String(255), nullable=False)
00039     role = Column(String(255), nullable=False)
00040
00041     addresses = relationship("Address", back_populates="user")
00042     orders = relationship("Orders", back_populates="user")
00043     reviews = relationship("Review", back_populates="user")
00044     user_devices = relationship("UserDevice", back_populates="user")
00045     user_faqs = relationship("UserFAQ", back_populates="user")
00046     chat_sessions = relationship("ChatSession", back_populates="user")
00047     notifications = relationship("Notification", back_populates="user")
00048     loyalty_points = relationship("LoyaltyPoint", back_populates="user")
00049     social_shares = relationship("SocialShare", back_populates="user")
00050
00051 class Shipper(Base):
00052     """ORM model cho bảng shipper, quản lý thông tin người giao hàng."""
00053     __tablename__ = "shipper"
00054     id = Column(Integer, primary_key=True, index=True)
00055     name = Column(String(255), nullable=False)
00056     phone = Column(String(255), nullable=False)
00057     rating = Column(Integer)
00058     description = Column(Text)
00059     status = Column(String(255), nullable=False)
00060     deliveries = relationship("Delivery", back_populates="shipper")
00061
00062 class Promotion(Base):
00063     """ORM model cho bảng promotion, mã khuyến mãi và giảm giá."""
00064     __tablename__ = "promotion"
00065     id = Column(Integer, primary_key=True, index=True)
00066     code = Column(String(255))
00067     discounttype = Column(String(255))
00068     discountvalue = Column(Integer)
00069     expiredate = Column(Date)
00070     minordervalue = Column(Integer)
00071     status = Column(String(255), nullable=False)
00072     used_promotions = relationship("UsedPromotion", back_populates="promotion")
00073

```

```

00074 class Restaurant(Base):
00075     """ORM model cho bảng restaurant, thông tin nhà hàng."""
00076     __tablename__ = "restaurant"
00077     id = Column(Integer, primary_key=True, index=True)
00078     name = Column(String(255), nullable=False)
00079     image_url = Column(Text)
00080     address = Column(String(255))
00081     rating = Column(Integer)
00082     open_time = Column(Time)
00083     close_time = Column(Time)
00084     phone_number = Column(String(255), nullable=False)
00085     status = Column(String(255), nullable=False)
00086     description = Column(Text)
00087
00088     menu_items = relationship("MenuItem", back_populates="restaurant")
00089     orders = relationship("Orders", back_populates="restaurant")
00090     reviews = relationship("Review", back_populates="restaurant")
00091     social_shares = relationship("SocialShare", back_populates="restaurant")
00092
00093 class MenuItem(Base):
00094     """ORM model cho bảng menuitem, món ăn thuộc nhà hàng."""
00095     __tablename__ = "menuitem"
00096     id = Column(Integer, primary_key=True, index=True)
00097     name = Column(String(255), nullable=False)
00098     image_url = Column(Text)
00099     price = Column(Integer)
00100     is_available = Column(Boolean, default=True)
00101     description = Column(Text)
00102     restaurantid = Column(Integer, ForeignKey("restaurant.id"))
00103     categoryid = Column(Integer, ForeignKey("category.id"))
00104
00105     restaurant = relationship("Restaurant", back_populates="menu_items")
00106     category = relationship("Category", back_populates="menu_items")
00107     order_items = relationship("OrderItem", back_populates="menu_item")
00108     reviews = relationship("Review", back_populates="menu_item")
00109     social_shares = relationship("SocialShare", back_populates="menu_item")
00110
00111 class Address(Base):
00112     """ORM model cho bảng address, địa chỉ giao hàng của người dùng."""
00113     __tablename__ = "address"
00114     id = Column(Integer, primary_key=True, index=True)
00115     detail = Column(Text)
00116     phone = Column(String(255))
00117     userid = Column(Integer, ForeignKey("User.id"))
00118
00119     user = relationship("User", back_populates="addresses")
00120     orders = relationship("Orders", back_populates="address")
00121
00122 class Orders(Base):
00123     """ORM model cho bảng orders, đơn hàng đặt món."""
00124     __tablename__ = "orders"
00125     id = Column(Integer, primary_key=True, index=True)
00126     status = Column(String(255), nullable=False)
00127     createdat = Column(DateTime, server_default=func.now())
00128     preorderdate = Column(Date)
00129     preordertime = Column(Time)
00130     totalprice = Column(Integer)
00131     restaurantid = Column(Integer, ForeignKey("restaurant.id"))
00132     addressid = Column(Integer, ForeignKey("address.id"))
00133     userid = Column(Integer, ForeignKey("User.id"))
00134
00135     user = relationship("User", back_populates="orders")
00136     restaurant = relationship("Restaurant", back_populates="orders")
00137     address = relationship("Address", back_populates="orders")
00138     order_items = relationship("OrderItem", back_populates="order")
00139     payments = relationship("Payment", back_populates="order")
00140     delivery = relationship("Delivery", back_populates="order")
00141     used_promotions = relationship("UsedPromotion", back_populates="order")
00142     notifications = relationship("Notification", back_populates="order")
00143
00144 class OrderItem(Base):
00145     """ORM model cho bảng orderitem, chi tiết từng món trong đơn hàng."""
00146     __tablename__ = "orderitem"
00147     id = Column(Integer, primary_key=True, index=True)
00148     quantity = Column(Integer, nullable=False)
00149     price = Column(Integer, nullable=False)
00150     menuitemid = Column(Integer, ForeignKey("menuitem.id"))
00151     orderid = Column(Integer, ForeignKey("orders.id"))
00152
00153     order = relationship("Orders", back_populates="order_items")
00154     menu_item = relationship("MenuItem", back_populates="order_items")
00155
00156 class Payment(Base):
00157     """ORM model cho bảng payment, thông tin thanh toán đơn hàng."""
00158     __tablename__ = "payment"
00159     id = Column(Integer, primary_key=True, index=True)
00160     status = Column(String(255), nullable=False)

```

```

00161     method = Column(String(255))
00162     orderid = Column(Integer, ForeignKey("orders.id"))
00163     order = relationship("Orders", back_populates="payments")
00164
00165 class Delivery(Base):
00166     """ORM model cho bảng delivery, thông tin giao hàng."""
00167     __tablename__ = "delivery"
00168     id = Column(Integer, primary_key=True, index=True)
00169     status = Column(String(255), nullable=False)
00170     deliverytime = Column(Time)
00171     orderid = Column(Integer, ForeignKey("orders.id"))
00172     shipperid = Column(Integer, ForeignKey("shipper.id"))
00173
00174     order = relationship("Orders", back_populates="delivery")
00175     shipper = relationship("Shipper", back_populates="deliveries")
00176
00177 class Review(Base):
00178     """ORM model cho bảng review, đánh giá của người dùng về món ăn và nhà hàng."""
00179     __tablename__ = "review"
00180     id = Column(Integer, primary_key=True, index=True)
00181     rating = Column(Integer)
00182     comment = Column(Text)
00183     menuitemid = Column(Integer, ForeignKey("menuitem.id"))
00184     restaurantid = Column(Integer, ForeignKey("restaurant.id"))
00185     userid = Column(Integer, ForeignKey("User.id"))
00186     orderid = Column(Integer, ForeignKey("orders.id"))
00187
00188     user = relationship("User", back_populates="reviews")
00189     menu_item = relationship("MenuItem", back_populates="reviews")
00190     restaurant = relationship("Restaurant", back_populates="reviews")
00191     order = relationship("Orders")
00192
00193 class Notification(Base):
00194     """ORM model cho bảng notification, thông báo đẩy đến người dùng."""
00195     __tablename__ = "notification"
00196     id = Column(Integer, primary_key=True, index=True)
00197     title = Column(String(255))
00198     type = Column(String(255))
00199     content = Column(Text)
00200     isread = Column(Boolean, default=False)
00201     createdat = Column(DateTime, server_default=func.now())
00202     userid = Column(Integer, ForeignKey("User.id"))
00203     orderid = Column(Integer, ForeignKey("orders.id"), nullable=True)
00204     sessionid = Column(Integer, ForeignKey("chatsession.id"), nullable=True)
00205
00206     user = relationship("User", back_populates="notifications")
00207     order = relationship("Orders", back_populates="notifications")
00208     session = relationship("ChatSession", back_populates="notifications")
00209
00210 class ChatSession(Base):
00211     """ORM model cho bảng chatsession, phiên trò chuyện với chatbot AI."""
00212     __tablename__ = "chatsession"
00213     id = Column(Integer, primary_key=True, index=True)
00214     createdat = Column(DateTime, server_default=func.now())
00215     status = Column(String(255), nullable=False)
00216     userid = Column(Integer, ForeignKey("User.id"))
00217
00218     user = relationship("User", back_populates="chat_sessions")
00219     messages = relationship("ChatMessage", back_populates="session")
00220     notifications = relationship("Notification", back_populates="session")
00221
00222 class ChatMessage(Base):
00223     """ORM model cho bảng chatmessage, tin nhắn trong phiên trò chuyện."""
00224     __tablename__ = "chatmessage"
00225     id = Column(Integer, primary_key=True, index=True)
00226     senderrole = Column(String(255)) # "user" hoặc "bot"
00227     message = Column(Text)
00228     sentat = Column(DateTime, server_default=func.now())
00229     sessionid = Column(Integer, ForeignKey("chatsession.id"))
00230
00231     session = relationship("ChatSession", back_populates="messages")
00232
00233 class UserDevice(Base):
00234     """ORM model cho bảng userdevice, FCM token thiết bị của người dùng."""
00235     __tablename__ = "userdevice"
00236     id = Column(Integer, primary_key=True, index=True)
00237     device_token = Column(String(255), nullable=False)
00238     device_type = Column(String(255))
00239     last_active = Column(DateTime, server_default=func.now())
00240     userid = Column(Integer, ForeignKey("User.id"))
00241
00242     user = relationship("User", back_populates="user_devices")
00243
00244 class UserFAQ(Base):
00245     """ORM model cho bảng userfaq, lịch sử xem FAQ của người dùng."""
00246     __tablename__ = "userfaq"
00247     id = Column(Integer, primary_key=True, index=True)

```

```

00248 viewedat = Column(DateTime, server_default=func.now())
00249 userid = Column(Integer, ForeignKey("User.id"))
00250 faqid = Column(Integer, ForeignKey("faq.id"))
00251
00252 user = relationship("User", back_populates="user_faqs")
00253 faq = relationship("FAQ", back_populates="user_faqs")
00254
00255 class UsedPromotion(Base):
00256     """ORM model cho bảng usedpromotion, mã khuyến mãi đã sử dụng."""
00257     __tablename__ = "usedpromotion"
00258     id = Column(Integer, primary_key=True, index=True)
00259     usedat = Column(Date)
00260     promotionid = Column(Integer, ForeignKey("promotion.id"))
00261     orderid = Column(Integer, ForeignKey("orders.id"))
00262
00263     promotion = relationship("Promotion", back_populates="used_promotions")
00264     order = relationship("Orders", back_populates="used_promotions")
00265
00266 class LoyaltyPoint(Base):
00267     """ORM model cho bảng loaltypoint, điểm tích lũy thành viên."""
00268     __tablename__ = "loaltypoint"
00269     id = Column(Integer, primary_key=True, index=True)
00270     points = Column(Integer, default=0)
00271     updatedat = Column(Date)
00272     userid = Column(Integer, ForeignKey("User.id"))
00273     menuitemid = Column(Integer, ForeignKey("menuitem.id"))
00274
00275     user = relationship("User", back_populates="loyalty_points")
00276     menu_item = relationship("MenuItem")
00277
00278 class SocialShare(Base):
00279     """ORM model cho bảng socialshare, lượt chia sẻ mạng xã hội."""
00280     __tablename__ = "socialshare"
00281     id = Column(Integer, primary_key=True, index=True)
00282     sharetype = Column(String(255))
00283     platform = Column(String(255))
00284     context = Column(Text)
00285     createdat = Column(Date)
00286     userid = Column(Integer, ForeignKey("User.id"))
00287     restaurantid = Column(Integer, ForeignKey("restaurant.id"))
00288     menuitemid = Column(Integer, ForeignKey("menuitem.id"))
00289
00290     user = relationship("User", back_populates="social_shares")
00291     restaurant = relationship("Restaurant", back_populates="social_shares")
00292     menu_item = relationship("MenuItem", back_populates="social_shares")
00293

```

9.107 backend/notification_service.py File Reference

Namespaces

- namespace [notification_service](#)

Functions

- [notification_service.send_fcm_notification](#) (str token, str title, str body, dict data=None)
- [notification_service.notify_user](#) (Session db, int user_id, str title, str body, int order_id=None)

9.108 notification_service.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Module: notification_service.py
00003
00004 Service gửi thông báo đẩy qua Firebase Cloud Messaging (FCM).
00005 Lưu thông báo vào database và gửi push notification đến tất cả thiết bị của người dùng.
00006 """
00007
00008 from firebase_admin import messaging
00009 import firebase_admin

```

```

00010 from firebase_utils import init_firebase
00011 from sqlalchemy.orm import Session
00012 import models
00013
00014 def send_fcm_notification(token: str, title: str, body: str, data: dict = None):
00015     """Gửi thông báo tới một thiết bị Android cụ thể qua FCM."""
00016     if not firebase_admin._apps:
00017         init_firebase()
00018
00019     message = messaging.Message(
00020         notification=messaging.Notification(
00021             title=title,
00022             body=body,
00023         ),
00024         data=data,
00025         token=token,
00026     )
00027
00028     try:
00029         response = messaging.send(message)
00030         print(f"Successfully sent message: {response}")
00031         return True
00032     except Exception as e:
00033         print(f"Error sending FCM message: {e}")
00034         return False
00035
00036 def notify_user(db: Session, user_id: int, title: str, body: str, order_id: int = None):
00037     """Gửi thông báo cho tất cả thiết bị của một người dùng."""
00038     # 1. Lưu thông báo vào bảng Notification trong DB
00039     db_notification = models.Notification(
00040         title=title,
00041         content=body,
00042         type="order_update",
00043         userid=user_id,
00044         orderid=order_id,
00045         isread=False
00046     )
00047     db.add(db_notification)
00048     db.commit()
00049
00050     # 2. Lấy danh sách FCM Token của người dùng đó
00051     devices = db.query(models.UserDevice).filter(models.UserDevice.userid == user_id).all()
00052
00053     # 3. Gửi thông báo thực tới điện thoại
00054     for device in devices:
00055         send_fcm_notification(
00056             token=device.device_token,
00057             title=title,
00058             body=body,
00059             data={"order_id": str(order_id)} if order_id else None
00060         )

```

9.109 backend/payment_service.py File Reference

Namespaces

- namespace [payment_service](#)

Functions

- [payment_service.generate_vnpay_url](#) (str order_id, int amount, str ip_address)

Variables

- str [payment_service.VNP_URL](#) = "https://sandbox.vnpayment.vn/paymentv2/vpcpay.html"
- str [payment_service.VNP_TMNCODE](#) = "2HON5OA2"
- str [payment_service.VNP_HASH_SECRET](#) = "UGDPB7W11M55W6UTQ0N69X36I2VIE0UL"
- str [payment_service.VNP_RETURN_URL](#) = "http://localhost:8000/vnpay_return"

9.110 payment_service.py

[Go to the documentation of this file.](#)

```
00001 """
00002 Module: payment_service.py
00003
00004 Service tạo URL thanh toán VNPay (sandbox).
00005 Xây dựng payment URL với chữ ký HMAC-SHA512 để xác thực giao dịch.
00006 """
00007
00008 import hashlib
00009 import hmac
00010 import urllib.parse
00011 from datetime import datetime
00012
00013
00014 # Thông tin VNPay Sandbox
00015 VNP_URL = "https://sandbox.vnpayment.vn/paymentv2/vpcpay.html"
00016 VNP_TMNCODE = "2HON5OA2" # lấy từ VNPay Test Portal
00017 VNP_HASH_SECRET = "UGDPB7W11M55W6UTQ0N69X36I2VIE0UL" # lấy từ VNPay Test Portal
00018 VNP_RETURN_URL = "http://localhost:8000/vnpay_return"
00019
00020
00021
00022
00023 def generate_vnpay_url(order_id: str, amount: int, ip_address: str):
00024     """Tạo URL thanh toán VNPay với chữ ký HMAC-SHA512 cho đơn hàng."""
00025
00026     params = {
00027         "vnp_Version": "2.1.0",
00028         "vnp_Command": "pay",
00029         "vnp_TmnCode": VNP_TMNCODE,
00030         "vnp_Amount": int(amount) * 100,
00031         "vnp_CurrCode": "VND",
00032         "vnp_TxnRef": order_id,
00033         "vnp_OrderInfo": f"Thanh toan don hang {order_id}",
00034         "vnp_OrderType": "other",
00035         "vnp_Locale": "vn",
00036         "vnp_ReturnUrl": VNP_RETURN_URL,
00037         "vnp_IpAddr": ip_address,
00038         "vnp_CreateDate": datetime.now().strftime("%Y%m%d%H%M%S"),
00039     }
00040
00041
00042     # sort params
00043     sorted_params = sorted(params.items())
00044
00045
00046     query_string = urllib.parse.urlencode(sorted_params)
00047
00048
00049     # tạo hash
00050     hash_value = hmac.new(
00051         VNP_HASH_SECRET.encode(),
00052         query_string.encode(),
00053         hashlib.sha512
00054     ).hexdigest()
00055
00056
00057     payment_url = f"{VNP_URL}?{query_string}&vnp_SecureHash={hash_value}"
00058
00059
00060     return payment_url
```

9.111 backend/schemas.py File Reference

Classes

- class [schemas.CategoryBase](#)
- class [schemas.CategoryCreate](#)
- class [schemas.Category](#)
- class [schemas.MenuItemBase](#)
- class [schemas.MenuItemCreate](#)
- class [schemas.MenuItem](#)

- class [schemas.MenuItemWithRestaurant](#)
- class [schemas.RestaurantBase](#)
- class [schemas.RestaurantCreate](#)
- class [schemas.Restaurant](#)
- class [schemas.UserBase](#)
- class [schemas.UserCreate](#)
- class [schemas.User](#)
- class [schemas.Token](#)
- class [schemas.TokenData](#)
- class [schemas.AddressBase](#)
- class [schemas.AddressCreate](#)
- class [schemas.AddressUpdate](#)
- class [schemas.Address](#)
- class [schemas.OrderItemBase](#)
- class [schemas.OrderItemCreate](#)
- class [schemas.OrderItem](#)
- class [schemas.OrderBase](#)
- class [schemas.OrderCreate](#)
- class [schemas.Order](#)
- class [schemas.OrderItemDetail](#)
- class [schemas.OrderDetail](#)
- class [schemas.OrderStatusUpdate](#)
- class [schemas.UserProfileSummary](#)
- class [schemas.ChatMessageBase](#)
- class [schemas.ChatMessageCreate](#)
- class [schemas.ChatMessage](#)
- class [schemas.ChatSessionCreate](#)
- class [schemas.ChatSession](#)
- class [schemas.UserDeviceBase](#)
- class [schemas.UserDeviceCreate](#)
- class [schemas.UserDevice](#)
- class [schemas.UserFAQBase](#)
- class [schemas.UserFAQCreate](#)
- class [schemas.UserFAQ](#)
- class [schemas.FAQBase](#)
- class [schemas.FAQCreate](#)
- class [schemas.FAQ](#)
- class [schemas.ShipperBase](#)
- class [schemas.ShipperCreate](#)
- class [schemas.Shipper](#)
- class [schemas.PromotionBase](#)
- class [schemas.PromotionCreate](#)
- class [schemas.Promotion](#)
- class [schemas.PaymentBase](#)
- class [schemas.PaymentCreate](#)
- class [schemas.Payment](#)
- class [schemas.DeliveryBase](#)
- class [schemas.DeliveryCreate](#)
- class [schemas.Delivery](#)
- class [schemas.ReviewBase](#)
- class [schemas.ReviewCreate](#)
- class [schemas.Review](#)
- class [schemas.ReviewResponse](#)
- class [schemas.MenuItemWithReviews](#)
- class [schemas.NotificationBase](#)

- class [schemas.NotificationCreate](#)
- class [schemas.Notification](#)
- class [schemas.UsedPromotionBase](#)
- class [schemas.UsedPromotionCreate](#)
- class [schemas.UsedPromotion](#)
- class [schemas.LoyaltyPointBase](#)
- class [schemas.LoyaltyPointCreate](#)
- class [schemas.LoyaltyPoint](#)
- class [schemas.SocialShareBase](#)
- class [schemas.SocialShareCreate](#)
- class [schemas.SocialShare](#)

Namespaces

- namespace [schemas](#)

9.112 schemas.py

[Go to the documentation of this file.](#)

```
00001 """
00002 Module: schemas.py
00003
00004 Định nghĩa tất cả Pydantic v2 schemas cho request/response validation.
00005 Bao gồm schema cho Category, MenuItem, Restaurant, User, Order, Payment, Review, v.v.
00006 """
00007
00008 from pydantic import BaseModel, ConfigDict
00009 from typing import List, Optional
00010 from datetime import date, time, datetime
00011 from decimal import Decimal
00012
00013 # Base Schemas
00014
00015 class CategoryBase(BaseModel):
00016     """Schema cơ sở cho danh mục món ăn."""
00017     name: str
00018     type: Optional[str] = None
00019     description: Optional[str] = None
00020
00021 class CategoryCreate(CategoryBase):
00022     pass
00023
00024 class Category(CategoryBase):
00025     id: int
00026     model_config = ConfigDict(from_attributes=True)
00027
00028 class MenuItemBase(BaseModel):
00029     """Schema cơ sở cho món ăn trong thực đơn."""
00030     name: str
00031     image_url: Optional[str] = None
00032     price: int
00033     is_available: bool = True
00034     description: Optional[str] = None
00035     restaurantid: int
00036     categoryid: int
00037
00038 class MenuItemCreate(MenuItemBase):
00039     pass
00040
00041 class MenuItem(MenuItemBase):
00042     id: int
00043     model_config = ConfigDict(from_attributes=True)
00044
00045 class MenuItemWithRestaurant(MenuItemBase):
00046     id: int
00047     restaurant_name: Optional[str] = None
00048     model_config = ConfigDict(from_attributes=True)
00049
00050 class RestaurantBase(BaseModel):
00051     """Schema cơ sở cho nhà hàng."""
00052     name: str
```



```

00053     image_url: Optional[str] = None
00054     address: Optional[str] = None
00055     rating: Optional[int] = None
00056     open_time: Optional[time] = None
00057     close_time: Optional[time] = None
00058     phone_number: str
00059     status: str
00060     description: Optional[str] = None
00061
00062 class RestaurantCreate(RestaurantBase):
00063     pass
00064
00065 class Restaurant(RestaurantBase):
00066     id: int
00067     menu_items: List[MenuItem] = []
00068     model_config = ConfigDict(from_attributes=True)
00069
00070 class UserBase(BaseModel):
00071     """Schema cơ sở cho người dùng."""
00072     name: str
00073     username: str
00074     email: str
00075     phone: str
00076     role: str
00077
00078 class UserCreate(UserBase):
00079     password: str
00080
00081 class User(UserBase):
00082     id: int
00083     model_config = ConfigDict(from_attributes=True)
00084
00085 class Token(BaseModel):
00086     """Schema phản hồi JWT token khi đăng nhập thành công."""
00087     access_token: str
00088     token_type: str
00089
00090 class TokenData(BaseModel):
00091     """Schema dữ liệu giải mã từ JWT token."""
00092     email: Optional[str] = None
00093
00094 class AddressBase(BaseModel):
00095     """Schema cơ sở cho địa chỉ giao hàng."""
00096     detail: str
00097     phone: Optional[str] = None
00098     userid: int
00099
00100 class AddressCreate(AddressBase):
00101     pass
00102
00103 class AddressUpdate(BaseModel):
00104     detail: Optional[str] = None
00105     phone: Optional[str] = None
00106
00107 class Address(AddressBase):
00108     id: int
00109     model_config = ConfigDict(from_attributes=True)
00110
00111 class OrderItemBase(BaseModel):
00112     """Schema cơ sở cho chi tiết món trong đơn hàng."""
00113     quantity: int
00114     price: int
00115     menuitemid: int
00116
00117 class OrderItemCreate(OrderItemBase):
00118     pass
00119
00120 class OrderItem(OrderItemBase):
00121     id: int
00122     model_config = ConfigDict(from_attributes=True)
00123
00124 class OrderBase(BaseModel):
00125     """Schema cơ sở cho đơn hàng."""
00126     status: str
00127     preorderdate: Optional[date] = None
00128     preordertime: Optional[time] = None
00129     totalprice: int
00130     restaurantid: int
00131     addressid: int
00132     userid: int
00133
00134 class OrderCreate(OrderBase):
00135     order_items: List[OrderItemCreate]
00136
00137 class Order(OrderBase):
00138     id: int
00139     createdat: datetime

```

```

00140     order_items: List[OrderItem] = []
00141     model_config = ConfigDict(from_attributes=True)
00142
00143 class OrderItemDetail(BaseModel):
00144     """Schema chi tiết món ăn trong đơn hàng kèm tên và ảnh."""
00145     id: int
00146     quantity: int
00147     price: int
00148     menuitemid: int
00149     menuitem_name: Optional[str] = None
00150     image_url: Optional[str] = None
00151
00152 class OrderDetail(OrderBase):
00153     """Schema chi tiết đơn hàng đầy đủ kèm tên nhà hàng và địa chỉ."""
00154     id: int
00155     createdat: datetime
00156     restaurant_name: Optional[str] = None
00157     address_detail: Optional[str] = None
00158     order_items: List[OrderItemDetail] = []
00159
00160 class OrderStatusUpdate(BaseModel):
00161     """Schema cập nhật trạng thái đơn hàng."""
00162     status: str
00163
00164 class UserProfileSummary(BaseModel):
00165     """Schema tóm tắt hồ sơ người dùng: điểm, đơn hàng, tổng chi tiêu."""
00166     user_id: int
00167     user_name: str
00168     points: int
00169     delivered_orders: int
00170     total_spent: int
00171
00172 # Chat Schemas
00173
00174 class ChatMessageBase(BaseModel):
00175     """Schema cơ sở cho tin nhắn chat."""
00176     message: str
00177
00178 class ChatMessageCreate(ChatMessageBase):
00179     pass
00180
00181 class ChatMessage(ChatMessageBase):
00182     id: int
00183     senderrole: str
00184     sentat: datetime
00185     sessionid: int
00186     model_config = ConfigDict(from_attributes=True)
00187
00188 class ChatSessionCreate(BaseModel):
00189     """Schema tạo phiên trò chuyện chatbot mới."""
00190     userid: int
00191
00192 class ChatSession(BaseModel):
00193     id: int
00194     createdat: datetime
00195     status: str
00196     messages: List[ChatMessage] = []
00197     model_config = ConfigDict(from_attributes=True)
00198
00199 # UserDevice Schemas
00200
00201 class UserDeviceBase(BaseModel):
00202     """Schema cơ sở cho thiết bị người dùng (FCM token)."""
00203     device_token: str
00204     device_type: Optional[str] = None
00205
00206 class UserDeviceCreate(UserDeviceBase):
00207     userid: int
00208
00209 class UserDevice(UserDeviceBase):
00210     id: int
00211     last_active: datetime
00212     userid: int
00213     model_config = ConfigDict(from_attributes=True)
00214
00215 # UserFAQ Schemas
00216
00217 class UserFAQBase(BaseModel):
00218     """Schema cơ sở cho lịch sử xem FAQ của người dùng."""
00219     userid: int
00220     faqid: int
00221
00222 class UserFAQCreate(UserFAQBase):
00223     pass
00224
00225 class UserFAQ(UserFAQBase):
00226     id: int

```

```

00227     viewedat: datetime
00228     model_config = ConfigDict(from_attributes=True)
00229
00230 #     FAQ Schemas
00231
00232 class FAQBase(BaseModel):
00233     """Schema cơ sở cho câu hỏi thường gặp."""
00234     question: str
00235     answer: str
00236     isactive: bool = True
00237
00238 class FAQCreate(FAQBase):
00239     pass
00240
00241 class FAQ(FAQBase):
00242     id: int
00243     model_config = ConfigDict(from_attributes=True)
00244
00245 #     Shipper Schemas
00246
00247 class ShipperBase(BaseModel):
00248     """Schema cơ sở cho người giao hàng."""
00249     name: str
00250     phone: str
00251     rating: Optional[int] = None
00252     description: Optional[str] = None
00253     status: str
00254
00255 class ShipperCreate(ShipperBase):
00256     pass
00257
00258 class Shipper(ShipperBase):
00259     id: int
00260     model_config = ConfigDict(from_attributes=True)
00261
00262 #     Promotion Schemas
00263
00264 class PromotionBase(BaseModel):
00265     """Schema cơ sở cho mã khuyến mãi."""
00266     code: str
00267     discounttype: str
00268     discountvalue: int
00269     expiredate: date
00270     minordervalue: int
00271     status: str
00272
00273 class PromotionCreate(PromotionBase):
00274     pass
00275
00276 class Promotion(PromotionBase):
00277     id: int
00278     model_config = ConfigDict(from_attributes=True)
00279
00280 #     Payment Schemas
00281
00282 class PaymentBase(BaseModel):
00283     """Schema cơ sở cho thông tin thanh toán."""
00284     status: str
00285     method: str
00286     orderid: int
00287
00288 class PaymentCreate(PaymentBase):
00289     pass
00290
00291 class Payment(PaymentBase):
00292     id: int
00293     model_config = ConfigDict(from_attributes=True)
00294
00295 #     Delivery Schemas
00296
00297 class DeliveryBase(BaseModel):
00298     """Schema cơ sở cho thông tin giao hàng."""
00299     status: str
00300     deliverytime: Optional[time] = None
00301     orderid: int
00302     shipperid: int
00303
00304 class DeliveryCreate(DeliveryBase):
00305     pass
00306
00307 class Delivery(DeliveryBase):
00308     id: int
00309     model_config = ConfigDict(from_attributes=True)
00310
00311 #     Review Schemas
00312
00313 class ReviewBase(BaseModel):

```

```

00314     """Schema cơ sở cho đánh giá của người dùng."""
00315     rating: int
00316     comment: Optional[str] = None
00317     orderid: int
00318     userid: int
00319     menuitemid: Optional[int] = None
00320     restaurantid: Optional[int] = None
00321
00322 class ReviewCreate(ReviewBase):
00323     pass
00324
00325 class Review(ReviewBase):
00326     id: int
00327     model_config = ConfigDict(from_attributes=True)
00328
00329 class ReviewResponse(BaseModel):
00330     id: int
00331     rating: int
00332     comment: Optional[str] = None
00333     userid: int
00334     user_name: Optional[str] = None
00335     model_config = ConfigDict(from_attributes=True)
00336
00337 class MenuItemWithReviews(MenuItemBase):
00338     """Schema món ăn kèm danh sách đánh giá và điểm trung bình."""
00339     id: int
00340     restaurant_name: Optional[str] = None
00341     reviews: List["ReviewResponse"] = []
00342     avg_rating: float = 0.0
00343     model_config = ConfigDict(from_attributes=True)
00344
00345 # Notification Schemas
00346
00347 class NotificationBase(BaseModel):
00348     """Schema cơ sở cho thông báo đẩy."""
00349     title: str
00350     type: str
00351     content: str
00352     isread: bool = False
00353     userid: int
00354     orderid: Optional[int] = None
00355     sessionid: Optional[int] = None
00356
00357 class NotificationCreate(NotificationBase):
00358     pass
00359
00360 class Notification(NotificationBase):
00361     id: int
00362     createdat: datetime
00363     model_config = ConfigDict(from_attributes=True)
00364
00365 # UsedPromotion Schemas
00366
00367 class UsedPromotionBase(BaseModel):
00368     """Schema cơ sở cho mã khuyến mãi đã sử dụng."""
00369     usedat: date
00370     promotionid: int
00371     orderid: int
00372
00373 class UsedPromotionCreate(UsedPromotionBase):
00374     pass
00375
00376 class UsedPromotion(UsedPromotionBase):
00377     id: int
00378     model_config = ConfigDict(from_attributes=True)
00379
00380 # LoyaltyPoint Schemas
00381
00382 class LoyaltyPointBase(BaseModel):
00383     """Schema cơ sở cho điểm tích lũy thành viên."""
00384     points: int = 0
00385     userid: int
00386     menuitemid: int
00387
00388 class LoyaltyPointCreate(LoyaltyPointBase):
00389     pass
00390
00391 class LoyaltyPoint(LoyaltyPointBase):
00392     id: int
00393     updatedat: date
00394     model_config = ConfigDict(from_attributes=True)
00395
00396 # SocialShare Schemas
00397
00398 class SocialShareBase(BaseModel):
00399     """Schema cơ sở cho lượt chia sẻ mạng xã hội."""
00400     sharetype: str

```

```

00401     platform: str
00402     context: Optional[str] = None
00403     userid: int
00404     restaurantid: int
00405     menuitemid: int
00406
00407 class SocialShareCreate(SocialShareBase):
00408     pass
00409
00410 class SocialShare(SocialShareBase):
00411     id: int
00412     createdat: date
00413     model_config = ConfigDict(from_attributes=True)

```

9.113 backend/view_data.py File Reference

Namespaces

- namespace [view_data](#)

Functions

- [view_data.get_all_tables\(\)](#)
- [view_data.view_table_data](#) (Session db, model_class, str [table_name](#), int [limit](#)=10)
- [view_data.view_all_data\(\)](#)
- [view_data.view_specific_table](#) (str [table_name](#), int [limit](#)=20)

Variables

- [view_data.table_name](#) = sys.argv[1]
- int [view_data.limit](#) = int(sys.argv[2]) if len(sys.argv) > 2 else 20

9.114 view_data.py

[Go to the documentation of this file.](#)

```

00001 """
00002 Script đã sửa để hiển thị chi tiết thuộc tính của từng bản ghi
00003 """
00004
00005 from sqlalchemy.orm import Session
00006 from database import SessionLocal, engine
00007 from models import (
00008     Category, FAQ, User, Shipper, Promotion, Restaurant, MenuItem,
00009     Address, Orders, OrderItem, Payment, Delivery, Review, Notification,
00010     ChatSession, ChatMessage, UserDevice, UserFAQ, UsedPromotion,
00011     LoyaltyPoint, SocialShare
00012 )
00013 from sqlalchemy import inspect
00014
00015 def get_all_tables():
00016     """Lấy danh sách tất cả các bảng"""
00017     inspector = inspect(engine)
00018     return inspector.get_table_names()
00019
00020 def view_table_data(db: Session, model_class, table_name: str, limit: int = 10):
00021     """Xem dữ liệu của một bảng với đầy đủ thuộc tính"""
00022     print(f"\n{'='*60}")
00023     print(f"BẢNG: {table_name}")
00024     print(f"{'='*60}")
00025
00026     try:
00027         total = db.query(model_class).count()
00028         print(f"Tổng số bản ghi: {total}")
00029

```

```

00030     records = db.query(model_class).limit(limit).all()
00031
00032     if not records:
00033         print("Không có dữ liệu trong bảng này.")
00034         return
00035
00036     print(f"\nHiển thị {len(records)} bản ghi đầu tiên:")
00037     print("-" * 60)
00038
00039     # SỬA ĐOẠN NÀY ĐỂ HIỂN THỊ THUỘC TÍNH
00040     for i, record in enumerate(records, 1):
00041         # Lấy danh sách các cột thực tế từ Model
00042         columns = [column.key for column in inspect(record).mapper.column_attrs]
00043
00044         # Tạo dictionary chứa dữ liệu của các cột đó
00045         data = {col: getattr(record, col) for col in columns}
00046
00047         # In ra dưới dạng đẹp hơn
00048         print(f"[{i}] {data}")
00049
00050     except Exception as e:
00051         print(f"Lỗi khi truy vấn bảng {table_name}: {e}")
00052
00053 # ... (Các hàm view_all_data và view_specific_table giữ nguyên như cũ) ...
00054
00055 def view_all_data():
00056     """Xem dữ liệu tất cả các bảng"""
00057     db = SessionLocal()
00058     try:
00059         print("\n" + "="*60)
00060         print("DỮ LIỆU HIỆN TẠI TRONG DATABASE")
00061         print("="*60)
00062         tables = [
00063             (Category, "category"), (FAQ, "faq"), (User, "User"),
00064             (Shipper, "shipper"), (Promotion, "promotion"), (Restaurant, "restaurant"),
00065             (MenuItem, "menuitem"), (Address, "address"), (Orders, "orders"),
00066             (OrderItem, "orderitem"), (Payment, "payment"), (Delivery, "delivery"),
00067             (Review, "review"), (Notification, "notification"), (ChatSession, "chatsession"),
00068             (ChatMessage, "chatmessage"), (UserDevice, "userdevice"), (UserFAQ, "userfaq"),
00069             (UsedPromotion, "usedpromotion"), (LoyaltyPoint, "loyaltypoint"), (SocialShare, "socialshare"),
00070         ]
00071         for model_class, table_name in tables:
00072             view_table_data(db, model_class, table_name, limit=20)
00073     finally:
00074         db.close()
00075
00076 def view_specific_table(table_name: str, limit: int = 20):
00077     """Xem dữ liệu của một bảng cụ thể theo tên"""
00078     db = SessionLocal()
00079     try:
00080         table_map = {
00081             "category": Category, "faq": FAQ, "user": User, "shipper": Shipper,
00082             "promotion": Promotion, "restaurant": Restaurant, "menuitem": MenuItem,
00083             "address": Address, "orders": Orders, "orderitem": OrderItem,
00084             "payment": Payment, "delivery": Delivery, "review": Review,
00085             "notification": Notification, "chatsession": ChatSession, "chatmessage": ChatMessage,
00086             "userdevice": UserDevice, "userfaq": UserFAQ, "usedpromotion": UsedPromotion,
00087             "loyaltypoint": LoyaltyPoint, "socialshare": SocialShare,
00088         }
00089         model_class = table_map.get(table_name.lower())
00090         if not model_class:
00091             print(f"Không tìm thấy bảng '{table_name}'")
00092             return
00093         view_table_data(db, model_class, table_name, limit)
00094     finally:
00095         db.close()
00096
00097 if __name__ == "__main__":
00098     import sys
00099     if len(sys.argv) > 1:
00100         table_name = sys.argv[1]
00101         limit = int(sys.argv[2]) if len(sys.argv) > 2 else 20
00102         view_specific_table(table_name, limit)
00103     else:
00104         view_all_data()

```

Index

- `__init__`
 - `chatbot_service.ChatBotService`, [103](#)
- `__tablename__`
 - `models.Address`, [90](#)
 - `models.Category`, [97](#)
 - `models.ChatMessage`, [107](#)
 - `models.ChatSession`, [112](#)
 - `models.Delivery`, [117](#)
 - `models.FAQ`, [123](#)
 - `models.LoyaltyPoint`, [129](#)
 - `models.MenuItem`, [135](#)
 - `models.Notification`, [146](#)
 - `models.OrderItem`, [159](#)
 - `models.Orders`, [166](#)
 - `models.Payment`, [171](#)
 - `models.Promotion`, [176](#)
 - `models.Restaurant`, [182](#)
 - `models.Review`, [190](#)
 - `models.Shipper`, [198](#)
 - `models.SocialShare`, [204](#)
 - `models.UsedPromotion`, [213](#)
 - `models.User`, [219](#)
 - `models.UserDevice`, [226](#)
 - `models.UserFAQ`, [231](#)
- `_call_gemini_sdk`
 - `chatbot_service.ChatBotService`, [103](#), [105](#)
- `_upload_image_sync`
 - `firebase_utils`, [54](#)
- `ACCESS_TOKEN_EXPIRE_MINUTES`
 - `auth`, [48](#)
- `address`
 - `models.Orders`, [166](#)
 - `models.Restaurant`, [182](#)
 - `schemas.RestaurantBase`, [187](#)
- `address_detail`
 - `schemas.OrderDetail`, [158](#)
- `AddressAdapter`, [31](#)
- `AddressAdapter.AddressViewHolder`, [31](#)
- `addresses`
 - `models.User`, [219](#)
- `addressid`
 - `models.Orders`, [166](#)
- `ALGORITHM`
 - `auth`, [48](#)
- `allow_credentials`
 - `main`, [77](#)
- `allow_headers`
 - `main`, [77](#)
- `allow_methods`

- `main`, [77](#)
- `allow_origins`
 - `main`, [77](#)
- `answer`
 - `models.FAQ`, [123](#)
- `ApiService`, [31](#)
- `app`
 - `main`, [78](#)
- `app Directory Reference`, [24](#)
- `app/app Directory Reference`, [24](#)
- `app/app/src Directory Reference`, [29](#)
- `app/app/src/main Directory Reference`, [27](#)
- `app/app/src/main/java Directory Reference`, [26](#)
- `app/app/src/main/java/com Directory Reference`, [25](#)
- `app/app/src/main/java/com/example Directory Reference`, [26](#)
- `app/app/src/main/java/com/example/myapp Directory Reference`, [27](#)
- `app/app/src/main/java/com/example/myapp/adapters Directory Reference`, [23](#)
- `app/app/src/main/java/com/example/myapp/adapters/AddressAc Directory Reference`, [237](#)
- `app/app/src/main/java/com/example/myapp/adapters/ChatMess Directory Reference`, [238](#)
- `app/app/src/main/java/com/example/myapp/adapters/FAQAdap Directory Reference`, [238](#), [239](#)
- `app/app/src/main/java/com/example/myapp/adapters/MenuItem Directory Reference`, [239](#)
- `app/app/src/main/java/com/example/myapp/adapters/MenuItem Directory Reference`, [241](#)
- `app/app/src/main/java/com/example/myapp/adapters/MyFirebas Directory Reference`, [242](#)
- `app/app/src/main/java/com/example/myapp/adapters/Notificatio Directory Reference`, [243](#)
- `app/app/src/main/java/com/example/myapp/adapters/Restauran Directory Reference`, [244](#)
- `app/app/src/main/java/com/example/myapp/api Directory Reference`, [24](#)
- `app/app/src/main/java/com/example/myapp/api/ApiService.kt, Directory Reference`, [245](#)
- `app/app/src/main/java/com/example/myapp/api/FcmTokenRegis Directory Reference`, [249](#)
- `app/app/src/main/java/com/example/myapp/api/NetworkConfig.k Directory Reference`, [251](#)
- `app/app/src/main/java/com/example/myapp/api/RetrofitClient.k Directory Reference`, [251](#), [252](#)
- `app/app/src/main/java/com/example/myapp/screens Directory Reference`, [28](#)

- get_current_user, [47](#)
 - get_password_hash, [47](#)
 - oauth2_scheme, [49](#)
 - pwd_context, [49](#)
 - SECRET_KEY, [49](#)
 - verify_password, [48](#)
- avg_rating
 - schemas.MenuItemWithReviews, [144](#)
- backend Directory Reference, [25](#)
- backend/auth.py, [320](#)
- backend/chatbot_service.py, [321](#), [322](#)
- backend/check_db.py, [323](#)
- backend/database.py, [324](#)
- backend/firebase_utils.py, [324](#), [325](#)
- backend/main.py, [326](#), [327](#)
- backend/models.py, [336](#), [337](#)
- backend/notification_service.py, [340](#)
- backend/payment_service.py, [341](#), [342](#)
- backend/schemas.py, [342](#), [344](#)
- backend/view_data.py, [349](#)
- Base
 - database, [53](#)
- bind
 - main, [78](#)
- bindOrder
 - AppCompatActivity, [32](#)
- bindOrderCard
 - AppCompatActivity, [33](#)
- bindTopTwoOrders
 - AppCompatActivity, [33](#)
- build_menuitem_with_reviews
 - main, [58](#)
- CartItemAdapter, [49](#)
- CartItemAdapter.CartItemViewHolder, [49](#)
 - CartItemViewHolder, [50](#)
- CartItemViewHolder
 - CartItemAdapter.CartItemViewHolder, [50](#)
- category
 - models.MenuItem, [135](#)
- categoryid
 - models.MenuItem, [135](#)
- chat_sessions
 - models.User, [219](#)
- chat_with_bot
 - main, [59](#)
- chatbot_service, [50](#)
 - GEMINI_API_KEY, [51](#)
- chatbot_service.ChatBotService, [102](#)
 - __init__, [103](#)
 - _call_gemini_sdk, [103](#), [105](#)
 - client, [105](#)
 - db, [105](#)
 - get_app_context, [103](#)
 - get_response, [104](#)
 - user_id, [105](#)
- ChatMessageAdapter, [51](#)
- ChatMessageAdapter.MessageViewHolder, [51](#)
- check_db, [51](#)
 - conn, [51](#)
 - cursor, [51](#)
 - fcml, [51](#)
 - notifs, [52](#)
 - tables, [52](#)
- client
 - chatbot_service.ChatBotService, [105](#)
- close_time
 - models.Restaurant, [182](#)
 - schemas.RestaurantBase, [187](#)
- code
 - models.Promotion, [176](#)
- comment
 - models.Review, [190](#)
 - schemas.ReviewBase, [194](#)
 - schemas.ReviewResponse, [197](#)
- conn
 - check_db, [51](#)
- content
 - models.Notification, [146](#)
- context
 - models.SocialShare, [204](#)
 - schemas.SocialShareBase, [209](#)
- create_access_token
 - auth, [46](#)
- create_address
 - main, [60](#)
- create_menu_item
 - main, [60](#)
- create_notification
 - main, [61](#)
- create_order
 - main, [62](#)
- create_payment
 - main, [62](#)
- create_restaurant
 - main, [63](#)
- create_review
 - main, [64](#)
- create_user
 - main, [64](#)
- createdat
 - models.ChatSession, [112](#)
 - models.Notification, [146](#)
 - models.Orders, [166](#)
 - models.SocialShare, [205](#)
- currentAddress
 - AppCompatActivity, [42](#)
- cursor
 - check_db, [51](#)
- database, [52](#)
 - Base, [53](#)
 - engine, [53](#)
 - get_db, [52](#)
 - SessionLocal, [53](#)
 - SQLALCHEMY_DATABASE_URL, [53](#)
- db

- chatbot_service.ChatBotService, 105
- deliveries
 - models.Shipper, 198
- delivery
 - models.Orders, 167
- deliverytime
 - models.Delivery, 117
 - schemas.DeliveryBase, 120
- description
 - models.Category, 97
 - models.MenuItem, 135
 - models.Restaurant, 182
 - models.Shipper, 198
 - schemas.CategoryBase, 101
 - schemas.MenuItemBase, 139
 - schemas.RestaurantBase, 187
 - schemas.ShipperBase, 202
- detail
 - models.Address, 90
 - schemas.AddressUpdate, 96
- device_token
 - models.UserDevice, 226
- device_type
 - models.UserDevice, 226
 - schemas.UserDeviceBase, 229
- discounttype
 - models.Promotion, 176
- discountvalue
 - models.Promotion, 176
- displayMenuItems
 - AppCompatActivity, 34
- displayRestaurants
 - AppCompatActivity, 34
- edtPhone
 - AppCompatActivity, 42
- email
 - models.User, 219
 - schemas.TokenData, 212
- engine
 - database, 53
- expiredate
 - models.Promotion, 176
- faq
 - models.UserFAQ, 231
- FAQAdapter, 53
- FAQAdapter.FAQViewHolder, 53
- faqid
 - models.UserFAQ, 231
- fcm
 - check_db, 51
- FcmTokenRegistrar, 53
- FIREBASE_CREDENTIALS_PATH
 - firebase_utils, 55
- FIREBASE_STORAGE_BUCKET
 - firebase_utils, 55
- firebase_utils, 53
- _upload_image_sync, 54
- FIREBASE_CREDENTIALS_PATH, 55
- FIREBASE_STORAGE_BUCKET, 55
- init_firebase, 54
- upload_image, 55
- FirebaseMessagingService, 56
 - onMessageReceived, 56
 - showNotification, 56
- firstOrderId
 - AppCompatActivity, 42
- foodId
 - AppCompatActivity, 42
- GEMINI_API_KEY
 - chatbot_service, 51
- generate_vnpay_url
 - payment_service, 82
- get_all_tables
 - view_data, 85
- get_app_context
 - chatbot_service.ChatBotService, 103
- get_current_user
 - auth, 47
- get_db
 - database, 52
- get_password_hash
 - auth, 47
- get_response
 - chatbot_service.ChatBotService, 104
- host
 - main, 78
- icCart
 - AppCompatActivity, 43
- icHeart
 - AppCompatActivity, 43
- icHome
 - AppCompatActivity, 43
- icProfile
 - AppCompatActivity, 43
- id
 - models.Address, 90
 - models.Category, 97
 - models.ChatMessage, 107
 - models.ChatSession, 112
 - models.Delivery, 117
 - models.FAQ, 123
 - models.LoyaltyPoint, 129
 - models.MenuItem, 136
 - models.Notification, 146
 - models.OrderItem, 159
 - models.Orders, 167
 - models.Payment, 171
 - models.Promotion, 176
 - models.Restaurant, 182
 - models.Review, 190
 - models.Shipper, 199
 - models.SocialShare, 205
 - models.UsedPromotion, 213

- models.User, 219
- models.UserDevice, 226
- models.UserFAQ, 232
- if
 - AppCompatActivity, 34
- image_url
 - models.MenuItem, 136
 - models.Restaurant, 182
 - schemas.MenuItemBase, 139
 - schemas.OrderItemDetail, 165
 - schemas.RestaurantBase, 187
- init_firebase
 - firebase_utils, 54
- is_available
 - models.MenuItem, 136
 - schemas.MenuItemBase, 140
- isactive
 - models.FAQ, 123
 - schemas.FAQBase, 126
- isread
 - models.Notification, 147
 - schemas.NotificationBase, 151
- isRequestingVnPay
 - AppCompatActivity, 43
- last_active
 - models.UserDevice, 226
- limit
 - view_data, 88
- loadOrders
 - AppCompatActivity, 35
- login_for_access_token
 - main, 65
- loyalty_points
 - models.User, 219
- main, 57
 - allow_credentials, 77
 - allow_headers, 77
 - allow_methods, 77
 - allow_origins, 77
 - app, 78
 - bind, 78
 - build_menuitem_with_reviews, 58
 - chat_with_bot, 59
 - create_address, 60
 - create_menu_item, 60
 - create_notification, 61
 - create_order, 62
 - create_payment, 62
 - create_restaurant, 63
 - create_review, 64
 - create_user, 64
 - host, 78
 - login_for_access_token, 65
 - mark_notification_as_read, 66
 - port, 78
 - read_all_menu_items, 66
 - read_categories, 67
 - read_faqs, 67
 - read_menu_item_detail, 67
 - read_order_detail, 68
 - read_promotions, 69
 - read_restaurant, 69
 - read_restaurant_menu, 69
 - read_restaurants, 70
 - read_root, 70
 - read_user_addresses, 70
 - read_user_notifications, 71
 - read_user_orders, 71
 - read_user_profile_summary, 71
 - read_users, 72
 - read_users_me, 72
 - search_menu_items_by_category_name, 72
 - search_menu_items_by_name, 73
 - search_restaurants_by_name, 74
 - update_address, 74
 - update_device_token, 75
 - update_order_status, 75
 - vnpay_return, 76
- mark_notification_as_read
 - main, 66
- menu_item
 - models.LoyaltyPoint, 129
 - models.OrderItem, 159
 - models.Review, 190
 - models.SocialShare, 205
- menu_items
 - models.Category, 98
 - models.Restaurant, 182
 - schemas.Restaurant, 185
- menuitem_name
 - schemas.OrderItemDetail, 165
- MenuItemAdapter, 78
- MenuItemAdapter.MenuViewHolder, 78
- MenuItemAdapterSmall, 78
- MenuItemAdapterSmall.MenuViewHolder, 79
- menuitemid
 - models.LoyaltyPoint, 129
 - models.OrderItem, 160
 - models.Review, 190
 - models.SocialShare, 205
 - schemas.ReviewBase, 194
- menuItems
 - AppCompatActivity, 43
- message
 - models.ChatMessage, 107
- messages
 - models.ChatSession, 113
 - schemas.ChatSession, 114
- method
 - models.Payment, 171
- minordervalue
 - models.Promotion, 176
- model_config
 - schemas.Address, 92
 - schemas.Category, 99

- schemas.ChatMessage, 109
- schemas.ChatSession, 114
- schemas.Delivery, 119
- schemas.FAQ, 125
- schemas.LoyaltyPoint, 131
- schemas.MenuItem, 138
- schemas.MenuItemWithRestaurant, 143
- schemas.MenuItemWithReviews, 144
- schemas.Notification, 149
- schemas.Order, 154
- schemas.OrderItem, 162
- schemas.Payment, 173
- schemas.Promotion, 178
- schemas.Restaurant, 185
- schemas.Review, 193
- schemas.ReviewResponse, 197
- schemas.Shipper, 201
- schemas.SocialShare, 207
- schemas.UsedPromotion, 215
- schemas.User, 222
- schemas.UserDevice, 228
- schemas.UserFAQ, 234
- models, 79
- models.Address, 89
 - __tablename__, 90
 - detail, 90
 - id, 90
 - orders, 90
 - phone, 90
 - user, 91
 - userid, 91
- models.Category, 96
 - __tablename__, 97
 - description, 97
 - id, 97
 - menu_items, 98
 - name, 98
 - type, 98
- models.ChatMessage, 106
 - __tablename__, 107
 - id, 107
 - message, 107
 - senderrole, 107
 - sentat, 107
 - session, 107
 - sessionid, 107
- models.ChatSession, 111
 - __tablename__, 112
 - createdat, 112
 - id, 112
 - messages, 113
 - notifications, 113
 - status, 113
 - user, 113
 - userid, 113
- models.Delivery, 116
 - __tablename__, 117
 - deliverytime, 117
 - id, 117
 - order, 117
 - orderid, 117
 - shipper, 117
 - shipperid, 117
 - status, 117
- models.FAQ, 122
 - __tablename__, 123
 - answer, 123
 - id, 123
 - isactive, 123
 - question, 123
 - user_faqs, 123
- models.LoyaltyPoint, 128
 - __tablename__, 129
 - id, 129
 - menu_item, 129
 - menuitemid, 129
 - points, 129
 - updatedat, 129
 - user, 129
 - userid, 130
- models.MenuItem, 134
 - __tablename__, 135
 - category, 135
 - categoryid, 135
 - description, 135
 - id, 136
 - image_url, 136
 - is_available, 136
 - name, 136
 - order_items, 136
 - price, 136
 - restaurant, 136
 - restaurantid, 136
 - reviews, 137
 - social_shares, 137
- models.Notification, 145
 - __tablename__, 146
 - content, 146
 - createdat, 146
 - id, 146
 - isread, 147
 - order, 147
 - orderid, 147
 - session, 147
 - sessionid, 147
 - title, 147
 - type, 147
 - user, 147
 - userid, 148
- models.OrderItem, 158
 - __tablename__, 159
 - id, 159
 - menu_item, 159
 - menuitemid, 160
 - order, 160
 - orderid, 160

- price, 160
- quantity, 160
- models.Orders, 165
 - __tablename__, 166
 - address, 166
 - addressid, 166
 - createdat, 166
 - delivery, 167
 - id, 167
 - notifications, 167
 - order_items, 167
 - payments, 167
 - preorderdate, 167
 - preordertime, 167
 - restaurant, 167
 - restaurantid, 168
 - status, 168
 - totalprice, 168
 - used_promotions, 168
 - user, 168
 - userid, 168
- models.Payment, 170
 - __tablename__, 171
 - id, 171
 - method, 171
 - order, 171
 - orderid, 171
 - status, 171
- models.Promotion, 175
 - __tablename__, 176
 - code, 176
 - discounttype, 176
 - discountvalue, 176
 - expiredate, 176
 - id, 176
 - minordervalue, 176
 - status, 177
 - used_promotions, 177
- models.Restaurant, 181
 - __tablename__, 182
 - address, 182
 - close_time, 182
 - description, 182
 - id, 182
 - image_url, 182
 - menu_items, 182
 - name, 183
 - open_time, 183
 - orders, 183
 - phone_number, 183
 - rating, 183
 - reviews, 183
 - social_shares, 183
 - status, 183
- models.Review, 189
 - __tablename__, 190
 - comment, 190
 - id, 190
 - menu_item, 190
 - menuitemid, 190
 - order, 190
 - orderid, 190
 - rating, 191
 - restaurant, 191
 - restaurantid, 191
 - user, 191
 - userid, 191
- models.Shipper, 197
 - __tablename__, 198
 - deliveries, 198
 - description, 198
 - id, 199
 - name, 199
 - phone, 199
 - rating, 199
 - status, 199
- models.SocialShare, 203
 - __tablename__, 204
 - context, 204
 - createdat, 205
 - id, 205
 - menu_item, 205
 - menuitemid, 205
 - platform, 205
 - restaurant, 205
 - restaurantid, 205
 - sharetype, 205
 - user, 206
 - userid, 206
- models.UsedPromotion, 212
 - __tablename__, 213
 - id, 213
 - order, 213
 - orderid, 213
 - promotion, 213
 - promotionid, 214
 - usedat, 214
- models.User, 218
 - __tablename__, 219
 - addresses, 219
 - chat_sessions, 219
 - email, 219
 - id, 219
 - loyalty_points, 219
 - name, 219
 - notifications, 220
 - orders, 220
 - password, 220
 - phone, 220
 - reviews, 220
 - role, 220
 - social_shares, 220
 - user_devices, 220
 - user_faqs, 221
 - username, 221
- models.UserDevice, 225

- __tablename__, 226
 - device_token, 226
 - device_type, 226
 - id, 226
 - last_active, 226
 - user, 226
 - userid, 226
- models.UserFAQ, 230
 - __tablename__, 231
 - faq, 231
 - faqid, 231
 - id, 232
 - user, 232
 - userid, 232
 - viewedat, 232
- món
 - AppCompatActivity, 43
- name
 - models.Category, 98
 - models.MenuItem, 136
 - models.Restaurant, 183
 - models.Shipper, 199
 - models.User, 219
- nay
 - AppCompatActivity, 44
- NetworkConfig, 79
- notification_service, 79
 - notify_user, 80
 - send_fcm_notification, 80
- NotificationAdapter, 81
- NotificationAdapter.NotificationViewHolder, 81
- notifications
 - models.ChatSession, 113
 - models.Orders, 167
 - models.User, 220
- notifs
 - check_db, 52
- notify_user
 - notification_service, 80
- oauth2_scheme
 - auth, 49
- onCreate
 - AppCompatActivity, 35
- onMessageReceived
 - FirebaseMessagingService, 56
- open_time
 - models.Restaurant, 183
 - schemas.RestaurantBase, 187
- openOrderDetail
 - AppCompatActivity, 36
- order
 - models.Delivery, 117
 - models.Notification, 147
 - models.OrderItem, 160
 - models.Payment, 171
 - models.Review, 190
 - models.UsedPromotion, 213
- order_items
 - models.MenuItem, 136
 - models.Orders, 167
 - schemas.Order, 154
 - schemas.OrderCreate, 156
 - schemas.OrderDetail, 158
- orderId
 - AppCompatActivity, 44
- orderid
 - models.Delivery, 117
 - models.Notification, 147
 - models.OrderItem, 160
 - models.Payment, 171
 - models.Review, 190
 - models.UsedPromotion, 213
 - schemas.NotificationBase, 151
- orders
 - models.Address, 90
 - models.Restaurant, 183
 - models.User, 220
- password
 - models.User, 220
- payment_service, 81
 - generate_vnpay_url, 82
 - VNP_HASH_SECRET, 82
 - VNP_RETURN_URL, 82
 - VNP_TMNCODE, 83
 - VNP_URL, 83
- payments
 - models.Orders, 167
- phone
 - models.Address, 90
 - models.Shipper, 199
 - models.User, 220
 - schemas.AddressBase, 94
 - schemas.AddressUpdate, 96
- phone_number
 - models.Restaurant, 183
- platform
 - models.SocialShare, 205
- points
 - models.LoyaltyPoint, 129
 - schemas.LoyaltyPointBase, 133
- pointsCard2
 - AppCompatActivity, 44
- pointsCard3
 - AppCompatActivity, 44
- pointsCard4
 - AppCompatActivity, 44
- port
 - main, 78
- preorderdate
 - models.Orders, 167
 - schemas.OrderBase, 155
- preordertime
 - models.Orders, 167
 - schemas.OrderBase, 155
- price

- models.MenuItem, 136
 - models.OrderItem, 160
- promotion
 - models.UsedPromotion, 213
- promotionid
 - models.UsedPromotion, 214
- pwd_context
 - auth, 49
- quantity
 - models.OrderItem, 160
- question
 - models.FAQ, 123
- rating
 - models.Restaurant, 183
 - models.Review, 191
 - models.Shipper, 199
 - schemas.RestaurantBase, 187
 - schemas.ShipperBase, 202
- read_all_menu_items
 - main, 66
- read_categories
 - main, 67
- read_faqs
 - main, 67
- read_menu_item_detail
 - main, 67
- read_order_detail
 - main, 68
- read_promotions
 - main, 69
- read_restaurant
 - main, 69
- read_restaurant_menu
 - main, 69
- read_restaurants
 - main, 70
- read_root
 - main, 70
- read_user_addresses
 - main, 70
- read_user_notifications
 - main, 71
- read_user_orders
 - main, 71
- read_user_profile_summary
 - main, 71
- read_users
 - main, 72
- read_users_me
 - main, 72
- recyclerView
 - AppCompatActivity, 44
- reorderCurrentOrder
 - AppCompatActivity, 37
- resolveUserIdForPoints
 - AppCompatActivity, 37
- restaurant
 - AppCompatActivity, 44
 - models.MenuItem, 136
 - models.Orders, 167
 - models.Review, 191
 - models.SocialShare, 205
- restaurant__name
 - schemas.MenuItemWithRestaurant, 143
 - schemas.MenuItemWithReviews, 145
 - schemas.OrderDetail, 158
- RestaurantAdapter, 83
- RestaurantAdapter.RestaurantViewHolder, 83
- restaurantId
 - AppCompatActivity, 45
- restaurantid
 - models.MenuItem, 136
 - models.Orders, 168
 - models.Review, 191
 - models.SocialShare, 205
 - schemas.ReviewBase, 194
- restaurants
 - AppCompatActivity, 45
- RetrofitClient, 83
- reviews
 - models.MenuItem, 137
 - models.Restaurant, 183
 - models.User, 220
 - schemas.MenuItemWithReviews, 145
- role
 - models.User, 220
- rvAddresses
 - AppCompatActivity, 45
- rvCart
 - AppCompatActivity, 45
- rvNotifications
 - AppCompatActivity, 45
- rvPreOrderCart
 - AppCompatActivity, 45
- schemas, 83
- schemas.Address, 91
 - model_config, 92
- schemas.AddressBase, 93
 - phone, 94
- schemas.AddressCreate, 94
- schemas.AddressUpdate, 95
 - detail, 96
 - phone, 96
- schemas.Category, 98
 - model_config, 99
- schemas.CategoryBase, 100
 - description, 101
 - type, 101
- schemas.CategoryCreate, 101
- schemas.ChatMessage, 108
 - model_config, 109
- schemas.ChatMessageBase, 109
- schemas.ChatMessageCreate, 110
- schemas.ChatSession, 114
 - messages, 114

- model_config, 114
- schemas.ChatSessionCreate, 115
- schemas.Delivery, 118
 - model_config, 119
- schemas.DeliveryBase, 119
 - deliverytime, 120
- schemas.DeliveryCreate, 121
- schemas.FAQ, 124
 - model_config, 125
- schemas.FAQBase, 125
 - isactive, 126
- schemas.FAQCreate, 127
- schemas.LoyaltyPoint, 130
 - model_config, 131
- schemas.LoyaltyPointBase, 132
 - points, 133
- schemas.LoyaltyPointCreate, 133
- schemas.MenuItem, 137
 - model_config, 138
- schemas.MenuItemBase, 139
 - description, 139
 - image_url, 139
 - is_available, 140
- schemas.MenuItemCreate, 140
- schemas.MenuItemWithRestaurant, 142
 - model_config, 143
 - restaurant_name, 143
- schemas.MenuItemWithReviews, 143
 - avg_rating, 144
 - model_config, 144
 - restaurant_name, 145
 - reviews, 145
- schemas.Notification, 148
 - model_config, 149
- schemas.NotificationBase, 150
 - isread, 151
 - orderid, 151
 - sessionid, 151
- schemas.NotificationCreate, 151
- schemas.Order, 153
 - model_config, 154
 - order_items, 154
- schemas.OrderBase, 154
 - preorderdate, 155
 - preordertime, 155
- schemas.OrderCreate, 155
 - order_items, 156
- schemas.OrderDetail, 157
 - address_detail, 158
 - order_items, 158
 - restaurant_name, 158
- schemas.OrderItem, 161
 - model_config, 162
- schemas.OrderItemBase, 162
- schemas.OrderItemCreate, 163
- schemas.OrderItemDetail, 164
 - image_url, 165
 - menuitem_name, 165
- schemas.OrderStatusUpdate, 169
- schemas.Payment, 172
 - model_config, 173
- schemas.PaymentBase, 173
- schemas.PaymentCreate, 174
- schemas.Promotion, 177
 - model_config, 178
- schemas.PromotionBase, 179
- schemas.PromotionCreate, 180
- schemas.Restaurant, 184
 - menu_items, 185
 - model_config, 185
- schemas.RestaurantBase, 186
 - address, 187
 - close_time, 187
 - description, 187
 - image_url, 187
 - open_time, 187
 - rating, 187
- schemas.RestaurantCreate, 188
- schemas.Review, 192
 - model_config, 193
- schemas.ReviewBase, 193
 - comment, 194
 - menuitemid, 194
 - restaurantid, 194
- schemas.ReviewCreate, 195
- schemas.ReviewResponse, 196
 - comment, 197
 - model_config, 197
 - user_name, 197
- schemas.Shipper, 200
 - model_config, 201
- schemas.ShipperBase, 201
 - description, 202
 - rating, 202
- schemas.ShipperCreate, 202
- schemas.SocialShare, 206
 - model_config, 207
- schemas.SocialShareBase, 208
 - context, 209
- schemas.SocialShareCreate, 209
- schemas.Token, 210
- schemas.TokenData, 211
 - email, 212
- schemas.UsedPromotion, 214
 - model_config, 215
- schemas.UsedPromotionBase, 216
- schemas.UsedPromotionCreate, 217
- schemas.User, 221
 - model_config, 222
- schemas.UserBase, 223
- schemas.UserCreate, 224
- schemas.UserDevice, 227
 - model_config, 228
- schemas.UserDeviceBase, 228
 - device_type, 229
- schemas.UserDeviceCreate, 229

- schemas.UserFAQ, 233
 - model_config, 234
- schemas.UserFAQBase, 234
- schemas.UserFAQCreate, 235
- schemas.UserProfileSummary, 236
- search_menu_items_by_category_name
 - main, 72
- search_menu_items_by_name
 - main, 73
- search_restaurants_by_name
 - main, 74
- searchByCategory
 - AppCompatActivity, 38
- searchMenuItems
 - AppCompatActivity, 38
- searchRestaurants
 - AppCompatActivity, 39
- SECRET_KEY
 - auth, 49
- selectedPromotion
 - AppCompatActivity, 45
- send_fcm_notification
 - notification_service, 80
- senderrole
 - models.ChatMessage, 107
- sentat
 - models.ChatMessage, 107
- session
 - models.ChatMessage, 107
 - models.Notification, 147
- sessionid
 - models.ChatMessage, 107
 - models.Notification, 147
 - schemas.NotificationBase, 151
- SessionLocal
 - database, 53
- sharetype
 - models.SocialShare, 205
- shipper
 - models.Delivery, 117
- shipperid
 - models.Delivery, 117
- show
 - AppCompatActivity, 40
- showNotification
 - FirebaseMessagingService, 56
- social_shares
 - models.MenuItem, 137
 - models.Restaurant, 183
 - models.User, 220
- SQLALCHEMY_DATABASE_URL
 - database, 53
- startActivity
 - AppCompatActivity, 40
- status
 - models.ChatSession, 113
 - models.Delivery, 117
 - models.Orders, 168
 - models.Payment, 171
 - models.Promotion, 177
 - models.Restaurant, 183
 - models.Shipper, 199
- table_name
 - view_data, 88
- tables
 - check_db, 52
- title
 - models.Notification, 147
- totalprice
 - models.Orders, 168
- type
 - models.Category, 98
 - models.Notification, 147
 - schemas.CategoryBase, 101
- update_address
 - main, 74
- update_device_token
 - main, 75
- update_order_status
 - main, 75
- updateAddress
 - AppCompatActivity, 41
- updatedat
 - models.LoyaltyPoint, 129
- updateOrderStatusToCODConfirmed
 - AppCompatActivity, 41
- upload_image
 - firebase_utils, 55
- used_promotions
 - models.Orders, 168
 - models.Promotion, 177
- usedat
 - models.UsedPromotion, 214
- user
 - models.Address, 91
 - models.ChatSession, 113
 - models.LoyaltyPoint, 129
 - models.Notification, 147
 - models.Orders, 168
 - models.Review, 191
 - models.SocialShare, 206
 - models.UserDevice, 226
 - models.UserFAQ, 232
- user_devices
 - models.User, 220
- user_faqs
 - models.FAQ, 123
 - models.User, 221
- user_id
 - chatbot_service.ChatBotService, 105
- user_name
 - schemas.ReviewResponse, 197
- userid
 - models.Address, 91
 - models.ChatSession, 113

- models.LoyaltyPoint, [130](#)
 - models.Notification, [148](#)
 - models.Orders, [168](#)
 - models.Review, [191](#)
 - models.SocialShare, [206](#)
 - models.UserDevice, [226](#)
 - models.UserFAQ, [232](#)
- username
 - models.User, [221](#)
- verify__password
 - auth, [48](#)
- view__all__data
 - view__data, [85](#)
- view__data, [85](#)
 - get__all__tables, [85](#)
 - limit, [88](#)
 - table__name, [88](#)
 - view__all__data, [85](#)
 - view__specific__table, [86](#)
 - view__table__data, [87](#)
- view__specific__table
 - view__data, [86](#)
- view__table__data
 - view__data, [87](#)
- viewedat
 - models.UserFAQ, [232](#)
- VNP_HASH_SECRET
 - payment_service, [82](#)
- VNP_RETURN_URL
 - payment_service, [82](#)
- VNP_TMNCODE
 - payment_service, [83](#)
- VNP_URL
 - payment_service, [83](#)
- vnpay_return
 - main, [76](#)